



MONASH University

High-Level Modelling and Solving for Online, Real-Time, and Multiagent Combinatorial Optimisation

— by —

Alexander Ek

*Bachelor & Master of Science, Computer Science,
Uppsala University, Uppsala, Sweden*

A thesis submitted for the degree of Doctor of Philosophy at
Monash University in 2022
Faculty of Information Technology
Department of Data Science and AI

October 2022

© ALEXANDER EK 2022

I certify that I have made all reasonable efforts to secure copyright permissions for third-party content included in this thesis and have not knowingly added copyright content to my work without the owner's permission.

ABSTRACT

Combinatorial problems are notoriously hard decision-making problems to solve and appear in many fields and industries. Examples include vehicle routing in logistics, scheduling in manufacturing, resource allocation in finance and humanitarian efforts, rostering staff in health institutions, and control systems in automation endeavours. The field of combinatorial optimisation develops techniques that help find quality solutions to these kinds of problems. The importance of finding high-quality solutions to these problems has resulted in impressive advances in this field, including solving technologies such as CP, MIP, and SAT, as well as modelling languages such as MiniZinc, Essence, OPL, and AMPL.

Many modern combinatorial problems are *dynamic*, i.e., new information continually arrives, and the solution must repeatedly be refined to update existing decisions and make new ones if needed. Typically, building a system to solve these problems requires substantial ad hoc coding, despite commonalities therebetween. Unfortunately, combining dynamism and combinatorial optimisation reveals a gap in the literature that precludes using the modelling and solving technologies mentioned above.

This thesis helps fill this gap by presenting a novel framework for modelling and solving dynamic combinatorial problems that avoids ad hoc coding by taking advantage of and extending previous advances in modelling languages. The framework allows modellers to declaratively specify the dynamism of the problem being solved, which is then used to solve the problem with online and real-time optimisation automatically. This framework is then enhanced with a systematic *garbage collection* mechanism, which ensures that the internal representation of the evolving and growing problem remains small, even as time progresses. This is complex because such a mechanism must account for how past decisions affect future ones. Without garbage collection, internal representations quickly get too large, impeding compilation and solving time. The enhanced framework is implemented and evaluated empirically.

Many combinatorial problems concern multiple stakeholders with conflicting demands, making fairness and balance essential. Substantial work exists in adjacent

areas, such as economics, social choice theory, game theory, and multi-objective optimisation. However, to the best of our knowledge, the intersection of combinatorial optimisation, fairness, and dynamism has never been studied. Because of the multi-faceted complexity, formal guarantees on fairness are hard to give, and empirical experimentation becomes necessary. This thesis presents a new framework (in which the aforementioned framework can be used) capable of addressing the issue of fairness in dynamic combinatorial problems, and which allows easy experimentation with different ideas of fairness. The framework is founded upon an adaptive priority re-balancing principle that gives unjustly treated stakeholders higher priority in future decision-making. The results show that the framework can easily be used to achieve remarkable increases in fairness without significant loss in overall resource utilisation efficiency.

Even without dynamism, there is a significant gap in combinatorial optimisation with fairness considerations of inequality. This thesis presents two novel constraint propagation algorithms for two inequality metrics: variance and the Gini coefficient. They can produce explanations required for clause learning, making them usable in many state-of-the-art solving methods. These algorithms allow speedups in solving problems in which using these two inequality metrics is desirable.

DECLARATION

This thesis is an original work of my research and contains no material which has been accepted for the award of any other degree or diploma at any university or equivalent institution and that, to the best of my knowledge and belief, this thesis contains no material previously published or written by another person, except where due reference is made in the text of the thesis.

Alexander Ek

Friday 21 October 2022

CONTENTS

Acknowledgements	8
Publications During Enrolment	10
1 Introduction	11
1.1 Motivation and Problem Statement	14
1.2 Outline and Contribution to Knowledge	18
2 Background	23
2.1 Combinatorial Optimisation	23
2.2 Modelling Combinatorial Problems	38
2.3 Uncertainty and Dynamism	48
2.4 Online Combinatorial Optimisation	52
2.5 Fairness, Balance, and Dispersion	60
2.6 Concluding Remarks	62
3 High-Level Modelling for Online Combinatorial Optimisation	65
3.1 Related Work	66
3.2 A New Formalism	70
3.3 Modelling Language Design	78
3.4 Implementation	93
3.5 Empirical Evaluation	96
3.6 Discussion and Future Work	99
3.7 Chapter Conclusion	100
4 Automatic Garbage Collection for Online Combinatorial Optimisation	103
4.1 Preliminaries	104
4.2 Garbage Collection: A Naïve Approach	105
4.3 A Formalism of Garbage Collection	108

4.4	Aggregation Types and Rules	113
4.5	Implementation in Online MiniZinc	129
4.6	Experimental Results	134
4.7	Related Work	139
4.8	Discussion and Future Work	141
4.9	Chapter Conclusion	141
5	Fairness in Online Combinatorial Optimisation	143
5.1	Multiagent Job-Shop Scheduling	145
5.2	Background	146
5.3	Fairness in Online Combinatorial Optimisation	162
5.4	Related Work	171
5.5	Towards an Online Fairness Framework	174
5.6	Experimental Evaluation	180
5.7	Discussion and Future Work	190
5.8	Chapter Conclusion	193
6	Explaining Propagators for Gini and Spread with Variable Mean	195
6.1	Preliminaries	196
6.2	The Spread Constraint	200
6.3	The Gini Coefficient Constraint	205
6.4	Experimental Evaluation	211
6.5	Related Work	216
6.6	Chapter Conclusion	217
7	Thesis Conclusion	219
7.1	Modelling and Solving	219
7.2	Fairness and Balance	221
A	Review of Notation	225

ACKNOWLEDGEMENTS

This thesis is the culmination of more than three and a half years' worth of research done throughout my candidature. Many people helped make this candidature what it is. I want to thank everybody involved, no matter how minor.

First and foremost, I would like to express my deepest gratitude to my supervisors, Maria Garcia de la Banda, Andreas Schutt, Peter J. Stuckey, and Guido Tack; I could not have wished for a better team. Your guidance, advice, and feedback have been invaluable, and it has shown that you have cared about my progress and research.

I must also extend special thanks to my collaborators, Alexander Schiendorfer and Alexander Knapp, for the valuable discussions, insights, and supervision. In addition, I would like to extend extra gratitude to Mark Wallace, Graeme Gange, and Michael Wybrow for keeping an eye on my progress via the many milestone review seminars and giving valuable advice.

I would be amiss to not also extend my gratitude to the supervisor team of my Master thesis project, Justin Pearson and Pierre Flener, for setting me up for success. You helped me find this wonderful research area called combinatorial optimisation and this wonderful research group called the Monash Optimisation Group.

I would also like to acknowledge the authors of doctoral theses that inspired me and helped me structure this thesis: Peter Backeman, Gustav Björdal, Geoffrey G. Chu, Jip J. Dekker, David Hemmi, Edward Lam, Kevin Leo, Christopher David Mears, Joseph Scott, and Guido Tack.

Proper thanks and acknowledgements are due to Allison Mitchell of Monash University and Anna-Maria Johansson of Uppsala University. Your administrative magic has been invaluable.

I would like to acknowledge Monash University and CSIRO Data61 for offering me this fantastic opportunity and enabling this journey of research and self-development. This candidature has been jointly funded by Monash University, The Faculty of IT of Monash University, and CSIRO Data61. I would also like to acknowledge the Monash

University Graduate Research Completion Award for its support. Finally, I would like to acknowledge the Association for Constraint Programming for their invaluable travel-enabling financial support.

All my friends and colleagues at Monash University deserve my gratitude as well. In (somewhat) chronological order of me getting to know you, I would like to thank: Jip, David, Kevin, Pierre, Graeme, Kiana, Senthoo, Eddie, Mariam, Anjana, Dora, Arthur, Dan, Dani, Mark, Saman, Ali, Semini, Peter, Ariel, Gleb, Linnea, Alexey, Henk, Jason, Emely, Kelvin, Zahra. Your shenanigans have kept me sane. I thank you for your interest in my research and our intriguing discussions. In a similar vein, I would like to extend my sincere thanks to my friends overseas: Dennis, Peyman, Filip, Micke, Adde, Rautila, Scherman, and Sander. You have likewise kept me sane.

I cannot forget Russell and Michelle. I am incredibly grateful to you for welcoming me into your family when I first came to Australia as a lost PhD student. Thanks to you, I have always had the comfort of knowing that a warm home (and a cold beer) is a stone's throw away.

My success would not have been possible without the constant, unwavering support of my mother, Diana, and my father, Anders. Similarly, I must also thank Mari. You three have helped me look to the future, focus on what is important, and become a better version of myself.

Most of all, I would like to express my appreciation to my dearest, Evelina. Your love and support have been immeasurable and inveterate. I love you to Orion and back.

— Alexander Ek

PUBLICATIONS DURING ENROLMENT

Much of the content of this dissertation has already been presented in the published works listed below. For all these publications, I, Alexander Ek, have been the lead author, and the other authors have taken on a supervisory role. Some passages have been quoted verbatim from these sources into this dissertation.

- [I] Alexander Ek, Maria Garcia de la Banda, Andreas Schutt, Peter J. Stuckey and Guido Tack. ‘Modelling and Solving Online Optimisation Problems’. In: *AAAI’20*. AAAI Press, 2020, pp. 1477–1485 (cit. on pp. 19, 20, 65, 103, 134).
- [II] Alexander Ek, Maria Garcia de la Banda, Andreas Schutt, Peter J. Stuckey and Guido Tack. ‘Aggregation and Garbage Collection for Online Optimization’. In: *Principles and Practice of Constraint Programming*. LNCS 12333. Springer, 2020, pp. 231–247 (cit. on pp. 19, 20, 65, 103, 134).
- [III] Alexander Ek, Andreas Schutt, Peter J. Stuckey and Guido Tack. ‘Explaining Propagation for Gini and Spread with Variable Mean’. In: *Principles and Practice of Constraint Programming*. LIPIcs 235. Schloss Dagstuhl, 2022, 21:1–21:16 (cit. on pp. 21, 195).

INTRODUCTION

There is a lot of complexity hidden in day-to-day life. Everywhere and all the time, decisions, small and big, have to be made to solve problems. While many are small decisions—what type of coffee to order at the local café, for example—many are big decisions that are critical to how society runs. Examples include how to route delivery vehicles most efficiently, allocate disaster funds most fairly, schedule processes at a shared manufacturing plant most efficiently and fairly, and position and dispatch ambulances most efficiently. Bad decisions will have dire consequences, such as monetary loss, wasted resources, unjust outcomes, or even preventable deaths. However, what it means for decisions to be ‘good’ depends on how one defines ‘most efficiently’ and ‘most fairly’ in the above examples, which is not always that clear-cut.

Increasingly, computers are being used to aid humans in solving complex decision-making problems. Approximate methods like machine learning work well in cases where there are few if any complex, ‘hard’ constraints, that is, constraints that must be satisfied for a solution to be even remotely viable. Otherwise, more rigorous search and inference methods are utilised. Enter *combinatorial optimisation* [159, 137], a field that is dedicated to understanding, formulating, and solving complex decision-making problems where one must take discrete decisions that satisfy given constraints and usually optimise an objective function (i.e., minimising or maximising it, depending on the problem), which measures the quality of any solution. Such problems are more specifically called *combinatorial problems*. Combinatorial problems occur in nearly all fields and organisations, such as supply chain management and logistics, telecommunications, scheduling and planning, bioinformatics, timetabling and rostering, resource allocation, and robotics and control systems, to name a few [54, 162].

Combinatorial optimisation, and automated decision-making in general, is one of the central areas of artificial intelligence. Given a description of a decision-making problem, the aim is for a computer, with little or no human intervention, to provide one or more (near) optimal solutions (each as a set of decisions) that can then

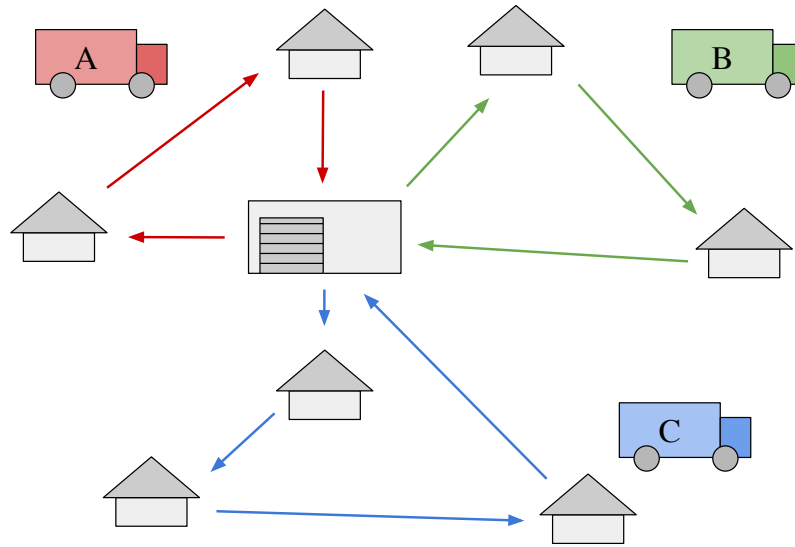


Figure 1.1: An example of a vehicle routing

either be used directly, or can inform a human decision-maker. The kind of problems addressed range from the strategic (long-term) level, such as planning the construction of a minimal footprint power-plant or a future-proof transportation system, where there is a generous amount of time before the decisions need to be executed; to the operational (short-term) level, such as routing delivery trucks to minimise cost or scheduling operations in a manufacturing setting, where the decisions need to be executed within a short time interval (e.g., a day); to the control (real-time) level, such as schedule repair during disruptions, where the decisions will be executed as quickly as possible.

Let us consider a well-known example of a combinatorial problem in more detail: *vehicle routing*, where one is tasked with designing routes for a fleet of vehicles to visit several locations under various restrictions. The major decisions to take are which vehicles to dispatch, when, servicing which locations, and in which order. The objective is often to design cost-effective routes. First, an *instance* of this problem needs to be provided, describing the exact details and parameters to use in an otherwise broad problem class. Consider the particular instance shown in Figure 1.1. In this instance, the fleet consists of three vehicles—a red one (A), a green one (B), and a blue one (C); seven locations (or customers) depicted as houses; and a vehicle depot positioned in the middle. The arrows illustrate a possible solution: vehicle A services the upper-left locations, vehicle B services the upper-right locations, and vehicle C services the bottom three locations. All three vehicles start from the depot, visit their respective locations, and finally finish at the same depot from which

they started. There are typically also many constraints in combinatorial problems, disallowing most of the possibilities. In the vehicle routing example, we could have regulatory constraints that enforce drivers to have breaks and shifts within specific time limits; in package delivery problems, we could have capacity constraints that limit the number of locations a vehicle can visit (as the packages need to fit in the vehicle); or we could have emissions constraints that limit stop-frequency and the use of small, low-speed roads.

Combinatorial problems are notoriously challenging to solve, as the number of possibilities often grows (at least) exponentially when the size of the problem grows linearly, and many of the possibilities are disallowed by the problem's constraints. To illustrate this, consider a vehicle routing problem with one vehicle, one depot, and five customers. The number of possible routes, visiting all customers exactly once and returning to the depot, is $5! = 120$. Just doubling the number of locations to a mere 10 locations results in over 3.5 million possibilities. This is a growth of $\mathcal{O}(n!)$ possibilities for n locations. In formal terms, the computational challenge comes from combinatorial problems often being NP-hard [137]. This means that every known algorithm for these problems will (at least) increase exponentially in runtime as the instances increase linearly in size.

Since the advent of computers, significant advances in addressing combinatorial problems have emerged. Using various kinds of well-thought-out search, inference, and heuristics, one can implement algorithms capable of solving large problems remarkably quickly. Unfortunately, implementing new algorithms is a very time-consuming and error-prone task. Luckily, there are general-purpose solving techniques readily available as stand-alone programs, referred to as *solvers*. Each solver is typically founded on a particular *solving technology*, which refers to principles of representing a problem and implementing search, inference, and heuristics on top of that representation. The most common solving technologies include *Boolean satisfiability* (SAT) [21], *satisfiability modulo theories* (SMT) [89], *mixed integer programming* (MIP) [136], and *constraint programming* (CP) [129]. To use a solver on a problem, one must first (declaratively) model the problem in a language and representation that the solver can understand.

All solvers operate with different representations and different languages. This complicates experimentation across solvers. Experimentation is vital because where one solver struggles, another may shine due to their differences. Tools for modelling combinatorial problems in a unified language have thus emerged. Such modelling languages are *solver-independent*, meaning that a problem can be modelled once and

solved with a wide range of different solvers. Examples of such modelling languages include *MiniZinc* [107], *Essence* [58], *OPL* [156], and *AMPL* [51]. These modelling languages are human-readable, allow exploitation of problem structures (only visible in high levels of representation) to improve solving, and have interfaces to many modern solvers.

Nevertheless, modelling languages and solvers often fall short in real-world applications where uncertainty and dynamism are prevalent. A problem is called *static* if it does not change with time or contains uncertainties, and a solving method is called *offline* if it considers the whole problem simultaneously and produces a complete solution. Conversely, a problem is called *dynamic* if it has uncertainties or does change with time, while a solving method is called *online* if it considers parts of the problem, considers updates happening to the problem during solving, and successively produces partial solutions, arriving at a complete solution if the problem concludes.

In addition to the above shortfall, modelling languages and solvers often fall short in situations where balance and fairness considerations are essential, e.g., among multiple stakeholders' conflicting wishes and goals regarding desired solutions. A problem is called *single-objective* if there is a single, clear goal expressible as a single objective function. Conversely, a problem is called *multi-objective* if there are several conflicting objective functions without a clear hierarchy among them, and a compromise must be found. Further, in this thesis, a problem is called *multiagent* if each objective function represents the *utility* a stakeholder derives from any solution, and a compromise needs to exhibit some kind of fairness or low inequality among the utilities.

Current modelling languages and solvers often assume that problems can be expressed fully and deterministically and that they only have one stakeholder. The following section discusses the resulting shortcomings.

1.1 MOTIVATION AND PROBLEM STATEMENT

This thesis focuses primarily on *online combinatorial optimisation* for solving *dynamic combinatorial problems*. These often appear in an operational or control setting where new information continually arrives, and the optimiser needs to react, adapt, and make the best decisions possible. In control settings, it is often referred to as *real-time combinatorial optimisation*. However, as we will soon see, it can also be used as a solving technique in large-scale strategic settings. Consider, again, the vehicle routing example. If disruptions were to happen during the execution of a plan, how could we

then adapt and re-plan most optimally? And if there are so many uncertainties that a complete plan becomes impractical, how would we continually plan throughout the execution process? For example, consider a situation where any location can at any time cancel the servicing request, and assume that the last location of the route that vehicle C is about to visit cancels while the penultimate location is being serviced. It would then likely be wasteful to continue with the originally planned path on the way back to the depot. Therefore, it would be advantageous to model the problem such that it can re-route vehicles when a change occurs to try to continue finding a better route (in this case, a quicker path back to the depot). So the question we need to ask is: *how can one model these problems?*

1.1.1 Modelling and Solving

There is a good selection of mature high-level modelling and solving tools for conventional combinatorial optimisation. However, these tools are still in their infancy for online combinatorial optimisation. Further, despite the strong similarities between all online optimisation methods, most current methods are *problem-specific*, meaning that they are tailored to one kind of problem, cannot be applied to other problems, and are not easily generalised to be applicable. This is because every time new information arrives, an online approach has to react and re-plan, where some previous decisions cannot be changed while others can, which can be a complex procedure. To illustrate, suppose that in the above vehicle routing problems, the packages that need to be delivered can change dynamically. Additionally, a package may have a delivery window, that is, the timeframe in which the package must be delivered; these windows may also change dynamically. We cannot add more locations to a vehicle's route after it has left the depot because the necessary packages are still at the depot, and the vehicle is not. Thus, we need to be careful not to dispatch a delivery vehicle prematurely. On the other hand, we may be able to change the order in which the (remaining) locations are visited, allowing us to satisfy new delivery windows. The shortcomings of these problem-dependent approaches are discussed in more length in Section 3.1.

The goal is to devise a *problem-independent* approach: a framework that applies to any kind of problem. In addition, the framework should also solver-independent, just as the modelling languages for static combinatorial problems are. Achieving problem independence is difficult because declaring which decisions can and cannot be changed is a complex procedure. It requires specifying how the uncertainties that can manifest themselves in the problem (i.e., how new data gets added and existing

data gets modified) interact with the variables and constraints used to model the problem. This specification is usually done by the modeller, who understands this interaction and can implement, what is referred to in this thesis as an *update-model*, that is, a model that combines the results of previously executed decisions with the newly arriving data. An iterative algorithm can then be used to repeatedly instantiate and solve this update-model with the new data received since the last iteration. Thus, we arrive at the first research question.

RESEARCH QUESTION 1. What are the characteristics of the dynamic problems and online methods in the literature, how can these be extracted to a high-level modelling framework, and what are the necessary components for making such a framework useful?

In addition, many static optimisation problems are too large to be solved efficiently with conventional techniques. A common technique for finding good solutions quickly is to model them as dynamic problems by decomposing them into smaller, simpler problems along an (often artificial) timeline. This popular approach is called *rolling horizon* (or *sliding window*) [100, 14], where the problem is decomposed into (usually overlapping) planning time horizons (or windows), each solved in increasing time order. This effectively converts the static problem into a dynamic one, where each new horizon refers to some old parts of the problem (those overlapping with the previous horizon) and to some new (the rest in the new horizon). This leads to the second research question.

RESEARCH QUESTION 2. How can one decompose a static problem into a dynamic problem to be solved online, in a problem-independent and systematic way?

These two research questions are addressed in Chapter 3 of this thesis.

Problem-independent online combinatorial optimisation brings with it a new issue: over time, many problem variables progressively become *realised*, i.e., their values were fixed in the past iterations, and they can no longer affect the solution. Put simply, the instances grow in size with time. The growth impedes solving performance, which is especially unwelcome in real-time contexts, where solutions must be rapidly produced. Consequently, our problem-independent framework needs to remove old and irrelevant data. Unfortunately, simply removing realised variables can be incorrect, as they might affect unrealised decisions. Thus, a method that summarises the realised data's effects on non-realised decisions is essential. Such summaries would make the underlying data safely removable. In online optimisation, there is no established, problem-independent methodology for identifying realised parts nor how to remove

them correctly. Understanding how to ‘summarise’ and ‘remove’ is a complex task currently performed manually in a problem-specific and time-consuming way. Thus, we arrive at the third research question.

RESEARCH QUESTION 3. How, and to what degree, can one summarise the effects of realised data in online optimisation, leading to contained instance sizes, in a problem-independent and systematic way?

We call this notion *garbage collection*. This research question is addressed in Chapter 4 of this thesis.

1.1.2 Fairness and Balance

Optimisation problems involving multiple stakeholders often require the solution to be balanced and fair. This is the case, for example, when balancing tardiness in scheduling, unwanted shifts in rostering, and desired resources in resource allocation, or minimising deviation from a baseline in schedule repair, to name a few. Further, the emergence of production paradigms such as cloud or shared manufacturing points to fairness becoming increasingly important in online combinatorial optimisation. The fairness literature often ignores problems with uncertainties and combinatorial structures, while the combinatorial optimisation literature often ignores issues of fairness and balance. For a comprehensive literature review, see Section 5.2.

The study of fairness and balance has a long history in economics [141]. Perhaps the most straightforward principle of addressing problems with multiple stakeholders, or *agents*, is *utilitarianism*. According to *utilitarianism*, the best solution is to maximise the overall happiness of everybody or, more specifically, the sum of everyone’s utility. This approach may lead to highly imbalanced allocations of the shared resources and unfair utility distributions because it may favour specific agents at the expense of others. For example, agents who declare lower utility from using the shared resources may never be allocated any, while those declaring disproportionately larger utility may be allocated all. Other factors, such as search heuristics or incomparable utilities across agents, can also result in imbalances. The need for fairness is particularly interesting in an online optimisation setting due to two main factors. First, balance and fairness may need to be measured over multiple time periods, while decisions are made myopically within each period. Second, while the online setting enables unfairness to be magnified over time, it also provides an opportunity to flexibly compensate for any previous unfairness. There is no investigation into the important intersection between online combinatorial optimisation and fairness in the literature.

There is often a focus on simpler problems, where stronger theoretical conclusions can be drawn. Thus, we arrive at the fourth research question.

RESEARCH QUESTION 4. How can the established concepts of fairness in simpler problems be applied to online combinatorial optimisation to increase fairness without inhibiting overall utility?

This research question is addressed in Chapter 5 of this thesis.

Even in offline combinatorial optimisation, there are gaps in the literature. An important one is the need to measure inequality, a subcomponent of fairness, which requires considering the statistical dispersion (sometimes called variability or spread) of the resource allocations in a solution. To solve problems with dispersion considerations efficiently, solvers must be able to effectively reason about dispersion, e.g., to make strong inferences, reducing the search time. Lately, solvers combining the CP and SAT technologies, arriving at what is called *clause-learning* CP solvers, have shown to be effective, especially for problems with components of scheduling [III, 148]. Previous work includes no inference algorithms on any dispersion metric applicable to clause-learning CP solvers. While some dispersion metrics have inference algorithms applicable to pure CP solvers, there is still a big gap with some common dispersion metrics. Further, most previous work has studied the case where the arithmetic mean of the distribution is fixed, but this may not be possible for many problems, e.g., scheduling where the total utility depends on the final schedule. Cases with a variable mean are overly complicated, have never been implemented in practice, or are inefficient. Thus, we arrive at the fifth and final research question that will be considered in this thesis

RESEARCH QUESTION 5. How can clause-learning CP solvers effectively reason about dispersion metrics?

This research question is answered in Chapter 6 of this thesis.

1.2 OUTLINE AND CONTRIBUTION TO KNOWLEDGE

Combinatorial optimisation problems are ubiquitous, and the shortcomings of current modelling and solving methods regarding uncertainty and fairness need to be addressed. This can be achieved by answering the research questions above. In doing so, this thesis arrives at tools and techniques improving support for modelling and solving in online, real-time, and multiagent combinatorial optimisation. All the

research questions are addressed both formally and empirically. The structure and the contributions of this thesis are summarised below.

Chapter 2: Background. The chapter first provides a brief history and an overview of the combinatorial optimisation field, including a summary of the most relevant solving and modelling approaches. This includes descriptions of the solving technologies CP, MIP, and SAT and the modelling language MiniZinc used throughout this thesis. It then discusses how uncertainty and dynamism can manifest in combinatorial optimisation and how to address them in online optimisation. The chapter concludes with a brief overview of fairness and balance considerations in combinatorial optimisation.

Chapter 3: High-Level Modelling for Online Combinatorial Optimisation. Typically, building an online decision-making system requires substantial ad hoc coding to ensure the solution of a dynamic combinatorial optimisation problem is continually adjusted and re-solved. The chapter defines a general framework for automatically solving dynamic combinatorial optimisation problems with online (including real-time) optimisation. This is achieved by extending a model of the static version of the problem, from which a dynamic version is automatically constructed, thus requiring no further modelling effort. The resulting framework enables modellers to specify the online aspects of a problem declaratively and automates the online solving process. In doing so, it formalises many aspects that arise in online combinatorial optimisation. The same framework can be applied to automatically create rolling horizon decompositions for large problems.

The chapter addresses Research Questions 1 and 2, and is partly based on Papers [I] and [II] (primarily the former). The main contributions of the chapter are: (i) a framework for declarative modelling for online and rolling-horizon optimisation, identifying common interactions of models with time; (ii) an automatic approach to transforming an annotated static model into the update-model needed to solve the problem online, taking into account new data and the results of previously executed decisions; and (iii) an implementation of the framework in the MiniZinc toolchain. Experiments show we can automatically create efficient online and rolling-horizon solutions for both dynamic and static combinatorial optimisation problems. The possibility, utility, and foundations of problem-independent online combinatorial optimisation are established. As a result, online combinatorial optimisation is made more efficient for experienced modellers and more accessible for novices.

Chapter 4: Automatic Garbage Collection for Online Combinatorial Optimisation. In online optimisation, the amount of realised data grows over time. If these realised variables and associated data are not removed, i.e., garbage collected, performance will slow down significantly. As simply removing them can be incorrect, it has primarily remained a complex, manual, problem-specific, and time-consuming task. It requires understanding how time and data interact with the variables and constraints used to model the problem. The chapter proposes a problem-independent framework that extends the one proposed in Chapter 3, to identify realised data and decisions and later remove them by compactly summarising them. The modeller simply provides minimal information about the relation between time, data, and variables. The framework performs three main steps. First, the modeller’s information is used to analyse each constraint, seeing what is irrelevant or can be made irrelevant. Second, the constraints are reformulated to summarise as much of the realised information as possible. Third, the garbage from all constraints, i.e., the information fully covered by the summaries, is collected and removed.

The chapter addresses Research Question 3 and is partly based on Papers [I] and [II] (primarily the latter). The main contributions of the chapter are: (i) a systematic way of modelling and inferring when each part of a model is realised and how to utilise this; (ii) for several kinds of constraints, methods of summarising the effect that realised parts have on the future without keeping them, attained by reformulating the model; (iii) a garbage collection mechanism that analyses a high-level constraint model, identifies which parts are never used again, and safely removes them from future sessions; and (iv) an analysis of what can and cannot be garbage collected. Empirical evaluation of the proposed approach shows substantially improved model performance. The end product is an automatic garbage collection mechanism for generic online (and real-time) combinatorial optimisation.

Chapter 5: Fairness in Online Combinatorial Optimisation. Many recent developments point to fairness becoming increasingly important in online combinatorial optimisation, but this combination remains a gap in the literature. The need is particularly interesting in the online setting, where there is flexibility to counteract past unfairness. The chapter presents a novel and generic algorithmic framework for exploring different combinations of fairness and overall utility in online multi-agent combinatorial problems. The framework is a problem- and solver-independent priority-balancing framework. Ideas from process scheduling, multi-objective optimisation, fair division, and economics are utilised. Intuitively, the framework performs three core operations per solving iteration. First, the problem in its current state is

solved. Second, the agents are ranked based on some measure of utility achieved so far. Third, the optimisation objective for the next iteration is re-balanced to favour the lower-ranked agents.

The chapter addresses Research Question 4. The main contributions of the chapter are: (i) a problem- and solver-independent modelling framework for achieving both fairness and utility when solving dynamic multiagent combinatorial optimisation problems using adaptive priority re-balancing; and (ii) a discussion of how unfairness and imbalance can manifest in online combinatorial optimisation, and how to address them. The chapter empirically explores several common fairness metrics applicable to online shared resource utilisation problems and re-balancing strategies. The experiments show that this approach can yield fair allocations according to multiple fairness measures while maintaining high utility. More broadly, the proposed framework allows further empirical evaluation in contexts where it is difficult to make formal guarantees on fairness.

Chapter 6: Explaining Propagators for Gini and Spread with Variable Mean.

In optimisation problems involving multiple agents, we often want to ensure that the solution is balanced and fair. Typically, we want to maximise the total utility subject to an upper bound on the statistical dispersion of the utility given to different agents or minimise the dispersion subject to some lower bounds on utility. The chapter presents two inference algorithms, commonly called *constraint propagators*, for statistical dispersion. The propagators can explain their inferences in logic, making them usable in many state-of-the-art clause-learning CP solvers. They both run in log-linear time. Their use yields speedups in solving combinatorial problems with dispersion constraints, objectives, or both.

The chapter addresses Research Question 5 and is partly based on Paper [III]. The main contributions of the chapter are the mathematical and algorithmic development of efficient and explaining inference algorithms, filtering the lower bound of (i) spread (i.e., variance and standard deviation), and (ii) the Gini coefficient. We implement the two propagators in the clause-learning CP solver Chuffed. Algorithms for (i) exist in the literature but do not produce explanations required for clause-learning solvers and often assume a fixed mean, while algorithms for (ii) have not been previously studied. The chapter shows through experimentation that clause learning can substantially improve the solving performance in dispersion problems.

Chapter 7: Thesis Conclusion. Finally, the thesis concludes by reiterating the discoveries and contributions of the thesis and outlining their actual and potential im-

pacts. The chapter ends with a list of unanswered questions, limitations, and directions for future work.

BACKGROUND

This chapter contains the necessary foundations upon which the concepts and discussions of the rest of this thesis are based. The basic concepts, along with terminology and notation, are presented. See Appendix A for a list of notation. Note that all the concepts and definitions in this chapter follow those in the well-known literature (such as [21, 129, 136, 137, 158]).

This chapter is organised as follows. Section 2.1 begins with an overview and a brief history of combinatorial optimisation, followed by a more detailed description of constraint programming, global constraints, and other solving technologies. Section 2.2 introduces concepts relating to modelling combinatorial problems, and Section 2.3 examines the issues that occur when the problem to be solved contains a part with an uncertain or dynamic nature. In Section 2.4, a more in-depth discussion of online combinatorial optimisation is provided, including a short investigation into an interesting special case thereof, namely real-time combinatorial optimisation. The chapter ends with an overview of fairness and balance in Section 2.5 and some final remarks about our assumptions moving forward in Section 2.6.

2.1 COMBINATORIAL OPTIMISATION

An optimisation problem is characterised first and foremost by a set of unknown variables, the values of which we need to find, an objective function to optimise (either minimise or maximise), and constraints (e.g., equations that the variables need to satisfy). Combinatorial optimisation, as a field, is mostly focused on understanding how to effectively and efficiently solve hard decision-making problems. When *combinatorial*, the unknown variables in the optimisation problem represent the decisions of the problem as discrete mathematical elements. These elements belong to finite (discrete) sets representing the possibilities for each decision.

Under the umbrella of combinatorial optimisation also sit problems with no objective, but where the constraints make finding one or more solutions difficult. Such problems are called *constraint satisfaction problems*. Sometimes, to clearly indicate that the discussion includes both satisfaction and optimisation problems, the term *combinatorial problems* is used.

2.1.1 A Brief History

Combinatorial optimisation is a bustling amalgamation of several research fields and technologies, all aimed at modelling, solving, and analysing hard optimisation problems. But how did we end up here? Let us briefly investigate the history of optimisation as it is known today.

The study of optimisation started with mathematics (such as calculus and graph theory) but for a long time remained a loose collection of independent lines of problem-specific research, especially for combinatorial optimisation [138]. The formation of a coherent field began around the 1940s, heavily influenced by the increasing complexities and demands of the transportation industry and the Second World War. This is when the early formalisations of *linear programming* (LP) came (see [86, 74]) and the later named *simplex algorithm* for solving linear programs was introduced by Dantzig (see [36]). A *linear program* (an LP) is an optimisation problem where the variables can take on any value from the reals, and the constraints are all linear equations and inequations. As LP and the simplex algorithm were applied to solve all kinds of optimisation problems, often with an improvement over the status quo, the optimisation field was propelled forward. Still, solving LPs remained a mainly manual task since no general implementation of the simplex algorithm existed until at least a few years into the 1950s [113]; regardless, there was little availability of electronic computers for quite some time after that.

Two large issues remained: the handling of integral variables and non-linear constraints. A solution for the integrality problem started emerging during the mid-1950s with techniques and frameworks put forward (see, e.g., [35, 99]). This is when the term *mixed-integer linear programming* or simply *mixed-integer programming* (MIP) was born, referring to the solving of *mixed-integer programs* (MIPs) [136]. A MIP is an extension of an LP where some variables have the extra constraint of being integral, making it much harder to solve. A general solution to the integrality problem was finally reached in 1960 with Land and Doig combining the aforementioned techniques into a coherent algorithm for solving generic MIPs, later to be known as *branch-and-bound* [91]. Around the same time, Gomory introduced an elegant *cutting-plane method* to solve

the integrality problem [67, 68], but it was quickly deemed impractical and solely a theoretical result [7].¹ Proper implementation of the branch-and-bound algorithm into what one could call the first MIP solver came around the mid-1960s (see, e.g., [11]).

Also in the 1960s, as the electronic computer became more widespread and powerful, a new branch of computer science emerged: automated *Boolean satisfiability* (SAT) [21]. A SAT problem consists of Boolean (true or false) variables and logic formulas, which must be satisfied. The main interest is not in optimisation per se but rather in constraint satisfaction and the algorithmisation of logic, which is still of interest in combinatorial optimisation in general due to the often complex nature of the constraints. At this time, several important techniques in SAT solving were developed, such as the DPLL procedure (see [38]) and the resolution calculus for first-order logic (see [127]).

Alongside these developments, the field of computational complexity theory developed. This is a field concerned with how a computer program's input size relates to the runtime and space usage necessary to obtain the program's output and how different algorithms aimed at obtaining the same output can differ in this relation. After a modest beginning in the 1930s with Turing's formalisation of the computing machine, the field took off during the 1960s [50] with the runtime of an algorithm being generally split into three complexity classes: P, NP, and NP-hard [137]. Problems in P can be solved within polynomial time, which means that the runtime grows at most $\mathcal{O}(n^k)$ as n (the size of the instance) grows linearly, where k is some constant. Problems in NP can be verified (check if a candidate solution is, in fact, a solution or not) in polynomial time. Note that $P \subseteq NP$, but whether every problem in NP is also in P is unproven; however, it is widely believed not to be the case (i.e., $P \neq NP$). Thus, in practice, problems in NP not (known to be) in P must be solved with algorithms that grow quicker than polynomial time (but at most exponential time $\mathcal{O}(k^n)$ where k is some constant) as n (instance size) grows linearly. Problems in NP-hard include the hardest problems in NP (those we only know exponential time algorithms for) and all harder problems (e.g., those with algorithms growing at least $\mathcal{O}(n!)$). In the early 1970s, several results highlighted that most combinatorial problems of interest fall into the NP-hard complexity classes [33, 87, 95]. Thus, at some point, different for each problem class, the instances generally become too big to solve within a reasonable time. However, using an appropriate approach for solving a problem, it is possible to delay the point at which the problem takes too long to solve. It was thus clear that solving technologies had to become better and more accessible.

¹Only decades later did cutting-plane methods become important for improving the performance of state-of-the-art MIP solvers.

Also around the early 1970s, the field of *constraint programming* (CP) started emerging from the constraint logic programming, constraint satisfaction, and artificial intelligence communities [9, 129]. The aim was to improve the modelling and solving of complex non-linear constraint satisfaction problems with integral variables. Three important techniques were developed and combined here: the formalisation of generic *backtracking search*, the introduction of *constraint propagation* (also called *domain filtering* or *inference*), and the introduction of high-level, declarative programming with constraints. However, it was not until the late 1980s that the field of CP, and a general-purpose CP solver, materialised (see [80, 157, 81]).

Today, there is a rich set of concepts and tools that one can use to understand and solve combinatorial problems. The remainder of this section discusses combinatorial problems and the concepts introduced above with this rich and modern viewpoint.

2.1.2 Constraint Satisfaction and Optimisation

A common mathematical formalism used to analyse combinatorial problems is the *constraint satisfaction problem* (CSP).

Definition 2.1 (CSP). A *constraint satisfaction problem* (CSP) is a triple $P = \langle X, D, C \rangle$ where X is a set of *decision variables*,² function D is a mapping from each decision variable $x \in X$ to a finite discrete set of values called the *domain* of x , denoted $D(x)$, and C is a set of relations over X called the *constraints*. An *assignment* $\mathcal{A}$ of P is a set $\{x \mapsto D(x) \mid x \in X\}$ of mappings from each variable $x \in X$ to a value in its domain. Each constraint $c \in C$ acts on a subset of the variables $\text{vars}(c) \subseteq X$. Over these variables, c specifies the allowed (partial) assignments $\text{sols}(c) \subseteq \{\mathcal{A}' \mid \mathcal{A}' = \{x \mapsto D(x) \mid x \in \text{vars}(c)\}\}$. We say that a (non-partial) assignment $\mathcal{A}$ satisfies constraint c if $\{x \mapsto \mathcal{A}(x) \mid x \in \text{vars}(c)\} \in \text{sols}(c)$. An assignment is called a *solution* to P if it satisfies all constraints in C .

Given a CSP, the aim is to find one or more of its solutions. Notice that the candidate solutions are contained in the domains of the variables. We refer to the set of all assignments $\{\mathcal{A} \mid \mathcal{A} = \{x \mapsto D(x) \mid x \in X\}\}$ as the *search space* of the problem P . We define the *solution space* of P as the set of all solutions of P , that is:³

$$\text{sols}(P) = \{\mathcal{A} \mid \mathcal{A} = \{x \mapsto D(x) \mid x \in X\}, \text{ s.th. } \forall c \in C, \exists \mathcal{A}' \in \text{sols}(c), \mathcal{A}' \subseteq \mathcal{A}\}$$

²Throughout this thesis, *decision variables* are often simply called *variables*, when the context is clear.

³Throughout this thesis, we will use ‘s.th.’ in mathematical contexts to mean ‘such that’.

If the solution space is empty, then we say that the problem (or problem instance) is *unsatisfiable*.

Example 2.1. Suppose one is tasked with colouring a map of the states and territories of Australia. No two neighbouring states or territories can have the same colour. Australia has six states—New South Wales (NSW), Queensland (QLD), South Australia (SA), Tasmania (TAS), Victoria (VIC), and Western Australia (WA)—and two territories located on the mainland—the Australian Capital Territory (ACT) and the Northern Territory (NT). Each state and territory can be represented as a node in a graph, as seen in Figure 2.1, where an edge between two nodes models a border. For the purposes of colouring, TAS is considered a neighbouring state to VIC, even though they do not share a land border, because of their proximity.

This colouring task can be represented as a CSP: nodes are the variables, the colours to colour them are the domains of these variables, respectively, and the restriction that two neighbouring nodes cannot be the same colour becomes our constraints. Hence, there is one variable per node:

$$X = \{x_{\text{NSW}}, x_{\text{QLD}}, x_{\text{SA}}, x_{\text{TAS}}, x_{\text{VIC}}, x_{\text{WA}}, x_{\text{ACT}}, x_{\text{NT}}\}.$$

To guarantee satisfiability, there must be at least as many colours as nodes. For example, let the domain of each variable $x \in X$ consist of eight colours:

$$D(x) = \{\text{blue, cyan, green, magenta, orange, purple, red, yellow}\}.$$

Finally, there is one disequation for each neighbouring state or territory, e.g., $x_{\text{NSW}} \neq x_{\text{QLD}}$.

Colouring each node a unique colour is obviously satisfiable, but it is often possible to use fewer colours than the number of nodes. However, as soon as two nodes are connected, one cannot use only one colour since the two connected nodes would violate the colouring constraint.

Often, one is not only interested in finding any solution, but a very good or even the best solution possible; in such cases, there is a notion of one solution being *better* than others. Typically, this requires one to first specify a quantitative *objective* for evaluating and comparing solutions. For example, we may want to minimise the number of colours used in Example 2.1.

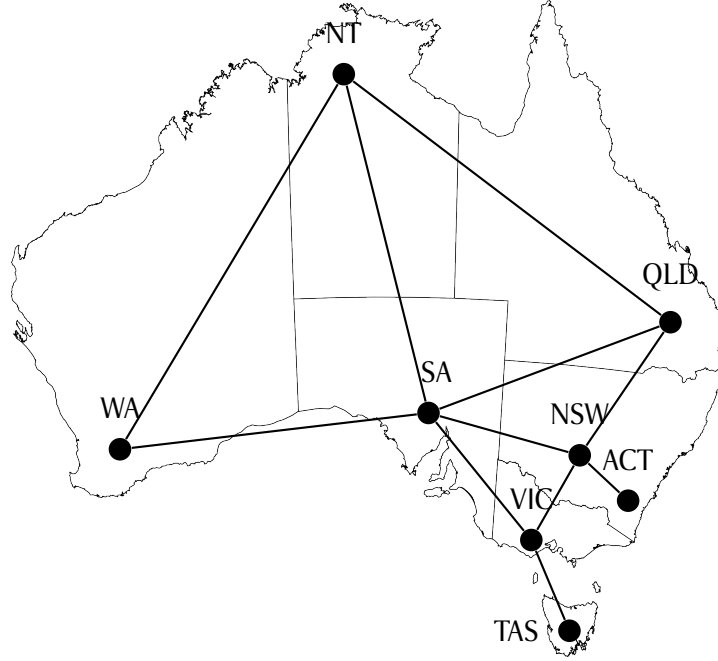


Figure 2.1: A map of Australia with a graph representation, connecting the states and territories, above it

Definition 2.2 (COP). A *constrained optimisation problem* (COP) is a quadruple $P = \langle X, D, C, f \rangle$ where $P' = \langle X, D, C \rangle$ is a CSP and f is the *objective function* of P . Operating on a subset $\text{vars}(f) = \{x_1, \dots, x_k\}$ of the variables X , the function $f : D(x_1) \times \dots \times D(x_k) \rightarrow \mathbb{R}$ maps each element in the search space to a numeric value. Given an assignment $\mathcal{A}$, let $f(\mathcal{A})$ be a shorthand for $f(\mathcal{A}(x_1), \dots, \mathcal{A}(x_k))$. The goal is to either maximise or minimise (which is predetermined) the value of f while solving the COP. Unless otherwise specified, we assume that the objective function is to be maximised. The solution space $\text{sols}(P)$ is defined as $\text{sols}(P')$. A solution to P is called an *optimal solution* if it maximises (or minimises) f within the solution space, and $\text{opts}(P)$ denotes the set of all optimal solutions to P .

Except where explicitly stated, all ideas presented in this thesis apply to both CSPs and COPs. For brevity, however, only COPs will be used for most of the discussion in the remainder of this thesis.

A COP can be solved by using *brute-force search*, that is, by enumerating all solutions, evaluating them one by one, and keeping the best valid solution encountered throughout the process (if any). This can be achieved by traversing the so-called *search tree*, which can be defined as follows. Suppose we have a COP $\langle X, D, C, f \rangle$

and some ordering $x_1, \dots, x_n$ of the variables in X , then x_i corresponds to the i th level in the tree and each node in that level connects to the next level with $|D(x_i)|$ edges: one for each possible assignment to x_i . The number of nodes will rapidly increase as we move down the tree; at the k th level, there will be $\prod_{i=1}^{k-1} |D(x_i)|$ nodes, which is in $\mathcal{O}(n^k)$ where n is the largest domain of any variable in the COP.

The infeasibility of such an approach is unsurprising. One can, however, augment it to create a more feasible approach for solving COPs, as shown next.

2.1.3 Constraint Programming

Constraint programming (CP) provides an often effective way to solve COPs. It builds upon backtracking search, which is a general algorithmic framework. Backtracking search is an enhanced variation of brute-force search, where candidate (partial) solutions are incrementally built during search and the algorithm *backtracks* as soon as it is determined that no solution can result from the current candidate solution being constructed. Thus, compared to brute-force search, many fewer leaf and inner nodes (corresponding to solutions and partial assignments, respectively) of the search tree are expected to be visited. In CP, the domains of the variables are progressively reduced while moving down the search tree, eventually arriving at a solution (if one exists). This provides, among other things, increased flexibility compared to brute-force search: a variable $x_i \in X$ need not be assigned a value when going from level i to $i + 1$, instead arbitrary reductions of the domain of x can be imposed. The only requirement is for all branches from every node to be mutually exclusive and collectively exhaustive. For clarity, let us introduce two terms: *initial domains*, meaning the domains provided as part of the problem specification, and *current domains*, meaning the domain values left to consider at a given point in a search tree. If the current domain of a variable contains only one value, then that domain, or variable, is said to be *fixed*. We assume that all constraints are automatically reduced to correspond to the current domain at each point in the search tree.

In CP, each constraint $c \in C$ has a *constraint propagator* p_c , which uses specialised algorithms and logic to check or reduce the domains of the variables in c . The simplest form of a constraint propagator, and its bare minimum requirement, is a *constraint checker*.

Definition 2.3 (Constraint Checker). Given a COP $\langle X, D, C, f \rangle$, a constraint $c \in C$, and current domains $D'(x)$ such that $\emptyset \neq D'(x) \subseteq D(x)$ for all $x \in X$, where $\emptyset$ denotes the empty set, a constraint checker p_c must (a) return true if there is a solution for c in D' , and must (b) return false if all variables in $\text{vars}(c)$ are fixed with no solution for c .

Note that $D'(x)$ is known never to be empty, as this is immediately recognised as false for all checkers and propagators. Essentially, a constraint checker merely *checks* whether the current domains satisfy the constraint in question or not, without reducing the domains of its variables. Note that the conditions (a) and (b) are not collectively exhaustive, meaning that there is no requirement for false to be returned when there is no solution in D' but an unfixed variable in $\text{vars}(c)$. It is generally beneficial for a checker to do this, and many checkers do it in practice. Constraint checkers that never return false unless all its variables are fixed are called *minimal*, as it is the minimal requirement for correctness. If a constraint checker returns false at any point in the search tree, then the search will not continue from this point, and instead, backtracking is initiated. If all variables are fixed, and no constraint checker returns false, then a solution to the problem has been found.

Often we can do more than just checking. For example, if we have the constraint $x + y = z$ and the domains of x and y are fixed, then we can calculate the domain of z and reduce it accordingly. We could similarly analyse the upper and lower bounds or go through all possibilities of a constraint, removing the domain values that are guaranteed to lead to no solution.

Definition 2.4 (Constraint Propagator). Given a COP $\langle X, D, C \rangle$ and one of its constraints $c \in C$, a *constraint propagator* p_c for the constraint c is a function that maps a given domain mapping D to a new (possibly reduced) domain mapping $p_c(D)$ such that the following criteria are satisfied:

Checking. p_c must be a (minimal) constraint checker; and if the checker returns false, then we set $p_c(D)(x) = \emptyset$ for at least one $x \in \text{vars}(c)$, i.e., the empty domain.

Monotonic. For every $x \in X$, we must have $p_c(D)(x) \subseteq D(x)$, i.e., it can only remove domain values, not add new ones.

Correct. All solutions for c in D must be in $p_c(D)$, i.e., it cannot remove values that appear in valid solutions for c . Formally, this can be defined as: $\{\mathcal{A}' \mid \mathcal{A}' = \{x \mapsto p_c(D)(x) \mid x \in \text{vars}(c)\}\} = \text{sols}(c)$.

If $f_c(D)(x) = \emptyset$ for some $x \in X$, then we say that we have reached a *failure* because there is no solution in D . Note that applying the same propagator consecutively may lead to new results, that is, $f_c(f_c(D))$ and $f_c(D)$ need not be equal. When $f_c(D) = D$ holds, we say that we have reached a propagator *fixpoint*.

Constraint propagators can have different strengths. We have already seen the weakest one, which is a constraint checker. The stronger the propagator, the more values can be removed from the domains, but also, the more time is generally spent on removing said values. Thus, there is a balance in the amount of time spent in search versus propagation. Let us define the strongest kind of propagator: a *domain consistent* propagator, sometimes called a *generalised arc consistent* propagator.

Definition 2.5 (Domain Consistency). A domain D is *domain consistent* for a constraint c with $\text{vars}(c) = \{x_1, \dots, x_n\}$ if for every variable $x_i \in \text{vars}(c)$ and every domain value $d_i \in D(x_i)$, every other variable $x_j \in \text{vars}(c)$ (where $i \neq j$) has a domain value $d_j \in D(x_j)$ such that $\{x_i \mapsto d_i, x_j \mapsto d_j\}$ can be extended to an assignment $\mathcal{A} \in \text{sols}(c)$ that satisfies c , i.e., $\{x_i \mapsto d_i, x_j \mapsto d_j\} \subseteq \mathcal{A}$. If a propagator f_c always returns a domain consistent domain $f_c(D)$ for c when at fixpoint, we say that f_c *maintains domain consistency* or *is domain consistent*.

Notice that domain consistent propagators are the strongest possible; to be stronger, it would have to remove more values from the domains and, thus, void the correctness criterion of propagators. Note also that we may have to run the propagator multiple times to reach domain consistency.

To help form an intuition, consider the following example. We have three variables x_1 , x_2 , and x_3 , each of which have three values, 1, 2, and 3, in their initial domain. There are three constraints, namely $x_1 \neq x_2$, $x_1 \neq x_3$, and $x_2 \neq x_3$. This example can be thought of as an instance of the map colouring problem seen previously, where the three variables are the nodes to colour, they are all adjacent, and the three numbers are the colours available. Figure 2.2 depicts a search tree of this problem with a solver using merely the minimal constraint checker as propagator. The nodes are labelled in alphabetical order of being considered by the solver. The green square, red triangular, and black circular nodes respectively represent solution states, failure states, and states that are not solutions and not known to be failures. In the root node (a), no inference can be made. Thus, we need to search, that is, make a guess. Suppose that the search procedure used to solve our example takes the first unfixed variable in the list and assigns it the first available value in its domain. If such a guess leads to a failure, then the negation of that guess is used in a second branch. This

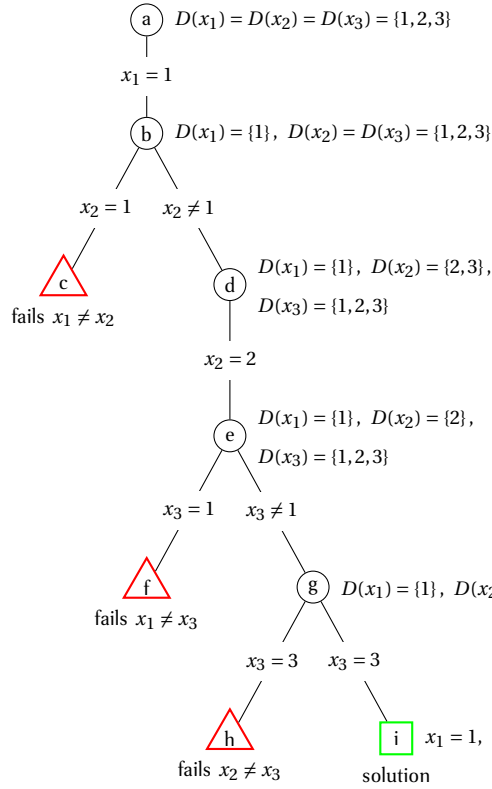


Figure 2.2: A search tree with minimal constraint checkers

procedure leads to node (b), where x_1 is set to equal 1. Still, the minimal constraint checkers cannot infer anything at this point, which leads to the next guess: setting x_2 to 1. This leads to node (c). Here, the checker for $x_1 \neq x_2$ fails because there is no solution, meaning that backtracking is in order. Generally, if one of the checkers (or propagators) fails (i.e., returns the empty set), a failure state has been reached, and we need to backtrack. The second branch from node (b) will then remove 1 from the domain of x_2 , trying the other possibilities. We are brought to node (d). This pattern continues with two more backtrackings before finally reaching a solution, namely $x_1 = 1$, $x_2 = 2$, and $x_3 = 3$. In general, a solution has been reached if we reach a state where all variables are assigned and no propagator fails.

By contrast, consider the search tree in Figure 2.3, in which the solver uses propagators maintaining domain consistency. Generally, at the root node, we queue all propagators and run them one after another to reduce the domains as much as possible before the search begins. The first propagator in the queue, say p , gets removed from the queue and starts running. If p changes the domain of a variable x , then we

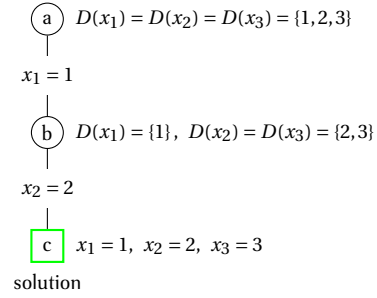


Figure 2.3: A search tree with filtering propagators

queue every propagator p' where such a change could lead to p' being able to remove values, unless it is already in the queue. Note that all queued propagators p' could include the propagator p that just ran. In our example, no propagation can happen at the root node (a). In general, if the queue of propagators is empty, meaning the propagators can make no more inferences, and we do not have a solution or failure yet, then we need to *search* for a solution. The first guess is, again, to set x_1 to 1. This change in variable domains queues propagators that may remove further values using inference, which happens analogously to what happens in the root node. In our example, the propagators for $x_1 \neq x_2$ and $x_2 \neq x_3$ can remove value 1 from the domains of x_2 and x_3 because of the obvious inconsistencies. We can see that the next guess is $x_2 = 2$, leading to node (c). At this point, further propagation can be made on the domain of x_3 , resulting in the solution $x_1 = 1$, $x_2 = 2$, and $x_3 = 3$.

When a solution is found, the solver either backtracks and looks for a better solution (in the case of optimisation) or terminates (in the case of satisfaction). Heuristics for guiding the search are often used to try to find a solution with few search steps. If all branches have been exhausted with no solution, then the problem is unsatisfiable, and if all branches have been exhausted with a (best) solution, then we have proven it to be optimal.

Many CP solvers are available and routinely used today. Among them are Gecode [64] and IBM ILOG CP Optimizer [79]—which are pure CP solvers—and Choco [119], Chuffed [29], Geas [61], and OR-Tools [69]—which are based on CP technology but is (or can readily be) combined with other solving technologies, such as SAT and MIP.

2.1.4 Global Constraints

One of the principal powers of constraint programming comes from having a single constraint that simultaneously constrains a large set of variables instead of having numerous simpler constraints (e.g., ' $x + y = z$ ' and ' $x < y$ ') that, in conjunction, represent the same constraint. A single constraint that is logically equivalent to the conjunction of simpler constraints is called a *global constraint*, while the conjunction of the simpler constraints is called a *decomposition* of the global constraint. In decompositions, intermediate variables may have to be introduced to capture its associated global constraint fully.

Global constraints are often introduced to better address common combinatorial substructures that occur across many kinds of problems. One can then have a dedicated algorithm—a specialised propagator—for just that combinatorial substructure. Possibly the best known global constraint is the *alldifferent* constraint, which con-

strains a set of variables to be assigned different values. Suppose we have a set of n variables $X = \{x_1, \dots, x_n\}$ that we want to constrain to be all different. We can express this constraint as the following conjunction of binary disequations:

$$\bigwedge_{i=1}^n \bigwedge_{j=i+1}^n x_i \neq x_j,$$

or as the global constraint:

$$\text{alldifferent}(X).$$

The most apparent difference is that using a global constraint is more *high-level*, i.e., the relations among the variables are captured more succinctly and intuitively.

There are three main benefits of using global constraints. We have just seen the first one: they make our model higher-level and easier to read. Second, they allow a solver to have its own customised propagator: the one most suited to how the solver works. And in cases where a solver does not natively support a global constraint, it could have specialised rules to decompose it down to something that the solver does support and is well suited for. Third, it allows for high-level analysis that would not be possible if the model was written directly with the corresponding decompositions. We will discuss this in Chapter 4.

In this thesis, the notation $\llbracket \dots \rrbracket$ represents the truth (or 0-1) value of the mathematical relation inside the brackets. For example, $\llbracket x \leq y + 5 \rrbracket$ is true if and only if the inequation $x \leq y + 5$ is true. The notation $l..u$ will also be used, representing the set of integers $\{l, l+1, \dots, u-1, u\}$, where l and u are integers.

The following is a list of the common global constraints used in this thesis, along with descriptions of the rules they impose.

alldifferent(X). Given a set of variables X , ensure that no two variables of X are assigned the same value.

table(X, T). Given a list of variables $X = \langle x_1, \dots, x_n \rangle$, and a set of lists $T = \langle T_1, \dots, T_m \rangle$, where each list $T_i = \langle d_1, \dots, d_n \rangle$ with $d_j \in D(x_j)$, ensure that the variables in X are assigned to the values of some $T_i \in T$.

cumulative(X, Y, R, b). Given a set of n start time variables $X = \{x_1, \dots, x_n\}$, a set of n duration time variables $Y = \{y_1, \dots, y_n\}$ defining tasks $1, \dots, n$, where task i starts at x_i and ends at $x_i + y_i$, a set of n resource usage variables $R = r_1, \dots, r_n$, and a resource budget b , where a resource amount of r_i is used while task i is run, enforce that no tasks simultaneously running will have a collective resource

usage above the resource budget b . More formally, the constraint ensures

$$\sum_{i=1}^n \llbracket x_i \leq t < x_i + y_i \rrbracket \times r_i \leq b \quad \forall t \in -\infty.. \infty$$

$\text{disjunctive}(X, Y)$. Defined as $\text{cumulative}(X, Y, \{1, \dots, 1\}, 1)$.

$\text{circuit}(X)$. Given a list of n successor variables $X = \{x_1, \dots, x_n\}$, where $x_i = j$ means that j is the successor of i , ensure that repeatedly following the successors, starting from any variable, results in traversing all other variables in X exactly once before ending up back at the start.

$\text{nvalue}(m, X)$. Given a set of variables X , ensure that m is equal to the number of distinct values that all variables across X are assigned.

2.1.5 Linear and Mixed-Integer Programming

Let us step back in time from CP, which is the newest of the solving technologies described here, to the first: linear programming (LP). Linear programs (LPs) are not combinatorial per se—they do not require us to search through combinations—instead, they can be solved in polynomial time using mathematical methods.

Assume we have n decision variables $x_1, \dots, x_n$, and m constraints. A generic LP can be formulated in its canonical form as:

$$\begin{aligned} \text{find: } & x_1, \dots, x_n \\ \text{minimise: } & \sum_{i=1}^n c_i x_i \\ \text{subject to: } & L_j \leq \sum_{i=1}^n a_{ij} x_i \leq U_j \quad \forall j \in 1..m \\ & x_i \in \mathbb{R} \quad \forall i \in 1..n \end{aligned}$$

Usually, the ‘find’ line is omitted, and the variables are explained in the text.

LPs cannot fully express many combinatorial problems because all variables are assumed to be from the set of reals. LP can be extended, however, with an additional constraint that all (or some) variables must take on integer variables. This will yield a *mixed-integer program* (MIP).

MIP is seen as a lower-level formulation of a problem than CP since it requires the problem to be formulated using only linear inequalities and equalities; non-linear

constraints and complex high-level global constraints (which CP allows) are not permitted.

Example 2.2. Suppose we want to model the graph colouring problem of Example 2.1 as a MIP. Recall that our nodes are the states and territories X . As we cannot use disequalities directly, we must encode the constraints using linear inequalities. To do so, we must introduce a 0-1 variable for each node-colour pair. For example, we have a 0-1 variable representing whether Victoria is coloured red, one representing whether Victoria is coloured blue, and so on. Here, 0 means false, and 1 means true.

Let us represent each node (state and territory) and each colour by a number. Let the 0-1 variable x_{vc} represent whether node v is coloured with colour c , and let e_{uv} represent whether there is an edge between node u and node v . We can now set up the MIP as follows, where m is the number of colours used, and 8 is the number of colours allowed as well as the number of nodes considered.

$$\begin{array}{ll} \text{minimise:} & m \\ \text{subject to:} & c \times x_{vc} \leq m \qquad \qquad \qquad \forall v, c \in 1..8 \\ & x_{vc} + x_{uc} \leq 1 \qquad \qquad \forall u, v, c \in 1..8, u < v, e_{uv} = 1 \end{array}$$

The first constraint says that at most m colours are used if only colours up to $c \leq m$ are used. The second constraint says that at most one of every two neighbouring nodes can have any given colour.

2.1.6 Boolean Satisfiability

In Boolean satisfiability (SAT), we operate at a much lower level than both CP and MIP. Every variable has the domain $\mathbb{B}$ (i.e., it must be either true or false) and a *literal* is either a variable (x , representing the fact that x is true) or the negation of a variable ($\neg x$, representing the fact that x is false). Usually, SAT solvers operate on formulas in conjunctive normal form (CNF), that is, each constraint is a set of literals interpreted as their disjunction (in other words, one of the literals must be true so that the constraint is true). Although some extensions introduce optimisation abilities (e.g., MaxSAT [96]), SAT is mainly known for satisfaction problems.

Example 2.3. Suppose, again, that we want to colour the map of Australia, as in Example 2.1. Because SAT does not support disequalities, we must introduce binary (true-false) variables. We can do this using the same variables from Example 2.2: x_{vc} , which represents whether node v is coloured with colour c . In addition, we define a set E such that $(u, v) \in E$ if and only if there is an edge between node u and node v , and $u < v$.

$$\bigwedge_{(u,v) \in E} \bigwedge_{c=1}^8 (\neg x_{uc} \vee \neg x_{vc})$$

These constraints ensure that for every pair of adjacent nodes and every colour c , at least one of the nodes must not be coloured with colour c .

In this thesis, the notation $\llbracket \dots \rrbracket$ will also be used in SAT contexts for a literal representing the truth value of the mathematical relation inside the brackets.

2.1.7 More Examples of Combinatorial Problems

Let us consider a few more examples of combinatorial problems, which will be used throughout this thesis. First, consider the *job-shop scheduling problem*.

Example 2.4 (Job-Shop Scheduling). We are given a set of machines $MACs$ and a set of jobs $JOBS$. Each job $j \in JOBS$ must be run on every machine $m \in MACs$ exactly once, in some specified order specific to the job. We can represent this by dividing each job into an ordered set (i.e., list) of tasks $TSKs$, one for each machine, and say that the first task of the job must run first, the second must run second, and so on. For each job $j \in JOBS$ and task $t \in TSKs$ we have a machine requirement req_{jt} , which specifies the machine the job's task must run on, and associate it with a processing duration dur_{jt} . The decisions to be made are the start times $start_{jt}$ for each task $t \in TSKs$ and job $j \in JOBS$. While the task's starting must satisfy the aforementioned task ordering constraint, it must also satisfy the machine usage constraint, which says that a machine can only process one job at a time. The objective is to minimise the total makespan of the solution, i.e., make the latest task start as early as possible.

Next, let us consider the *vehicle routing problem*.

Example 2.5 (Vehicle Routing). We are given a set of vehicles $VEHs$ and a set of locations (i.e., requests) $LOCs$ to serve. We are also given a set of vehicle depots $DEPs$, which, together with the locations $LOCs$, form a set of nodes in a network $NODs$. Each pair of nodes $x, y \in NODs$ has an associated travel distance $dist_{xy}$ given in time units. Our goal is to route every vehicle $v \in VEHs$ from its initial depot $sdep_v$ to a series of locations, serving the customers there, and finish at its end depot $edep_v$. No two vehicles can service the same location, and every location must be served. A location $l \in LOCs$ requires to be serviced for $wait_l$ time units. For each node $x \in NODs$, we need to decide what vehicle $serv_x$ it should be served by and which other node $next_x$ should be served next by the same vehicle. The objective is usually to minimise total travel time.

Often, we can decide how many of our available vehicles to dispatch, which can drastically affect the possible total travel times.

2.2 MODELLING COMBINATORIAL PROBLEMS

Modelling, in the mathematical and scientific sense, is the art and science of reducing otherwise complex systems (e.g., reality) into brief, manageable, and quantifiable abstractions. There are many sorts and varieties of models, each with its own purpose. In this thesis, a *model* refers to a *prescriptive* model, which attempts to prescribe what one ought to do to attain the best outcome.

In the previous sections, we have seen three generic solving techniques for combinatorial problems: CP, MIP, and SAT. So far, the focus has been chiefly on *how* to solve combinatorial problems. However, before one can solve a problem, one must *model* it, i.e., mathematically formalise *what* the problem is. Additionally, while a combinatorial problem (in the most abstract sense) is unique, we must make design choices once we try to formalise it. These choices affect how we understand the problem and, more importantly for this chapter, how a solver understands and solves the problem. Formulating a good problem model is crucial since two models of the same problem can have vastly different performances with the exact same solver.

2.2.1 Problems, Models, Instances, and Solvers

Before addressing the world of modelling, let us first establish the basic terms and concepts used throughout this section.

A *combinatorial problem* (or simply *problem* when unambiguous) is what one actually is interested in solving. For example, we are dealing with a problem if we need to route vehicles or schedule jobs. To solve a problem, we first need to *model* it. A *model* of a combinatorial problem is an actual formalisation of that problem in, for example, mathematics. Every problem can be modelled in a multitude of ways. A model consists of several unknowns, i.e., mathematical variables, for which we need to determine a value. Often, we want a model to be *parametric*, which means we can simply declare some unknowns as *parameters*, whose value is to be provided as input data. The rest of the unknowns are *decision variables*, or simply *variables* when the context is clear. The benefit of having a parametric model is that one then changes the value of a parameter, instead of rewriting the model, to solve a slight variation of the problem in question. Thus, a model is often a representation of a *class* of problems of the same kind. For example, a parameter could be the number of available vehicles and the current traffic conditions in vehicle routing, or the number of jobs to schedule, and their length, in job scheduling. When a value for each parameter is provided, we obtain a *model instance* (or simply *instance* when unambiguous). A *solver*, which could be based on CP, MIP, or SAT technology, or a combination thereof, takes an *instance* and attempts to solve it by finding suitable values for the decision variables.

Let us look closer at the distinction between a *model* of a problem and an *instance* of a model. The former is parametric and requires input data to be given in order to be solved. The latter is an instantiated parametric model, where all input data has been provided and, as such, only contains variables and constraints that do not depend on unknown parameters. An instance can be represented as a COP, whereas a model can be represented as a parameterised COP [10].

Definition 2.6 (Parameterised COP). A parameterised COP, or simply called a *model*, $M[\Delta] = \langle X[\Delta], D[\Delta], C[\Delta], f[\Delta] \rangle$ is a family of COPs parameterised by Δ , called its *parameter space*. For every *data instance* $\delta \in \Delta$, $M[\delta]$ is a *model instance* (i.e., a conventional COP) written as $\langle X[\delta], D[\delta], C[\delta], f[\delta] \rangle$.

With this formalism, we can separately talk about models and instances. Note the conflating terminology of *data instance* and *model instance*. Both of these concepts are often simply called *instance*, which is usually clear from (or irrelevant within) the context. For brevity henceforward, we will also refer to both these concepts as *instance* unless clear differentiation is important.

The parameterised COP formalism is enriched with further structure in this thesis. To do this, we partition the data $\delta \in \Delta$ provided to a model into two kinds: *named objects* (or, simply, *objects*), and *data points*. The *named objects* are categorised into *object sets* in the model, where the number of object sets is specified by parameter space Δ , and the actual objects in any one set are specified by the particular instance $\delta \in \Delta$ provided to the model. In a vehicle routing problem, for example, the parameter space could declare two object sets, one for vehicles and one for locations, while each instance of this parameter space provides actual vehicle objects and location objects. Generally, we consider one object set per type of object, e.g., one object set for the jobs in a job scheduling problem, one for the product orders in a product manufacturing problem, and one for the locations to service in a vehicle routing problem. The data points denote the particular givens in an instance, often associated with an object. For example, the current location of all vehicles to route, the current expected travel times between points in a traffic network, or the durations of the jobs to schedule. The motivation behind this structure is to allow a connection between instances of the same parameterised COP, enabling a concept of two instances sharing variables and data points. Typically, the object sets are the backbone of the COP models, as they *index* most loops and arrays of decision variables and data points. For example, the set of jobs will *index* the array of precedences among jobs, and the set of locations to service will *index* the array containing the amount of time required by each location. The same named object can occur in many instances, as well as the variables and data points indexed by that object. This is necessary if one wants to consider an alternative circumstance, often a future state of the same problem in this thesis, that shares objects with the current circumstance. It is also why the objects in each object set of Δ must be uniquely named, with their number often determining the size of the instance. Let us illustrate with an example:

Example 2.6. Consider a case where we want to create a parameterised COP for generic graph colouring problems (including Example 2.1). Let us first define the parameter space Δ . The nodes that need to be coloured and the colours that are available to do so affect the size of the search space. That is to say, the more nodes are given and the more colours available, the more possible assignments one needs to consider. As such, the nodes will be represented by a set N of objects, and the colours will be represented by a set L of objects. We also have data points, e_{uv} for every $u, v \in N$, denoting whether the nodes u and v are adjacent, not affecting the size of the search space. With $\Delta = (N, L; e)$, where the semicolon partitions Δ

into named objects (those in object sets N and L) and the data points (those in e), the resulting parameterised COP could be expressed as:

$$\begin{aligned}
 M[N, L; e] &= \langle X[N, L; e], D[N, L; e], C[N, L; e] \rangle \\
 \text{where } e: N \times N &\rightarrow \mathbb{B}, \\
 X[N, L; e] &= \{x_v \mid v \in N\} \cup \{n\}, \\
 D[N, L; e](x) &= L \quad \forall x \in X[N, L; e], \\
 D[N, L; e](n) &= \mathbb{N}, \\
 C[N, L; e] &= \{x_v \neq x_u \mid u, v \in N \text{ s.th. } e_{uv}\} \cup \{\text{nvalue}(n, x)\}, \text{ and} \\
 f[N, L; e] &= n \text{ (to be minimised).}
 \end{aligned}$$

In this example, x_v is a variable denoting the colour of node v , and n is the total number of colours used (notice the use of the global constraint `nvalue`, constraining n).

Syntactically, notice that the object sets are written in upper case, and the data points are written in lower case. This notational standard will be used throughout this thesis.

2.2.2 Towards Solver-Independent Modelling

Traditionally, to use a solver, one had to learn, at least superficially, how its internals worked and then write the model directly for that solver in a language specific to it. During the early 1990s, an increasing number of off-the-shelf solvers became available, based on different solving technologies and even hybrids of them. But because of the lack of interoperability between solvers and their models, one often had to commit to a solver before knowing if it would perform well on the problem in question. Alternatively, one had to spend extra time learning another solver's internals and then translating the model to the other solver. This translation could be very time-consuming if the solvers used vastly different technologies (e.g., MIP and CP). Modellers quickly realised that this hampered the ability to experiment with different solvers.

One may ask why it is important to experiment with using different solvers. Recall, from the preceding section, that most combinatorial problems of interest are NP-hard, which means we do not know a polynomial-time algorithm to solve the problem. Inevitably, a problem instance will be too large to solve in a feasible time frame. Recall also that using an appropriate approach can delay the point at which this inevitable infeasibility begins. In our context, this means using a solver with the appropriate

types of inferences, search strategies, and heuristics for the problem in question. However, determining whether a solver is more appropriate than another requires empirical evaluation since no method dominates all others in general.

In conclusion, each solver has its benefits and drawbacks, and one must experiment with them to find one suitable for a problem. This can be very time-consuming if the model has to be rewritten for each solver. To overcome this, problem-independent modelling languages surfaced. These promised the ability to model once and, with a press of a button, run any off-the-shelf solver.

2.2.3 Algebraic Modelling

Algebraic modelling languages are chiefly aimed at the MIP solving technology. And this is, indeed, the most popular way of describing combinatorial problems throughout the broader literature. The way we *mathematically* modelled MIPs in the preceding section is essentially the way an algebraic model looks: very similar to mathematics. Algebraic modelling languages include AMPL [5], AIMMS [128], and GAMS [25]. Just as in MIP, the requirement is for variables to be either continuous or discrete and for constraints and the objective to be linear equations or inequations. Note, however, that there is some support for non-linear constraints and objectives: if possible, they can translate the constraints down to their linear counterparts or use solvers (extending MIP) that support some non-linearity.

2.2.4 High-Level Constraint-Based Modelling

Algebraic modelling solves the interoperability issue of models, but only for MIP solvers. Recall from the prior section how vastly differently CP, MIP, and SAT encode the same problem. To rewrite a CP model into a MIP model, one needs to linearise all constraints, for example. Thus, switching between solving technologies generally requires more work (regardless of whether the process is manual or automatic) than merely switching between solvers within the same solving technology.

To address this, attempts at general-purpose modelling languages emerged throughout the 2000s. Inspired by the CP view of combinatorial problems, the development of constraint-based modelling languages began. CP, as opposed to MIP and (especially) SAT, can have a high-level encoding of a combinatorial problem. The idea behind constraint-based modelling is that one expresses the problem as if one were to use a CP solver, with (not necessarily linear) mathematical equations, inequations, and disequations; array lookups; non-numeric variables (e.g., set variables); and global

constraints. Sometimes even higher-level constructs than some CP solvers natively supports are utilised. If necessary, high-level modelling constructs will be decomposed into logically equivalent but lower-level constructs that the target solver supports. For example, if a global constraint in a model has a dedicated propagator for it (or another way of handling it directly), then it will be used. However, if there is no such support, then the global constraint will be decomposed into something the solver does support. This means that each global constraint will have a decomposition prepared, as a backup, if there is no direct support for it in the target solver. Ideally, this decomposition is specially constructed for the target solver to be (generally) as effective as possible for this solver.

In constraint-based modelling, a constraint is typically one of three kinds: a *problem constraint*, a *redundant constraint* (sometimes called an *implied constraint*), or a *symmetry-breaking constraint*. *Problem constraints*, as the name indicates, are the necessary constraints that directly specify the problem being solved. *Redundant constraints*, on the other hand, are not necessary for correctness. They are used to help the solver with extra inference during solving. *Symmetry-breaking constraints* are also not necessary for correctness. They are used to eliminate equivalent assignments with different internal representations, only leaving one such representation for the solver to consider. Having multiple representations of equivalent assignments will make the search space unnecessarily large since the solver may waste time revisiting equivalent solutions and non-solutions.

There are many available medium- to high-level constraint-based modelling languages such as MiniZinc [107], Essence [58], ESRA [49], and OPL [156].

2.2.5 The MiniZinc Toolchain

This thesis extensively uses the popular MiniZinc modelling language and its toolchain. Generally, in this thesis, we will use MiniZinc syntax when describing models and use mathematical parameterised COP notation for general mathematical definitions and discussions. While most of the syntax of MiniZinc is self-explanatory, let us use the job-shop scheduling problem (Example 2.4, page 37) to illustrate the syntax. In this thesis, a typewriter font is used for all MiniZinc code, and we use green colour for keywords, blue for functions names, and grey (with a preceding ‘%’) for comments.

A MiniZinc model of the job-shop scheduling problem is given in Figure 2.4. The first seven lines declare the *parameters* of the problem, i.e., the input data that must be provided to obtain a model instance that can be solved. Lines 2 and 3 declare that the model is given a set of jobs *JOBs* and a set of machines *MACs* as input.

```
1 % PARAMETERS
2 enum MACs;           % set of machines
3 enum JOBS;           % set of jobs
4 enum TSKs = T(MACs); % set of tasks (one per machine)
5
6 array [JOBS,TSKs] of MACs: req; % machine requirements
7 array [JOBS,TSKs] of int: dur;  % processing durations
8
9 % VARIABLES
10 array [JOBS,TSKs] of var 0...: start; % start times
11
12 % CONSTRAINTS
13 %% task ordering
14 constraint forall (j in JOBS, t in ..<max(TSKs)) (
15   start[j,t] + dur[j,t] <= start[j,enum_next(t)] );
16
17 %% machine usage
18 include "disjunctive.mzn";
19 constraint forall (m in MACs) (
20   disjunctive([start[j,t] | j in JOBS, t in TSKs where req[j,t]=m]
21             , [dur[j,t] | j in JOBS, t in TSKs where req[j,t]=m]
22             ) );
23
24 % OBJECTIVE (makespan)
25 solve minimize max (j in JOBS) (
26   start[j,max(TSKs)] + dur[j,max(TSKs)] );
```

Figure 2.4: A MiniZinc model for the job-shop scheduling problem

The `enum` keyword denotes that the set contains uniquely identifiable names. Line 4 defines the set of tasks $TSKs$, computed in the declaration to be of equal size as $MACs$. Note that T is a function that creates a bijection between $MACs$ and $TSKs$, introduced and defined by this. Lines 6 and 7 declare the machine requirement req_{jt} and processing duration dur_{jt} for every job $j \in JOBS$ and task $t \in TSKs$. To declare these in MiniZinc, we use the `array` construct. The first array has domain $MACs$, while the second has domain $\mathbb{Z}$. Both are indexed first by $JOBS$ and then by $TSKs$. We use array access in MiniZinc and subscripts of mathematical symbols analogously: for example, $req[i, j]$ is used to denote the value of req_{ij} . Line 10 declares the array $start$ of decision variables (using the keyword `var`) denoting the start times for the tasks. Note that this array also has domain $\mathbb{Z}$ but with a lower bound of 0 and no upper bound (denoted as $0..$). This is because time step 0 is the first time step where we can schedule anything.

Let us now attend to the constraints of the model. The two constraints in lines 14-22 enforce the task ordering and machine usage constraints. To do this in MiniZinc,

we use the `constraint` keyword, followed by the Boolean expression that must hold according to the constraint. Lines 14 and 15 post the task ordering constraint, using the `forall` function to post a linear inequation for every task (except the last one) in a job, ensuring that the task finishes before the next task starts. Note that this requires us to have some sort of ordering on the object set *TSKs*. By default, every object set has an ordering imposed by the order the objects are provided to the model. The `..max(TSKs)` construct generates a subset of *TSKs* where the last (maximum) element of it is missing. The `enum_next(t)` call returns the next task in the ordered set *TSKs*. Line 18 contains an `include` statement to include the library describing the disjunctive constraint and its decomposition, which will be used for the machine usage constraint. Lines 19–22 post the machine usage constraint, again using the `forall` function as we want one disjunctive per machine. We use an array comprehension to generate the appropriate start and duration arrays for each disjunctive constraint inside the `forall`. In general, an array comprehension is formatted as `[E | G where P]`. Here, *E* represents an expression (of some type), *G* represents a list of generators defining variables that can be used by *E*, and *P* represents a local constraint over the variables of *G* that must hold for a particular generator variable assignments to be included in the generated list. In our example above, for every machine *m*, the first list comprehension results in an array of start times variables for every job and every task of that job that requires *m*. Note that, in our example, there is exactly one task requiring *m* per job.

Finally in the model, lines 25 and 26 declare our objective: to minimise the makespan of the overall schedule, that is, to make the last of all jobs finish as early as possible. To declare an objective in MiniZinc one must use the `solve` keyword. Since we want to minimise, we use it together with the `minimize` keyword. We use the `max` function to obtain the finishing time of the latest final task over all jobs.

See, also, Figure 2.5 for a complete model of the vehicle routing problem (Example 2.5). In particular, let us highlight the construct in line 5, which defines a new object set *NODs* based on vehicles *VEHs* and locations *LOCs*. Suppose we have two vehicles and two locations, which results in the first two elements of *NODs* representing the start depots of the vehicles, the next (middle) two elements representing the end depots of the vehicles, and the final two elements representing the two locations. In addition, the following common constructs of the MiniZinc language will be used throughout this thesis:

- conditional statements, e.g., `if b then t else e endif`, which yields t if b is true and e otherwise, where e is by definition set to true if the else branch is omitted;
- array concatenation, e.g., `a ++ b`, which builds an array by concatenating array b at the end of array a ; and
- existential quantification, e.g., `exists (i in A) ( P(i) )`, which holds if for some $i \in A$ the condition $P(i)$ holds.

Notice the congruence between our mathematical concepts and MiniZinc: an `enum` declares an object set, and the other parameters without the `var` keyword declare data points.

MiniZinc is more than just a modelling language; it is a toolchain that includes a compiler to transform a high-level MiniZinc model (representing a parameterised COP) into a low-level language suitable for solvers, called FlatZinc (representing COP). The compilation process requires values to be given to all parameters since FlatZinc is not parameterised. It also requires users to specify the solver to be used so that the FlatZinc model can be tailored to that solver. The toolchain also has an integrated development environment (IDE) with a user interface for writing and running MiniZinc models.

Every year, a competition called the MiniZinc challenge is held [148]. In this competition, problem sets and solvers are provided and compete to be the fastest and best. We will use some of the problems and instances found in the MiniZinc benchmark suite, where all problems and instances throughout the MiniZinc challenge are found.⁴

⁴<https://github.com/MiniZinc/minizinc-benchmarks>

```

1 % PARAMETERS
2 enum VEHS; % set of vehicles
3 enum LOCs; % set of locations to visit
4
5 enum NODs = S(VEHS) ++ E(VEHS) ++ L(LOCs); % nodes
6
7 array [VEHS] if int: ready; % time veh ready to use
8 array [NODs] of int: wait; % wait time per node
9 array [NODs,NODs] of int: dist; % distance between nodes
10
11 % VARIABLES
12 array [NODs] of var int: arri; % arrival at node
13 array [NODs] of var VEHS: serv; % node served by
14 array [NODs] of var NODs: next; % successor of node
15
16 % CONSTRAINTS
17 %% link vehicle with its start and end
18 constraint forall (v in VEHS) ( serv[S(v)] = serv[E(v)] );
19
20 %% same vehicle serves whole tour
21 constraint forall (n in S(VEHS) ++ L(LOCs)) (
22   serv[n] = serv[next[n]] );
23
24 %% cannot leave before ready (assumes wait is 0)
25 constraint forall (v in VEHS) ( arri[S(v)] = ready[v] );
26
27 %% travel time and wait time constraints
28 constraint forall (n in S(VEHS) ++ L(LOCs)) (
29   arri[n] + wait[n] + dist[n,next[n]] = arri[next[n]] );
30
31 %% hamiltonian grand circuit constraint
32 include "circuit.mzn";
33 constraint circuit(succ);
34
35 %% symmetry-breaking constraints for grand circuit
36 constraint next[E(max(VEHS))] = S(min(VEHS));
37 constraint forall (v in ..<S(max(VEHS))) (
38   next[E(v)] = S(enum_next(v)) );
39
40 % OBJECTIVE (minimize total travel cost)
41 solve minimize sum (v in VEHS) ( arri[E(v)] - ready[v] );

```

Figure 2.5: A MiniZinc model for the vehicle routing problem

2.3 UNCERTAINTY AND DYNAMISM

In this chapter, we have thus far made two implicit assumptions about the nature of combinatorial problems, which are often held implicitly in the combinatorial optimisation community at large. The two assumptions are that combinatorial optimisation problems are:

Certain. The problem description is complete and deterministic, i.e., there is no *uncertainty*.

Static. The problem description remains identical from the beginning to the final execution of the solution, i.e., there is no *dynamism*.

For many practical and important real-world applications, these two assumptions are too strong [24]. Let us first understand when these two assumptions do not hold with an example.

Suppose we are dealing with a vehicle routing problem and must pick up and deliver goods at different locations. If we only have rough estimates of the travel times between these locations and need to plan according to these, then the problem is not certain, that is, it has uncertainty. However, if we were to have a perfect forecast of the travel times at every time, then the problem would be certain. If a new request may appear at any time, and we need to react to new requests when they do appear, then the problem is not static; it is instead dynamic.

On a final note, the real world is indisputably uncertain and dynamic, meaning that real-world applicable solving methods must be able to address uncertainty and dynamism.

2.3.1 Definitions

First, let us acknowledge that the terms ‘uncertainty’ and ‘dynamism’ are hard to separate and often conflate. In the literature, a problem that exhibits any or both of these is sometimes called other names as well, such as *online problems*, *stochastic problems*, or *evolving problems*. In the remainder of this thesis, we will use the terms ‘uncertainty’ and ‘dynamism’ interchangeably, indicating that neither of the aforementioned assumptions may hold. Next, we will define exactly what it means, in the context of this thesis, for neither of the two assumptions to hold.

When dealing with uncertainty, the concept of time becomes central. As time passes, uncertainties become realised, that is, we are no longer unsure what is about

to happen; instead, we can look back at what did happen. Similarly, as time passes, events will occur that alter the nature of the problem we are solving and become a ‘static’ part of our past. Throughout this thesis, we will denote meta-variables representing time points by the symbol ‘ τ ’, including any subscripts or prime marks.

Mathematically, one can formalise uncertainty within combinatorial optimisation using the *dynamic constrained optimisation problem* (dCOP) formalism.

Definition 2.7 (dCOP). The *dynamic constrained optimisation problem* (dCOP) is a (possibly infinite) list of COPs $Q = \langle P_1, P_2, \dots \rangle$, where P_k represent the state of the problem at the k th time step, in which new changes to the problem have occurred, uncertainties have newly been realised, or both.

A solution to a dCOP is different from a solution to a conventional COP. Solving a dCOP requires having solutions for every P_k in the list. If we have solutions $\mathcal{A}_1, \dots, \mathcal{A}_{k-2}, \mathcal{A}_{k-1}$ for the problems up to P_{k-1} and now need to solve P_k we say that we have an *incumbent* solution $\mathcal{A}_{k-1}$ that we may need to alter to solve P_k .

We highlight the difference between *solving* a problem P_k and *executing* that solution in the *problem milieu* (i.e., the environment in which the problem evolves, e.g., a corresponding problem simulation or real-world setting). The solver can only *recommend* what decision to take, and during execution, we try to match this recommendation as closely as possible. We say that we have *perfect execution* if the recommendations of the solver are followed flawlessly at all times. This is unlikely in real-world settings. We often, instead, have *uncertainty in execution*, under which the execution may deviate from the recommended decisions. This means that external sources can influence all our decisions until they are *realised*, i.e., they have been executed and observed. For example, if the solver schedules a job to start at precisely one o’clock, but the job actually starts a few minutes after one o’clock in the problem milieu, then we have had uncertainty in execution. This is inevitable in the real world because of ever-present chaos. The state of being *realised* also applies to data points. For example, a job may have an uncertain processing duration, but as soon as the job is finished, we have fully observed the actual processing duration, that is, the duration is *realised*. In this thesis, we extend the definition of being realised to include data points that are guaranteed never to be used and can thus be safely ignored.

There are generally two categories of uncertainty: *exogenous* and *endogenous*. *Exogenous* uncertainty comes from uncontrollable factors (e.g., weather or traffic). *Endogenous* uncertainty comes (to some degree) from factors controllable by the decision-taker. In this case, the outcome depends on the action taken, but it may

be only somewhat predictable. For example, deciding to increase production may (unpredictably) negatively impact the prices of the products and thus profit, or it may negatively impact the supply chain. We assume that both categories of uncertainty take place in our problems. As such, in a dCOP $\langle P_1, P_2, \dots \rangle$, the decisions we make when solving P_k may influence the structure and nature of the subsequent instances.

One risk with solving problems full of uncertainties is that decisions we take *now* could lead to irreparable situations and points of no return. Recovering from these situations is a complex task and out of the scope of this thesis. Herein, we assume that the problems we solve exhibit *complete recourse* [37], which means no decisions we make when solving P_k can contribute to any subsequent instance becoming unsatisfiable. This assumption allows us to concentrate on problems where decisions taken at any point cannot lead to such irreparable situations, and instead simply lead to either higher or lower objective value in the future.

It is also useful to divide events into two categories: those that add or remove objects and those that change the value of data points. For example, a scheduling problem with a never-ending influx of jobs to schedule is an example of events that adds objects. By contrast, a scheduling problem where the machines can require unpredictable maintenance at any time, being unusable for a period, is an example of events that change the value of a data point (i.e., the status of a machine). Object sets for which dynamic events can add or remove objects are referred to as *dynamic object sets*.

2.3.2 Addressing Uncertainty: A Need for New Frameworks

The following example provides some intuition for the challenges and approaches to addressing optimisation problems with uncertainty. Suppose one is tasked with designing package delivery routes for the next few days for a postal service company. Every morning the delivery trucks have to leave the (only) city depot, which contains all packages to be delivered and deliver them to the different addresses around town. The first question regards how many trucks to send out. Counting the night before how many packages there are to deliver is not suitable as an unknown amount of new express-class packages could arrive in the morning that need to be delivered the same day. This is one uncertainty of the problem. The second question regards where to send out the trucks. Precomputing all the routes the night before is again unsuitable due to the possible arrival of express-class packages. Further, vehicle routing is NP-hard, i.e., it takes time to find good solutions, so it may be difficult to find a good routing even when disregarding all uncertainty. With the addition of the new

packages arriving in the morning, the optimal routings may drastically change. This is yet another form of uncertainty. The third question is what order each truck must follow when delivering the packages. When a truck leaves the depot, the packages it carries cannot be altered, but the order in which it visits its (remaining) delivery addresses can be changed. If any unexpected traffic jams or accidents happen, the ability to change the order of visits (or even not visit some of the remaining locations at all) can be useful to keep in mind. The best method may be to react dynamically to any of the above events or plan meticulously beforehand, covering all possibilities.

Deciding how to deal with these uncertainties depends on what our goal is. Our goal could be to provide a plan that is likely to be good enough no matter what happens, to provide a plan that is statistically optimal given the expected outcomes, or to do as little preplanning as possible (e.g., because it would be wasted effort to do so). Of course, this is an oversimplification, but it provides a flavour of the many choices when solving uncertain and dynamic problems.

In order to tackle optimisation problems with uncertainty, new frameworks have emerged: *stochastic* optimisation (also known as *stochastic programming*), *robust* optimisation, and *online* optimisation (also known as *reactive* or *dynamic* optimisation).

Let us first discuss *stochastic* optimisation, which attempts to fully determine the different changes that can happen, when they can happen, and each of their probabilities. Instead of producing a single solution, the stochastic information is used to create a *policy* of solutions, mapping each possible change to the best course of action therefrom. This creates a tree-like structure branching out at all possible points of change. Thus, stochastic optimisation operates from a proactive standpoint, i.e., we want to proactively create a plan with enough detail to be applicable regardless of what happens in the problem milieu. This method is best suited for problems where the uncertainties are well-behaved and, to some degree, predictable and where the goal is to ensure some probability on the quality of the end solutions. If the problem is too dynamic or goes on for too long, then the policy of solutions quickly becomes infeasibly large and complex to execute.

Next, let us discuss *robust* optimisation [31], which attempts to produce a solution that is minimally impacted or rarely invalidated by any changes that could happen. Thus, the goal is to make re-planning a rare event. We say that an incumbent solution is *robust* if it has a high likelihood of remaining a solution after a dynamic event and *flexible* if it is easily altered to remain a solution after a dynamic event. This method is, again, best suited for problems where the uncertainties are well-behaved

and, to some degree, predictable. Note that robust optimisation also operates from a proactive standpoint.

Finally, we discuss *online* optimisation. The idea here is to solve the problem with the best currently available information but initiate a new solve as soon as significant changes happen. This method is best suited for problems where the uncertainties are too many and too hard to predict. We say that a method of solving a dynamic problem is *online* if it reacts to any change and re-solves; conversely, we say that a method is *offline* if it solves only once, before execution commences. Stochastic optimisation is chiefly offline as it aims to produce a self-sustained policy of solutions. Robust optimisation is also chiefly offline but has room for online methods insofar as repairing flexible solutions is concerned. Stochastic and robust optimisation often require prior expert analysis to start the solving, to qualify and quantify the uncertainties fully. However, this is not needed for online optimisation, where one only needs to describe how to react to changes in the problem. Note that conventional combinatorial optimisation, solving problems without uncertainty, is considered to be offline.

This thesis focuses on online combinatorial optimisation, which is applicable in many real-world scenarios. For example, if we are to allocate disaster funds, then online optimisation is often helpful as predicting disasters and the global economy are difficult. Similarly, scheduling processes at a manufacturing plant may call for online optimisation as unpredictable disruptions and requests may appear at any time. Likewise, staff rostering, especially during epidemics and pandemics where anyone may call in sick at any time, calls for online optimisation as we cannot plan too far into the future and must respond to changes in staff availability while following regulations.

2.4 ONLINE COMBINATORIAL OPTIMISATION

An algorithm is said to be *online* if it does not consider the whole problem and its information when making decisions but instead successively considers the input as it is fed to the algorithm. Thus, the output is generated successively in a similar manner. Online combinatorial optimisation, put simply, is about using online algorithms to solve combinatorial problems. In highly dynamic situations, it is difficult to escape using online algorithms to some degree since countless unpredictable events, changing the nature of the problem, are fed to the optimisation algorithm over time. The study of online combinatorial optimisation had a slow start in the 1960s but gained formality

and some popularity in the mid-1980s [70]. Today, there are many online optimisation algorithms specialised for specific problems (see, e.g., [19, 97, 120, 158]).

If one had perfect knowledge and infinite time, using an optimal offline algorithm would always be better (or equal) to using an online algorithm. However, we often have imperfect knowledge, little time, or both. Thus, we need to periodically or continuously revisit the problem and quickly revise the incumbent solution to match and best fit the current state of the problem.

2.4.1 The Rolling Horizon Viewpoint

One can look at online optimisation through the lens of the *rolling horizon* viewpoint, also sometimes called the *sliding window* viewpoint [142]. This viewpoint is based on an iterative approach that ‘rolls’ a *planning horizon* forward along a timeline in discrete solving segments called *sessions*. The viewpoint is visualised in Figure 2.6. At the top, three sessions are shown—in each session, a horizon, split into four sub-horizons, is depicted—and each session covers more data than the previous one; hence, the ‘rolling’. Below this, a single horizon is visualised in more detail. Let us consider the sub-horizons from left to right. First, the grey-shaded sub-horizon consists of the *realised*, data and decisions, i.e., that which have been executed by actors in the problem milieu. Naturally, the optimisation algorithm cannot revise the realised decisions, and it can be sure that no realised data or decision will ever change. The data left of the dotted horizontal line were realised in previous sessions, and the data right of it is newly realised. Second, the grey-hatched sub-horizon is the *commit horizon*, which consists of decisions that the solver is no longer allowed to revise but is not executed in the problem milieu (and thus may change externally). Similarly, the decisions left of the dotted line were committed in previous sessions, and the ones right of it are newly committed. Third, the blank sub-horizon with a thick border is the *decision horizon*, which contains the decisions that the solver can control, subject to the problem constraints. The decisions left of the dotted line were available in previous sessions and can be revised, and the ones right of it are fresh decisions. The first two (with possibly part of the third) sub-horizons form the *communication horizon*. This is the information that is communicated to the actors in the problem milieu, allowing them to prepare or execute accordingly. The fourth sub-horizon is the *lookahead* (also known as *forecast*), which only contains data about what future decisions and events may occur. The commit horizon, decision horizon, and lookahead form the *planning horizon* (also known as the *visibility horizon*), which

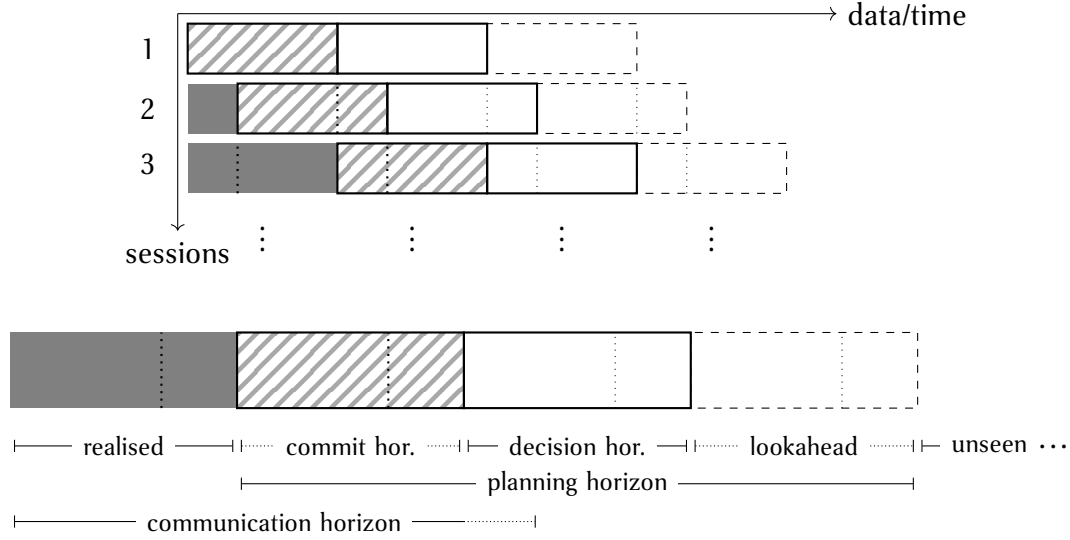


Figure 2.6: A graphical representation of the parts and terminology of rolling horizon. In each session, the planning horizon rolls forward in both time and data.

is the subproblem that the solver currently considers and operates on. Finally, beyond the lookahead, is the unknown or *unseen* future.

Each time the planning horizon rolls forward, new data and decisions will be put into each horizon. That is to say, data from the unseen future is moved to the lookahead; data from the lookahead is moved into the decision horizon, where new the new decisions are revealed; decisions are moved from the decision horizon to the commit horizon; and, at the left-most part of the horizon, decisions exit the commit horizon once they are realised, along with any other newly realised data. In particular, some data and decisions remain in the planning horizon from the previous session and can be reexamined and revised. We call this the *overlap* of two sessions. Data and decisions that overlap in two sessions are called *old* in the later session, and the others' are called *new*. Old decisions have an incumbent assignment (i.e., what they were assigned before solving started) that may need to be considered during solving. Due to dynamic events, old data and decisions may have changed since the last session. We call the number of decisions within the decision horizon the *width* of the planning horizon (or session), and the number of decisions in the planning horizon that becomes realised in the next session the *step size*.

The lookahead is optional. It may consist of predictions from stochastic data. In the lookahead, constraints are sometimes relaxed to strike a balance between fast but myopic solving and slow but better or more robust solving in the decision horizon. The decision horizon is not optional, but it may be empty in some sessions

depending on the decisions currently available. The commit horizon is not optional either because, at some point, the solver must *commit* to a decision, promising it will not revise it [88]. At the very least, this occurs at realisation (often referred to as having no commit horizon) but can occur earlier. Having no commit horizon makes the actors in the problem milieu less able to prepare for execution. A long commit horizon means that the solver commits early, allowing actors in the problem milieu to prepare adequately, but it limits the solver's adaptivity to unpredicted events. Designing a good commit horizon remains a fundamental question for many problems and applications. Finally, as the communication horizon may include parts of the decision horizon, some sort of stability criteria may be desired when going from the incumbent solution to the new solution [31], making sure that the plan does not change too drastically going from one solution to the next. This helps the actors in the problem milieu plan more rigorously.

Interestingly, online optimisation can be used not only for problems that have intrinsic uncertainty but also for those with no uncertainty whatsoever. In such cases, the online nature of the solution method *introduces* a kind of artificial uncertainty into the problem. This can be desirable if a static problem is too large to solve when considering all the information simultaneously. In such cases, we use the rolling-horizon viewpoint to break up the problem into smaller subproblems and solve them one by one, obtaining a solution for each subproblem that can be put together, creating a solution for the whole problem. Uncertainty is introduced because we need to make decisions for the early subproblems without knowing what is to come in later ones. We say that the problem is transformed into a *rolling-horizon decomposition*. In this case, the lookahead is optional and may contain actual data about what is next to come. Further, commitment and realisedness are indistinguishable concepts as there is no uncertainty in execution.

2.4.2 Paradigms of Online Optimisation

There are generally three modus operandi of online optimisation [70]; which we will refer to as paradigms:

The Pure Sequence Paradigm. A single decision at a time is considered and needs to be taken and committed to before the next decision can be taken (see, e.g., [42]).

The Stage-based Sequence Paradigm. A set of decisions are considered simultaneously and must be committed before the next set of decisions is considered.

That is, it splits up the problem into a sequence of decision stages instead of a sequence of individual decisions (see, e.g., [126, 166]).

The Time-stamp Paradigm. Decisions and data become available at given (or unknown) times, and we must make decisions while new data is revealed. In this paradigm, the same decision may be revisited multiple times as more information becomes available before we finally commit (see, e.g., [88]).

Next, we will see how these three paradigms fit into the rolling horizon viewpoint.

Looking at the pure sequence paradigm through the rolling horizon viewpoint, the width of each session is one, and so is the step size. There may or may not be lookahead into what decisions will be or likely be revealed next. In the stage-based sequence paradigm, each session's width and step size are greater than one, but there is similarly no overlap between sessions. That is to say, the step size is equal to the width. By contrast, overlap between sessions is possible in the time-stamp paradigm. In fact, the width, overlap, and step size can be set to any value and even dynamically change throughout the solving process; thus, multiple decisions are often considered simultaneously.

Let us illustrate the pure sequence paradigm with an example. Consider the *online bin-packing problem* explained as follows [140]:

Example 2.7 (Online Bin Packing). We have a sequence of items $t_1, t_2, \dots$ to pack into bins. We have an unlimited number of bins of equal capacity q at our disposal. Each item t_i have the size denoted $|t_i|$, where $|t_i| \in \mathbb{R}$ and $0 < |t_i| \leq q$. Each item must be put into exactly one bin, and the sum of the item sizes in a bin must not overreach its capacity q . The goal is to minimise the number of bins that have one or more items in them, i.e., pack everything in the least possible number of bins. The items are considered one by one and must be assigned (and committed) to a bin before the next item in the sequence is considered. There is no lookahead.

The above is a prime example of purely sequential online optimisation; in fact, the problem description forces the use of online optimisation. Pure sequence problems are often pretty simple, with few constraints.

The stage-based sequence paradigm, assuming non-trivially sized stages, is not as widely applicable as the pure sequence paradigm since not all problems can be nicely split into stages. In practice, each stage is often NP-hard. We can transform the online bin-packing example into a stage-based problem as follows. Instead of considering

items one by one, we consider a batch of items simultaneously and commit to an assignment before the next batch is revealed.

The time-stamp paradigm can either be *preemptive* or *non-preemptive*. In the *preemptive* version, a session can be cancelled at any time during solving, and a new solving session may be started, considering new data, immediately after. This adds flexibility, e.g., in the case of time-sensitive dynamic events, but also adds a new layer of complexity. More often, one considers the *non-preemptive* version, where once a session starts, it must run in full and return its result before a new session can start. Regardless, we must decide when to prompt the solving (i.e., start the next solving session) as the solver is always allowed to wait until enough requests have arrived before triggering a new solving session. Notice that in the time-stamp paradigm, we not only need to decide the value of each decision variable but also *decide when to commit to it*.

Let us illustrate the time-stamp paradigm with two examples often addressed using this paradigm: the *dynamic job-shop scheduling problem* (extension of Example 2.4) and the *dynamic vehicle routing problem* (extension of Example 2.5). These examples will be used throughout this thesis.

Example 2.8 (Dynamic Job-Shop Scheduling Problem). Just as Example 2.4, we have a set of machines $MACs$ a set of jobs $JOBs$, and a set of tasks $TSKs$. In this dynamic version, $MACs$ (and thus $TSKs$) are static, but $JOBs$ is dynamic, i.e., new jobs can be added anytime. Again, we have the machine requirement req_{jt} processing duration dur_{jt} , for every job $j \in JOBs$ and task $t \in TSKs$. For simplicity, we assume that each req_{jt} is static (i.e., the requirements never change), but each dur_{jt} is assumed to be dynamic (i.e., the durations may change at any time). The decision variables are again the start times $start_{jt}$, and we must satisfy the task ordering and the machine requirement constraints. The objective is to minimise the total makespan of the currently available jobs.

In any session, we may have multiple jobs that need to be scheduled and possibly multiple committed jobs to adhere to. Moreover, we may have jobs where some tasks are committed and some are not. The key question is when to prompt the solver: do we wait until we have many jobs, risking a later execution, or do we solve aggressively, ensuring an early execution but increasing the risk of regretting the decisions once the next jobs are revealed?

Next, let us look closer at dynamic vehicle routing.

Example 2.9 (Dynamic Vehicle Routing Problem). Just as Example 2.5, we have a set of vehicles *VEHs*, and a set of locations *LOCs*, a set of vehicle depots *DEPs*, and a set of nodes *NODs* (containing *LOCs* and *DEPs*). In the dynamic version, we assume that *VEHs* and *LOCs* (and thus *NODs*) are dynamic, i.e., new vehicles and locations can arrive at any time, and we assume that *DEPs* is static. The travel distance time $dist_{xy}$ between each pair of nodes $x, y \in NODs$ is dynamic, i.e., the time it takes to travel between them can change at any time. The service time $wait_l$ for location l is also dynamic. Recall that for each node $x \in NODs$, we need to decide what vehicle $serv_x$ it should be served by and which other node $next_x$ should be served next by the same vehicle. Once a vehicle leaves the depot, we cannot change the locations it will serve. However, we can change the successor of a not yet visited node. The same constraints apply in the dynamic version as in the static version.

There are a few objectives that could be considered, perhaps in combination: minimising average or maximum latency (i.e., the time between a service request is revealed and actually served), maximise throughput (i.e., number of requests served per unit time), minimise total travel time, or minimise vehicle usage.

Again, in any session, we may have multiple vehicles, perhaps halfway through their journey, and multiple requests to service. While a vehicle that has left the depot cannot change its locations to service, the order of the remaining locations to be serviced can be changed. This can be useful in case of, e.g., traffic disruptions. A similar question of when to prompt the solving also applies here.

2.4.3 Real-Time Online Combinatorial Optimisation

A special and particularly interesting case of online optimisation is *real-time online optimisation*, or simply *real-time optimisation* [70]. Let us highlight three key differences between real-time and non-real-time online optimisation:

1. In real-time optimisation, the decisions must be executed *as soon as possible*, while in non-real-time, there may be a delay before the decisions can even begin to execute.
2. In real-time optimisation, the solving and execution of the solutions occurs *simultaneously*, while in non-real-time, the execution may pause and wait for (updated) instructions or be so far delayed as to become irrelevant.

3. In real-time optimisation solving, increasing the solving time will not only lead to later execution but to decisions produced *becoming outdated or even invalid* because of events happening in the background if solving for too long, while in non-real-time, this issue is less likely and the threshold for ‘too long’ is higher.

Note that this is not another paradigm; it is a special case that can be applied to all three paradigms we discussed in the previous section.

There is a major consequence of the above three points that warrants further discussion. Recall that online optimisation consists of a series of solving sessions. In real-time optimisation, *how long* to run a session for is a central question. We call this the session’s *length*. A session’s length significantly impacts what is realised and committed since the longer we solve, the later the results will be announced to the problem milieu, resulting in more decisions and data being realised compared to having a short session. It is difficult to adaptively change the session length mid-solving while reusing any inference since having a longer session could change the circumstances significantly. Thus, we need to decide on a session’s length before the session starts. Solving for longer rather than shorter, aside from increasing the risk of decisions becoming outdated, leads to the decisions becoming executed later. A solving session that is too short leads to little time for finding good solutions and may eliminate chances of finding any. Thus, we want to solve for long enough that we can find good solutions but short enough to execute the resulting decisions as early as possible. Going too far in either direction leads to an overall poor solution across the entire problem horizon. Another nuance to highlight is how many decisions we must consider in each session. If we have short and more frequent solving sessions, then the sessions will likely be easier to solve than if we have long and less frequent ones. The less frequently our sessions occur, the more new data and decisions we must consider in each new session. We call this the *meticulousness-quickness trade-off*.

2.4.4 Competitive Analysis of Online Algorithms

As mentioned afore, an optimal offline algorithm with perfect knowledge and unlimited time will always give better results than an online algorithm. The *competitive ratio* of an online algorithm denotes *how much worse* than an optimal offline algorithm it is. We say that an online algorithm is *competitive* in general if its competitive ratio is bounded, i.e., constant as the instance size grows. The analysis of these values and algorithms is called *competitive analysis*.

Specialised insights into the problem being solved are often required to design a competitive online algorithm. In other words, we must design bespoke strategies

for each problem to obtain good competitiveness. For example, there is an online algorithm for the online bin-packing problem (of Example 2.7) that has a competitive ratio of 1.58889 [140]. However, the competitive ratio of any algorithm solving the dynamic job-shop problem (of Example 2.8) is unbounded [70]. General-purpose online algorithms, therefore, will always have an unbounded competitive ratio.

While it may be interesting to perform competitive analysis of general-purpose online algorithms given specific heuristics and on a small class of problems, even if unbounded, this is outside the scope of this thesis and remains future work.

2.5 FAIRNESS, BALANCE, AND DISPERSION

So far, we have seen many kinds of optimisation problems, but every one of them has had a single objective function. In many real-world applications, however, there are multiple stakeholders (agents), each with their own views and preferences. In other words, each agent has its own objective. Similarly, even in single-agent problems, multiple, perhaps conflicting, objectives may need to be considered simultaneously. Thus, we often want to ensure that the solution produced is, to some degree, balanced or fair across these objectives. However, what the terms ‘balanced’ and ‘fair’ mean is debatable, and there are several established schools of thought on the matter. This section will establish the statistical and other fundamental concepts regarding fairness and balance in optimisation problems.

To provide some intuition, let us consider an example. Suppose we are solving a static problem with multiple objectives (each representing the preferences of an agent), each objective is to be maximised, and they conflict (i.e., increasing one objective may decrease another). The simplest way to consider all the objectives simultaneously is often by summing all the objectives up and then solving the problem, maximising this sum. This is called the *utilitarian* approach. The upside of this approach is that it is simple and wastes little resources (i.e., maximises total output); the downside is that any agent can get arbitrarily little, or even nothing. In other words, there is no guarantee on fairness or balance.

2.5.1 Statistical Preliminaries

One of the most common ways to statistically summarise a set of numbers is that of its central tendency, and arguably the most prevalent of those is the *arithmetic mean*. Suppose we have a *population* X of values $x_1, \dots, x_n \in \mathbb{R}$. We respectively denote the

arithmetic mean and the *sum* of these as:

$$\mu_X = \frac{1}{n} \sum_{i=1}^n x_i, \quad M_X = \sum_{i=1}^n x_i.$$

For brevity, the subscript ‘X’ is lost when there is little room for ambiguity.

A *measure of dispersion* is another way of statistically summarising a set of numbers. It describes how different or similar values of X are. The first measure of dispersion we consider is the *population variance*, or simply *variance*. The *variance* of X is defined as the average squared difference from the arithmetic mean, with its conventional and alternative formulations as follows:

$$\sigma_X^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2, \quad \sigma_X^2 = \left(\frac{1}{n} \sum_{i=1}^n x_i^2 \right) - \mu^2.$$

A related measure of dispersion is the *standard deviation*, which is the square-root of variance:

$$\sigma_X = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2},$$

and observe that this is a monotonic transformation of variance. As such, minimising (and maximising) standard deviation is equivalent to minimising (and maximising) variance. Similarly, any constraints specified on the standard deviation can be squared to obtain the equivalent constraint on the variance. The variance and standard deviation values are bounded below by zero and unbounded from above.

The second measure of dispersion considered in this thesis is the *coefficient of variation*. The *coefficient of variation* (CV) of X is defined as the standard deviation normalised by the mean of X as follows:

$$CV(X) = \frac{1}{\mu_X} \sqrt{\sum_{i=1}^n (x_i - \mu)^2}.$$

Because of the division and the possibility of the mean taking on a negative sign, the coefficient of variation is unbounded in both directions.

The third measure of dispersion we consider is the *Gini coefficient*. The *Gini coefficient* of X is defined as:

$$\text{Gini}(X) = \frac{\sum_{i=1}^n \sum_{j=i+1}^n |x_i - x_j|}{n \sum_{j=1}^n x_i}.$$

The Gini coefficient, originally developed for measuring income inequality, measures how far a distribution X deviates from a totally equal distribution. Generally, the Gini coefficient is only well-defined for positive values in X . With this restriction, the Gini coefficient is always between zero and one.

2.5.2 Agents, Utilities, and Multiple Objectives

If we have multiple stakeholders (each with an objective) or just multiple objectives for a single stakeholder in an optimisation problem, then we will in this thesis refer to this problem as having multiple agents and let each of the objectives be associated with a unique agent. Suppose we have a multiagent problem with a set of n agents $A = \{a_1, \dots, a_n\}$. Each agent $a_k \in A$ has a utility function u_k that provides a cardinal representation of the benefit (or happiness) that a_k draws from a given solution to the considered problem. More formally, such problems can be described with the COP formalism as follows:

Definition 2.8 (maCOP). A *multiagent constrained optimisation problem* (maCOP) with n agents is a tuple $P = \langle X, D, C, u_1, \dots, u_n \rangle$ where there is no strict total ordering of importance of the objectives. We define $\text{sols}(P)$ as the solution space of the CSP $\langle X, D, C \rangle$. Note that for each $1 \leq k \leq n$ the quadruple $\langle X, D, C, u_k \rangle$ is a conventional COP.

The question is how to simultaneously solve the hard underlying problem while ensuring some balance between (and sufficient consideration to) all the utility functions. By convention, the higher the utility, the better it is for the agent in question. We will see how this definition can be combined with the dCOP formalism in Chapter 5.

2.6 CONCLUDING REMARKS

We have now had an in-depth walkthrough of the relevant topics and concepts established in the optimisation (and related) fields. Before we dissect the contributions of this thesis, let us clearly state our assumptions moving forward.

The methods and techniques presented in this thesis are developed from a pure reaction viewpoint. This means that we assume that the algorithms have no lookahead into, and no statistical information or predictions of, the future states of the problem. There is no reasoning about whether (or how) the information may change or may have changed. In pure reaction, one assumes the world is fixed when solving and then react to any changes (i.e., re-solve) when they happen. In a similar vein, we ignore

robustness and stability criteria. Lastly, we also assume that our online optimisation algorithms are non-preemptive, i.e., once solving starts, we must wait until it finishes before we can start solving again. We believe, however, that extending the concepts presented in this thesis to the broader domain will be relatively simple. It remains future work, nevertheless.

HIGH-LEVEL MODELLING FOR ONLINE COMBINATORIAL OPTIMISATION⁵

This chapter presents a new modelling framework capable of modelling and solving dynamic problems using online optimisation and at a high level of abstraction. This new framework provides a more generic approach to online optimisation than those existing in the literature, as it enables modellers to specify the dynamic aspects of a problem declaratively and automates the solving and re-solving process.

To achieve this, a modeller first creates a model of a static version of the problem being addressed, where no dynamism or uncertainty occurs. Then, the modeller adds annotations to specify how the data, variables, and constraints interact with time. The main goal is to have a general-purpose modelling and solving framework for dynamic combinatorial optimisation problems. The framework supports the three online optimisation paradigms we discussed in Section 2.4:

- the pure sequence paradigm, where one decision at a time is considered and needs to be committed to before the next decision can be taken;
- the stage-based sequence paradigm, where a set of decisions are considered simultaneously, and they all need to be committed before the next set of decisions is considered; and
- the time-stamp paradigm, where decisions and data become available at given (or unknown) times, and we must make decisions while new data is revealed.

The framework also supports real-time optimisation in these three paradigms, where decisions must be executed as soon as possible and solving co-occurs with execution. Finally, the framework supports rolling-horizon decompositions in the three paradigms, enabling one to solve a large static problem with online optimisation.

⁵The content of this chapter is partly based on the published work of Papers [I] and [II].

The rest of this chapter is structured as follows. First, related work and its shortcomings are explained in Section 3.1. Then, Section 3.2 describes the new formalism behind the framework, followed by Section 3.3 describing the actual modelling language support required to implement the framework in MiniZinc, which we call *Online MiniZinc*. Section 3.4 describes how this framework is implemented on top of the MiniZinc toolchain. Section 3.5 shows what the framework is capable of via an empirical evaluation, Section 3.6 outlines future work and open questions, and Section 3.7 concludes.

3.1 RELATED WORK

Dynamic combinatorial problems and online solution methods have been well studied (see, e.g., [39, 158, 24, 160]). Notwithstanding the strong similarities between all dynamic combinatorial problems, current approaches to solving them online are *problem-specific* (as we will see in this section). As the name implies, a problem-specific approach cannot easily be applied to other problems. Doing so either requires the approach to be significantly changed, which can be a time-consuming task, or may not even be possible if the approach relies too heavily on leveraging the structure of the problem. By contrast, we want to design a *problem-independent* solving approach, which is agnostic to the problem in question and works as-is.

Two of the key goals in combinatorial optimisation research to make its technology more accessible to non-experts and more applicable to real-world problems are to (i) increase high-level modelling capabilities and (ii) increase capabilities for addressing dynamic (and real-time) optimisation. Since the mid-1990s, these two goals, among others, have been repeatedly put forth in the literature, with some progress made (see, e.g., [53, 55, 109, 56]). We must progress towards (i) because formulating good models currently requires significant expertise. Tools that support high-level modelling are necessary for making quality modelling more accessible. Important steps in the right direction include the solver-independent modelling toolchains MiniZinc [107], Essence [58], and AMPL [51], as well as the reformulation tools Globalizer [93] and ModelSeeker [12], among others [108, 2]. We must also progress towards (ii) because understanding, specifying, and properly reacting to uncertainty in combinatorial optimisation are difficult tasks, unavoidable in real-world applications [160, 70, 24, 75]. Despite recent progress on each goal separately, little progress has been made on tools and techniques that simultaneously address (i) and (ii). The contributions in this chapter, and the thesis more broadly, help fill this gap.

3.1.1 Existing Solving Approaches

Two main approaches are currently found in the literature for addressing dynamic problems: (a) to use an off-the-shelf solver with an ad hoc online algorithm wrapped around it (e.g., [14, 122, 30]), and (b) to develop an algorithm specific to the problem in question (e.g., [120, 78]). Let us discuss these two approaches.

Approach (a) requires the implementation of an iterative re-solve algorithm wrapped around a particular solver where the new data arrives between re-solves. This approach requires the model to combine the old data and decisions with the new ones and be able to reason about what decisions can change (and how). To do this, essentially, two copies of all data and decisions must be maintained, one with the old or incumbent and one with the new. In addition, one must properly link the old and new information. Further, there must be constraints that disallow changes to committed variables, among other things. While this approach is less time-consuming and more generalisable than (b), due to all of the above required to make the model properly work in a rolling-horizon environment, it also obfuscates the model and the underlying problem. Another downside of this approach is that the wrapper algorithm itself is often problem-specific. This results in a (still) time-consuming process that is not high-level.

Examples of approach (a) include the work of Bertsimas et al. [19] for solving a dynamic vehicle routing which they represent in terms of a graph. New information is represented by adding nodes and edges to the graph. They develop their own iterative online wrapper algorithm around a MIP solver and include heuristics that enhance the solver's performance. Another example is the online scheduling approach of [97], in which a MIP solver with large neighbourhood search is run in each solving session.

Examples of approach (b) are more widespread and include the algorithms for online vehicle routing given in the survey by Jaillet et al. [82]. Chien et al. presented an iterative repair method for planning and scheduling NASA robots online [28], using repair algorithms of [121]. *Iterative repair* is a method that reacts to any dynamic event that causes conflicts (i.e., violated constraints) by untangling each of the conflicts one by one. While methods of approach (b) are more efficient as they are more specialised than counterparts of (a), they are more time-consuming to implement and less generalisable. For an overview of specialised algorithms that also use stochastic data, we refer the reader to [158]. See also [118] for an overview of specialised algorithms and concepts under the pure sequence paradigm.

In some situations, the same (or similar) decisions have to be repeatedly taken, over time, in real-time. For example, there could be a set of control variables that need to be adjusted depending on sensor information, and how the control variables can be changed does not depend on their previous assignments. This case is often addressed by developing fast single-point algorithms or models that can be used to re-solve with the latest data as desired, followed by replacing the old decisions with the new ones [72, 5]. Such algorithms could be implemented using either approach (a) or (b) and align with the stage-based sequence paradigm previously discussed.

Not all works fit nicely into one of the two above categories. This is the case of [16], which introduces a general-purpose online optimisation algorithm. However, this algorithm is limited to the pure sequence paradigm. Recall that our goal is to have a general-purpose modelling and solving language for online optimisation, capable of supporting all three paradigms, real-time optimisation, and rolling-horizon decompositions.

3.1.2 Existing Modelling Support

A thorough search through the relevant literature yielded no results for a high-level problem-independent framework (solver-independent or not) that enables the modelling of dynamic combinatorial optimisation problems to be solved online (including real-time) and rolling-horizon decompositions. The closest work is that of modelling language AIMMS, which supports the modelling and use of rolling-horizon decompositions of static problems with a time component [128]. This is done by first coding how all the parts of a model can be divided into multiple (possibly overlapping) windows and then coding an iteration script that iterates through all these windows, solves them, and makes any necessary changes between the iterations. Hence, it is really an example of approach (a). The rolling-horizon feature of AIMMS has been used in several works, such as [100, 17].

The GHOST framework is also somewhat relevant as it is a C++ library for modelling problems in strategic video games and solving them under very strict time constraints [126]. It comes bundled with a meta-heuristic solver. The framework is limited, however, in that it can only express problems in the stage-based sequence paradigm.

This thesis proposes an extension to the MiniZinc toolchain. Although other extensions of MiniZinc can be found in the literature, none of them are targeted at online optimisation. For example, there is a problem-independent large-neighbourhood search (LNS) extension to MiniZinc [40]. LNS is similar to a rolling-horizon decom-

position in that some decisions are fixed, some are free, and solving is performed iteratively. However, LNS starts from an initial solution and simply iteratively improves that solution, with no concept of time-dependent decisions.

3.1.3 Existing Formalisms

There are numerous ways of formalising combinatorial problems with uncertainty [160]. Online MiniZinc is based on the dynamic constrained optimisation problem (dCOP) formalism already discussed in Section 2.4 (see Definition 2.7). This section provides an overview of other formalisms and discusses why they are not as suitable as dCOP.

A *branching constraint satisfaction problem* (bCSP) [52] is a CSP where decisions are taken sequentially, and new decision variables are added into consideration, from a set of unseen variables, after each decision is taken. At each step, a bCSP has a pool of unassigned variables, and only one variable may be assigned a value. Once a variable is assigned a value, it can no longer be changed, and the problem evolves with the addition of new decision variables to the pool. Exactly which new decision variables are added is uncertain: at each step, a probability of being added to the pool is associated with each unseen variable, possibly depending on the decision taken in the current step. A utility is associated with each variable-value assignment. The goal is to maximise the global expected utility. This formalism is unsuitable for the problems addressed in this thesis for three reasons. First, it requires probabilities to be attached to the addition of new variables, which is not always possible in highly uncertain situations. Second, it does not allow existing constraints or assigned variables to be modified or updated, which is required when there is uncertainty in execution, and unexpected disruptions can happen. Third, and most importantly, it is restricted to the pure or stage-based sequence paradigms, while this thesis's goal is also to handle the time-stamp paradigm.

Another formalism is that of a *mixed constraint satisfaction problem* (mCSP) [47]. An mCSP has (controllable) decision variables and uncontrollable ones, which are unknown. The goal is often to achieve a degree of robustness with respect to the uncontrollable variables or find an assignment for the controllable variables that holds for all possible assignments of the uncontrollable. This formalism does not lend itself very easily to problems with a time component and is more suited for problems where we cannot react to the uncertainty. Thus, it is not as suitable as the more general dCOP formalism.

Yet another formalism is that of a *probabilistic constraint satisfaction problem* (pCSP) [46]. Similarly to the mCSPs formalism, pCSPs are meant for situations where

we cannot react to the uncertainties and hence not as suitable as the dCOP formalism. A pCSP extends a CSP such that each constraint is associated with a probability (that it must be satisfied), and the objective is to maximise the expected satisfaction. For example, if a solution violates only one constraint, which has a probability of 5%, then the expected satisfaction is 95%.

The *stochastic constraint satisfaction problem* (sCSP) formalism [163, 152] combines and extends the pCSP and mCSP ones, making it better suited for problems where one can react to dynamic events. Again, the variables are divided into controllable and uncontrollable, but now the uncontrollable variables are stochastic (i.e., each has a probability distribution associated with it). As in pCSP, the constraints have an associated probability, and the objective is to maximise the expected satisfaction. Further, there are several *decision stages* in the problem, and each variable (controllable and uncontrollable) is associated with a stage. In each stage, the controllable variables are assigned first, and then the uncontrollable variables are observed, moving then on to the next stage. We obtain a ‘*decide, observe, decide, observe, ...*’ sequence, where each ‘decide’ can regard multiple decision variables. Thus, this formalism is only suitable for the pure or stage-based sequence paradigms. In addition, the formalism does not allow existing constraints or assigned variables to be modified or updated.

The above formalisms are best suited for problems with uncertainty where knowledge about when, where, and how the uncertainties manifest themselves is present. Such knowledge is essential for proactive approaches such as robust and stochastic optimisation. In this thesis, however, reactive approaches are utilised. Thus, the dCOP formalism is best suited for this thesis.

Another formalism is *constraint networks on timelines* [161], which was introduced to unify planning and scheduling. It allows specifying and solving problems where the number of steps required to find a solution must be determined during solving. This can be useful for offline solving of problems with temporal components and uncertainty. Dunke and Nickel also presented a somewhat related formalism [42], which considers dynamic problems solved in non-real-time under the pure sequence paradigm with lookahead.

3.2 A NEW FORMALISM

This section presents a new formalism that enriches the dCOP formalism with the concepts commonly used in online optimisation.

3.2.1 Parameterised Dynamic COPs and Update-Models

Recall the concepts of the parameterised COP (Definition 2.6) and dynamic COP (Definition 2.7) formalisms. Let us combine these two, arriving at a *parameterised dynamic COP*. The basic idea is that the parameters change, but the underlying model remains the same instead of having the problem change arbitrarily.

Definition 3.1 (Parameterised Dynamic COP). A *parameterised dynamic COP* (or *parameterised dCOP*) is a possibly infinite list of COPs:

$$Q = \langle P_1, P_2, \dots \rangle = \langle M[\delta_1], M[\delta_2], \dots \rangle$$

where $M[\Delta]$ is a parameterised COP and $\delta_1, \delta_2, \dots \in \Delta$ denote the evolution of the objects and data points in the dynamic problem. $M[\Delta]$ instantiated with data δ_s , yields the problem P_s to solve in session s . The data instance δ_s may depend on the solution produced by solving $M[\delta_{s-1}]$.

Each problem P_s to solve (i.e., *session* s) is connected to the problems solved in other sessions through the underlying parameterised COP $M[\Delta]$. Recall that the parameters in Δ can be classified into two distinct types: *objects* and *data points*. The former typically specifies the size, and the latter specifies the details of a problem. The evolution of a parameterised dCOP can be visualised as follows:

$$M[\delta_{s-1}] = P_{s-1} \xrightarrow{\text{solve}} \mathcal{A}_{s-1} \xrightarrow{\text{execute}} (\text{milieu evolving}) \xrightarrow{\text{observe}} \delta_s \xrightarrow{\text{instantiate}} M[\delta_s] = P_s$$

First, the problem P_{s-1} is solved in session $s-1$, resulting in an assignment $\mathcal{A}_{s-1}$. Next, this assignment is executed in the problem milieu, which will evolve until the solving session s starts. Therefore, session s can only start after session $s-1$ finishes, either directly or after a specific waiting period (or some other triggering criteria) is reached. Before solving starts in session s , the milieu is observed and put into δ_s , which is then used to instantiate the problem $M[\delta_s]$. And the ‘solve-observe’ cycle continues. When $s = 1$, that is, for the first session of the parameterised dCOP, the ‘observe’ step is the first one taken. Note that, because there may be uncertainty in execution, the values of the parameters between consecutive COPs, i.e., δ_{s-1} and δ_s , can be modified by external sources.

As an example of a parameterised dCOP, consider a scheduling problem where new tasks arrive dynamically and must be scheduled in an ongoing manner. The parameter space Δ would encompass all possible scenarios for the scheduling prob-

lem, e.g., the kind of new tasks that can arrive, and the parameterised COP $M[\Delta]$ would define parameterised constraints and decision variables based on the given parameters. Suppose that $\Delta = (T; d)$, where T is the set of tasks to schedule, and d is a one-dimensional array indexed by T , where d_t is the processing duration of task $t \in T$. The decision variables $X[T] = \{x_t \mid t \in T\}$ of the parameterised COP $M[T; d]$ would be the starting time x_t of task t , with domains $D(x_t) = 0.. \infty$ for all $t \in T$. This formalism would be sufficient for applications where a fresh scheduling problem is solved in each session, and no connection is required between the current and previous sessions. More concretely, in our example, there is no notion of tasks already being scheduled but not yet fully processed that need to be considered when scheduling new tasks.

For problems that require connecting the past with the present (and eventually, the future), some constraints in $M[\Delta]$ will need to refer to both the incumbent (i.e., before solving) and produced (i.e., result of solving) states of the problem. Thus, the parameterised dCOP formalism must be enriched with the concept of incumbent assignments.

Definition 3.2 (Incumbent Assignments). A decision variable $x \in X[\delta]$ of a COP $M[\delta] = \langle X[\delta], D[\delta], C[\delta], f[\delta] \rangle$, with δ from parameter space Δ , may have an *incumbent assignment* $\text{inc}(x)$ (which is undefined if not), where $\text{inc}(x) \in \delta$. We will denote by $\text{old}(X[\delta])$ the subset of all $x \in X[\delta]$ for which $\text{inc}(x)$ is defined. i.e., the set of variables with incumbent assignments. For ease of notation, we say $\text{inc}(p) = p$ when p is a data point.

Consider now a definition of Δ in the scheduling example above: $\Delta = (T; d, \text{inc})$, where $\text{inc}: X[T; d] \rightarrow \mathbb{Z}$ is a function such that $\text{inc}(x_t)$ refers to the incumbent assignment of the start time of task t . We can now add to $M[\Delta]$ constraints to make the problem comply with incumbent assignments for all $x_t \in X[T; d, \text{inc}]$: if $\text{inc}(x_t)$ is undefined, i.e., x_t is a fresh decision, then no constraint is added; otherwise constraint ' $x_t = \text{inc}(x_t)$ ' is added to $C[T; d, \text{inc}]$.

Incumbent assignments and associated constraints allow us to tackle dynamic problems that need to be solved in the pure or stage-based sequence paradigms. However, this does not fully capture what is possible in online optimisation. Time can interact not only with the *evolution of* the problem but also with the *solving of* the problem and the *execution of* the *solutions*, as explained in the background chapter. The parameterised dCOP formalism must therefore be enriched with the concept of

time, necessary for the time-stamp paradigm and real-time optimisation, connecting the solving and the execution.

Definition 3.3 (Time Parameters). A parameterised dCOP, with underlying parameterised COP $M[\Delta]$, declares *time parameters* $\tau^{\text{beg}}, \tau^{\text{len}}, \tau^{\text{end}} \in \Delta$ if, for every session s , the following four conditions hold for $M[\delta_s]$. First, parameter $\tau_s^{\text{beg}} \in \delta_s$ provides the start time of session s , called its *beginning*. Second, parameter $\tau_s^{\text{len}} \in \delta_s$ provides the duration allotted to solving session s , called its *length*. Third, parameter $\tau_s^{\text{end}} \in \delta_s$ provides the deadline for reporting a solution to the problem of session s , called its *end*. Fourth, the relationship $\tau_s^{\text{beg}} + \tau_s^{\text{len}} = \tau_s^{\text{end}}$ holds.

These three time parameters synchronise the solving with the problem milieu. Note that the time unit used by the time parameters can be anything. In non-real-time situations, e.g., where the execution pauses during solving, the parameter τ_s^{len} , for all sessions s , is set to 0. In real-time situations, however, decisions regarding or associated with time, in session s , are constrained by τ_s^{end} in one way or another. For example, decisions that are now in the past cannot be revised by the solver. However, what ‘the past’ refers to ought to be clarified: session s ends at time τ_s^{end} , which is also the time in the problem milieu when a solution from session s can be executed; thus, any solution produced must use τ_s^{end} as an indicator for what is past, present, and future. Hence, times less than (i.e., before) τ_s^{end} are considered the past, and times equal to or greater than (i.e., after) τ_s^{end} are considered the present and the future, respectively. In the following subsection, we will see more examples of constraints based on time. But first, let us return to our simple scheduling example.

The concept of time parameters allows us to define Δ as $(T; d, \text{inc}, \tau^{\text{end}}, \tau^{\text{beg}}, \tau^{\text{len}})$, where τ_s^{beg} , τ_s^{end} , and τ_s^{len} respectively denote the beginning, end, and length of session s . We now have all the necessary tools to support the pure sequence, stage-based sequence, and time-stamp paradigms, as well as real-time and non-real-time optimisation. For example, we can add constraints to make the problem adhere to incumbent assignments, but only if the incumbent assignment refers to a point in time of the past and disallows points in the past when making new decisions.

What the preceding discussions arrived at—the enriched parameterised dCOP formalism—will be referred to as an *update-model* in this thesis.

Definition 3.4 (Update-Model). Given a parameterised dCOP $Q = \langle M[\delta_1], \dots \rangle$, its underlying parameterised COP $M[\Delta] = \langle X[\Delta], D[\Delta], C[\Delta], f[\Delta] \rangle$ is an *update-model*

if it declares time parameters and, for all potential variables $X[\Delta]$, an incumbent assignment. Thus, all $\delta \in \Delta$ have defined time parameters and an incumbent assignment for all variables in $\text{old}(X[\delta])$. To signify that a parameterised COP is an update-model, the notation $\vec{M}[\Delta]$ is used.

An update-model accounts for past states along with new information to fully capture what can be changed by the solver (and how) and what cannot be changed by it. In the next subsection, we will discuss the most common types of time-based constraints (which connects the past and the present) that can appear in update-models.

3.2.2 Locking, Committing, and Forbidding

Thus far, we have discussed the concept of decisions being committed, i.e., ones that the solver cannot modify, somewhat vaguely; it is often a vague concept in the literature, too. Here, formal definitions for this and related concepts are presented.

Consider, again, the scheduling problem from the previous subsection, where we had $\Delta = (T; d, \text{inc}, \tau^{\text{end}}, \tau^{\text{beg}}, \tau^{\text{len}})$. An update-model of this problem must fully specify how the incumbent solution affects the decision-making possibilities of the current solving session. A first reasonable restriction in this example is only to allow changes in the scheduling of tasks that have not yet started. This suggests the notion of a decision variable being *locked*, i.e., a variable whose assignment the solver can no longer change, after a specific point in time or after a specific condition holds. In our context, we will assume that if a decision variable is locked, then its assignment must be the same as its incumbent assignment. This will be enforced by variable locking constraints defined as follows:

Definition 3.5 (Locked Decision Variables). Consider an update-model $\vec{M}[\Delta]$ with components $\langle X[\Delta], D[\Delta], C[\Delta], f[\Delta] \rangle$ and a function $\mathcal{L}_X: X[\Delta] \times \Delta \rightarrow \mathbb{B}$ indicating whether a decision variable is *locked*. The *variable locking constraints* enforce that the assignment of every locked decision variable is not altered, which are, for a given instance δ , more formally specified as:

$$\mathcal{L}_X(x, \delta) \rightarrow \mathcal{A}(x) = \text{inc}(x) \quad \forall x \in \text{old}(X[\delta]).$$

Note that only variables with an incumbent assignment have a variable locking constraint. As a result, fresh variables cannot be locked. This concept is often called *commitment*, or that a variable is *committed*, in the literature. By contrast, this thesis

defines the term *commitment* in a more nuanced manner, as we will see in Definition 3.7.

A more complex form of time-based constraints typical of dynamic problems involves checking the *domain values* of a variable: while some of these values might need to be locked once selected, others might become unavailable as time passes. To illustrate this, consider the dynamic vehicle routing problem of Example 2.9, where, given a set of locations and a set of vehicles, all locations must be visited once by one of the vehicles. Suppose this is a package delivery problem, where each vehicle is loaded with packages at the start depot and then delivers these to its locations, arriving empty at the end depot. A central decision in this problem is on which vehicle to load a given package, i.e., each package is represented by a variable, the domain of which is the set of available vehicles. After a vehicle leaves the depot, no more packages can be loaded onto the vehicle, and thus the domain value representing that vehicle becomes locked for the variables associated with the packages in that vehicle. This suggests a notion of specific domain values of decision variables being *locked*. Formally, this concept can be defined as follows:

Definition 3.6 (Locked Domain Values). Consider an update-model $\vec{M}[\Delta]$ with components $\langle X[\Delta], D[\Delta], C[\Delta], f[\Delta] \rangle$ and a function $\mathcal{L}_D: X[\Delta] \times D[\Delta] \times \Delta \rightarrow \mathbb{B}$ indicating whether a domain value is *locked* for a particular variable. The *value locking constraints* enforce that no variable is assigned a locked domain value unless incumbent, in which case its assignment cannot be altered, which are, for a given instance δ , more formally specified as:

$$\mathcal{L}_D(x, d, \delta) \rightarrow \left(\mathcal{A}(x) = d \leftrightarrow \text{inc}(x) = d \right) \quad \forall x \in \text{old}(X[\delta]), \forall d \in D[\delta](x).$$

Notice how this definition subsumes the locking of decision variables presented before: if the locking of domain values is defined such that either all or none of the domain values are locked, then variable locking is effectively defined.

Value locking conflates two different kinds of constraints: committing and forbidding. These respectively indicate that a decision cannot be changed and that it is no longer available as time passes. Consider the package delivery problem again. When deciding which vehicle should visit each location, it is unrealistic to add (or remove) a location to (or from) a vehicle if this vehicle is about to leave the depot, since it takes time to load (or unload) packages. Assuming we need less time to pack for a new delivery than to find and remove a packed delivery, we want to *commit* to (at least)

	Committed	Forbidden	Locked
$\text{inc}(x) = d$	$\mathcal{A}(x) = d$	No constraint added	$\mathcal{A}(x) = d$
$\text{inc}(x) \neq d$	No constraint added	$\mathcal{A}(x) \neq d$	$\mathcal{A}(x) \neq d$

Figure 3.1: An overview of the constraint posted for value committing, forbidding, and locking with respect to a variable x and a domain value d

the current set of locations before *forbidding* the addition of new locations. These two concepts can be defined formally as follows:

Definition 3.7 (Committed and Forbidden Domain Values). Consider an update-model $\vec{M}[\Delta]$ with components $\langle X[\Delta], D[\Delta], C[\Delta], f[\Delta] \rangle$, a function $\mathcal{F}_D: X[\Delta] \times D[\Delta] \times \Delta \rightarrow \mathbb{B}$ indicating whether a domain value is *forbidden* for a particular variable, and a function $\mathcal{C}_D: X[\Delta] \times D[\Delta] \times \Delta \rightarrow \mathbb{B}$ indicating whether a domain value is *committed* for a particular variable. The *value forbidding constraints* enforce that no variable is assigned a forbidden domain value unless incumbent, which are, for a given instance δ , more formally specified as:

$$\mathcal{F}_D(x, d, \delta) \rightarrow (\mathcal{A}(x) = d \rightarrow \text{inc}(x) = d) \quad \forall x \in \text{old}(X[\delta]), \forall d \in D[\delta](x),$$

and the *value committing constraints* enforce that the assignment of every variable assigned a committed domain value is not altered, which are, for a given instance δ , more formally specified as:

$$\mathcal{C}_D(x, d, \delta) \rightarrow (\mathcal{A}(x) = d \leftarrow \text{inc}(x) = d) \quad \forall x \in \text{old}(X[\delta]), \forall d \in D[\delta](x).$$

Intuitively, a variable cannot be assigned a forbidden value unless it is its incumbent assignment, and a variable assignment cannot be revised if its incumbent assignment is a committed value. Note how a value may be forbidden for one variable but not for another. The constraint posted for each case is summarised in Figure 3.1. The rows indicate whether the incumbent assignment of x is d or not, where x is any variable and d is any domain value of x . The columns indicate whether d is committed, forbidden, or both (i.e., locked) for the variable x . Recall that variable committing (and forbidding) constraints only apply to variables with an incumbent assignment.

Note also that forbidding and committing are concepts only the solver must adhere to; the problem milieu and the execution need not adhere to these solving-only constraints. Because of uncertainty in execution, variables or domain values may be forbidden or committed in session s but not so in session $s+1$. Further, unsatisfiable

assignments are also forbidden, and all realised assignments (i.e., executed in the problem milieu) are also committed, but not the other way around.

With these two concepts—value committing and forbidding—most, if not all, time-related constraints in dynamic combinatorial problems are covered. Importantly, *when* a variable or a domain value is committed, forbidden, or locked is not defined inside but outside this formalism, allowing for maximum flexibility. These definitions will, however, most likely be based on τ^{beg} and τ^{end} .

Another strength of this formalism is that it allows a separation of parameterised dCOPs and update-models. A parameterised COP is a model for a static problem, while a (corresponding) update-model is a model for a dynamic variation of the static problem. Often, a static problem has more than one possible dynamic variation. There are nearly endless possibilities for which parts to make dynamic and uncertain, and which parts to leave static. The concepts of update-models are defined on top of the parameterised COP formalism. Thus, one can define a parameterised COP—a conventional, static problem—and systematically transform it into an update-model. To do this, one need only provide what is committed and forbidden (which allows the generation of committing and forbidding constraints), the time parameters, and incumbent assignments.

Definition 3.8 (Update-Models with Committing and Forbidding). Suppose that $M[\Delta]$ is a parameterised COP, which we want to transform into an update-model. Δ is extended to Δ' to accommodate time parameters, incumbent assignments, and declarations for $\mathcal{C}_D$ and $\mathcal{F}_D$ (as in Definition 3.7). The transformation rule can be defined as a function transform as follows:

$$\vec{M}[\Delta'] = \text{transform}(M[\Delta])$$

which results in an update-model with domain value committing and forbidding constraints, with a $\delta' \in \Delta'$ indicating what is and is not committed and forbidden.

The above formalism assumes that each variable must be assigned exactly one value from its domain. An obvious example that fits this description is integer variables, where each variable is assigned an integer. In some contexts and modelling languages, other constructs may be used, such as variables that can be assigned any number of values (e.g., set variables) or variables that can be assigned one or no value (e.g., optional variables). There is no loss of generality in the presented formalism, however, since it can easily be extended to set variables and optional variables. This

extension can be achieved by, for example, exchanging the equalities ('=') for leftward set inclusions (' $\supseteq$ ') in Definition 3.7.

3.3 MODELLING LANGUAGE DESIGN

This section outlines how an actual modelling language can be designed to implement the new formalism discussed in the previous section. To exemplify this design, we introduce *language extensions* to the MiniZinc language; but note that analogous designs also apply to other high-level constraint-based modelling languages. The language extensions we introduce enable the declarative modelling of dynamic optimisation problems in a high-level and solver-independent manner according to the formalism of Definition 3.8. These language extensions are centred around *annotations* that modellers can add to a conventional, static MiniZinc model to capture the dynamic aspects in a concise and declarative way. Behind each of these annotations is a constraint or model transformation that enables a high-level MiniZinc model to be transformed into a lower-level update-model. We say that an update-model exists at a lower level of abstraction because of its explicit incumbent assignments along with committing and forbidding constraints. We will refer to this language extension as *Online MiniZinc*.

Recall that we can access the incumbent assignment of a decision variable x as $\text{inc}(x)$. Let these be accessed in Online MiniZinc as x and `inc( $x$ )`. This is similar to the use of the function `sol()` in MiniSearch [124] and other extensions of MiniZinc [40] to refer to the previous solution to a problem. Moreover, an extra function is provided to allow checking whether a variable has an incumbent assignment `has_inc( $x$ )` (using `sol()` on a variable without incumbent assignment will lead to a runtime error) or not, and the τ^{beg} , τ^{len} , τ^{end} time parameters are also provided. Throughout this section, the session subscript for τ^{beg} , τ^{len} , and τ^{end} will be omitted as they will always be used in reference to the current session.

A modeller must specify when the locking of a variable or the locking, committing, and forbidding of a domain value occurs. The specifications are provided to the update-model as Boolean expressions that are determined before solving starts in session s based on the data points (including time parameters and incumbent assignments) provided in session s . Intuitively, the decisions taken in session s cannot influence what is and is not committed, forbidden, or locked in session s .

Note that, as mentioned before, domain value forbidding and committing subsumes value and variable locking. Nevertheless, from a modelling perspective, it will often

be helpful for modellers to have access to both general and specific constructs, as this allows modellers to strike a balance between brief, readable models and model specificity.

3.3.1 *Dynamic Objects Sets and Data Points*

First, one must declare *what the dynamisms are* in a model, i.e., how the object sets and data points may change over time. While all data points are assumed to be dynamic automatically, object sets are assumed to be static. Thus, Online MiniZinc requires a new annotation, called `online`, which declares that the annotated object set (i.e., `enum`) can be extended with new objects at any new session. For example, recall our MiniZinc model of the job-shop scheduling problem shown in Figure 2.4 (page 44). Declaring the jobs `JOBS` to be a dynamic object set would require the modeller to modify line 3 to be `enum JOBS ::online;`. Arrays indexed by an enum annotated as `online` will be automatically declared as a dynamically sized array. For example, since `JOBS` is dynamic, the arrays `req`, `dur`, and `start` will allow new indices to be added in its first index set, dynamically, from one session to the next.

The presented formalism also supports multiple dynamic object sets. For example, in the vehicle routing MiniZinc model of Figure 2.5 (page 47), we could declare both the locations and the vehicles in lines 2 and 3, respectively, to be dynamic object sets by declaring them instead as `enum VEHs ::online;` and `enum LOCs ::online;`. This would align with, for example, gig economies (e.g., Uber and the like) where new locations (i.e., customers to service) and new vehicles (i.e., workers) could appear, unannounced, at any time.

In general the `online` annotation is simply used as: `enum E ::online;`

3.3.2 *Decisions Regarding Time*

Many decision variables in a dynamic optimisation problem will be affected by time and other external factors. Examples include unexpected disruptions that one must react to and the inability to revise the assignment of locked variables. The most obvious case of time interacting with decisions is when the associated variables directly represent time, that is, variables whose domains are time itself. An example of such variables is the start time of a job in dynamic job-shop scheduling. Thus, the next new annotation in Online MiniZinc is `time`, which declares the annotated decision variable to represent time directly. This annotation, in effect, locks (i.e., both forbids

and commits) domain values less than τ^{end} . For example, in the job-shop example of Figure 2.4 (page 44), we would annotate the start time variables in line 10 as:

```
array [JOBS, TSKs] of var 0..: start ::time;
```

When a task has started executing before the current session, we can no longer revise the start time of that task. Formulating this as a constraint, we obtain the following:

```
constraint forall (j in JOBS, t in TSKs) (
  if has_inc(start[j,t]) /\ inc(start[j,t]) <  $\tau^{\text{end}}$ 
  then start[j,t] = inc(start[j,t])
  else start[j,t] >=  $\tau^{\text{end}}$  endif );
```

Put simply, for each task of each job, the if-condition checks whether the task is in the past and (if true) the then-branch commits tasks (scheduled in the past); otherwise, the else-branch forbids the past when revising the task's start time.

In general the `time` annotation is used as:

```
array [E] of var int: x ::time;
```

and is automatically transformed into a constraint for each variable, using the following predicate:⁶

```
predicate time(array [E] of var int: x) =
  forall (i in E) (
    if has_inc(x[i]) /\ inc(x[i]) <  $\tau^{\text{end}}$ 
    then x[i] = inc(x[i])
    else x[i] >=  $\tau^{\text{end}}$  endif );
```

Intuitively, the above constraint asserts that if there is an incumbent assignment, whose value is a time that is now in the past, then we commit the decision, else we forbid assigning a value in the past. Note that this annotation only applies to `var int` types.

Essentially, the `time` annotation locks the variable in question depending on its own incumbent value compared to the current session deadline. This annotation can be generalised into one where an offset is provided. Consider the job-shop example of Figure 2.4 (page 44) again. Having the solver reschedule a task that is just about to be executed is often undesirable as it may cause instability and make the actors in the problem less able to prepare. To address this, we introduce an annotation, which

⁶For brevity, this section will primarily define predicates for one-dimensional arrays. Such predicates can be used on multidimensional arrays straightforwardly by collapsing any number of m dimensions $E_1, \dots, E_m$ into E . Furthermore, any non-array variable x can be easily converted into a one-dimensional array with one element $[x]$.

we call `lock_self` for convenience and is similar to the `time` annotation but takes an offset. For example, we could annotate the `start` array as:

```
array [JOBS, TASKS] of var 0...: start ::lock_self(+5);
```

meaning that the start time of a task is locked if it is 5 time units (e.g., minutes) or less between the end of the session τ^{end} (i.e., the latest time at which the solution is communicated to the problem milieu) and the planned start time (i.e., the incumbent assignment). That is, we lock 5 time units earlier than normally. This would provide more stability, as tasks that are 5 minutes or less from being executed cannot be rescheduled. By contrast, setting the offset to 0 would mean locking `start` just before execution, having the same effect as the `time` annotation.

In general, the `lock_self` annotation is used as:

```
array [E] of var int: x ::lock_self(ofs);
```

which is automatically transformed into a constraint for each variable, using the following predicate:

```
predicate lock_self(array [E] of var int: x,
                    array [E] of int: ofs) =
  forall (i in E) (
    if has_inc(x[i]) /\ inc(x[i]) <  $\tau^{\text{end}}$  + ofs[i]
    then x[i] = inc(x[i])
    else x[i] >=  $\tau^{\text{end}}$  + ofs[i] endif );
```

where the variable `ofs[i]` denotes the locking offset, i.e., how far ahead in time the variable `x[i]` will be locked. Negative offsets result in locking after the fact (we look back in time), and a positive offset results in locking ahead of time (we look forward in time). For brevity, we will allow modellers to provide a single number as an offset, instead of an array, as a shorthand for providing an array of identical elements (i.e., the provided number).

Analogous to what we discussed in the definition of the new formalism (Section 3.2), the `lock_self` annotation can be split into two complementary annotations, `commit_self` and `forbid_self`, which can be added to the model instead. This is useful if one wants to specify just one of them or to have separately defined commit and forbidding offsets. Again, let us illustrate this via the job-shop example of Figure 2.4 (page 44). Above, we considered a locking offset of 5 minutes; however, postponing a task has often less drastic effects on stability than moving it earlier. In addition, we could have a ‘minimal postponement’ to make room for any additional preparations required. To address this, we could have separate offsets for committing and forbidding. For example, we could annotate the `start` array as:

```
array [JOBS,TSKs] of var 0..: start
  ::commit_self(+5) ::forbid_self(+10);
```

meaning that a task cannot be postponed if there are 5 minutes or less between the end of the session τ^{end} and the planned start time; if a task is postponed, then it must be to a time 10 minutes or more after the end of the session τ^{end} ; and, similarly, if a task is moved earlier, then it must be to a time 10 minutes or more after τ^{end} .

In general, the `commit_self` annotation is automatically transformed into a constraint for each variable, using the following predicate:

```
predicate commit_self(array [E] of var int: x,
                      array [E] of int: ofs) =
  forall (i in E) (
    if has_inc(x[i]) /\ inc(x[i]) <  $\tau^{\text{end}}$  + ofs[i]
    then x[i] = inc(x[i]) endif );
```

Note the lack of an else-case, meaning that the if-statement is vacuously true whenever the if-condition is false. The `forbid_self` annotation is, similarly, automatically transformed into a constraint for each variable, using the following predicate:

```
predicate forbid_self(array [E] of var int: x,
                      array [E] of int: ofs) =
  forall (i in E) (
    if has_inc(x[i]) /\ inc(x[i]) <  $\tau^{\text{end}}$  + ofs[i]
    then x[i] = inc(x[i]) /\ x[i] >=  $\tau^{\text{end}}$  + ofs[i]
    else x[i] >=  $\tau^{\text{end}}$  + ofs[i] endif );
```

This is a perfect opportunity to highlight how forbidding works: both branches in the if-statement forbid the past (including the offsets), except that the then-branch allows us to keep a forbidden value if it is an incumbent assignment. That is, only *new* assignments in what is forbidden of the timeline are disallowed.

3.3.3 Variables Affected by Time

While the domain of some variables is not time itself, it may nevertheless represent values that the solver cannot change after a certain point in time. These variables do not represent time directly but are still affected by it. For example, in a delivery truck routing problem, the locations that a truck visits must be determined before this vehicle starts its tour; otherwise, we cannot ensure that the packages to be delivered are loaded on the appropriate vehicle. Such decisions are locked by virtue of another variable that does represent time. In the delivery truck example, the selection of locations a truck will visit could be locked via the departure time variable.

To achieve this connection between variables, Online MiniZinc allows modellers to annotate variables using the `lock_var_at` annotation.

Let us illustrate this annotation via an example. In the open-shop scheduling problem, which is similar to that of Figure 3.2 (page 91), the allocation of tasks to machines is not fixed; it is instead a decision. This means that the array of parameters `req` is now declared as an array of variables. Clearly, once a task starts to be processed, the machine that processes it cannot change. Hence, the model for this problem has the following extra annotation:

```
array[JObS,TSKs] of var MACs: req ::lock_var_at(start);
```

Note that the dimensions of arrays `start` and `req` are the same. Formulating this as a constraint, we obtain the following:

```
constraint forall (j in JOBS, t in TSKs) (
  if has_inc(req[j,t]) /\ has_inc(start[j,t])
  /\ inc(start[j,t]) <  $\tau^{\text{end}}$ 
  then req[j,t] = inc(req[j,t]) endif );
```

Put simply, for each task of each job, the if-condition checks whether the task is in the past and (if true) the then-branch ensures the machine chosen for this task cannot be changed.

In general, this annotation is used as:

```
array [E] of var D: x ::lock_var_at(at, ofs);
```

which is automatically transformed into a constraint for each variable, using the following predicate:

```
predicate lock_var_at(array [E] of var D: x,
                      array [E] of var int: at,
                      array [E] of int: ofs) =
  forall (i in E) (
    if has_inc(x[i]) /\ has_inc(at[i])
    /\ inc(at[i]) <  $\tau^{\text{end}}$  + ofs[i]
    then x[i] = inc(x[i]) endif );
```

where `at[i]` is a variable whose domain is time and whose value is the time point after which the decision for `x[i]` cannot be changed. Intuitively, the above predicate ensures that the value of every variable `x[i]` that is locked, cannot be revised (i.e., must be the same as `inc(x[i])`). Note the omission of the locking offset (`ofs`) parameter in our job-shop example above. We will assume that all the annotations and predicates where the offset is not provided are defined to have an offset of 0 for all variables.

Worth highlighting is the possibility of using negative offsets. This could be useful for a variety of reasons, such as locking a decision based on events that have happened earlier. Consider an extension of the job-shop scheduling problem, where a reviewer must be assigned to each job to verify the quality of the resulting product. We can model this by introducing a new enum (`enum PERs`) to represent the set of persons that can review as well as an array of variables (`array [JOBS] of var PERs: review`), mapping jobs to reviewers. Suppose we want the review to take place at least 24 hours after the completion of the job in question, then we could annotate the variable declaration, looking back in time, as follows:

```
array [JOBS] of var PERs: review
  ::lock_val_at([ start[j,max(TSKs)] + dur[j,max(TSKs)]
                | j in JOBS ], -24);
```

assuming that one time unit is equal to one hour. This annotation will be transformed into the constraint:⁷

```
constraint forall (j in JOBS) (
  if has_inc(review[j,t]) /\ has_inc(start[j,max(TSKs)])
  /\ has_inc(dur[j,max(TSKs)])
  /\ inc(start[j,t]) + dur[j,max(TSKs)] <  $\tau^{\text{end}}$  - 24
  then review[j] = inc(review[j]) endif );
```

Put simply, for each job, the if-condition checks whether the job finished 24 hours (or more) ago and (if true) the then-branch enforces that the allocated reviewer remains allocated.

3.3.4 Domain Values Affected by Time

As discussed in Section 3.2, time-based constraints concerning the domain values of a variable are typical of dynamic problems. To achieve this in Online MiniZinc, modellers can annotate a variable with the `lock_val_at` annotation. In general, it is used as:

```
array [E] of var D: x ::lock_val_at(at, ofs);
```

which is automatically transformed into a constraint for each variable, using the following predicate:

```
predicate lock_val_at(array [E] of var D: x,
                     array [D] of var int: at,
                     array [D] of int: ofs) =
```

⁷For brevity, the constraint has been slightly simplified by unrolling the array comprehension.

```

forall (d in D) (
  if has_inc(at[d]) /\ inc(at[d]) <  $\tau^{\text{end}}$  + ofs[d]
  then forall (i in E) (
    (has_inc(x[i]) /\ inc(x[i]) = d) <-> x[i] = d )
  endif );

```

Intuitively, for every locked domain value d , if it is already picked by a variable x , then we must keep this assignment; otherwise (i.e., if fresh or if not picked), we must not assign the value d to x .

To illustrate this with an example, recall the dynamic vehicle routing problem of Figure 3.3 (page 92), and suppose that the problem is applied to package delivery, i.e., packages are loaded onto vehicles at the depot and delivered to their respective locations. We can annotate three of the declarations in lines 7–9 as follows:

```

array [NODs] of var 0..: arri ::time;
array [NODs] of var NODs: next ::lock_val_at(arri);
array [NODs] of var VEHs: serv ::lock_val_at(ready);

```

The first annotation indicates that the arrival time at each node is a time variable. The second one indicates that deciding where to go next from node n is locked at the time point when the vehicle arrives at n . The third one indicates, as the packages must be loaded onto the vehicles at the depot, that deciding which customers to visit is locked when leaving the depot.

Similar to the case of variables, we can commit and forbid values via the use of the annotations `commit_val_at` and `forbid_val_at`. They are used analogously to the `lock_val_at` annotation and are automatically transformed into a constraint for each variable, using the following predicates, respectively:

```

predicate commit_val_at(array [E] of var D: x,
                        array [D] of var int: at,
                        array [D] of int: ofs) =

forall (d in D) (
  if has_inc(at[i]) /\ inc(at[i]) <  $\tau^{\text{end}}$  + ofs[i]
  then forall (i in E) (
    (has_inc(x[i]) /\ inc(x[i]) = d) -> x[i] = d )
  endif );

predicate forbid_val_at(array [E] of var D: x,
                       array [D] of var int: at,
                       array [D] of int: ofs) =

forall (d in D) (
  if has_inc(at[i]) /\ inc(at[i]) <  $\tau^{\text{end}}$  + ofs[i]
  then forall (i in E) (

```

```
(has_inc(x[i]) /\ inc(x[i]) = d) <- x[i] = d )
endif );
```

Note that the only difference between the two predicates lies in the direction of the implication.

Consider the aforementioned package delivery variation of the dynamic vehicle routing problem (Figure 3.3, page 92), where packages are loaded onto vehicles at the depot and delivered to their respective locations. When deciding which vehicle should visit each customer, it is unrealistic to add (or remove) a customer to (or from) vehicle v if v is just about to leave the depot since it takes time to load (or unload) a package. Assume we need 5 minutes to pack a new delivery and 15 minutes to find and remove a packed delivery. This can be expressed by substituting the annotation on lines 13 and 14 of `serv` (which is a variable array that maps each node to a vehicle that will serve the node) by the following:

```
array[NODs] of var VEHs: serv ::commit_val_at(ready, +15)
                        ::forbid_val_at(ready, +5);
```

This reflects the desired behaviour: the annotations state that the decision of assigning a customer to a vehicle $v = \text{serv}[n]$ cannot be changed by the solver if, in the current solution, v is about to leave in the next 15 minutes, and a customer cannot be (newly) assigned to a vehicle that is about to leave in the next 5 minutes.

One thing to note is that locking, forbidding, and committing variable domains require the domains to be finite and relatively small for good performance. This is because a constraint is added per domain value, which is only possible for finite domains, and may negatively impact the solving performance for large domains. Value committing and forbidding for infinite domains is most commonly applied to variables that model time, in which case the `time` (and derived) annotations are to be used instead.

3.3.5 Boolean Arrays Affected by Time

A common practice in constraint-based modelling is experimenting with different viewpoints, i.e., ways of encoding the same meaning. This is because the way the constraints and variables are formulated can affect solver performance, despite representing the same underlying problem. This subsection shows how Online MiniZinc supports viewpoints with Boolean arrays.

Consider our package delivery variation of the dynamic vehicle routing problem again (Figure 3.3, page 92), where packages are loaded onto vehicles at the depot

and delivered to their respective locations. The `serv` array in the example, which maps each node to the vehicle that serves it, could have been modelled using a two-dimensional Boolean viewpoint, i.e., mapping each vehicle-node pair to true if the node is served by the vehicle, and false otherwise. We could declare the following two-dimensional Boolean array:

```
array [VEHs, NODs] of var bool: serv_bool;
```

where `serv_bool[v, n]` is true if and only if vehicle `v` will serve node `n`. Of course, we want to be able to lock, commit, and forbid `serv_bool` like we have previously done with other viewpoints. To do this, we introduce another annotation `lock_aofm_at` where `a` and `m` are replaced with numbers ($1 \leq a \leq m$) as follows:

```
array [E1, E2] of var bool: x :: lock_1of2_at(at, ofs);
array [E1, E2] of var bool: x :: lock_2of2_at(at, ofs);
array [E1, E2, E3] of var bool: x :: lock_1of3_at(at, ofs);
array [E1, E2, E3] of var bool: x :: lock_2of3_at(at, ofs);
array [E1, E2, E3] of var bool: x :: lock_3of3_at(at, ofs);
...
```

and so on, where `m` is the dimensionality of the annotated array and `a` is the index set which we lock in respect to, using `at`. For example, if we annotate the array `serv_bool` as follows:

```
array [VEHs, NODs] of var bool: serv_bool :: lock_1of2_at(ready);
```

then it means that when the time `ready[i]` is met, the *i*th object of `VEHs` will be locked, that is, every variable in the *i*th row will be locked to its truth value. As a result, we cannot assign the nodes visited by the *i*th vehicle to another vehicle, nor can we assign any new nodes to the *i*th vehicle.

In general, an annotation `lock_aofm_at` on a (*m*-dimensional) array `x` means that every element that is indexed by element `i` of its *a*th index set is locked if and only if `at[i]` is in the past (including offset). More formally, the annotation is automatically transformed into a constraint for each variable, using the following predicate:

```
predicate lock_aofm_at(array [E1, ..., Ea, ..., Em] of var bool: x,
                        array [Ea] of var int: at,
                        array [Ea] of int: ofs) =
forall (i in Ea) (
    if has_inc(at[i]) /\ inc(at[i]) <  $\tau^{\text{end}}$  + ofs[i]
    then forall (j1 in E1, ..., ja-1 in Ea-1, ja+1 in Ea+1, ..., jm in Em) (
        ( has_inc(x[j1, ..., ja-1, i, ja+1, ..., jm])
          /\ inc(x[j1, ..., ja-1, i, ja+1, ..., jm]) )
        <-> x[j1, ..., ja-1, i, ja+1, ..., jm] ) endif );
```

As before, we split this annotation into its two counterparts for committing and forbidding: `commit_aofm_at` and `forbid_aofm_at`, respectively. The only difference is that using the following predicate for the constraint in the committing counterpart uses an implication ‘ $\rightarrow$ ’ instead of an equivalence ‘ $\leftrightarrow$ ’, and the forbidding counterpart uses a leftward implication ‘ $\leftarrow$ ’. Consider, again, the variation of the package delivery problem where we need 5 minutes to pack a new delivery and 15 minutes to find and remove a packed delivery. If the Boolean array viewpoint is used, then we can annotate the declaration as follows:

```
array[VEHs,NODs] of var bool: serv_bool ::commit_lof2_at(ready, +15)
                                ::forbid_lof2_at(ready, +5);
```

3.3.6 Other Constructs and Final Remarks

So far, we have discussed how to declare decisions and how they relate to time directly. However, time has been defined somewhat arbitrarily. The benefits of the definitions so far are that they apply not only to time but to any increasing dynamic parameter. It could be length, height, or an arbitrary total order. This is useful for addressing problems in environments that do not directly evolve with time but some other parameter. Also, it is useful for defining rolling-horizon decompositions for problems that do not relate to time (this will be explored in more detail in Section 3.5). To demonstrate this case, consider a carpet cutting problem in which a long stretch of carpet, a carpet roll, has to be cut into pieces that will be used to install a fitted carpet in a building. Suppose this is a dynamic problem, where the requirements are uncertain, and cut pieces of carpet must be produced continually. While time does play a role in the evolution of such a problem, the important markers on which to base forbidding and committing decisions need not be time per se. Imagine that the carpet roll is actively being cut from one end while decisions on *how to* cut are being made regarding the other end. The parameter τ^{beg} would, here, denote the point in the carpet at which cutting has not yet been executed. Cutting has commenced for all points in the carpet before τ^{beg} , and for all points after τ^{beg} , decisions are still being made. Although τ^{beg} will move from one end of the carpet roll to the other based on time, this movement need not be linear in time or even predictable.

For maximum flexibility, Online MiniZinc allows for an outside process, more complex in-model constraints, or both to determine whether a decision is locked, committed, or forbidden. This is referred to as *index-based* committing, forbidding, and locking. Variations of all the lock, commit, and forbid annotations shown so far are introduced, where the `_at` suffix is replaced with a `_for` suffix. With these annota-

tions, a modeller simply needs to provide an array of Boolean data points (instead of times) that determines whether a given decision is locked, committed, or forbidden. In addition, modellers can also provide their own rules regarding how the Boolean data points are given values. For example, consider the following `lock_var_for` declaration:

```
array [E] of var D: x :: lock_var_for(y);
```

which is automatically transformed into a constraint for each variable, using the following predicate:

```
predicate lock_var_for(array [E] of var D: x,
                      array [E] of var bool: y)
forall (i in E) (
  if has_inc(y[i]) /\ inc(y[i]) /\ has_inc(x[i])
  then x[i] = inc(x[i]) endif );
```

Note that, in contrast to the definition of the predicate `lock_var_at`, there is no need for array `ofs` and no need for testing whether anything is before τ^{end} . This is because the array `y` holds the information of which variables in `x` are locked; thus, modellers have to ensure somewhere in the model (or externally, if provided as data points) `y`, `inc(y)`, and `has_inc(y)` agree with the effect they want to model. Corresponding variations of all the other annotations are analogously defined.

All the annotations introduced in this chapter capture the most common constraints that arise when solving a dynamic problem using online methods. For the rare problem-specific cases, where these annotations are not sufficient, Online Mini-Zinc gives modellers access to the `inc()` and `has_inc` functions, and the τ^{beg} , τ^{len} , and τ^{end} parameters introduced above. Consider, for example, a variation of the package delivery problem, where we inform customers of the expected arrival times within the next 24 hours. After informing customers about their arrival times, we want to ensure that later solutions do not delay these arrival times by more than an hour per solving session. A modeller can express this in the model as follows:

```
constraint forall (l in LOCs where has_inc(arri[l])) (
  if inc(arri[l]) <  $\tau^{\text{end}}$  + 24*60
  then inc(arri[l]) <= arri[l]
  /\ arri[l] <= inc(arri[l]) + 60 endif );
```

where one time unit is one minute. For example, having one solving session per 8 hours (three per 24 hours) would result in a maximum total delay of 4 hours. This is because, in the first session, the arrival is a maximum of 24 hours in the future; thus, we can postpone it three times (3 hours) in that 24 hours, pushing it to be a maximum

of 3 hours in the future at the start of a fourth session, where a final postponement may occur, leading to a total delay of 4 hours. Furthermore, modellers can check whether a constraint holds before the solving starts, i.e., incumbent constraints. For example, the incumbent value of the constraint $x < y$ can be checked using $\text{inc}(x) < \text{inc}(y)$, where x and y are decision variables.

Worth noting, since only modellers know how τ^{end} , τ^{len} , and τ^{beg} relate to time in the problem milieu, is that they are the ones who must define the values of these parameters and how they are provided to each solving session.

Complete, annotated Online MiniZinc models of the dynamic job-shop scheduling problem and the dynamic vehicle routing problem, extending the MiniZinc models in Figure 2.4 (page 44) and Figure 2.5 (page 47), are respectively provided in Figure 3.2 and Figure 3.3. The only two differences in the dynamic job-shop model are that the job `enum` is annotated with `online` and the start time (decision) of each task is annotated with `time`. The dynamic vehicle routing model has more annotations. Both the vehicle `enum` and the location `enum` are annotated with `online`; the arrival time decision for each node is annotated with `time`; the decisions of what vehicle will service what node is annotated with `commit_val_at(ready, 15)` and `forbid_val_at(ready, 15)`; and lastly, the successor variables are annotated with `lock_var_at(arri)`. Notice the tiny changes required in modelling. This comes from the fact that Online MiniZinc transforms these annotations into their constraint counterparts and adds declarations for incumbent assignments as well as time parameters, creating an update-model. This update-model can then be used inside an iterative solving algorithm that simply provides the current problem milieu to MiniZinc, but more on that in the next chapter.

Before we close this section, recall that rolling-horizon decompositions are a common approach for solving large static optimisation problems, where the problem is decomposed into a series of subproblems restricted to a small planning horizon that slides forward (e.g., in time) during the process. All decisions before the planning horizon are locked, and all decisions (and their constraints) after it are ignored. Online MiniZinc allows a modeller to solve a static problem via a rolling-horizon decomposition approach. This is achieved simply by appropriately splitting the data into parts, and supplying them one by one to Online MiniZinc. Similarly to dynamic problems, we can lock, commit, and forbid decisions based on time (or any other parameter) or based on index values. Because of the choice of how to split data, there is an extra layer of decisions to make. In other words, the evolution of the problem milieu is controllable. Thus, experimentation is often required to find a good decomposition strategy, which is a strong case for Online MiniZinc.

```

1 % PARAMETERS
2 enum MACs;           % set of machines
3 enum JOBS ::online;  % set of jobs (online)
4 enum TSKs = T(MACs); % set of tasks (one per machine)
5
6 array [JOBS,TSKs] of MACs: req; % machine requirements (assumed
   static)
7 array [JOBS,TSKs] of int: dur;  % processing durations
8
9 % VARIABLES
10 array [JOBS,TSKs] of var 0...: start ::time; % start times
11
12 % CONSTRAINTS
13 %% task ordering
14 constraint forall (j in JOBS, t in ..<max(TSKs)) (
15   start[j,t] + dur[j,t] <= start[j,enum_next(t)] );
16
17 %% machine usage
18 include "disjunctive.mzn";
19 constraint forall (m in MACs) (
20   disjunctive([start[j,t] | j in JOBS, t in TSKs where req[j,t]=m]
21             , [dur[j,t] | j in JOBS, t in TSKs where req[j,t]=m]
22             ) );
23
24 % OBJECTIVE (makespan)
25 solve minimize max (j in JOBS) (
26   start[j,max(TSKs)] + dur[j,max(TSKs)] );

```

Figure 3.2: An annotated Online MiniZinc model for the dynamic job-shop scheduling problem

```
1 % PARAMETERS
2 enum VEHS ::online; % set of vehicles (online)
3 enum LOCs ::online; % set of locations to visit (online)
4
5 enum NODs = S(VEHS) ++ E(VEHS) ++ L(LOCs); % nodes
6
7 array [VEHS] if int: ready; % time veh ready to use
8 array [NODs] of int: wait; % wait time per node
9 array [NODs,NODs] of int: dist; % distance between nodes
10
11 % VARIABLES
12 array [NODs] of var int: arri ::time; % arrival at node
13 array [NODs] of var VEHS: serv % node served by
14 ::commit_val_at(ready, 15) ::forbid_val_at(ready, 5);
15 array [NODs] of var NODs: next % successor of node
16 ::lock_var_at(arri);
17
18 % CONSTRAINTS
19 %% link vehicle with its start and end
20 constraint forall (v in VEHS) ( serv[S(v)] = serv[E(V)] );
21
22 %% same vehicle serves whole tour
23 constraint forall (n in S(VEHS) ++ L(LOCs)) (
24   serv[n] = serv[next[n]] );
25
26 %% cannot leave before ready (assumes wait is 0)
27 constraint forall (v in VEHS) ( arri[S(v)] = ready[v] );
28
29 %% travel time and wait time constraints
30 constraint forall (n in S(VEHS) ++ L(LOCs)) (
31   arri[n] + wait[n] + dist[n,next[n]] = arri[next[n]] );
32
33 %% hamiltonian grand circuit constraint
34 include "circuit.mzn";
35 constraint circuit(succ);
36
37 %% symmetry-breaking constraints for grand circuit
38 constraint next[E(max(VEHS))] = S(min(VEHS));
39 constraint forall (v in ..<S(max(VEHS))) (
40   next[E(v)] = S(enum_next(v)) );
41
42 % OBJECTIVE (minimize total travel cost)
43 solve minimize sum (v in VEHS) ( arri[E(v)] - ready[v] );
```

Figure 3.3: An annotated Online MiniZinc model for the dynamic vehicle routing problem

3.4 IMPLEMENTATION

This section presents some of the more interesting implementation details of Online MiniZinc. In particular, it discusses how an annotated high-level model is transformed into a lower-level update-model and how to use Online MiniZinc in general.

All paradigms and approaches explained in Chapter 2 are supported by the Online MiniZinc framework. The framework emulates the pure sequential, stage-based sequential, and time-stamp online optimisation paradigm mentioned in the introduction of this chapter. It also supports all parts of the rolling-horizon viewpoint, except a high-level way to represent lookahead (defined in Section 2.4). Lookahead still has to be managed manually via low-level modifications, including modelling weakened constraints for the lookahead and how the lookahead data influences the decisions in the planning horizon. It remains future work to automate this in the framework.

3.4.1 Using The Framework

A modeller must first understand how to connect Online MiniZinc with the problem milieu. The pseudocode in Algorithm 1 depicts the general program structure required to use Online MiniZinc. We need to provide four arguments to the `OnlineSolve` procedure: an annotated MiniZinc model $M[\Delta]$; a function `get` that returns the current state of the problem milieu; a function `put` that transmits the decisions to the problem milieu (to be executed); and a function `wait` that waits until it is time to solve (e.g., waits for a trigger from the problem milieu) and eventually returns true if it is time to solve, or returns false if there are no more solving sessions. Modellers must define these three functions, connecting the model $M[\Delta]$ to the problem milieu. In doing so, they define how time passes and how their problem evolves. For problem milieux that are not in real-time, the function `put` will have to make sure that the horizon rolls forward.

Let us illustrate the use of this framework by means of two examples. In our first example, we want to create a rolling-horizon decomposition. We could define `wait()` to instantaneously return true until we have solved the entire problem, since there is no need to wait for a re-solving trigger in non-real-time situations. We could also define `put( $\mathcal{A}$ )` to apply solution $\mathcal{A}$ to the problem and roll the planning horizon forward (the desired step size), the result of which would be returned by `get()`.

```

1: procedure OnlineSolve( $M[\Delta]$ , get, put, wait)
2:    $\vec{M}[\Delta'] \leftarrow \text{transform}(M[\Delta])$ 
3:   while wait() do
4:      $\delta \leftarrow \text{get}()$ 
5:      $\mathcal{A} \leftarrow \text{solve}(\vec{M}[\delta])$ 
6:     put( $\mathcal{A}$ )

```

Algorithm 1: Online MiniZinc Framework

In the second example, we want to solve a dynamic task scheduling problem with Online MiniZinc. The function `wait()` will instantaneously return true if there are unscheduled, fresh tasks; it will wait if there are no such tasks (but there soon will be); and it will return false if all tasks have been considered. The function `put( $\mathcal{A}$ )` will transmit the generated $\mathcal{A}$ schedule to the problem milieu to be executed. *How* execution will transpire is defined outside the Online MiniZinc framework. Nevertheless, the framework provides an interface to the problem milieu via `put`. Finally, the function `get()` reads the current state of the problem milieu (e.g., what the current schedule being executed is, and what the plans of the unscheduled tasks are) into a format that MiniZinc understands. Again, how the problem milieu is read is defined outside the Online MiniZinc framework, with `get` forming the rest of the interface to the problem milieu.

3.4.2 Architecture and Model Transformations

Once a modeller has created an annotated model and wants to run it, the system will automatically transform it into an update-model. This is achieved by automatically (i) adding declarations for τ^{beg} , τ^{len} , and τ^{end} , (ii) adding declarations for the incumbent assignments of all variables, and (iii) transforming each annotation into its constraint counterpart. Note that (i) and (ii) regard parameter *declarations*, not *definitions*; providing values for definitions comes later, namely, at the start of each session.

The architecture of Online MiniZinc is visualised in Figure 3.4. One can see a split into three categories: the *MiniZinc front-end*, the *problem milieu*, and the *back-and solver-end*. In the front-end live processes that parse the user input, report any potential errors in the input, and convert it into an internal representation. In addition to what happens in conventional MiniZinc, there is a transformation step that produces an update-model (highlighted in boldface). The back- and solver-end consists of the translations (i.e., compilation) to code a solver can understand and

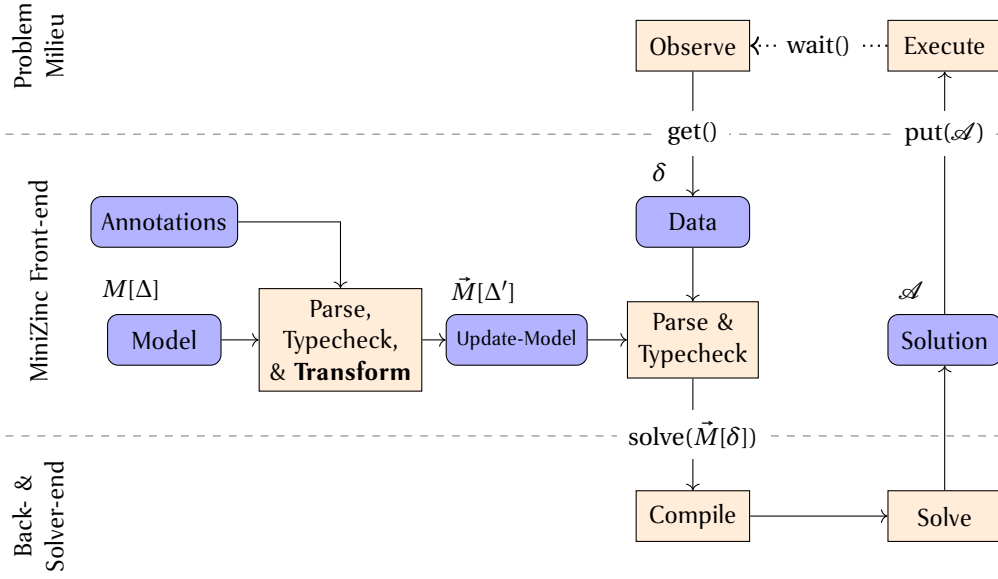


Figure 3.4: The architecture of Online MiniZinc

use, followed by the actual solving of the problem. The problem milieu part of the architecture involves three functions: one that can observe the current state of the problem milieu `get()`, one that submits a solution $\mathcal{A}$ for execution `put( $\mathcal{A}$ )`, and one that waits until it is time to observe again `wait()`.

Let us step through the figure in more detail, starting at the ‘Model’ node. The model $M[\Delta]$ and the annotations are first parsed and typechecked by the MiniZinc toolchain. Then, they are together transformed into an update-model $\tilde{M}[\Delta]$. The next steps involve combining the transformed model with the newest data δ (representing the current state of the problem milieu), creating an instance $\tilde{M}[\delta]$, parsing and typechecking this instance, and sending it to the MiniZinc compiler. The compiler translates the instance into a formulation understandable by the target solver, which in turn solves the instance, producing a solution $\mathcal{A}$. Then, this solution is sent to the problem milieu to be executed via `put( $\mathcal{A}$ )`. The problem milieu evolves and executes as much as possible of the solution. At this point, it depends on whether the solving of the problem has concluded. If not, and there is a trigger for re-solving, the current state is observed and transformed into a data file understandable by MiniZinc via the `get()` function. This produces the most up-to-date representation δ of the problem milieu, and the cycle continues.

3.5 EMPIRICAL EVALUATION

This section presents experimental results that demonstrate the proof-of-concept and the usefulness of Online MiniZinc.⁸ First, we present an experiment for real-time optimisation, and then we present one for rolling-horizon decompositions.

All experiments in this chapter were run on a 2.2 GHz Intel Core i7 processor with 16 GB RAM, using the lazy clause generation solver Chuffed (version bundled with the MiniZinc IDE Version 2.2.3). The MiniZinc interface (version 0.1.0) for Python (version 3.7.3) was used to simulate data changes between solving sessions.

3.5.1 Real-Time Online Job-Shop Scheduling

The first experiment uses the dynamic job-shop scheduling model shown in Figure 3.2. The online data used is constructed from the offline data file `ft20` of the MiniZinc benchmarks⁹ by repeatedly adding copies of the jobs from `ft20` into an endless queue. This endless queue is then iterated through as follows. The first i jobs in the queue form the initial solving session, where i is the greatest integer such that the first session can be solved to optimality within 5 seconds under perfect execution. After this, the next $i/2$ jobs from the queue form the next session consecutively, for as many sessions as desired.

The aim of this experiment is to show that the presented framework supports real-time optimisation with uncertain task durations. As we solve in real-time, τ^{len} is 5 seconds, i.e., $\tau^{\text{end}} = \tau^{\text{beg}} + 5$. Each task is given an ‘original’ duration computed as an expected value of an unknown distribution, which means that its realisation (i.e., the actual duration during execution) may differ from its given value. As a result, the realisation of a task’s start time is also uncertain, as it may have to be pushed forward during execution because a predecessor task has a longer duration than expected. We simulate the execution step in this experiment as follows. Starting from the solution found by solving the model, each task that will be finished in the next session (using the given task duration) is randomly assigned a (possibly) different task duration. These random durations are generated from a normal distribution with the mean parameter being the ‘original’ task duration and the standard deviation as one tenth of the average task duration in the instance file (`ft20`). If a task runs slower (or faster) than expected, then each task that depends on it (i.e., has to move) is set to its earliest possible start time (respecting all constraints), repairing the entire schedule.

⁸Code available at <https://gitlab.com/minizinc/online-minizinc>

⁹<https://github.com/MiniZinc/minizinc-benchmarks/tree/master/jobshop>

Note that, after each task move, tasks that appear later in the schedule may also have to be moved, which is why we start from the earliest task in the schedule when performing the above schedule repair. See online schedule generation [146] for more information regarding such an approach. The realised data and decisions are then read into the model via the `get()` function (see Algorithm 1) and accessed as normal via the `inc()` command.

Table 3.1 shows the results of the final makespan for scheduling 1,162 jobs in total (from `ft20`), with different time limits per session, and different rates of new jobs arriving per second. For this experiment, we assumed that 1 wall-clock second corresponded to 408 time units (based on the average task duration in the chosen benchmark). We can see how the choice of solving time allowed between sessions results in a trade-off between solution quality and how early the solution can be executed. This illustrates the meticulousness-quickness trade-off, introduced in Section 2.4. Assuming new jobs arrive every second, the more time we allow the solver to spend and find solutions, the further in the future these jobs will have to be scheduled (to account for the solving time), and the more new jobs will arrive for the next session to consider. However, the less time we give the solver, the more myopically it will solve, thus finding mediocre solutions or, in the worst case, no solution.

As we can see, if the rate of new jobs is low (e.g., 4 jobs per second), then using short solving sessions (e.g., of 0.5 seconds per session) yields the best overall makespan. However, too short a session (e.g., of 0.25 seconds per session) can result in a timeout with no solution. With a higher data rate (e.g., 8 jobs per second), a timeout of 4 seconds per session yields the best result. Being able to quickly experiment with these settings, by translating the external time automatically into model-specific time units, is another significant advantage of Online MiniZinc over manual approaches.

3.5.2 *Rolling Horizon Decomposition for Cargo Assembly Planning*

As mentioned earlier, a common approach for large, hard-to-solve, static optimisation problems is to solve a rolling-horizon decomposition of the problem. Recall that the framework can be used to directly implement and solve rolling-horizon decompositions simply by appropriately splitting the data.

To illustrate this, consider the cargo assembly planning problem (CAPP) from the MiniZinc benchmark suite,¹⁰ which is a simplified version of the problem described

¹⁰<https://github.com/MiniZinc/minizinc-benchmarks/tree/master/cargo>

time limit per session	makespan	
	4 jobs/s	8 jobs/s
0.25 s	—	—
0.50 s	117,845	—
1.00 s	117,973	70,663
2.00 s	118,170	68,549
4.00 s	118,749	67,892
8.00 s	119,581	68,632
16.00 s	121,352	71,687

Table 3.1: Real-time dynamic job-shop scheduling (file `ft20`). Showing average over 5 runs. The symbol ‘—’ denotes a timeout of a solving session along the way.

in [14]. In CAPP, vessels arrive at different times. Every vessel has a set of cargoes that has to be assembled in a stockyard, at an unoccupied part on a stacking pad, into a set of stockpiles. Each stockpile is then loaded onto its vessel. The aim is to minimise the total delay.

To decompose the problem, we annotate the model in the following ways: the number of vessels becomes an `online`-annotated enum, the stacking and loading start times for each stockpile are `time`-annotated, and the position of each stockpile is `lock_var_at`-annotated with the assembly start time of that stockpile.

Table 3.2 shows the results for all instances from the MiniZinc benchmarks with at least 16 vessels. We compare the offline solving approach (original), as a baseline, with a rolling horizon of 10 non-arrived vessels, adding 5 new vessels at each session. We arrived at these two numbers after some quick experimentation, as they seemed promising. A total timeout of 120 seconds (only instantiation and solving time) was used for each method, divided equally among the solving sessions.

Clearly, the rolling-horizon decomposition is usually better than solving the original problem; the exception is for the easiest of the problems, where the global viewpoint allows the solver to find a better solution.

instance name	full horizon		rolling horizon	
	time	delay	time	delay
07_1s_133	—	10,563	40.4	5,868
08_222f_3475	—	70,068	90.4	66,085
09_1s_18_OPT	—	7,117	0.4	22,958
10_15966f_2060	—	38,119	90.3	36,347
16_10720f_4243	—	—	80.7	88,862
19_31058f_2548	—	141,119	77.3	129,748

Table 3.2: Cargo Assembly Planning Problems solved as a single optimisation problem and using rolling-horizon decomposition. The symbol — under time indicates a timeout (120s) and under delay indicates no solution was found.

3.6 DISCUSSION AND FUTURE WORK

As we reach the end of this chapter, let us discuss a few directions for future work and open questions.

One direction for future work is to address the lack of support in Online MiniZinc for modelling any a priori known stochastic aspects of dynamic problems. For example, in job-shop scheduling, this could be estimates (along with probable deviation and confidence) of the number and durations of the jobs that will arrive in the next few sessions. With such information, we could strengthen the solving by taking it into account, either by solving for the robustness of the solution (e.g., leaving gaps between scheduled jobs for anticipated delays) or by optimising for the expected value (e.g., leaving room for anticipated difficult jobs, leading to a better schedule later). Relevant works include [125, 73, 158].

Related to robustness, allowing a modeller to specify stability criteria (see, e.g., [31]) is also a potential avenue for future work, where we want the incumbent solution to change as little as possible session to session. Currently, modellers can only specify these criteria manually in the objective function, using the functionality presently available in Online MiniZinc. This could be done by, for example, minimising a distance function measuring some kind of distance between the solution and the incumbent assignment. Another interesting direction for future work is understanding how to integrate methods like pattern learning and active prediction (e.g., some form of machine learning) as described in [15] with the presented framework. This could allow for incorporating predictions of future data while solving.

A full implementation of the framework inside the MiniZinc toolchain also remains future work. For example, the online solving loop is currently outside MiniZinc. This could be implemented as part of the MiniZinc toolchain, allowing in-model specifications for rolling-horizon decompositions at a high level as a solve-annotation, for example.

On the solving side, there are also several potential avenues for future work. The first question regards how to utilise warm-starts most effectively along with this framework. A warm-start is when the solver is provided with a variable assignment (not necessarily a solution) as a starting point. It then searches for similar solutions before searching for solutions more generally. Warm-starts can be very effective in some circumstances because it may be wasteful to search for solutions anew, blind to any previous insights and incumbent assignments, and it can also help with stability criteria. However, a warm-start can also mislead the solver, so modellers need to be careful when to use warm-starts (e.g., identifying sessions where it may be effective). The second question regards how to utilise local repair methods. A local repair method is a particular kind of warm-start method where the variable assignments that violate the constraints are revised (often one by one) until all constraints are satisfied. The goal is often to find a satisfying solution with as few changes as possible from the starting variable assignment. Local repair can be very quick in many cases, where starting a full search is unnecessary, making it possible to have a short session length and execute the decisions earlier. The third question regards adaptive online optimisation, where the sessions are dynamically sized, perhaps as a response to the predicted difficulty of the session. This approach would aim to find an optimal meticulousness-quickness trade-off.

3.7 CHAPTER CONCLUSION

This chapter has presented a systematic approach that significantly simplifies the modelling and solving of dynamic optimisation problems, making it more efficient for experienced modellers and more accessible for novices. A supporting formalism has been developed that captures the common concepts of online optimisation and dynamic combinatorial problems. Several annotations and predicates have been introduced that can be used to turn MiniZinc models into Online MiniZinc models. This is achieved via compilation techniques that automatically transform the annotations into predicate calls. This enables modellers to describe their problem and its dynamism at a high abstraction level and solve it online.

Experiments have shown the framework's usefulness for both a dynamic problem and rolling-horizon decompositions of a static problem. We believe that the generality of the introduced concepts enables this framework to be used for many other dynamic problems.

AUTOMATIC GARBAGE COLLECTION FOR ONLINE COMBINATORIAL OPTIMISATION¹¹

One inherent problem of high-level generic online optimisation is the increasing size of instances. As time passes, more objects are added, leading to more data points, variables, and constraints. This increase inevitably leads to a performance decay in both compilation and solving time. In addition, as time passes, previous decisions get executed, and all their implications get accounted for, i.e., they can never be changed and have no effect on new decisions. Thus, they are irrelevant and can be considered *garbage* when solving all current and future sessions. Therefore, they can be removed from all remaining instances to keep their size as small as possible and keep the solving of them as fast as possible.

This chapter proposes a problem-independent garbage collection framework for online combinatorial optimisation. The framework is designed for a modeller to provide minimal information about the relation between time, data, and variables, allowing the framework to automatically aggregate (i.e., simplify and remember the simplifications for later sessions) and garbage-collect the instances. The chapter proceeds as follows. First, the necessary preliminaries are presented (Section 4.1). Then, a naïve and problem-dependent approach to garbage collection is introduced, laying the foundation and motivation for this chapter as a whole (Section 4.2). The discussion then leads into the main contributions of this chapter, starting with a formalism of garbage collection (Section 4.3). Thereafter, aggregation rules (Section 4.4) and the overall implementation (Section 4.5) are presented in more detail. The chapter then rounds off with some experimental evaluation (Section 4.6), related work (Section 4.7), discussion and directions for future work (Section 4.8), and conclusion (Section 4.9).

¹¹The content of this chapter is partly based on the published work of Papers [I] and [II].

4.1 PRELIMINARIES

The contributions presented herein do not build upon the concept of garbage collection from the field of automatic (dynamic) memory management but are merely inspired by it. The field of automatic memory management is concerned with helping programmers with the often difficult task of writing programs that correctly and efficiently manage computer memory [83]. Thus, the ‘garbage’ collected refers to data in memory that is no longer needed for the continued, successful execution of the computer program. Methods like tracing or reference counting are utilised to automatically detect data that is no longer reachable from the program code (the garbage) and can be safely removed. In this chapter, *garbage* refers to realised decision variables and input data that do not occur in any constraints (or the objective) with non-realised decision variables. Recall that variables and data are *realised* when their assignments and values can no longer be changed by the solver or the problem milieu, e.g., they refer to the past. Thus, this garbage can be safely removed without affecting the correctness of the model.

Unfortunately, garbage collection is not straightforward as decision variables and data that were once non-realised and are now realised will still be present in constraints all across the generated instance. We can take inspiration from the aggregation process performed by some solvers for ‘fixed’ decision variables during search. Constraints with fixed variables can often be rewritten by *aggregating* the fixed variables into a single or a few values. Similarly, we denote by *aggregation* the process of simplifying a constraint by replacing any fixed decision variables with their value. Faster solving can be achieved if we precompute the aggregated value of the fixed variables in each constraint and remove these variables from the constraint. Consider for example a constraint containing the sum $\sum_i c_i \times x_i$. We can easily partition the coefficients c_i and variables x_i into two sets: those containing fixed decision variables F and the rest V . Then, we can substitute the sum $\sum_i c_i \times x_i$ by $C + \sum_{i \in V} c_i \times x_i$, where the constant value $C = \sum_{i \in F} c_i \times x_i$ is precomputed. This technique is common in many solvers, mainly copying solvers (e.g., Gecode [64]), which aggressively simplify all constraints before copying them. We can do something similar in online optimisation while remembering (and not redoing) the simplifications done in past sessions in addition and thus incrementally aggregating the results over time.

4.2 GARBAGE COLLECTION: A NAÏVE APPROACH

Let us consider the dynamic job-shop problem, formalised in the annotated Online MiniZinc model of Figure 3.2 (page 91), whose decision variables are declared in line 10 (the `start` variables). The question one must answer is when (and how) one can disregard a decision variable. Naturally, if a job j is still running, then j may still suffer uncertainty in execution, i.e., its start time variables and processing duration data points may change externally (as we assume that machine requirements are static). Conversely, if a job j is finished running, meaning that τ_s^{beg} (the start of session s) occurs later than the latest end time of any task in j , then j can have no further effect on any other jobs. That is to say, the job j has been fully executed in the problem milieu (i.e., all its tasks have finished) and is thus *realised*. We say that a job itself is *realised* if all its associated start times and durations are realised. Therefore, we can focus on the decision variables and data points of this model associated with any one job. Every realised decision variable and data point is a *candidate* for removal, and every non-realised one is not. We say it is a *candidate* because simply removing it may incorrectly alter the logical meaning of a constraint. As such, further investigation is needed. Let us illustrate this with the constraints in the job-shop model, starting with the task-ordering constraint.

```
constraint forall (j in JOBS, t in ..<max(TSKs)) (
  start[j,t] + dur[j,t] <= start[j,enum_next(t)] );
```

Each constraint generated by the `forall` loop only constrains variables (and uses data points) *within* a given job j , not *across* jobs. Therefore, in the context of this constraint, we can omit from *JOBS* any realised job j without adverse effects.

Consider now the temporal constraint posted by the `time` annotation.

```
array [JOBS,TSKs] of var 0..: start ::time; % start times
```

The annotated decision variables will generate, as explained in Chapter 3, a constraint as follows. Recall that τ^{end} denotes the end of the current solving session, and that `inc(x)` denotes the incumbent assignment for a variable x .

```
constraint forall (j in JOBS, t in TSKs) (
  if has_inc(start[j,t]) /\ inc(start[j,t]) <  $\tau^{\text{end}}$ 
  then start[j,t] = inc(start[j,t])
  else start[j,t] >=  $\tau^{\text{end}}$  endif );
```

As with the task-ordering constraint, this generated constraint only constrains variables (and only uses the data points) within a given job j . Thus, we can likewise omit from *JOBS* any realised job j .

Consider now the disjunctive constraint:

```
include "disjunctive.mzn";
constraint forall (m in MACs) (
  disjunctive([start[j,t] | j in JOBS, t in TSKs where req[j,t]=m]
    , [dur[j,t] | j in JOBS, t in TSKs where req[j,t]=m]
  ) );
```

which may constrain variables across jobs, potentially making garbage collection complex. However, let us make two observations that alleviate this. First, a job j is realised when the start time and duration of its final task are realised, meaning that j was scheduled and has finished processing before the current session starts, i.e., before τ^{beg} . In any other case, the job j could suffer uncertainty in execution. The second observation comes from the `time` annotation of the `start` variables. The constraint posted by this annotation ensures that any new or revised start time must be after the current session's end (i.e., after τ^{end}), which is, in turn, after τ^{beg} . Therefore, any restriction enforced by the disjunctive constraint between a realised and non-realised job is already enforced by the `time` annotation constraint. Accordingly, in the context of the disjunctive constraint, any realised job j can safely be omitted from *JOBS*.

Finally, let us consider the objective function. Often, the logic of the objective function is less affected than constraints by the removal of variables. For example, assume that a variable y is fixed to the value 5. A constraint $x + y = 10$ in this case is very different from a constraint $x = 10$, but maximising the expression $x + y$ is equivalent to maximising x when y is fixed. This behaviour is reflected in the makespan objective.

```
solve minimize max (j in JOBS) (
  start[j,max(TSKs)] + dur[j,max(TSKs)] );
```

Removing a realised job will not affect which solution in the non-realised search space is best; it will only reduce the actual objective value by a specific constant. In fact, removing a realised job will not affect the objective value at all if there are still jobs to schedule. This comes from the fact that any non-realised job must occur later in the schedule than the realised job we remove. In conclusion, no constraint is adversely affected by the removal of a realised variable, nor is the objective function.

The next step is to remove realised jobs from *JOBS*. Let us rewrite the job-shop model slightly, adding a new enum:

```
enum JOBsKeep = K( min({j | j in JOBs where not
                      realised(j)}) ..max(JOBs) );
```

where **K** is a function mapping objects from `JOBs` to `JOBsKeep`. This new enum is designed to start at the first non-realised job, according to the test `realised(j)`. This test will not involve decision variables, only incumbent assignments, in order to permit its evaluation at compile time. The aim is to replace every use of `JOBs` with `JOBsKeep` in the rest of the model and still ensure correctness. Given the analysis above and an appropriate definition of `realised(j)`, we are guaranteed this correctness. As a result, the actual solving will not consider the realised jobs before `realised(j)`, speeding up solving slightly. In the subsequent session, we can set the input of `JOBs` to be the result of the current session's `JOBsKeep` plus any fresh jobs. This allows us to remember the simplifications from one session to the next. The definition of `realised` will be as follows, essentially stating that a given job j is realised if its final task has finished processing before the current session begins.

```
test realised(JOBs: j) =
  inc(start[j,max(TSKs)]) + dur[j,max(TSKs)] <=  $\tau^{\text{beg}}$ ;
```

Note the use of the **test** keyword, which denotes a Boolean function that can be evaluated at compile time. The test will function as follows. Suppose that we enumerate the global set of jobs as $\{j_1, j_2, \dots, j_{k_i-1}, j_{k_i}, j_{k_i+1}, \dots, j_{n-1}, j_n\}$, where j_{k_i} is the latest job revealed in session s . Initially, `JOBsKeep` will start at j_1 . After instantiating the update-model, if this expression determines, for example, that job j_5 is the first non-realised job, then `JOBsKeep` will start at j_5 . In turn, the input set `JOBs` will start j_5 . At this point, the jobs $j_1, \dots, j_4$ will no longer be passed to future sessions: they are *garbage collected*. Things that are not garbage collected will be referred to as *alive*.

Note that this method always lags by one iteration: the model computes which data would have been irrelevant for the time point of the current iteration, and that data is then excluded at next the iteration. That is, garbage collection is part of the compilation process. We say this method is naïve because of two weaknesses. The first weakness is that if a very long-running job arrives early, it will keep all subsequent jobs alive until it is finally finished. In other words, this is effective only if the objects that appear earlier in the list are more likely to be realised earlier than objects later in the list. Otherwise, this approach will not be as powerful as if we could remove arbitrary objects. The second weakness is that all analysis and model transformation was manual. The question is whether garbage collection can be automated (or, if not entirely, then to what degree). Another question is how to handle more complicated

constraints, such as circuit and other global constraints, where the removal of realised variables and data is much more complex. In the coming sections, we will investigate these questions.

4.3 A FORMALISM OF GARBAGE COLLECTION

In this section, the formalism of Section 3.2 is expanded to include the concepts of realisation, garbage, and aggregation. These concepts lay the theoretical foundations for this chapter. First, the formalism established so far is summarised, with a few enhancements of the notation necessary for this chapter.

Recall that $M[\Delta]$ denotes a parameterised COP (simply referred to as a model when the context is clear) and is represented as a quadruple $\langle X[\Delta], D[\Delta], C[\Delta], f[\Delta] \rangle$ of parameterised variables, domain values, constraints, and a parameterised objective function. Using $M[\Delta]$ as an underlying model, $Q = \langle M[\delta_1], M[\delta_2], \dots \rangle$ denotes a parameterised dCOP, where the data instances $\delta_1, \delta_2, \dots$ represent the evolving problem milieu, and δ_s represents the state of the problem milieu at the commencement of solving session s . Let us extend the $\text{inc}(x)$ notation for the purposes in this chapter to have $\text{inc}_s(x)$ denote the incumbent assignment of variable x in session s . A data instance $\delta \in \Delta$ contains objects and data points. The objects can be partitioned into object sets, where the number of object sets is defined by Δ , but the sizes of these sets are defined by δ . A data point $p \in \delta$ is said to be *generated* by an object $a \in \delta$ if $p \notin \delta \setminus \{a\}$. Similarly, variables are generated by the objects, where a variable $x \in X[\delta]$ is said to be *generated* by an object $a \in \delta$ if $x \notin X[\delta \setminus \{a\}]$. Objects, data points, variables, constraints, and the objective function are *declared* in Δ , i.e., specified parametrically with respect to potential the objects in any $\delta \in \Delta$, while they are *defined* in δ , i.e., specified concretely with respect to the given objects in δ .

Note that the formalism, as is, does not enforce more data to be added as the problem evolves, in which case garbage collection would not be necessary. In some contexts, the same decisions must be revised repeatedly, but no new ones are ever added. However, in most contexts, new decisions are added with time. Let us formally define this as a problem with *growing instances*.

Definition 4.1 (Growing Instances). A parameterised dCOP $\langle M[\delta_1], M[\delta_2], \dots \rangle$ is defined to exhibit *growing instances* if and only if the number of objects (and thus associated data and variables) grows as the problem evolves, and no object is

removed. More formally, for $\delta_s, \delta_i \in \Delta$, this means $\delta_s \subseteq \delta_i$ for all sessions s and i , where $s \leq i$.

Notice that δ_s and δ_i are from the same parameter space Δ , meaning that the only way one can be a strict superset of the other is with one having more objects. Having more data points would require a Δ to change. A consequence in a parameterised dCOP with growing instances, is that no variables are removed from one session to the next, i.e., $X[\delta_s] \subseteq X[\delta_i]$ for all sessions s and i where $s \leq i$. For simplicity, the number of objects does not need to strictly grow as time progresses.

Let us illustrate this with two examples. Consider a problem that involves controlling air conditioning units in buildings, trying to maintain a comfortable temperature under some constraints. Such a problem would be dynamic and could be modelled as a parameterised dCOP. However, it would not exhibit growing instances, assuming (reasonably) that the number of air conditioning units remains the same. On the contrary, a scheduling problem with new jobs to schedule constantly arriving would exhibit growing instances.

4.3.1 Realisations and Garbage

Recall that a data point (or a decision variable) is *realised* if it is guaranteed never to change its value (or incumbent assignment) in the problem milieu or never affect a decision in any possible future solving session. Additionally, an object is said to be realised if all decision variables and data points generated by this object are realised. The motivation for solving constraint models is to determine what to assign the non-realised variables. Thus, if we can summarise the exact effect that the realised objects have on the non-realised variables without using these objects, then we can express an equivalent problem using fewer objects, variables, and data points: less garbage. The concept of realisations can be formalised as follows:

Definition 4.2 (Realisations). Suppose that $M[\Delta]$ is a parameterised COP with components $\langle X[\delta], D[\delta], C[\delta], f[\delta] \rangle$, that a corresponding parameterised dCOP with growing instances is $\langle M[\delta_1], M[\delta_2], \dots \rangle$, and that $\mathcal{R} : X[\Delta] \cup \Delta \times \Delta \rightarrow \mathbb{B}$ is a function that denotes whether a variable, data point, or object is *realised* or not. All the following conditions then hold for every session s :

1. For every variable $x \in X[\delta_s]$ where $\mathcal{R}(x, \delta_s)$ is true, $\text{inc}_s(x) = \text{inc}_i(x)$ holds for all potential future sessions $i \geq s$.

2. For every data point $p \in \delta_s$ where $\mathcal{R}(p, \delta_s)$ is true, at least one of these two criteria holds:
 - (a) the value of p is the same in δ_s and δ_i for all potential future sessions $i \geq s$; or
 - (b) changing the value of p to any other value, creating an altered data instance δ'_s as a result, will have no effect on the solution space, i.e., $\text{sols}(M[\delta_i]) = \text{sols}(M[\delta'_i])$ for all potential future sessions $i \geq s$.
3. For every object $a \in \delta_s$, the function $\mathcal{R}(a, \delta_s)$ is true if and only if all its generated variables and data points are realised.
4. If $\mathcal{R}(y, \delta_s)$ is true for any variable, data point, or object y , then $\mathcal{R}(y, \delta_i)$ is also true for all potential future sessions $i \geq s$.

The first condition says that realised decision variables are static, and the second that realised data points are either static or irrelevant. The third condition connects the realisedness of objects, data points, and variables. Finally, the fourth condition states that realisedness is monotonic. The problem milieu determines what is realised and what is not. In practice, this requires a modeller to understand the underlying problem being solved and how it can evolve. This is because, generally, one cannot conclude what is realised and not by simply analysing a model without context of the underlying problem.

Given a constraint c , if all its variables $\text{vars}(c)$ are realised, and no new variables can be added to it in a later session, then c is said to be a *realised constraint*. Realised constraints can be safely ignored for solving since they do not affect any remaining decisions. Given the assumption that we have complete recourse, no realised constraint can be unsatisfiable.¹² A decision variable is said to be *garbage* if it does not, and will never in the future, occur in any constraints (or the objective) that also constrains non-realised decision variables. In other words, a realised variable that only occurs in realised constraints (and only ever will) is garbage. The same idea applies to data points. Similarly, an object is said to be garbage if all variables and data points it generates are garbage. Garbage can be formally defined as follows:

Definition 4.3 (Garbage). Suppose that $Q = \langle M[\delta_1], M[\delta_2], \dots \rangle$ is a parameterised dCOP with growing instances, and that $M[\Delta]$ is its underlying model with compon-

¹²For problems without complete recourse, the framework would abort with an error message if a constraint with realised variables becomes unsatisfiable, and human intervention would be necessary.

ents $\langle X[\Delta], D[\Delta], C[\Delta], f[\Delta] \rangle$. Let $\mathcal{G} : X[\Delta] \cup \Delta \times \Delta \rightarrow \mathbb{B}$ be a function that denotes whether a variable, data point, or object is *garbage* and be defined as follows for every session s :

1. For every data point $p \in \delta_s$, if $\mathcal{R}(p, \delta_s)$ is true (i.e., p is realised), then $\mathcal{G}(p, \delta_s)$ is true.
2. No variable $x \in \text{vars}(f[\delta_s])$ has $\mathcal{G}(x, \delta_s)$ as true.
3. For every realised variable $x \in X[\delta_s] \setminus \text{vars}(f[\delta_s])$, if for every potential future sessions $i \geq s$ and every constraint $c \in C[\delta_i]$ with $x \in \text{vars}(c)$, each variable $x' \in \text{vars}(c)$ is realised in session i , i.e., $\mathcal{R}(x', \delta_i)$ is true, then x is *garbage* in session s , i.e., $\mathcal{G}(x, \delta_s)$ is true.
4. Every object $a \in \delta_s$ that only generates garbage variables and data points is also *garbage*, i.e., $\mathcal{G}(a, \delta_s)$ is true.
5. For all other cases, $\mathcal{G}(y, \delta_s)$ is false for every variable, data point, or object y .

The first condition states that all realised data points are also garbage. This is because every realised data point can be replaced by a constant in its respective constraint specifications. This mere replacement is not that useful in practice since the goal is to forget values and free up space. The second condition states that no variable (potentially) affecting the objective value is garbage. The third condition states that realised variables (not in the objective) that occur, and only will occur, in realised constraints are garbage. Finally, the fourth condition connects the objects with the data points and variables. Notice that we have extended the two definitions of being realised and garbage to operate on the objects. This is because we want to garbage-collect at the object level, making it easy to remove all its associated variables and data points once they are all realised.

Note also that, as the condition for being garbage is quite strict, many model instances will have no garbage for any particular object set. This is the case, for example, if a model has a global constraint encompassing an entire object set. Consider a simple sum constraint, ensuring that one variable equals the sum of all variables generated by some object set. As time passes, this object set will only get larger, and the sum constraint will always encompass all the generated variables. Thus, the constraint will never become realised if this object set can be expanded in the future. A resolution for this will be presented next.

4.3.2 Hidden Garbage and Aggregations

Often garbage will be *hidden* in an instance. *Hidden garbage* refers to decision variables, data points, and objects that are realised but not garbage, and that with a *reformulation* of the model become garbage. A *reformulation* of a model is a change in how the variables and constraints are declared that still produces equivalent solution spaces. That is, they model the same problem. For this thesis, constraint reformulation suffices. Reformulations can also utilise the introduction of new data points (but not new objects or object sets), the usefulness of which will be discussed later. We want to reformulate the models to allow as much hidden garbage as possible to become visible. Recall that $\text{sols}(P)$ denotes the solution space of a COP P and that $\mathcal{A}$ denotes a variable assignment, mapping each decision variable to a value in its domain. Model reformulation can be formalised as follows:

Definition 4.4 (Model Reformulation). Let $M[\Delta]$ be a parameterised COP with components $\langle X[\Delta], D[\Delta], C[\Delta], f[\Delta] \rangle$. Let $\Delta' \supset \Delta$ be an extension of Δ with new data point declarations that are never used in any constraints, domains, or the objective function. Thus, $M[\Delta']$ and $M[\Delta]$ describe the same parameterised COP in practice since the new data point declarations in Δ' are never used and can be ignored. Construct a new set of constraint declarations $C'[\Delta']$ and a new objective function declaration $f'[\Delta']$ such that $R[\Delta'] = \langle X[\Delta'], D[\Delta'], C'[\Delta'], f'[\Delta'] \rangle$ and $M[\Delta']$ specify the same solution space, i.e., $\text{sols}(R[\delta]) = \text{sols}(M[\delta])$ for every data instance $\delta \in \Delta'$, as well as corresponding solution qualities, i.e., $f'[\delta](\mathcal{A}) = f[\delta](\mathcal{A})$ for all solutions $\mathcal{A} \in \text{sols}(R[\delta])$. The resulting model $R[\Delta']$ is a *model reformulation* of $M[\Delta]$.

The reason for introducing extra data points will be clarified later in this subsection. Put simply, allowing the addition of extra data points makes more reformulations possible.

To maximise garbage collection is to find the model reformulation that uncovers the most hidden garbage, where hidden garbage is formalised as follows:

Definition 4.5 (Hidden Garbage). Let $M[\Delta]$ be a parameterised COP and $R[\Delta']$ a reformulation of $M[\Delta]$. For any data instance $\delta \in \Delta$, an object $a \in \delta$ is *hidden garbage* in $M[\delta]$ if it is not garbage in $M[\delta]$ but is garbage in $R[\delta]$.

We seek model reformulations where the objects increasingly become garbage from one session to the next. To achieve this, constraints need to be reformulated to allow

the use of *aggregated data*, that is, of a compact summary of all necessary information regarding the realised variables and data points. More formally, the data points introduced to enable a reformulation (see Definition 4.4) compose the *aggregated data*, and each such data point is called an *aggregate construct*. A constraint that can utilise aggregated data need not use the realised variables and data represented by said aggregated data. A constraint is said to be *aggregated* if it is reformulated to use one or more aggregate constructs. Each constraint can be systematically aggregated based on *aggregation rules*, which are laid out in Section 4.4.

If a model is such that more and more objects become garbage as the sessions progress, then it is said to be a *garbage-collecting model*. A formal definition follows.

Definition 4.6 (Garbage-Collecting Model). Let $Q = \langle M[\delta_1], M[\delta_2], \dots \rangle$ be a parameterised dCOP with growing instances. Let $M[\Delta]$ be its underlying model with components $\langle X[\Delta], D[\Delta], C[\Delta], f[\Delta] \rangle$. $M[\Delta]$ is a *garbage-collecting model* if and only if, for all sessions s and potential future sessions $i \geq s$:

$$\{a \mid a \in \delta_s, \mathcal{G}(a, \delta_s)\} \subseteq \{a \mid a \in \delta_i, \mathcal{G}(a, \delta_i)\}.$$

Put simply, in a garbage-collecting model, the set of garbage objects cannot decrease. Again, for simplicity, the set of garbage does not need to strictly grow as time progresses.

A garbage-collecting model allows a computer representation of the problem to ignore objects marked as garbage. As a result, the representations of the instances can remain compact.

4.4 AGGREGATION TYPES AND RULES

Let us investigate aggregation further, which is the most intricate concept in this chapter. We want to do be able to do what we did in Section 4.2, but automatically and for any kind of expression. The basic principle of aggregation is that one can summarise the effects of realised objects (including the data and variables they generate) as something more compact, i.e., they can be aggregated. In general, aggregation is constraint- and expression-specific. However, since constraint-based models are built from simpler constraints and expressions, there is a possibility of a general theory of aggregation. This chapter takes many important first steps towards such a general theory. To do this, we use the MiniZinc language to help exemplify aggregation more clearly; but note that aggregation concepts analogously apply to other high-level

constraint-based modelling languages. We present and classify aggregation rules into types, further classify many of the common constraints and constructs of MiniZinc into one of these types, and exact aggregation rules for a number of these as well as many generic rules. The presented aggregation method will modify the update-model to both make use of the modified form of constraints with aggregation and compute the new aggregate values as well as output these new aggregate values so that they are available to the next session.

Thankfully aggregation in the conventional sense is well understood, and modern solvers, such as Gecode [64], aggressively aggregate constraints during search. The extra challenge that arises in automatic garbage collection for online optimisation is that the results of aggregation need to be communicated *across* sessions. To achieve this, we need to introduce an *aggregate construct*: a summary of the realised data in a smaller format. The model must also be reformulated to allow its constraints and expressions to utilise these aggregate constructs instead of the realised data it represents. Each aggregate construct consists of two parts: an old part, which constitutes previously aggregated data and is provided as input to the model, and a fresh part, which constitutes data being aggregated in the current session. The aggregate constructs are then output at the end of the session (alongside the solution) to be sent to the next session, where further aggregation occurs.

First, we create a taxonomy of aggregation types based on the form of the aggregate construct for expressions and constraints common in MiniZinc models. The reason for the classification of a specific expression or constraint, along with specific rules of how to aggregate some of them, will be discussed later in this section.

1. *Vacuous Aggregation*: there is no need for any aggregate construct. This type mainly concerns the function `forall`.
2. *Value Aggregation*: the aggregate construct is a single value. This type includes the functions `sum`, `product`, `maximum`, and `minimum`.
3. *Set Aggregation*: the aggregate construct is a set of values, the ordering of which being irrelevant. This type includes the global constraint `alldifferent`.
4. *Strong Array Aggregation*: the aggregate construct is a list of values, where their ordering is relevant. We say it is *strong* because all realised objects can be part of the aggregation. This type includes the global constraints `disjunctive`, and `cumulative`.
5. *Weak Array Aggregation*: the aggregate construct is a list of values, where the ordering of them is relevant. We say it is *weak* because at least one realised

object needs to be kept outside the aggregation. This type includes the global constraint `circuit`.

6. *Non-Aggregatable*: expressions and constraints that cannot be aggregated in general but possibly so if extra context is provided. This type chiefly includes sorting constraints, such as `sort`.

Expressions can also be nested in other expressions, where both the outer expression and the inner one can be aggregated, making aggregation more complex. For example, if the inner expression is value or set aggregatable, then it automatically falls into one of the two array aggregation classes because of the added dimensionality. But more on that later. Let us discuss these aggregation types from the easiest to the most intricate.

For simplicity of notation, we extend the functions $\mathcal{R}$ and $\neg\mathcal{R}$ to respectively return the realised and non-realised objects when given an object set as input. Further, for any object set E , we assume that it is garbage-collected before being provided to the model, i.e., only the objects that will be used in a potential future session are provided to the next session.

4.4.1 Vacuous Aggregation

We have already seen vacuous aggregation: realised constraints that can be safely removed without reformulation or data being saved. These most commonly appear in the `forall` function. Let us consider the task ordering constraint of Figure 3.2 (page 91) again.

```
constraint forall (j in JOBS, t in ..<max(TSKs)) (
  start[j,t] + dur[j,t] <= start[j,enum_next(t)] );
```

Such a `forall` construction essentially posts a constraint for each job j in `JOBS`. If some j is realised, then that means that the data points $\text{start}[j,t]$ and $\text{dur}[j,t]$, as well as the variable $\text{start}[j,\text{enum_next}(t)]$ are realised regardless of t ; thus, the constraint posted for the job j is likewise realised. In that case, the constraint must be satisfied because of the complete recourse assumption, and we can replace it with a constant true. Further, a constant true constraint in a broader `forall` constraint is superfluous since it cannot affect the broader constraint's truth value, i.e., the constant true constraint can be straightforwardly removed. To put it concisely, all realised objects indexing a `forall` loop can be ignored without creating an aggregate construct.

We can define vacuous aggregation for any arbitrary, vacuously aggregatable function f . Suppose we have the current list of n variables $x_1, \dots, x_n$ partitioned into its realised part $\{y_1, \dots, y_k\}$ and its non-realised part $\{y_{k+1}, \dots, y_n\}$. For ease of notation, we assume that f has arbitrary arity, i.e., it is defined (analogously) for any number of provided arguments. The transformation is as follows:

$$f(x_1, \dots, x_n) \mapsto f(y_{k+1}, \dots, y_n)$$

4.4.2 Value Aggregation

Let us discuss the *value aggregation* type, which is when a single value is the construct necessary for aggregation. Let us do this by means of an example: the objective of the dynamic vehicle routing problem appearing in Example 2.9 (page 58) and the model in Figure 3.3 (page 92). This objective has the following sum:

```
sum (v in VEHs) ( arri[E(v)] - ready[v] );
```

which constructs a loop over the object set *VEHs* of vehicles. Partitioning *VEHs* into its realised and non-realised counterparts yields:

```
sum (v in R(VEHs)) ( arri[E(v)] - ready[v] )
+ sum (v in ¬R(VEHs)) ( arri[E(v)] - ready[v] );
```

To aggregate the first sum—the realised part—we simply precompute its value and keep it in the newly introduced data point parameter **int**: *agg*, which is the aggregate construct. The constraint can then be transformed as follows:

```
agg + sum (v in ¬R(VEHs)) ( arri[E(v)] - ready[v] );
```

where *agg* is the sum of everything aggregated so far in the whole problem lifetime. To define *agg* in MiniZinc one can write:

```
int: agg = old_agg + sum (v in R(VEHs)) ( arri[E(v)] - ready[v] );
```

where *old_agg* is the old aggregate value, i.e., the one passed on from the previous session. Note that this rule applies analogously to sums with more than one index. In that case, the aggregate sum would contain the terms that are indexed solely by realised objects.

We present a generic rule of value aggregation for any arbitrary function f . Suppose we have the current list of n variables $x_1, \dots, x_n$ partitioned into its realised part $\{y_1, \dots, y_k\}$ and its non-realised part $\{y_{k+1}, \dots, y_n\}$. Also, assume we have an aggregator for f , a function f' returning a single value. For ease of notation, we

assume that f and f' have arbitrary arity, i.e., they are defined (analogously) for any number of provided arguments. Hence, we can transform f as follows:

$$f(x_1, \dots, x_n) \mapsto f(f'(y_1, \dots, y_k), y_{k+1}, \dots, y_n)$$

We can replace $f'(y_1, \dots, y_k)$ with an aggregate construct: a value a . This value a can then be communicated to the next session, where further aggregation can ensue. Thus, a value a' is provided in most sessions, representing the aggregate value from the previous sessions. In that case, the transformation rule is as follows:

$$f(a', x_1, \dots, x_n) \mapsto f(f'(a', y_1, \dots, y_k), y_{k+1}, \dots, y_n)$$

where $f'(a', y_1, \dots, y_k)$ becomes the aggregate value for the next session. Because of the arbitrary arity, in the first session, a is omitted. Additionally, if there are no realised values in a session, then f' will only operate on a unless it is the first session, in which f' will be applied to no values. Thus, f' needs to have an ‘empty’ definition, i.e., a default value that does not change the output value of f if added as one of the arguments. For example, an empty sum is 0, and an empty product is 1.

In most cases, the functions f and f' are then same function, as was the case for `sum`. A few other value aggregatable functions include `maximum`, `minimum`, and `product`.

4.4.3 Set Aggregation

Let us now discuss the *set aggregation* type, where a set, instead of a value, must be computed, saved, and made accessible to the next session. Consider, for example, an `alldifferent` constraint, where an array x is indexed by dynamic object set E , as follows:

```
array [E] of var D: x;
constraint alldifferent(x);
```

where D is also an object set. We can interpret this as x being constructed by the array comprehension $[x[i] \mid i \text{ in } E]$. We can then partition E into its realised and non-realised counterparts, obtaining the following:

```
constraint alldifferent([x[i] \mid i in ¬ℛ(E)] ++ [x[i] \mid i in ℛ(E)]);
```

Because of the logic of the `alldifferent` constraint, this is equivalent to restricting the domain of the non-realised variables to not take on any value from the set

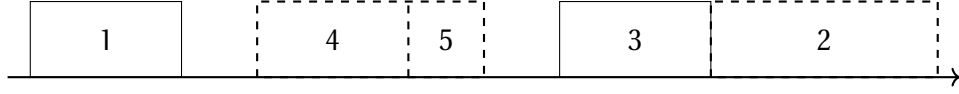


Figure 4.1: A disjunctive schedule before aggregation, with realised (thickly dashed; 4, 5, and 2) and non-realised tasks (rest; 1 and 3)



Figure 4.2: The disjunctive schedule after aggregation but before solving, where the thinly dashed tasks (a_1 and a_2) are the introduced aggregates and the tasks 2, 4, and 5 have become garbage

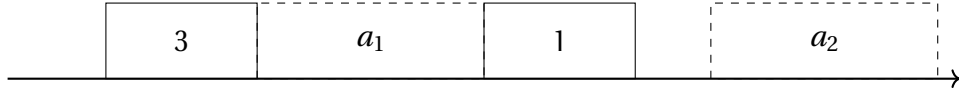


Figure 4.3: The aggregated disjunctive schedule after solving, where the two non-realised tasks have been rescheduled

$\{x[i] \mid i \text{ in } \mathcal{R}(E)\}$ of realised variables. Thus, we can make the corresponding adjustment in the declaration of the array x , obtaining the following:

```

1 set of int: old_agg;
2 set of int: agg = old_agg union {x[i] | i in  $\mathcal{R}(E)$ };
3 array [E] of var D diff agg: x;
4 constraint alldifferent ([x[i] | i in  $\neg\mathcal{R}(E)$ ]);

```

In line 1, we declare the set `old_agg`, which is the old aggregate set, i.e., the one passed on from the previous session. In line 2, we define the aggregate construct `agg` as a set that is the union of the old aggregate set `old_agg` and the set of the newly realised variables' values in x . In line 3, we have adjusted the array declaration such that the domain D does not include the values from the aggregate construct. Further, this would allow us to mark these objects as garbage, enabling garbage collection from D as well as from E .

4.4.4 Strong Array Aggregation

Let us discuss the *strong array aggregation* type, where all realised objects can (possibly) be turned into garbage, and the aggregate construct consists of one (or more) array. Consider, for example, a disjunctive constraint that has five tasks to schedule, with start times $s_1, \dots, s_5$ and corresponding processing durations $d_1, \dots, d_5$. Let the

incumbent assignment be as visualised in Figure 4.1. From earliest to latest: task 1 is non-realised, tasks 4 and 5 are realised and scheduled directly one after the other, task 3 is non-realised, and directly after it is task 2 scheduled and realised. The realised tasks can be replaced with aggregates. Further, any set of contiguously scheduled realised tasks need not be separate aggregate tasks: they can be represented by one task covering all of them in length. In this case, tasks 4 and 5 are replaced with aggregate task a_1 and task 2 is replaced with aggregate task a_2 . After aggregation, the resulting instance is visualised in Figure 4.2. After solving, where the non-realised tasks can be revised, the solution visualised in Figure 4.3 is a possibility. One can see that task 1 and task 3 have changed starting times while (as expected) the aggregate tasks remain static.

Let us discuss how to achieve aggregation of **disjunctive** in general. Consider the **disjunctive**_(s,d) constraint where s is an array of start times with domains D_s , and d is a corresponding array of processing durations with domains D_d . Let both of these be indexed by a dynamic object set E . A disjunctive constraint over these arrays can be expressed in MiniZinc as

```
array [E] of var Ds: s;
array [E] of var Dd: d;
constraint disjunctive(s, d);
```

As usual, the object set E can be partitioned into its realised $\mathcal{R}(E)$ and non-realised $\neg\mathcal{R}(E)$ parts. Hence, the disjunctive constraint can be rewritten as

```
constraint disjunctive([s[i] | i in  $\mathcal{R}(E)$ ] ++ [s[i] | i in  $\neg\mathcal{R}(E)$ ],
                      [d[i] | i in  $\mathcal{R}(E)$ ] ++ [d[i] | i in  $\neg\mathcal{R}(E)$ ]);
```

The next step is to aggregate the two realised arrays, which we do as follows. Let $[e_1, \dots, e_k]$ be a list of the elements of $\mathcal{R}(E)$ sorted by starting time s in ascending order. Traversing this list, for all $1 \leq i \leq k$, if two tasks e_i and e_{i+1} are adjacent (i.e., $s[e_i] + d[e_i] = s[e_{i+1}]$), then we can replace these two tasks by an aggregate task a_i with $s[a_i] = s[e_i]$ and $d[a_i] = d[e_i] + d[e_{i+1}]$. This results in a new list $[e_1, \dots, e_{i-1}, a_i, e_{i+2}, \dots, e_k]$. We can now repeat this whole step and continue the process with this new list, starting by comparing a_i and e_{i+2} . Once every element in the list has been traversed, the entire list has been aggregated, i.e., new aggregate tasks covering realised tasks. Let functions g_s and g_d respectively return the aggregate start times and aggregate durations resulting from the above process, given a list of either realised start times or durations. To achieve this, we can reformulate the model as follows:

```
1 array [E] of var Ds: s;
```

```

2 array [E] of var  $D_d$ : d;
3 array [int] of  $D_s$ : old_sagg;
4 array [int] of  $D_d$ : old_dagg;
5 array [int] of  $D_s$ : sagg =  $g_s$ (old_sagg ++ [s[i] | i in  $\mathcal{R}(\mathbf{E})$ ]);
6 array [int] of  $D_d$ : dagg =  $g_d$ (old_dagg ++ [d[i] | i in  $\mathcal{R}(\mathbf{E})$ ]);
7 constraint disjunctive(sagg ++ [s[i] | i in  $\neg\mathcal{R}(\mathbf{E})$ ],
8                        dagg ++ [d[i] | i in  $\neg\mathcal{R}(\mathbf{E})$ ]);

```

The start times and durations are declared as usual in the first two lines. In the lines 3 and 4, the old aggregate values are declared: `old_sagg` for s and `old_dagg` for d . In line 5, we define the aggregate construct `sagg`: an array that is the task aggregation of `old_sagg` and list the newly realised tasks. This is analogous for line 6. In the last two lines, the constraint has been reformulated to make use of the aggregate constructs `sagg` and `dagg`.

Since scheduling constraints such as `disjunctive` often have a component of time, the distant past is wholly irrelevant to the current decisions. We previously established this in Section 4.2. Hence, in such cases, one can enhance the aggregation by simply omitting aggregate tasks that have finished processing before the start of the session. Let us illustrate this with the example of Figure 4.1. If the current time (τ^{beg}) is later than the end of task 5, then tasks 1, 2, and 5 are all realised because they are in the past. Further, no new tasks can be scheduled to start before τ^{beg} (or even τ^{end}), meaning that creating contiguous blocks before τ^{beg} is futile and even wasteful. As such, one can enhance the garbage collection by altogether omitting these jobs and any associated aggregate tasks. In summary, there is a *cut-off* before which aggregate tasks will not be introduced or kept.

The aggregation is analogous for the cumulative constraint, but a resource profile is generated (possibly consisting of multiple aggregate tasks) for each contiguous block of realised tasks. Let us briefly illustrate this with an example. Consider the schedule in Figure 4.4. From earliest to latest: task 1 is realised and takes up one resource unit; one third through the duration of task 1, there is task 2, which is also realised, but takes up two resource units and ends at the same time as task 1; task 3 starts directly after tasks 1 and 2 ends, is realised, and takes up three resource units; task 4 starts directly after task 3 ends, is also realised and takes up three resource units, but is half the length of task 3. That summarises the realised tasks. After a small gap come two non-realised tasks: first comes task 5, which is non-realised and takes up two resource units; and finally, directly after task 5 ends, task 6 starts, which is also non-realised but takes up three resource units. As with `disjunctive`, we can aggregate the realised tasks. After aggregation, the resulting instance is visualised in Figure 4.5,

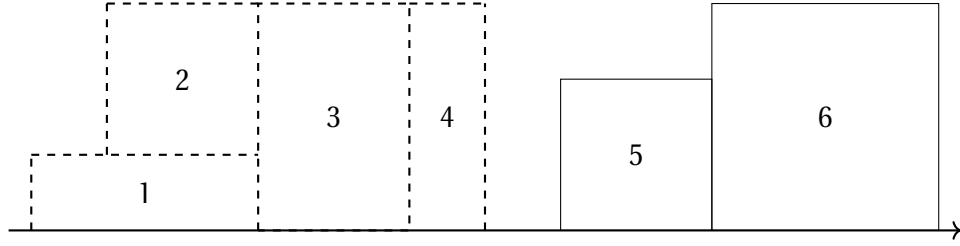


Figure 4.4: A cumulative schedule before aggregation, with realised (thickly dashed; 1, 2, 3, and 2) and non-realised tasks (rest; 1 and 3)

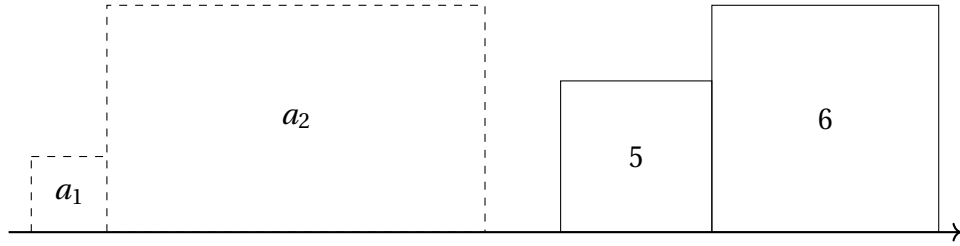


Figure 4.5: A cumulative schedule after aggregation, where the thinly dashed tasks (a_1 and a_2) are the introduced aggregates and the task 1, 2, 3, and 4 have become garbage

in which aggregate task a_1 partly covers task 1, and aggregate task a_2 covers the rest of task 1 and the entireties of tasks 2, 3, and 4.

Note that the ordering of the elements in the aggregate array is important, not because they need to be related correctly to the non-realised objects, but because the ordering of the elements is important within the aggregate construct itself. In the disjunctive example, there had to be an order correspondence between the aggregate starting times and the aggregate durations. This is the case for everything we have classified as strongly array aggregatable. In the contrary case, the expression is instead weakly array aggregatable, which will be discussed in the next subsection.

Before that, we present a general rule for strong array aggregation as follows. For any strongly array aggregatable function f , with variables $x_1, \dots, x_n$ split into realised $y_1, \dots, y_k$ and non-realised $y_{k+1}, \dots, y_n$, and supposing that ℓ aggregate variables (z_i for $1 \leq i \leq \ell$) are created, the transformation rule is as follows:

$$f(x_1, \dots, x_n) \longmapsto f(\overbrace{f'(y_1, \dots, y_k)}^{f'(y_1, \dots, y_k)}, z_1, \dots, z_\ell, y_{k+1}, \dots, y_n)$$

Again, we assume that f and f' have arbitrary arity and ‘empty’ definitions.

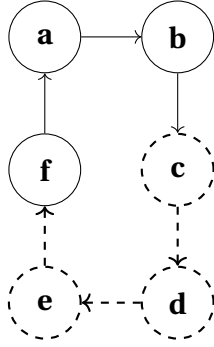


Figure 4.6: A circuit before aggregation, with realised (dashed) and non-realised successors

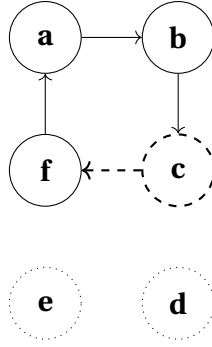


Figure 4.7: The circuit after aggregation (dotted nodes are garbage) but before solving

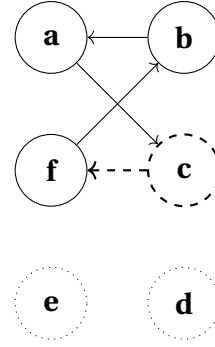


Figure 4.8: The aggregated circuit after solving, where non-realised successors have been revised

4.4.5 Weak Array Aggregation

Aggregation can be tremendously complex for constraints where the realised values are used to access other values. In this case, *weak* aggregation is often needed, that is, aggregation where not all realised objects can become garbage. Consider, for example, the circuit constraint for a graph with six nodes **a**, ..., **f**. Suppose the incumbent assignment is as visualised in Figure 4.6, where the incumbent successor of **a** is **b**, which then points to **c**, and so on. Notice that nodes **c**, **d**, and **e** are realised, i.e., the assignment of what node is their successor is realised. Aggregation for this circuit can then be achieved by short-circuiting it as visualised in Figure 4.7. Note that node **c** cannot be removed because a variable must point to **c** in future sessions. The nodes **d** and **e** can, however, be removed from the constraint and thus become candidate garbage. The successor assignments of the non-realised variables can be revised as usual, obtaining, for example, the solution in Figure 4.8, where pointing to **c** essentially means pointing to the realised subcircuit that starts at **c** and ends at **f**.

In general, consider the $\text{circuit}(x_1, \dots, x_n)$ constraint. Each consecutive subcircuit of realised nodes will be aggregated into its initial node with the final node's pointer (i.e., the pointer to the first non-realised node).¹³ More formally, suppose there are ℓ realised subcircuits, where each subcircuit starts at x_{r_i} and ends with a pointer to its first non-realised node x_{d_i} , for all $1 \leq i \leq \ell$. Starting at the first non-realised in the list, let the binary operator $y < z$ mean that node y is before z in the given

¹³This assumes that there is at least one non-realised node. This is a safe assumption as, otherwise, the constraint is garbage (fully realised, and we cannot add new nodes to it) and can be simply removed in full.

incumbent successor assignment. Let the notation of the realised subcircuits be such that $x_{r_1} < x_{d_1} < \dots < x_{r_\ell} < x_{d_\ell}$ holds. For the first realised subcircuit, starting at $x_{r_1} = x_i$ and ending with a pointer to $x_{d_1} = x_j$, we aggregate this into one node with index i but which points to x_j . As i refers to a named object indexing the list of nodes $x_1, \dots, x_n$, and as named objects remain from one session to the next, the object i will continue to refer to this aggregate node in all future sessions until further aggregation concerning i occurs. Let $y_1, \dots, y_n$ be the non-realised variables of the input list $x_1, \dots, d_n$ in the same relative order, and x_{e_i} be the predecessor of x_{d_i} for any $1 \leq i \leq \ell$. Further, let y_{r_i} be the last non-realised node before the realised node x_{r_i} in the list, and y_{d_i} be the first non-realised node after it. Note the key differentiation of a node being before or after another in the list $x_1, \dots, x_n$ (i.e., lower or higher index) and being before or after another in the circuit (which is independent of index). Given the above, the transformation rule of circuit can be formalised as follows:

$$f(x_1, \dots, x_n) \mapsto f(y_1, \dots, y_{r_1}, f'(x_{r_1}, \dots, x_{e_1}), y_{d_1}, \dots, y_{r_\ell}, f'(x_{r_\ell}, \dots, x_{e_\ell}), y_{d_\ell}, \dots, y_k)$$

Notice the importance of keeping the ordering of the nodes; this is an essential characteristic of weakly array aggregatable expressions. Any node in $x_1, \dots, x_n$ may already represent a previously aggregated (short-circuited) realised subcircuit. Again, we assume that f and f' have arbitrary arity and ‘empty’ definitions.

4.4.6 Aggregation for Forall-Nested Expressions

Often in constraint-based models, expressions are *nested* within other expressions and constraints. The simplest and most common example of this is forall-nested functions, as seen in both the dynamic job-shop (Figure 3.2, page 91) and dynamic vehicle routing (Figure 3.3, page 92) models.

When a **forall** is nested within another **forall**, no additional action need be taken. The constraints are still vacuously aggregatable. Examples of this are locking, forbidding, and committing constraints (presented in Section 3.3). These constraints are meant to connect the past, the present, and the future. By definition, all realised decisions are also locked; thus, all further locking, forbidding, or committing constraints are obsolete, i.e., they can have no effect. Thus, we can vacuously aggregate these constraints.

Aggregation becomes more complicated if the inner constraint is value, set, or array aggregatable: an aggregate construct is needed per loop index of the **forall**.

For example, a sum is aggregatable into a value, while a **forall** of sums must be aggregated into an array of values—one for each sum across the **forall**. This will make the aggregation classification of the outer expression become strongly array aggregatable.

MiniZinc has native support for arrays of values and sets; thus, value and set aggregation can be naturally extended to **forall**-nested value and set aggregation. In fact, multiply **forall**-nested set and value aggregation (e.g., a sum inside a **forall** itself inside a **forall**) is also straightforwardly dealt with: one array dimension per **forall**. Consider the following example:

```
1 array [E1] of var int: x;
2 array [E1, E2] of var int: y;
3 constraint forall (i in E1) (
4   x[i] = sum (j in E2) (y[i, j]) );
```

where we want to aggregate a **sum** nested within a **forall**. In line 1, a one-dimensional array x of variables is declared with index set E_1 . In line 2, a two-dimensional array y of variables is declared with index sets E_1 and E_2 . In lines 3 and 4, the **forall**-nested constraint is posted. If an object in E_1 is realised, then we can discard it via vacuous aggregation of the **forall**. Suppose that an object j in E_2 is realised, which means that $y[i, j]$ is realised for every object i in E_1 , regardless of whether i is realised. In that case, for each non-realised object in E_1 , we can summarise the sum for index j (plus any other realised objects of E_2) as an aggregate sum. We obtain the following from reformulation:

```
1 array [E1] of var int: x;
2 array [E1, E2] of var int: y;
3 array [E1] of int: old_agg;
4 array [¬ℛ(E1)] of int: agg =
5   [ old_agg[i] + sum (j in ℛ(E2)) (y[i, j]) | i in ¬ℛ(E1) ];
6 constraint forall (i in ¬ℛ(E1)) (
7   x[i] = agg[i] + sum (j in ¬ℛ(E2)) (y[i, j]) );
```

In the first two lines, x and y are defined as previously. In line 3, an array `old_agg` of aggregate sums is declared, which will be provided from the previous session. Note that this array is indexed by E_1 , some objects of which may not be realised and thus have no aggregate construct. In that case, the sum is set to zero. In lines 4 and 5, the new aggregate sums of the current sessions are constructed, using `old_agg`. Notice that these are only constructed for non-realised objects of E_1 . This is because the realised objects of E_1 are omitted by the vacuous aggregation of the **forall**. In lines 6 and 7, the constraint is reformulated. In line 6, notice that the objects of

E_1 used within the `forall` are restricted to be the non-realised ones because the realised ones can be omitted via vacuous aggregation. In line 7, notice that the sum has been rewritten according to value aggregation, i.e., the expression now uses the aggregate sum `agg[i]`, adding it to the rest of the (non-realised) sum.

A more complicated case is when a (weakly or strongly) array aggregatable expression appears inside a `forall`. Consider a disjunctive inside a `forall`, just as in the dynamic job-shop model (of Figure 3.2, page 91). Suppose there is a 2d-array of start times s , with domains D_s , and a corresponding array of processing durations d , with domains D_d . Let both be indexed by two dynamic object sets E_1 and E_2 , as shown in the following MiniZinc snippet.

```
array [E1, E2] of var Ds: s;
array [E1, E2] of Dd: d;
constraint forall (i in E1) (
    disjunctive ([s[i, j] | j in E2],
                [d[i, j] | j in E2]) );
```

The first step to reformulate this code is to follow the non-nested aggregation rules for disjunctive and then lift this to the nested case. Let us assume that MiniZinc supports arrays of arrays.¹⁴ Let, again, the functions g_s and g_d respectively return the aggregate start times and aggregate durations resulting from aggregating the disjunctive constraint, given a list of either realised start times or durations. Hence, we can reformulate the snippet for aggregation as follows:

```
1 array [E1, E2] of var Ds: s;
2 array [E1, E2] of Dd: d;
3 array [E1] of array [int] of Ds: old_sagg;
4 array [E1] of array [int] of Dd: old_dagg;
5 array [¬R(E1)] of array [int] of Ds: sagg =
6   [ g_s(old_sagg ++ [s[i, j] | i in R(E1)]) | i in ¬R(E1)];
7 array [¬R(E1)] of array [int] of Dd: dagg =
8   [ g_d(old_dagg ++ [d[i, j] | i in R(E1)]) | i in ¬R(E1)];
9 constraint forall (i in ¬R(E1)) (
10   disjunctive (sagg[i] ++ [s[i, j] | j in ¬R(E2)]
11               dagg[i] ++ [d[i, j] | j in ¬R(E2)]) );
```

Start time variables and duration data points are declared in the first two lines. In line 3, an array of arrays `old_sagg` is declared, through which the aggregate tasks' start times are passed on from the previous session. This is analogous for `old_dagg`

¹⁴If arrays of arrays are not natively supported, this can be emulated by one 'master' array, contiguously containing all the arrays. In that case, the concepts herein are defined analogously, and one just needs to keep track of at which index each array starts.

in line 4. In lines 5 and 6, an array of arrays `sagg` containing the start times of the current session's aggregate tasks is defined. This is analogous for `dagg` in lines 7 and 8. Notice that the outer array is constructed only for non-realised objects of E_1 . This is because the realised objects of E_1 are omitted by the vacuous aggregation of the `forall` and are thus made obsolete. In lines 9–11, the aggregated constraint is posted. In line 9, notice that the `forall` only concerns non-realised objects of E_1 due to vacuous aggregation. In lines 10 and 11, the aggregate constructs `sagg` and `dagg` are used, respectively combined with the non-realised start times and durations.

4.4.7 Aggregation for Arbitrarily Nested Expressions

Expressions can be nested quite arbitrarily in MiniZinc, not only within a `forall`. For example, one could have any kind of constraint in a `where` clause of an array or set comprehension. If such a constraint requires consideration over a dynamic object set, then the aggregation rules can quickly become very complex. The development of a comprehensive theory for aggregating any MiniZinc model, including arbitrarily nested expressions, remains future work. Instead, let us illustrate the basic intuition through an example. Consider the following MiniZinc snippet, constraining a variable y to be a sum of products:

```
var int: y;
array [E1, E2] of var int: x;
constraint y = sum (i in E1) (
  product (j in E2) (x[i, j]) );
```

Both the inner (`product`) and the outer (`sum`) expressions are aggregatable in their own right. The simplest way to address this is just to aggregate the outer `sum` as-is, ignoring the inner `product`. However, this may result in subpar aggregation. We can do better. We can use both their aggregation rules in the following manner: (a) the `sum` can be aggregated as usual, and (b) for each index in the sum, the `product` can be aggregated as usual, creating an array of product aggregates. That is, the model can be reformulated as follows:

```
1 var int: y;
2 array [E1, E2] of var int: x;
3 array [E1] of int: old_pagg;
4 array [E1] of int: pagg =
5   [ old_pagg[i] * product (j in  $\mathcal{R}(E_2)$ ) (x[i, j]) | i in E1 ];
6 int: old_sagg;
7 int: sagg = old_sagg + sum (i in  $\mathcal{R}(E_1)$ ) (
8   pagg[i] * product (j in  $\neg\mathcal{R}(E_2)$ ) (x[i, j]) );
```

```

9 constraint y = sagg + sum (i in  $\neg \mathcal{R}(E_1)$ ) (
10   pagg[i] * product (j in  $\neg \mathcal{R}(E_2)$ ) (x[i, j]) );

```

In the first two lines, the variable y and array x are declared as before. In line 3, an array of the old aggregate products is declared, i.e., those passed on from the previous session. Each object in E_1 that has no old aggregate associated with it is set to 1 as not to affect any of the products. In lines 4 and 5, the new aggregate products are defined. For each object i in E_1 , we obtain the old aggregate product `old_pagg[i]` and multiply it with the product of the newly realised $x[i, j]$. In line 6, the old aggregate sum is declared. In lines 7 and 8, the aggregate sum of the current session is defined by adding the old aggregate sum with the sum of the newly realised products. Note non-realised objects j in E_2 are used in this aggregate construct. This is allowed because all objects it indexes, all $x[i, j]$, are realised by virtue of i being realised. Finally, in lines 9 and 10, the aggregated constraint is presented: we constrain y to be the aggregate sum plus the non-realised sum, where the non-realised one is the sum of aggregate products multiplied with the corresponding non-realised product.

4.4.8 Non-Aggregatable Expressions

Some constraints and expressions cannot be aggregated in general. These are primarily sorting constraints and functions. For example, the constraint `sort(x, y)` takes two arrays, x and y and constrains that the latter takes on the same values as the former but sorted in increasing order. Adding a new element to the arrays can disrupt the order of y in essentially any way, making aggregation impossible since no part of the second array is ever realised. However, this is usually not a problem as `sort` (and similar constraints) are often used as symmetry-breaking constraints, that is, constraints that are used to eliminate other internal representations of equivalent variable assignments. We discuss this further in the following subsection.

4.4.9 A Note on Symmetry-breaking Constraints

Symmetry-breaking constraints need some extra attention. These constraints aim to disallow multiple internal representations for equivalent assignments; as a result, they disallow assignments not disallowed by the other constraints. Nevertheless, they are not necessary insofar as correctness is concerned but may be essential for speeding up solving. These constraints might need careful formulation as they may remove symmetries on which the aggregation rules rely. This can lead to artificial

unsatisfiability issues. Another key point regarding symmetry-breaking constraints is that they should only apply to non-realised decisions, as realised decisions should be taken as-is. That is to say, a symmetry-breaking constraint should be carefully designed to not rely on realised data points and decision variables if necessary. With this in mind, it is possible to use unaggregatable constraints such as `sort`, and symmetry-breaking constraints can often be left unaggregated.

For example, let us consider the vehicle routing model (of Figure 3.3). As we want the `next` array to form a single circuit, we have unintentionally introduced symmetries into the problem. To eliminate them, we have the following two symmetry-breaking constraints.

```
1 constraint next[E(max(VEHs))] = S(min(VEHs));  
2 constraint forall (v in ..<S(max(VEHs))) (  
3   next[E(v)] = S(enum_next(v)) );
```

In line 1, we set the successor of the (in `VEHs`) last vehicle's end depot to be the (in `VEHs`) first vehicle's start depot. This constraint will never be realised as long as new vehicles can be added. In lines 2 and 3, we set the successor of the (non-last) i th vehicle's end depot to be the $i + 1$ th vehicle's start depot. Our circuit aggregation is compatible with these constraints. First, when a new vehicle is added, it becomes the last vehicle in `VEHs` and remains that until another vehicle is added. Second, any short-circuit aggregation that will change the first vehicle will result in an aggregate construct taking its place, retaining the circuit. Similarly, if short-circuit aggregation affects the last vehicle, then the aggregate construct that replaces it will point towards the first vehicle's start depot. Additionally, if short-circuit aggregation affects both the last and first, a new first one will take its place (the previous one becomes garbage-collected), which will be the successor of the introduced aggregate construct. Third, a vehicle is only realised if its entire tour is realised, in which case the successors of all nodes from its start depot to (but not necessarily including) its end depot are realised. This means that any short-circuit aggregation will not infringe on correctness.

Note, however, that redundant constraints, which aim to speed up solving by improving inference but are not necessary for defining the problem, are aggregated as normal constraints. This is because there is no issue of conflict with the aggregation as they do not disallow any assignments already disallowed by the other constraints. We must aggregate them, though, as they may use realised objects like any other constraint.

4.5 IMPLEMENTATION IN ONLINE MINIZINC

This section provides the details for implementing the presented formalism and methods of realisedness, aggregation, and garbage collection in Online MiniZinc (presented in Chapter 3). The implementation reformulates the model in such a way that realisation analysis, aggregation, and garbage collection all occur as a part of the compilation phase of a MiniZinc model. The model reformulation process is divided into the following steps:

1. Identify and infer realisations:
 - (a) Identify realised variables and data points; and
 - (b) Infer realised objects from the above.
2. Aggregate the model:
 - (a) For each constraint, identify the realised objects that can be aggregated, and how;
 - (b) Reformulate the constraints and the objective function to use these; and
 - (c) Introduce and define the aggregate constructs.
3. Identify and omit garbage.

For each step, we will describe how to reformulate the model and what this will result in during compilation.

Step 1a. Identify Realised Variables and Data Points

This step requires the modeller to define a realisedness function $\mathcal{R}$ in MiniZinc for each decision variable and data point in the model. From this, its complement (i.e., the non-realisedness function) $\neg\mathcal{R}$ is automatically inferred to be data, variables, and objects not in $\mathcal{R}$.

If online optimisation is used to solve a static problem, then this can be done automatically: any data value parameter that becomes known and any decision variable that is locked is automatically realised, and all data points are automatically realised. For dynamic problems, however, a modeller's expert knowledge is needed to annotate the data value parameters and decision variables in the model to indicate when they are realised.

Let us illustrate the identification of realisedness by means of the dynamic vehicle routing model shown in Figure 3.3 (on page 92). This model has three arrays of

decision variables and three arrays of data points. Each of their realisations is provided by the modeller, as functions $\mathcal{R}$, as follows. First, a `ready` data point is observed (i.e., realised) when the vehicle in question is ready before the session starts; otherwise, it may still be postponed. This can be expressed as follows:

```
test  $\mathcal{R}(\text{ready}[v]) = \text{ready}[v] < \tau^{\text{beg}};$ 
```

Recall that the `test` keyword denotes a Boolean function that can be evaluated at compile time. Second, a `wait` data point is realised when the vehicle in question has finished waiting at the given node before the session starts. This can be expressed as follows:

```
test  $\mathcal{R}(\text{wait}[n]) = \text{inc}(\text{arri}[n]) + \text{wait}[n] < \tau^{\text{beg}};$ 
```

Third, a `dist` data point is realised when the distance has already been travelled before the session starts or when the distance is guaranteed to be never travelled in the future. This can be expressed as follows:

```
test  $\mathcal{R}(\text{dist}[n,m]) = \text{inc}(\text{arri}[\text{inc}(\text{next}[n])]) < \tau^{\text{beg}}$   

 $\quad \vee \text{inc}(\text{arr}[m]) < \tau^{\text{beg}};$ 
```

Notice that only one of two conditions in the disjunction must be met. The first condition refers to a vehicle having left node n and arrived at its successor (which may or may not be m), while the second condition refers to a vehicle having arrived at node m (and may or may not have come from n). As we only allow each node to be visited once, if either of these conditions is true, then it means that the distance between n and m has been travelled or that it will never be.

The arrays of the decision variables are defined analogously. First, an `arri` decision is realised when the vehicle in question has already arrived at the given node before the session starts; otherwise, it may still be postponed. This can be expressed as follows:

```
test  $\mathcal{R}(\text{arri}[n]) = \text{inc}(\text{arri}[n]) < \tau^{\text{beg}};$ 
```

Second, a `serv` decision is similarly realised when the vehicle in question has arrived at the given node (and thus begun serving it) before the session starts. Before this, external factors may cause the vehicle to, e.g., break down, requiring another vehicle to step in. This can be expressed as follows:

```
test  $\mathcal{R}(\text{serv}[n]) = \text{inc}(\text{arri}[n]) < \tau^{\text{beg}};$ 
```

Third, a `next` decision for node n is realised when the vehicle in question arrives at the incumbent successor of n , assuming that one cannot change successor once arrived. This can be expressed as follows:

```
test  $\mathcal{R}(\text{next}[n]) = \text{inc}(\text{arri}[\text{inc}(\text{next}[n])]) < \tau^{\text{beg}};$ 
```

All the above definitions use τ^{beg} , instead of τ^{end} , as a reference for settling on its inputs' realisedness. This is because objects (and associated data) must be determined if realised *before solving starts*, rather than before the produced solution's execution can start. In the latter case, data may still be dynamically altered during solving, leading to incorrectly ignoring this alteration if aggregated.

The above example shows why realisedness needs to be defined by a modeller. For example, the current model assumes that one cannot change the allocation of customers to vehicles after a vehicle has left the depot. This may be suitable for vehicles that need to pick up goods from the depot and deliver them to the customers. However, it may be unnecessarily restrictive for vehicles that provide a standardised service, in which case the rule can be changed as follows:

```
test  $\mathcal{R}(\text{serv}[n]) = \text{inc}(\text{arri}[n]) < \tau^{\text{beg}};$ 
```

Step 1b. Infer Realised Objects

An object is realised when all the data points and decision variables generated by the object are realised. Given the realisedness of all data points and decision variables, an object's realisedness can be inferred easily and automatically.

Continuing on the dynamic vehicle routing example, there are three object sets: vehicles *VEHs*, locations *LOCs*, and nodes *NODs*. One data point array is indexed by *VEHs* (namely, *ready*), while the others are indexed by *NODs*, including the decision variable arrays. Since *NODs* is constructed from *VEHs* and *LOCs*, we can infer that there must be some correspondence between them. We will come back to this correspondence later. Let us first review the realisedness of objects, ignoring the correspondence between them. This is called *local realisedness* of an object, denoted $\mathcal{R}^L$. Recall that an object is realised if and only if all its associated data and decision variables are realised; similarly, an object is *locally realised* if and only if all data points and decision variables it is an index of is realised (which ignores correspondences between objects).

Looking at the objects of *LOCs*, it can be concluded that all of them are locally realised because they are never *directly* used to generate an array: that is, $\mathcal{R}^L(\mathbf{l}) = \text{true}$ for all locations $\mathbf{l}$ of *LOCs*. However, objects of *VEHs* are used to generate the array *ready*. Each vehicle $\mathbf{v}$ of *VEHs* is locally realised if and only if *ready* $[\mathbf{v}]$ is realised. In MiniZinc, this can be defined thus:

```
test  $\mathcal{R}^L(\mathbf{v}) = \mathcal{R}(\text{ready}[\mathbf{v}]);$ 
```

Every node $n \in NODs$ is analogously realised, but for all data points and decision variables of which it is an index. The following definition is obtained:

```
test  $\mathcal{R}^L(n) = \mathcal{R}(\text{wait}[n]) \wedge \mathcal{R}(\text{arri}[n]) \wedge \mathcal{R}(\text{serv}[n]) \wedge \mathcal{R}(\text{next}[n])$ 
      /\ forall (m in NODs) (  $\mathcal{R}(\text{dist}[n,m])$  );
```

The **forall** for the distance data point `dist` is needed because *all* that is generated from an object must be realised in order for the object to be considered realised.

The final step is to define the actual realisations of all the objects via the local realisations. How the objects are connected is provided in the model; thereupon, the realisations of objects can be automated similarly to local realisations. For this particular example, the realisations can be as follows:

```
% locations l
test  $\mathcal{R}(l) = \mathcal{R}^L(l) \wedge \mathcal{R}^L(\mathbf{L}(l))$ ;

% vehicles v
test  $\mathcal{R}(v) = \mathcal{R}^L(v) \wedge \mathcal{R}^L(\mathbf{S}(v)) \wedge \mathcal{R}^L(\mathbf{E}(v))$ ;

% nodes n
test  $\mathcal{R}(n) = \mathcal{R}^L(n)$ 
      /\ (if  $v = \mathbf{S}^{-1}(n) \vee v = \mathbf{E}^{-1}(n)$  then  $\mathcal{R}^L(v)$  endif)
      /\ (if  $l = \mathbf{L}^{-1}(n)$  then  $\mathcal{R}^L(l)$  endif);
```

This indicates that a location (e.g., customer request) l is realised if both l and its associated node $\mathbf{L}(l)$ are locally realised; a vehicle v is realised if v , its start node, and its end node are locally realised; and a node n is realised if n is locally realised and its associated vehicle or customer is locally realised.

Steps 2a–2c. Aggregate the Model

The steps for how to (2a) identify realised objects, (2b) reformulate the constraints and objective function, and (2c) introduce and define the aggregate constructs follow from the discussion in Section 4.4. Let us briefly describe what happens during the compilation of such a reformulated model. First, the actual values of the aggregate constructs are calculated from the currently realised objects and the provided old aggregates. Second, the constraints are defined and posted using these aggregate constructs, constraining the currently non-realised variables.

Step 3. Identify and Omit Garbage

The final step is to perform the actual garbage collection. This is achieved by producing the objects that are not garbage (i.e., still used) as output along with the produced solution; hence, the objects not produced as output are automatically removed. As shown for the circuit constraint above, some realised (and aggregated) objects cannot be removed because they are still used. Therefore, there is a need to determine what can be safely removed and what cannot. That is, we need to infer the function $\mathcal{G}$ (that we introduced in Section 4.3). For brevity, we instead describe how to infer $\neg\mathcal{G}$ (i.e., the set of all data, variables, and objects that are not garbage). Nevertheless, either can be straightforwardly defined based on the other.

The garbage collection step for $P[\delta]$ is itself divided into three parts. The first part identifies the set of realised objects that are non-garbage for each constraint in $P[\delta]$ when considered *in isolation*, that is, the non-garbage objects local to each constraint. Note that these objects, together with the data value parameters and variables associated with them, are the only realised elements that have *not* been already aggregated during the previous steps. Formally, the identification of the non-garbage (i.e., still used) objects *local* to a given constraint in $P[\delta] = \langle X[\delta], D[\delta], C[\delta] \rangle$, where Ω is the set of dynamic object sets in δ (and $\bigcup \Omega$ is the union of all those object sets), is defined in terms of the function:

$$\neg\mathcal{G}^L: C[\delta] \cup \{f[\delta]\} \rightarrow \bigcup \Omega, \text{ where } \neg\mathcal{G}^L(\text{vars}(c)) \subseteq \text{vars}(c) \quad \forall c \in C[\delta] \cup \{f[\delta]\}.$$

This function returns the realised objects in every object set that the given constraint c (including objective function) still needs to use and have therefore not been aggregated by c . For example, in the dynamic vehicle routing model, if c is the objective function, then $\neg\mathcal{G}^L(c)$ will return the empty set since all realised objects are garbage for sums. However, if c is the circuit constraint, then $\neg\mathcal{G}^L(c)$ will return every realised node n whose predecessor node is not realised, that is, it returns:

$$\{n \in \mathcal{R}(N) \mid \text{next}[m] = n \text{ and } m \in \neg\mathcal{R}(N)\}$$

The second part of the garbage collection step collates the local non-garbage information to determine the realised objects that are non-garbage for the entire instance $P[\delta]$. To do this, a global non-garbage function is defined as follows:

$$\neg\mathcal{G}: \{S \mid S \in \Omega\} \rightarrow \bigcup \Omega, \text{ where } \neg\mathcal{G}(S) \subseteq S \quad \forall S \in \Omega,$$

which maps each object set $S \in \Omega$ to the objects in S that are non-garbage:

$$\neg \mathcal{G}(S) = \mathcal{R}(S) \cap \bigcup_{c \in C[\delta]} \neg \mathcal{G}^L(c), \quad \forall S \in \Omega.$$

The result is the subset of objects identified by any constraint c in the instance as non-garbage that are also realised objects of S . Notice that this function takes an object set and returns an object set for ease of notation, as opposed to $\mathcal{G}$ from Section 4.3, which takes an object and returns either true or false. Nevertheless, either version can be defined from the other.

The third and final part in the garbage collection step removes any garbage identified at the beginning of session s to create instance $P[\delta_s]$, thus eliminating the garbage from all future instances. This is already partially achieved in the aggregation steps by modifying the constraints to aggregate all realised data points and decision variables. In addition, we must remove any objects identified as garbage from the input object sets to complete the task. As a result, all associated data parameters and decision variables will also be removed. This removal is done by redefining each input object set S as $\neg \mathcal{G}(S) \cup \neg \mathcal{R}(S)$ and outputting it.

4.6 EXPERIMENTAL RESULTS

This section presents experimental results that demonstrate the effectiveness of the proposed garbage collection approach. For this purpose, we use the MiniZinc models of the dynamic job-shop problem (Figure 3.2, page 91) and the dynamic vehicle routing problem (Figure 3.3, page 92) and reformulated them (by hand) such that automatic aggregation and garbage collection can occur at compilation as explained in this chapter.¹⁵

All experiments in this chapter were run on a 2.2 GHz Intel Core i7 processor with 16 GB RAM. The MiniZinc interface for Python was used to simulate data changes between solving sessions. The first experiment used MiniZinc version 2.2.3 and the lazy clause generation solver Chuffed bundled with it. The last two experiments used MiniZinc version 2.4.3 and the CP solver Gecode 6.1.1 bundled with it. The differences in solver and MiniZinc version stem from the first experiment being performed earlier and as part of Paper [I], and the latter two experiments being part of Paper [II]. Gecode was chosen as opposed to Chuffed in the latter two experiments due to a bug in Chuffed causing a crash during more advanced aggregation. Nevertheless,

¹⁵See <https://github.com/aekh/CP2020-Models> for the resulting reformulated models.

as we do not compare results between these experiments, our conclusions are not invalidated.

4.6.1 Initial Experiments: Naïve Garbage Collection

The first experiment is based on the dynamic job-shop scheduling problem described throughout this thesis (Figure 3.2, page 91). The naïve garbage collection approach is evaluated for these initial experiments.

The dynamic data used is constructed from the offline data files `abz5`, `ft06`, `ft10`, and `ft20` of the standard MiniZinc benchmarks¹⁶ by, for each data file, repeatedly adding copies of the jobs into an endless queue, which is iterated as follows. The first i jobs in the queue form the initial iteration, where i is the greatest integer such that the first iteration can be solved to optimality within 5 seconds under perfect execution. (Recall that perfect execution means that produced solutions are executed flawlessly in the problem milieu, and no external sources can influence data points or incumbent assignments.) After this, the next $i/2$ jobs from the queue form the next iteration for as many iterations as desired. The τ^{beg} and τ^{end} parameters increase each by p time units after every session, where p is the lower bound makespan for scheduling the new jobs, assuming all tasks have the average processing duration of the given instance. For example, if there are three tasks with respective durations of 3, 5, and 13, then the average processing duration is $\frac{3+5+13}{3} = 7$. Additionally, with m machines, j jobs, and an average task processing duration of x , we calculate the lower bound of the schedule as $\lceil j/m \rceil mx$.

The main focus of this experiment is to illustrate the effect of garbage collection on runtime. This experiment is not run in real-time, i.e., the evolution of the problem milieu depends not on the *time* spent solving but only on the solutions produced. Figure 4.9 shows the compilation time (a) and solving time (b) for each iteration using file `abz5`, running for 300 iterations (i.e., 300 solving sessions). Without garbage collection, compilation time quickly starts dominating, highlighting the importance of garbage collection for long-running online optimisation applications. By contrast, compilation time with garbage collections is basically constant, shown at the very bottom of the plot. Solving time also increases without garbage collection, which is partly because a much larger initial model has to be parsed, and the additional propagation for parts of the model that are fixed needs to be performed. The peaks in the plot simply indicate that some instances are more challenging than others, and the recurring pattern of the peaks comes from `abz5` iteratively being reused until

¹⁶<https://github.com/MiniZinc/minizinc-benchmarks/tree/master/jobshop>

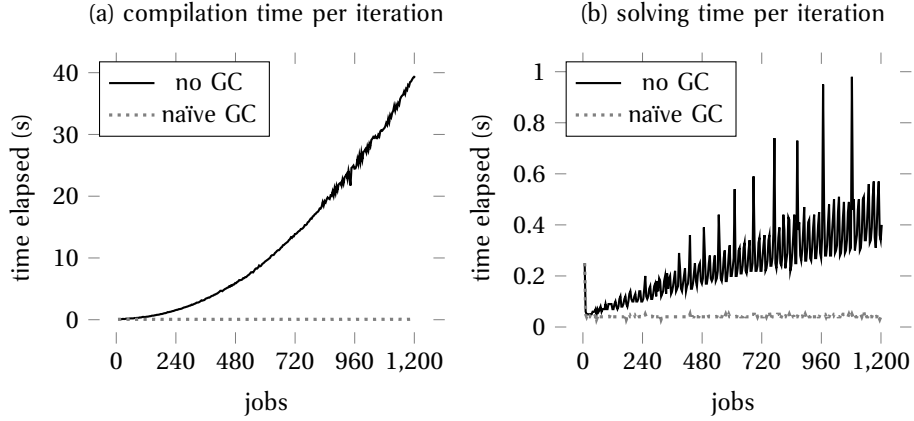


Figure 4.9: How (a) compilation time and (b) solving time per iteration changes over time, with and without garbage collection (GC), using abz5.

instance	naïve GC		no GC	
	comp. (s)	run (s)	comp. (s)	run (s)
abz5	19.00	32.01	3,913.48	3,989.12
ft06	18.91	33.47	1,363.91	1,420.54
ft10	19.86	36.90	3,882.49	4,004.08
ft20	18.84	1,486.40	941.64	2,432.50

Table 4.1: Accumulated compilation time and runtime (solving + compilation) with and without garbage collection (GC).

we reach 300 sessions. Nevertheless, there is an apparent upward trend. Table 4.1 shows the accumulated compilation time and runtime (i.e., compilation + solving) for multiple instances, each running for 300 iterations. We report runtime as opposed to solving time as time limits on sessions regard the entire runtime, not just compilation or solving time individually. We also report compilation time individually to illustrate that it is affected the most by garbage.

4.6.2 Naïve Versus Systematic: Long-Running Jobs

Again, the second experiment uses the dynamic job-shop scheduling problem described above. This experiment is modelled to highlight the advantage of using the proposed systematic approach over the naïve approach. In particular, some jobs are given very long-running times compared to other jobs. These long-running jobs should prevent effective garbage collection under the naïve policy since it cannot garbage-collect objects arbitrarily.

#	jobs	obj.	no GC		naïve GC			systematic GC		
			comp. (s)	run (s)	comp. (s)	run (s)	GC'd	comp. (s)	run (s)	GC'd
1	4	4278	0.09	0.11	0.09	0.11	0	0.08	0.10	0
2	8	4308	0.09	0.11	0.08	0.11	0	0.08	0.10	0
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
8	32	5428	0.46	0.50	0.47	0.51	0	0.27	0.30	7
9	36	5633	0.65	*1.00	0.68	*1.00	0	0.34	*1.00	9
10	40	5874	0.88	*1.00	0.90	*1.00	0	0.23	*1.00	18
11	44	8358	*1.00	*1.00	*1.00	*1.00	0	0.23	0.26	22
12	48	8388	—	—	—	—	—	0.17	0.20	30
13	52	8508	—	—	—	—	—	0.11	0.14	39
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
92	368	41028	—	—	—	—	—	0.19	0.22	350
93	372	41148	—	—	—	—	—	0.13	0.17	359
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮

Table 4.2: Shows the session number, total number of jobs added so far, best objective value (makespan) found (all methods reported the same makespan), compilation time, and runtime without garbage collection, plus the same information and the garbage collected jobs (GC'd) for the naïve and the systematic methods. An asterisk ‘*’ before a compilation time or runtime indicates a timeout.

To achieve this, we use the same model as before, a slight variation of the data file `ft20` from the MiniZinc benchmarks, and the same method to obtain an endless queue of jobs. In each session, the times τ^{beg} and τ^{end} are increased by 408 scheduling time units, the next four jobs are added to the queue, and a time limit of 1 second is set. The data file `ft20` is used to generate the instances per session as follows: The first job (and every 40th subsequent job) is a long-running job. Such jobs’ processing time is multiplied, over all machines, by 20.

Table 4.2 shows the benefit of using the proposed systematic method compared to not having garbage collection and to the naïve method. By reasoning about jobs individually (instead of finding the first non-realised job), the systematic approach can remove any job that becomes realised. The naïve approach, just like the approach without garbage collection, is timed out at the 12th solving session. At this point, nothing has been garbage collected by the naïve approach, while 30 tasks have been garbage collected by the systematic approach. The first (long-running) job is not realised in session 12. While this does not hinder the systematic approach, the naïve method is thus prevented from garbage-collecting any jobs, becoming too slow. These results highlight the strength of being able to reason directly about objects.

4.6.3 *Dynamic Vehicle Routing*

The final experiment is based on the dynamic vehicle routing model used throughout this thesis (Figure 3.3, page 92), and the class 1 input data file 100-0-rc101-1.¹⁷ The data file provides coordinates for each node, service times for each customer request (i.e., location), the number of vehicles available (16), the start and end depots (always the same), and the start time of each time window. The coordinates are used to compute the distance between any two nodes as their Euclidean distance and to set the time when a customer request arrives within a given time window as the start time of that time window. No other information in the data file was used. Note that whenever a tour ends, i.e., a vehicle arrives at its end depot, a new vehicle is added in that same session to represent a new tour. Note also that there may be some vehicles that remain unused in the solution produced a session (i.e., routed directly from the start depot to the end depot); these will be garbage collected in the next session and added anew. The time τ^{end} was increased by 50 time-units each session, which is equivalent to 1 minute wall-clock time. Thus, the fleet gets updated instructions each minute. The optimisation runtime timeout was set to 45 seconds to create a more realistic scenario, meaning that there are 15 extra seconds in a session, which can be used for (a) reading the problem milieu before compiling and (b) communicating the solution after solving. This means compilation and solving must be together completed in under 45 seconds. The experiment keeps running until all 100 customers have been added.

Recall that symmetry-breaking constraints need to work in concert with the aggregation rules. In this model, the symmetry-breaking constraints are compatible with the aggregations, as explained in the subsection of Section 4.4.

The naïve garbage collection method was not used for these experiments because we established before that it is inferior to the systematic garbage collection mechanism. The results in Table 4.3 show that the systematic garbage collection approach significantly improves the compilation time; without it, MiniZinc quickly gets overwhelmed with irrelevant tours and customers. Further, the systematic method makes the problem suitable to run within a 45-second timeout.

¹⁷<http://becool.info.ucl.ac.be/resources/benchmarks-dynamic-and-stochastic-vehicle-routing-problem-time-windows>

#	custs.	tours	with GC					without GC		
			obj.	comp. (s)	run (s)	GC'd <i>VEHs</i>	alive <i>LOCs</i>	obj.	comp. (s)	run (s)
1	2	16	64	0.18	*45.00	0	2	64	0.21	*45.00
2	3	31	128	0.46	*45.00	14	3	128	3.49	*45.00
3	10	47	572	0.55	*45.00	29	8	572	38.01	*45.00
4	18	63	861	0.76	*45.00	45	15	—	*45.00	*45.00
5	24	79	1346	0.99	*45.00	60	21	—	*45.00	*45.00
6	36	95	2010	1.61	*45.00	76	32	—	*45.00	*45.00
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
14	91	223	5003	10.57	*45.00	204	82	—	*45.00	*45.00
15	95	239	5352	13.34	*45.00	220	86	—	*45.00	*45.00
16	96	255	5370	13.46	*45.00	236	87	—	*45.00	*45.00
17	99	271	5432	13.52	*45.00	252	90	—	*45.00	*45.00
18	99	287	5458	15.38	*45.00	268	90	—	*45.00	*45.00
19	100	303	5483	15.61	*45.00	284	91	—	*45.00	*45.00

Table 4.3: Shows the session number, total number of customer requests so far, total number of tours added so far, (using garbage collection (GC) and aggregation:) best objective value found, compilation time, and runtime, number of garbage collected tours, and number of alive customers, (without GC and aggregation:) best objective value found, compilation time, and runtime. An asterisk “*” before a compilation time or runtime indicates a timeout.

4.6.4 Closing Remarks

As the experiments show, the overhead of the garbage collection steps is negligible compared to the overall compilation time and the performance gains in later sessions. Reformulation only occurs once, before solving commences. Thereafter, the MiniZinc model will calculate the values of the aggregate constructs and the garbage during the compilation part of each session. Since garbage collection aims to minimise compilation time in future sessions, it will be quick and worthwhile in practice.

4.7 RELATED WORK

Garbage collection is a well-studied topic for programming languages and runtime environments [83], where it refers to the elimination of program data in memory that is no longer needed for the continued, successful execution of the computer program. Methods like tracing or reference counting are utilised to automatically detect program data that is no longer reachable from the program code (the garbage) and can be safely removed. However, in the online optimisation context, the data and past decisions always have references, but they may still not be required for

the current and future sessions, depending on which decisions are realised. Hence, determining what is garbage is more complex. As we have seen above, the model may require transformations such as aggregation to remove references to past decisions and data.

While online solving methods have been studied in great detail, almost all of this work concentrates on particular application problems (e.g., [82, 120]). Any online solving method has to tackle the difficulty that, without deletion, the problem continues to grow. While for particular application problems, there may be simple rules that allow the modeller to define *garbage*, for a general definition, the interaction of realised decisions and earlier data with future decisions is complex.

Modelling support for online problem-solving is still in its infancy. The modelling language AIMMS has a notion of aggregation when using rolling horizon [128], which allows the effect of previous realised decisions to be accounted for in some parts of the model, such as the objective. The notion of realisation used there is much simpler, and the automated aggregation of complex constraints like circuit is not considered.

Open global constraints [8, 45] are a form of extensible constraint useful for dynamic optimisation problems whose size grows. The focus is on correct propagation of open constraints when not all information about them is available. In this chapter, a constraint defined over variables generated by a dynamic object set can be seen as being an open constraint. These works do not examine garbage collection since there is no notion of realisation in the general open-world setting. The constraints simply grow as time progresses.

There is surprisingly little published work on the aggregation of constraints. While simplifying constraints when their variables become fixed is well understood, it is rarely documented. There are preprocessing methods that consider this, such as Boolean equi-propagation [103], which simplifies the encoding of constraints for SAT solvers by detecting equivalences in the instance, and Gecode [64], which aggressively aggregates constraints during search. There has been work on eliminating variables during propagation [139], although we are not aware of a system that currently implements this. Regardless, these processes require an instance to be given. Thus, they are not directly applicable to online optimisation, where the aggregation rules must be *instance-independent*, and the result of aggregation needs to be stored and communicated *across* sessions. This kind of aggregation does not seem to have been considered before.

4.8 DISCUSSION AND FUTURE WORK

Before concluding this chapter, let us briefly discuss and investigate directions for future work.

The most obvious of these is to classify (into the different aggregation types) more constraints and expressions that are available in MiniZinc, including negated expressions and constraints. This requires formulating aggregation rules for these. Some may require complex aggregation and garbage collection schemes, similar to the scheme for circuit presented here, which may be interesting to investigate in their own right and may unveil new types of aggregation in the process. The goal would be to have a library of aggregation rules for typical MiniZinc constraints and functions.

Another avenue for future work concerns a closer investigation of how symmetry-breaking constraints and aggregation can co-exist in a model. How do we ensure that they are compatible, i.e., that the constraint does not remove the symmetries that the aggregation relies on for maintaining correctness?

All the above aim to eventually lead to a general and complete theory of aggregation and garbage collection. This is required to answer questions such as how to aggregate any arbitrarily nested expression. That is, we need a process that, given an arbitrary MiniZinc model, can derive steps on how to reformulate it for maximum aggregation. Once a general theory, or at least close thereto, is formulated, implementing the aggregation and garbage collection framework in MiniZinc, automating the reformulations, is a natural next step.

4.9 CHAPTER CONCLUSION

Building online optimisation systems has in the past been a rather complex process. Essentially, the modeller does not build a general model of the problem, but an update-model, which reasons about how to take information from the previous session and the new data to solve the new version of the problem. The previous chapter laid out how to construct the update-model automatically from the original model using annotations. This chapter laid out concepts of systematised aggregation and garbage collection. First, a modeller specifies the conditions for when data, decisions, and (ultimately) objects are realised. From this, the model can be systematically reformulated so that it uses as few realised objects as possible, by introducing aggregate constructs that summarise their effects. To achieve this, we tackle the key problem of aggregation, where we reformulate constraints to make use of aggregate constructs.

Finally, many realised objects become garbage, i.e., unused and can be removed from future sessions. We rely on uniquely named objects to ensure consistency of information across sessions.

FAIRNESS IN ONLINE COMBINATORIAL OPTIMISATION

Manufacturing is a major application area for combinatorial optimisation that is undergoing significant changes, as shown by two important emerging paradigms: *shared manufacturing* [167] and *cloud manufacturing* [98]. Shared manufacturing refers to several companies jointly using (or owning) the same factory resources, subject to availability and demand. Cloud manufacturing more generally applies principles from cloud computing to the manufacturing domain, e.g., small manufacturers can pay the factory owner ‘by the hour’ or ‘per use.’ Many of the operational problems arising in manufacturing are discrete and NP-hard, making combinatorial optimisation a major candidate for addressing them.

From the emerging manufacturing paradigms, combinatorial problems arise that must consider multiple heterogeneous objectives, resulting from multiple customers or stakeholders (i.e., *agents*) sharing and competing over resources under constraints. The interactions between the desires (*utility functions*) and needs (*constraints*) of these agents are varied and complex, making it difficult to find good solutions that are valuable to all agents simultaneously. This indicates that fairness is becoming increasingly important in combinatorial optimisation. Solving these multiagent problems with purely utilitarian objectives, which simply maximise the sum of the agents’ utilities, can be arbitrarily unfair. Thus, ensuring some form of fairness among the agents is imperative.

Because of the uncertainties and the complexity present in the manufacturing industry, online combinatorial optimisation is a major candidate for addressing the operation of manufacturing: repeatedly, intractable problems need to be solved under strict time constraints while the problem definition is dynamically changed by agents (submitting new requests) or other external factors. The online setting raises an interesting phenomenon regarding fairness: it allows any imbalances in the solutions

to become exacerbated over time, but it also allows for solutions that eventually provide fairness over a period of time despite being arbitrarily unfair at any one specific time.

Despite this, the fairness literature often ignores problems that are dynamic and have underlying NP-hard structures. Likewise, the online combinatorial optimisation literature often ignores issues of fairness and balance. Note that manufacturing is only one example, which will be primarily used in this chapter, but the marriage of fairness and online combinatorial optimisation is important also for many other application areas, such as staff rostering and other types of combinatorial problems involving resource allocation.

This chapter considers the issue of *fairness* in dynamic multiagent combinatorial problems like the ones described above. More concretely, a novel and generic algorithmic framework is presented that can be used to experiment with (and ultimately achieve) both fairness and overall efficiency (i.e., utility) in dynamic multiagent optimisation problems. The framework is presented as an extension of the Online MiniZinc system introduced in previous chapters, but applies to any solver-independent high-level constraint-based modelling language. Notably, the framework is *problem independent*, that is, it can be instantiated easily for any dynamic combinatorial optimisation problem, as long as each agent can be assigned a scalar utility. Due to this generality, no formal fairness guarantees are given for the framework, as they strongly depend on the problem. Instead, the framework is proposed as a tool for *empirically* evaluating and increasing the fairness of a given dynamic problem. Further, due to the use of MiniZinc, the framework is *solver independent*, since modellers can choose the most appropriate off-the-shelf solver for the underlying models that need to be solved in each time period. Experiments show that this approach can yield fair allocations according to multiple fairness measures while maintaining high utility.

The rest of this chapter is structured as follows. Section 5.1 begins by presenting a case study, *multiagent job-shop scheduling*, which will be used as a running example in this chapter as well as to position this chapter in the broader fairness literature. Section 5.2 continues by presenting preliminaries and a review of the relevant literature. This leads to an investigation, in Section 5.3, of the opportunities and challenges of achieving fairness in online multiagent combinatorial optimisation, followed by an exploration of related work, in Section 5.4. This provides the motivation for a generic fairness framework, which is presented in full detail in Section 5.5. Then, Section 5.6 provides an experimental evaluation of the framework, and Section 5.7 provides a

discussion of the assumptions, challenges, and open questions of online multiagent combinatorial optimisation. The conclusions for the chapter are given in Section 5.8.

5.1 MULTIAGENT JOB-SHOP SCHEDULING

Let us first clarify what we refer to as multiagent combinatorial optimisation precisely is, by means of an example: a *multiagent job-shop scheduling* problem. This problem is an extended variant of the job-shop problem presented in Example 2.4 (page 37). To make our example more concrete, we assume the job-shop is a manufacturing plant, and a solution assigns a starting time (day, hour, minute) to each job.

Example 5.1 (Multiagent Job-shop Scheduling). From Example 2.4, recall that we are given a set of machines $MACs$, a set of jobs $JOBS$, and a set of tasks $TSKs$; for each job $j \in JOBS$ and task $t \in TSKs$, we are given a machine requirement req_{jt} , and a processing duration dur_{jt} ; we need to decide when to start each task (i.e., $start_{jt}$); and we want to minimise the makespan. For the multiagent version of this problem, we introduce a set of agents A who share the manufacturing plant: each job $j \in JOBS$ has an owner agent $own_j \in A$, and the machines are the shared resources. The shop can be oversubscribed, and we allow each job to be scheduled at a particular starting time or cancelled (i.e., none of its tasks are assigned a starting time). To define the utility function of each agent, given a solution $\mathcal{A}$, we will use the costs and revenue associated with the jobs they own as follows. Each day, a shipment leaves the manufacturing plant at hour h . Each job j has a deadline due_j specifying the day of the week by which its last task must have finished processing before hour h (so that the product can be shipped). The revenue rev_j of processing job j is only granted if j is not cancelled in $\mathcal{A}$. Agents must pay for the storage costs for any job j they own, from when the last task in j is finished until the time j is shipped, given as fee per hour per job. In addition, the owner of any job j that misses its deadline must also pay a tardiness cost $tard_j$ per day between j 's deadline and its shipment day.

The set of jobs requested by the agents to be processed is called the *demand*, and the set of jobs used in a solution is called the *supply*. The extra decisions are thus which jobs to use in the schedule (denoted by the variable use_j) and which ones to cancel. The goal of each agent k is to maximise their utility u_k . For our example, we will assume utility is defined as individual profit, that is, the sum for

$\mathcal{A}$ of the revenue of all completed jobs owned by agent k minus the sum of all the associated costs.

We will occasionally use this example to help clarify the preliminary concepts of fairness and balance.

5.2 BACKGROUND

Fairness is a vague term that has given rise to many different schools of thought. While utter unfairness is easy to spot, the line between unfairness and fairness is diffuse. This section discusses the most prominent schools of thought, concepts, and theories scattered in the literature.

5.2.1 Foundational Economic Theory

The study of fairness and balance has a long history in economics [141, 18]. In typical welfare economics, a central planner must allocate goods in an economy to a set of agents. Given the agents' utility functions, the question is what a best allocation is. To find it, one must combine a population's preferences, opinions, and interests to reach a collective decision. The problem is usually stated as picking a point from a utility possibility set. This is analogous, from the viewpoint of this thesis, to picking a solution from a solution space.

A *scheme* is a principled way of managing the different utility functions and arriving at a solution. For example, picking a solution at random is a scheme that disregards all agents' utilities. Recall that a multiagent COP (or maCOP) is a tuple $P = \langle X, D, C, u_1, \dots, u_n \rangle$, where n is the number of agents. Given a solution $\mathcal{A}$ to P , we can evaluate the utility of agent k as $u_k(\mathcal{A})$. In this chapter, we will often shorten this to simply u_k to denote the utility of agent k under any current solution to P . A scheme is formally defined as follows:

Definition 5.1 (Scheme). A *scheme* $\mathcal{S}$ is a function that takes a maCOP $P = \langle X, D, C, u_1, \dots, u_n \rangle$, transforms it into a COP $\mathcal{S}(P) = \langle X, D, C, f \rangle$, where f is defined by $u_1, \dots, u_n$ based on some principle of compromise or fairness, and then solves this COP.

The principles that a scheme can be based on will be discussed in the rest of this section. Because a scheme $\mathcal{S}$, given a maCOP P , returns an assignment $\mathcal{S}(P) = \mathcal{A}$,

it can notationally be used analogously to any assignment (e.g., $u_k(\mathcal{S}(P))$ returns the utility of agent k for solution $\mathcal{A}$ of P produced using $\mathcal{S}$).

To evaluate a solution, perhaps produced using a particular scheme, a *welfare metric* is often used, which summarises the supposed welfare of all the agents as a single number, to aid a decision-maker in arriving at a suitable conclusion. The simplest welfare metric is *utilitarian welfare*, sometimes called *utilitarian efficiency*, or simply *efficiency*, because it shows how efficiently the available resources are used to maximise the overall utility of a system or population of agents. The associated *utilitarian scheme* can then be achieved by maximising $\langle X, D, C, f \rangle$, where f is defined as $u_1 + \dots + u_n$.

Definition 5.2 (Utilitarian Welfare and Scheme). Given a maCOP P with components $\langle X, D, C, u_1, \dots, u_n \rangle$, the *utilitarian scheme* aims to maximise the COP

$$\text{Util}(P) = \langle X, D, C, f \rangle, \text{ where } f = \sum_{k=1}^n u_k,$$

which also denotes the *utilitarian welfare*, i.e., $\text{Util}(\mathcal{A}) = f(\mathcal{A})$ (using the above definition of f) for every solution $\mathcal{A}$ to P . Solutions with the highest possible utilitarian welfare are called *utilitarian optimal*.

Notice how we use Util to denote the scheme when given a maCOP P and the welfare when given a solution to P . Using the utilitarian scheme for the multiagent job-shop problem (Example 5.1) may lead to significant differences in profits across agents. This is because the demands of the agents most likely conflict, and favouring some agents over others leads to overall higher utility. Further, some agents may also have a greater potential for high profits, creating a bias towards them in the solutions produced. The online nature of the solving may also lead to these differences getting magnified over time. Some kind of fairness is hence often needed to, for example, not violate agreements or make all customers (agents) relatively satisfied. This can be achieved by using a different welfare metric to rank solutions.

One could, for example, consider the *Rawlsian welfare* metric—which has an analogous scheme, the *Rawlsian scheme*—where the goal is to maximise the utility of the worst-off agent. The Rawlsian welfare metric, sometimes also called the *egalitarian welfare metric*, is based on the philosophical work of economist John Rawls [123]. The associated *Rawlsian scheme* aims to maximise the worst-off agent.

Definition 5.3 (Rawlsian Welfare and Scheme). Given a maCOP P with components $\langle X, D, C, u_1, \dots, u_n \rangle$, the *Rawlsian scheme* aims to maximise the COP

$$\text{Rawls}(P) = \langle X, D, C, f \rangle, \text{ where } f = \min_{k=1}^n u_k,$$

which also denotes the *Rawlsian welfare*, i.e., $\text{Rawls}(\mathcal{A}) = f(\mathcal{A})$ (using the above definition of f) for every solution $\mathcal{A}$ to P . Solutions with the highest possible Rawlsian welfare are called *Rawlsian optimal*.

Note that Rawlsian welfare is oblivious to agents' utilities as long as they are above the minimum. One can enhance the Rawlsian scheme to address this issue by first maximising the worst-off, then the next worst-off, and so on, that is, by lexicographically maximising the utilities in ascending order. This is called the *leximin scheme* which attains *leximin optimality* (sometimes referred to as *leximin fairness*, *maximin fairness* or *max-min fairness*). Leximin optimality can equivalently be described as a solution in which increasing the utility of any agent necessarily leads to a decrease in the utility of an agent with an equal or lower utility. Another way to enhance the Rawlsian scheme is to optimise for Rawlsian welfare first, obtaining a lower bound on all utilities, then optimise for utilitarian welfare subject to this lower bound.

Welfare metrics are complemented by *inequality metrics*. Such metrics measure, in one way or another, how resources are distributed or spread out across the agents, indicating how imbalanced or unequal the distribution is. Inequality metrics are often used to study income inequality. However, they can be applied to any other utility function. An inequality metric is simply a measure of statistical dispersion over a set of values. Two inequality metrics used in this chapter were already discussed in Chapter 2, but they are worth reiterating here to see how they can be used to evaluate a solution. The first one is the *coefficient of variation*, defined as follows.

Definition 5.4 (Coefficient of Variation). Given maCOP $P = \langle X, D, C, u_1, \dots, u_n \rangle$, the *coefficient of variation* of a solution $\mathcal{A}$ to P , using the previously defined function CV over a list of values, is defined as:

$$\text{CV}(\mathcal{A}) = \text{CV}(u_1, \dots, u_n) = \frac{\sqrt{\sum_{k=1}^n \left(u_k - \frac{1}{n} \sum_{i=1}^n u_i\right)^2}}{\frac{1}{n} \sum_{i=1}^n u_i}.$$

The second metric we saw in Chapter 2 is the Gini coefficient, defined as follows.

Definition 5.5 (Gini Coefficient). Given maCOP $P = \langle X, D, C, u_1, \dots, u_n \rangle$, the *Gini coefficient* of a solution $\mathcal{A}$ to P , using the previously defined function Gini over a list of values, is defined as:

$$\text{Gini}(\mathcal{A}) = \text{Gini}(u_1, \dots, u_n) = \frac{\sum_{k=1}^n \sum_{i=k+1}^n |u_k - u_i|}{n \sum_{k=1}^n u_k}.$$

The main difference between a welfare metric and an inequality metric is that a welfare metric is linked to an overall utility, while an inequality metric is only concerned with the utilities of the agents *relative* to each other. For example, solely minimising inequality will probably lead to a zero utility allocation: no one gets anything, which automatically means that everyone gets the same amount. Thus, inequality metrics do not lend themselves to be used as schemes directly. Note that there are plenty of other inequality metrics; some are complex, such as the Hoover index [76] and the Theil index [153], while others are simple, such as the mean relative difference between the max and the min. Note that the aim of this chapter is to propose a framework that supports the experimental evaluation of fairness and efficiency considerations, rather than exhaustively evaluate the different fairness measures. Thus, this chapter will only focus on two inequality metrics (the coefficient of variation and the Gini coefficient) and four welfare metrics (utilitarian and Rawlsian welfare defined above, plus the proportionality coefficient and Nash welfare, which we discuss later).

Lastly, we will discuss a property prevalent in economics: *Pareto optimality*, sometimes called *Pareto efficiency*, which states that no utility can be increased without simultaneously decreasing another utility.

Property 1 (Pareto Optimality). Given maCOP $P = \langle X, D, C, u_1, \dots, u_n \rangle$, a solution $\mathcal{A}$ is *Pareto optimal*, denoted $\text{Pareto}(\mathcal{A})$, if and only if, for all agents $k \in 1..n$:

$$u_k(\mathcal{A}') > u_k(\mathcal{A}) \rightarrow \exists i \in 1..n, i \neq k: u_i(\mathcal{A}') < u_i(\mathcal{A}) \quad \forall \mathcal{A}' \in \text{sols}(P)$$

Note that the utilitarian scheme and the leximin scheme adhere to the Pareto optimality property, i.e., all utilitarian optimal and leximin fair solutions are also Pareto optimal. However, the Rawlsian scheme does not adhere to Pareto optimality (unless combined with, e.g., utilitarianism) since it ignores the non-worst-off agents. For a multiagent COP $P = \langle X, D, C, u_1, \dots, u_n \rangle$, we say that a solution $\mathcal{A}$ *Pareto dominates* another solution $\mathcal{A}'$ if, for all $1 \leq k \leq n$, $\mathcal{A}(u_k) \geq \mathcal{A}'(u_k)$, where at least one inequality is strict.

Before reviewing the rest of the literature, let us try to understand why efficiency is at odds with fairness in general. Let us look at the standard utilitarian scheme a bit closer. As mentioned before, this scheme maximises efficiency and ignores fairness. The problem is simply solved by maximising the total summed utility obtained by each agent from using the shared resources. For example, if we can decrease the worst-off agent by a utility of 1 and increase the best-off by a utility of 2, then we will do so in the utilitarian scheme. This approach may lead to highly imbalanced allocations of the shared resources and unfair utility distributions because it may favour specific agents at the expense of others. Factors that can lead to this are, for example, when agents declare a lower utility from using the shared resources (thus, they may never be allocated any), while those declaring disproportionately larger utility may be allocated all. Other factors, such as search heuristics or the use of incomparable utilities across agents, can also result in imbalances.

5.2.2 Multi-Objective Optimisation

Optimisation with fairness considerations can be viewed through the lens of multi-objective optimisation, which is an active field of research and has many applications [32, 155]. However, the prime focus of multi-objective optimisation is not about balancing or ensuring fairness between the several objectives, but simply about optimising multiple objectives where there is (often) no clear hierarchy among them. As such, every Pareto optimal solution can be seen as an optimal compromise, where improving any of the objectives necessarily comes at the cost of one of the others. In this section, we assume (without loss of generality) that all objectives are to be maximised.

Described in [32], common concepts in multi-objective optimisation include the *Pareto frontier*, the *ideal point*, and the *nadir point*, which are used to help a human decision-maker arrive at a suitable compromise, as well as *a priori*, *interactive* (also called *progressive*), and *a posteriori approaches*, which are the three main ways of addressing multi-objective problems. These concepts will be presented slightly differently in this chapter, using this thesis's notation.

A *Pareto frontier* is the set of all Pareto optimal solutions and is often created for humans to consider multiple solutions simultaneously and converge on a compromise.

Definition 5.6 (Pareto Frontier). Given a maCOP $P = \langle X, D, C, u_1, \dots, u_n \rangle$ the *Pareto frontier* is defined as $\text{PF}(P) = \{\mathcal{A} \mid \mathcal{A} \in \text{sols}(P), \text{Pareto}(\mathcal{A}(P))\}$.

The Pareto frontier is easy to visualise when there are only two objectives: as shown by the solid dots in Figure 5.1. Note that in conventional multi-objective optimisation, the best choice is usually one of the Pareto optimal solutions. Because of additional fairness considerations, however, it may be more desirable to pick a Pareto non-optimal solution with a good balance of objective values than a Pareto optimal solution whose objective values are less balanced.

Two additional concepts that can help one evaluate the suitability of a compromise are the *ideal* and *nadir points*, defined as follows.

Definition 5.7 (Ideal Point). Given a maCOP $P = \langle X, D, C, u_1, \dots, u_n \rangle$, the *ideal point* gives a value for each of the objectives $u_1, \dots, u_n$ such that the value of u_k is its optimal when solving a COP $\langle X, D, C, u_k \rangle$. Formally, this can be expressed as an n -tuple:

$$\begin{aligned} \langle u_1, \dots, u_n \rangle &= \langle d_1, \dots, d_n \rangle \\ \text{where } d_k &= \max_{\mathcal{A} \in \text{sols}(X, D, C, u_k)} \mathcal{A}(u_k) \quad \forall k, 1 \leq k \leq n \end{aligned}$$

Note that the ideal point is often not a valid solution. If it were, the problem would not be harder than solving a single-objective problem.

Definition 5.8 (Nadir Point). Given a maCOP $P = \langle X, D, C, u_1, \dots, u_n \rangle$, the *nadir point* gives a value for each of the objectives $u_1, \dots, u_n$ such that the value of u_k is its worst across all Pareto optimal solutions. Formally, this can be expressed as an n -tuple:

$$\begin{aligned} \langle u_1, \dots, u_n \rangle &= \langle d_1, \dots, d_n \rangle \\ \text{where } d_k &= \min_{\mathcal{A} \in \text{PF}(P)} \mathcal{A}(u_k) \quad \forall k, 1 \leq k \leq n \end{aligned}$$

The nadir point is, similarly, often not a valid solution. Figure 5.1 shows the Pareto frontier, ideal point, and nadir point in a bi-objective problem. Notice the bounding box obtained from the nadir and ideal points, which can be seen as the solution subspace containing the desirable solutions.

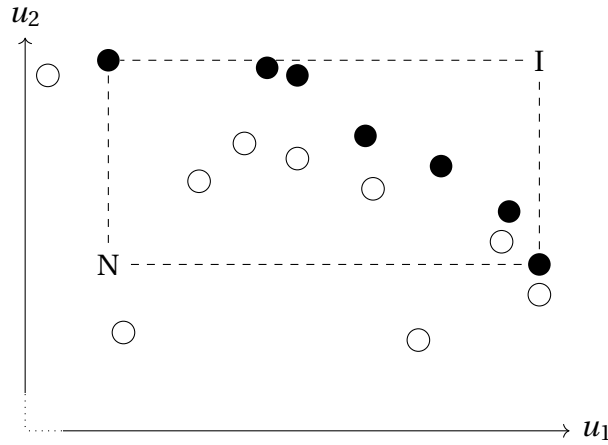


Figure 5.1: A visualisation of the Pareto frontier (black solid dots) of a maximisation problem along with the ideal (I) and nadir (N) points and Pareto non-optimal solutions (hollow dots) of a bi-objective (u_1 and u_2) optimisation problem

There are generally three kinds of approaches to solving a multi-objective optimisation problem: *a priori approaches*, *interactive approaches*, and *a posteriori approaches*. In an *a posteriori* approach, a set of (often Pareto optimal) solutions is presented to a decision-maker, who must pick the most desirable compromise from this set. Such approaches can also use information about the ideal and nadir points. In an *interactive approach*, a decision-maker participates in the solving process, guiding the search. These two kinds of approaches are less applicable to the problems of concern in this thesis than *a priori* approaches, which will be described next. This fundamentally comes down to the fact that these two methods can be incredibly time-intensive, especially for NP-hard problems, which is undesired in online optimisation under strict time constraints.

In an *a priori approach*, the relative importance of each objective is established before solving commences. This approach often requires experimentation with the relative importance to obtain solutions with reasonable compromises. Note that schemes, as defined, are *a priori* approaches. The most common *a priori* approaches are *scalar approaches*, where the multiple objectives are transformed into a single objective, for example, by assigning a weight to each objective and maximising the sum of objectives. Notice that this is equivalent to the utilitarian scheme if all weights are equal. See [43] for examples of scalar approaches, in which also other popular *a priori* approaches, based on converting objectives into constraints, are discussed. Such a constraint would specify a minimum bound of its associated objective. If all minimum bounds can be simultaneously satisfied, then the objective would specify some

overarching ‘nice-to-have’ goal, such as utilitarian efficiency. A prominent example of this includes ε -constraint method [71], in which all objectives are transformed into lower-bound constraints except the most important one, which is maximised on its own subject to all the constraints.

Often, however, all minimum bounds cannot be obtained simultaneously, or it is too computationally prohibitive to do so. In such cases, the constraints need to be *soft constraints* [57], where the objective is to satisfy them as much as possible, as opposed to *hard constraints*, which must always be satisfied. There are generally two approaches to addressing soft constraints. The first one is to try to satisfy as many soft constraints as possible (often referred to as *MaxCSP*). Even further, the soft constraints could have different priorities (i.e., weights), and the objective is to maximise the number of satisfied soft constraints according to the priorities (e.g., the sum of the weights of the satisfied constraints). This is often known as *weighted CSP* (see, e.g., [92]). The second approach is to create a *violation function* [131, 145] (also called a *penalty function*) for each soft constraint, denoting how far off a solution is from being satisfied by the constraint. The goal is often to minimise the total violation (e.g., how much the constraint are violated in total). For an overview of soft constraint approaches and an extension of MiniZinc to handle soft constraints, the reader is referred to [134].

The framework presented in this chapter is based on a scalar approach because it is flexible and easy to formulate. The other a priori approaches could likewise be applied to the presented framework, but are left to future work.

5.2.3 Social Choice Theory, Game Theory, and Decision Theory

Social choice theory—building on economics, mathematics, and political theory—regards questions on how to combine individual preferences and interests towards a collective choice [23]. Examples include how to design and evaluate voting rules and find stable matchings between two groups of people with preferences over each other. *Game theory* regards the study of mathematical models of *games*, in which multiple (often assumed rational) agents take turns acting, or act simultaneously, to each achieve an (often selfish) goal. In game theory, the behaviours and strategies of agents are often studied in depth. Social choice theory and game theory are closely related and share many concepts. Another closely related field is *decision theory*, which combines probability theory with a theoretical view of making decisions, where only one decision-maker (stakeholder) is considered.

Unsurprisingly, fairness plays a role in social choice theory but can also appear in game theory. Fairness appears in *cooperative* (also called *coalitional*) game theory, where the Shapley value [143] implements certain fairness axioms and determines payoffs of a group of agents as their mean marginal utility contribution to a coalition's value. These concepts have been applied to scheduling [144] and fair resource allocation [165]. A key difference to the approach presented in this chapter is that cooperative game theory assumes a jointly obtained profit of resource providers that needs to be divided fairly. This is not the case for many combinatorial problems, such as those based on a shared factory that serves each manufacturer independently.

Similarly, [114] suggests using social choice theory (in particular, *ordinal voting*, i.e., where each agent ranks the options from most preferred to least preferred) to find agreement over the ordering of jobs submitted to a single machine. In contrast to the applications of this chapter, no parallelism or combinatorial constraints are considered. *Condorcet voting* can be applied to constraint-based models [135], a kind of ordinal voting system, which results in the option that is majority-preferred above every other option when compared one-on-one. It has been shown to mitigate bias with utilitarian objectives and to obtain fairer solutions. However, that approach tends to strongly favour majority decisions at the expense of a few that pay a high price. Thus, fairness among agents is not considered and optimised explicitly. Note that voting systems are generally not applicable to the context of this chapter (i.e., time-constrained online solving) as they are very computationally prohibitive: they require the entire solution space to be enumerated and then have the agents rank every solution from most preferred to least preferred.

In games where agents can reveal information about their preferences to the group, for example, vote for an outcome or select a good to be allocated, an agent can either be truthful—revealing their actual, true preference—or lie to some degree. In some cases, it could be more lucrative for an agent to lie than to be truthful, that is, concocting a *strategy* can pay off. A game, or a scheme solving a game, is called *strategy-proof* [23] if no agent can do any better lying than being truthful. In multiagent combinatorial optimisation, strategies are manifested through the utility functions. Therefore, strategy-proofness can be defined in our context as follows:

Property 2 (Strategy-proofness). Let $P = \langle X, D, C, u_1, \dots, u_n \rangle$ be any maCOP where all agents report their utilities truthfully, and let $\tilde{P} = \langle X, D, C, u_1, \dots, \tilde{u}_k, \dots, u_n \rangle$ be a maCOP identical in every way except that agent k misrepresents their utility function, providing a misleading one $\tilde{u}_k$ instead of their truthful one u_k . A scheme

$\mathcal{S}$ is strategy-proof if and only if agent k does not gain utility from misrepresenting their utility, i.e., $u_k(\mathcal{S}(P)) \geq u_k(\mathcal{S}(\tilde{P}))$ for all such P and $\tilde{P}$.

The simplest strategy-proof scheme is a randomisation scheme, where a solution is picked randomly, disregarding the agents' utilities. Ensuring strategy-proofness is challenging when there are requirements for fairness and efficiency, even in highly specific games and situations. A classic example of a (non-random) strategy-proof scheme is majority voting [23], defined as follows: there are two options (i.e., solutions), every agent votes for one or the other option (i.e., provides utility 1 for one option and utility 0 for the other), and the option that gets the majority of the votes is the one selected (i.e., utilitarianism). This scheme is strategy-proof because voting for the option one prefers least of the two will only increase the chances of that one being selected. Voting insincerely cannot increase the chance that the option one truly prefers is selected. A classic example of a non-strategy-proof scheme is plurality voting [23], defined as follows: there are three or more options, every agent votes for exactly one option, and the option that gets the largest number of votes wins and gets selected. Suppose there are three options: a , b , and c , where one of the first two is expected to win, and the third, c , is expected to not come near winning. If an agent prefers c the most, but b over a , then voting for c will likely have little effect on which option will win, but by voting for b , they can increase the likelihood of a not winning. This shows that plurality voting is not strategy-proof.

In decision theory, the concept of *regret* [13] appears, defined as the difference between the option taken and the best possible option. With perfect information and rational agents, there is never any regret. However, with uncertainty, regret becomes relevant. For example, one can try to minimise expected regret when devising a strategy. Since multiagent combinatorial optimisation, as defined in this thesis, contains no interactivity on the part of the agents, regret could only appear when misreporting the utility function, i.e., when concocting a strategy (assuming that the scheme used to solve the problem is not strategy-proof). Recall that the agents provide the utility functions before the solving starts. For any particular agent, when providing the utility function, there is uncertainty as to what utility functions the other agents will provide and how the problem instance will look. Hence, regret could appear if an agent finds that there was another, more lucrative misreporting of its utility function than what it provided, given what the problem instance and the utility functions provided by the other agents.

5.2.4 Fair Division and Resource Allocation

The *fair division* field [164] emerges when a problem in game theory or social choice theory is an allocation problem, meaning (divisible or indivisible) goods or resources need to be assigned to one agent and excluded from the others. Fair division investigates allocating resources so that each agent perceives their share as fair or adequate, given their subjective valuations [105]. The agents can either compete for the resources [165] or own them [1]. While there has been a significant amount of research in the area of fair division, most approaches assume very lightly constrained problems, where the underlying problem structure is computationally easy (often polynomial) once the fairness aspect is disregarded. In fact, these problems often have a single constraint (which relates to the budget); they lack the more complex constraints typical in NP-hard problems, which include temporal precedences, deadlines, storage constraints, and mutual exclusions. Furthermore, many fair division protocols, such as divider-chooser methods [6] or sealed-bid division [147], work without an arbiter, which is generally not possible for problems where quantifying the utility for the agents is itself an NP-hard problem. Nevertheless, our approach draws from the rich literature in this area and generalises its ideas to the framework level.

Many desirable properties have been established in the fair division field. Some of these can be generalised to the combinatorial optimisation context with fairness considerations. One such property is *proportionality* [147], which states that each agent is allocated goods with a resulting proportional utility relative to its ideal utility and the number of agents. If the number of agents is n , then we say that a *proportional* allocation is reached if every agent k achieves at least a utility of $\text{ub}^*(u_k)/n$.

Property 3 (Proportionality). Given a maCOP $P = \langle X, D, C, u_1, \dots, u_n \rangle$, a solution $\mathcal{A}$ to P is *proportional* if each agent derives one n th of its maximum utility, that is:

$$u_k(\mathcal{A}) \geq \frac{\text{ub}^*(u_k)}{n} \quad \forall k \in 1..n$$

If the inequality is strict for every agent, the allocation is said to be *super-proportional*. These definitions for schemes follow straightforwardly.

For example, suppose there are three agents, where the first derives a utility of 10 out of a maximum of 30, the second derives 5 out of 15, and the third derives 4 out of 10, then the allocation would be proportional. This is because one third of 30 is 10, one third of 15 is 5, and one third of 10 is less than 4. If the third agent were

to derive a utility of 3 instead, all else being equal, then the allocation would not be proportional because 3 is less than a third of 10.

Proportionality can be generalised to a metric, called *relative concession* in the literature (see, e.g., [62]). The concept is often defined as the percentage from the ideal utility of an agent. This thesis defines a similar concept, called the proportionality coefficient, as follows:

Definition 5.9 (Proportionality Coefficient and Scheme). Given a maCOP $P = \langle X, D, C, u_1, \dots, u_n \rangle$, the *proportionality scheme* is obtained by maximising the COP

$$\text{Prop}(P) = \langle X, D, C, f \rangle \text{ where } f = n \min_{k=1}^n \frac{u_k}{\text{ub}^*(u_k)},$$

which also denotes the *proportionality coefficient*, i.e., $\text{Prop}(\mathcal{A}) = f(\mathcal{A})$ (using the above definition of f) for every solution $\mathcal{A}$ to P .

We multiply by n to obtain a ratio from exact proportionality. That is, the metric indicates to which degree all agents have achieved a proportional share, i.e., the worst-case utility conceded (or gained) relative to a proportional allocation. This can be seen as a Rawlsian aggregation of relative utilities, scaled by the number of agents. For example, if the proportionality coefficient is 1, then every agent received at least (with at least one precisely) their proportional share. If the proportionality coefficient is 0.5, then every agent received at least (with at least one precisely) half of their proportional share, meaning that there is a (worst-case) 50% concession from exact proportionality (or, equivalently, a worst-case $\frac{1}{2n}$ from the ideal utility). If the proportionality coefficient is 1.75, then every agent received at least (with at least one precisely) 75% more than their proportional share. Notice the undefinedness when an agent's maximum attainable utility $\text{ub}^*(u_k)$ is 0 due to a division by zero. In many situations, this undefinedness must be addressed. In this thesis, such cases are defined to be 0.

Another concept from fair division is *envy-freeness*, introduced in [60]. An allocation is *envy-free* if no agent desires someone else's allocation more than their own. For example, consider an allocation problem with two agents (agent 1 and 2) and three items a , b , and c . For each item, the agents have attached the following utility value:

	item a	item b	item c
agent 1	1	1	0
agent 2	0	0	3

Suppose that agent 1 is allocated item a and agent 2 is allocated items b and c . While such an allocation would violate Pareto optimality, it would satisfy envy-freeness. Agent 2 has utility 3, and by swapping, they would get utility 0; similarly, agent 1 has utility 1, and by swapping, they would get utility 1. Hence, no agent is better off by swapping their allocation with someone else: the solution is envy-free.

Envy-freeness is hard to generalise to the multiagent combinatorial optimisation context because the nature of the decisions is more complex in this general setting. In an allocation problem, the decisions are simple: for each item, we need to decide whom to allocate it to. Thus, in a solution, there is a notion of an agent *possessing* some resources to the exclusion of all others. With such a structure, it is clear what it means for one agent to *envy* another: one can swap what they possess and evaluate the utility of such a hypothetical scenario, seeing if that gives any agent higher utility. In general multiagent combinatorial optimisation, a decision could (with varying degrees) favour many agents simultaneously but still not be possessed by any one agent, making the concept of ‘envy’ hard to define. Consider, again, the voting examples we discussed earlier in this section. While an agent may be unhappy with the outcome of the voting scheme, no agent possesses the outcome, so there is no clear definition of what it would mean for that agent to ‘envy’ another agent. Important to note is that envy-freeness in fair division is not based on others’ utility, so envying that the outcome of a voting scheme made someone else happier than oneself would not be a reasonable definition of envy.

Limiting the definition of envy-freeness to decisions that directly regard possession of resources with the exclusion of others might not solve the issue, as such decisions may not even apply to another agent. Consider a scheduling problem where each agent submits jobs to be scheduled. For each job, resources and time must be *allocated* to accommodate the job. Notice the allocation problem within this scheduling problem; hence, there is a notion of possession. In order to evaluate envy, one must first understand if an agent’s utility function can be applied to another agent’s allocation or situation. This is because each agent may have unique constraints and circumstances (e.g., resource and time requirements of the jobs), so simply swapping allocations may lead to infeasible situations, which violate constraints. If it is not feasible for an agent k to swap with another agent i , then we could reasonably define them as not envying each other. However, we could perform a series of swaps and perhaps arrive at a feasible solution where agent k has i ’s allocation. This creates a computational hurdle to checking envy-freeness, in addition to the fact that the underlying problem is likely NP-hard.

5.2.5 Bargaining Theory and Properties of Schemes

Various desirable properties (sometimes called axioms) have been established in the literature to study and compare schemes. These have been intensively studied in *cooperative bargaining theory* [154], which is a subfield of game theory that is concerned with the mathematics of cooperating agents with disagreements that threaten the continuation of the cooperation if not properly addressed via bargaining (i.e., suitable schemes). Since there is no agreed-upon notion of fairness, these properties are far and wide, and because many are relatively restrictive, no single scheme can satisfy all of them. In [18], *Pareto optimality*, *symmetry*, *affine invariance*, and *independence of irrelevant alternatives* are described. These concepts will be presented slightly differently in this chapter, using this thesis's notation. We have already seen Pareto optimality. The other properties are as follows.

Property 4 (Symmetry). Let P and P' be any two maCOPs such that P' is obtained by permuting the list¹⁸ of variables, constraints, or utility functions of P . A scheme $\mathcal{S}$ has the *symmetry* property if and only if $\mathcal{S}(P) = \mathcal{S}(P')$ holds for all such P and P' .

In other words, a scheme is symmetric if the order of the variables, constraints, and utility functions are irrelevant.

Property 5 (Affine Invariance). Let P and P' any two maCOPs, with respective components $\langle X, D, C, u_1, \dots, u_n \rangle$ and $\langle X, D, C, u'_1, \dots, u'_n \rangle$, such that the utility functions $u'_1, \dots, u'_n$ are affine transformations of $u_1, \dots, u_n$, i.e., we have $u'_k = q_k u_k + b_k$, where q_k and b_k are constants. A scheme $\mathcal{S}$ has the *affine invariance* property if and only if $\mathcal{S}(P) = \mathcal{S}(P')$ holds for all such P and P' .

An affine invariant scheme will thus not be affected by agents that inflate, deflate, or otherwise offset their utility function.

For the next properties, a notion of *extending* a maCOP must be established. When a maCOP is extended, the solution space increases without removing any existing solutions and their utility values. For example, this could translate to adding more

¹⁸Note that the variables and constraints have been defined as (unordered) sets, not as lists, throughout this thesis, making permuting them appear nonsensical. However, in most if not all practical situations, every set must be ordered in one way or another to enable, e.g., a computer to run the scheme in question, as all variables and constraints cannot be addressed simultaneously. Thus, the permutation regards the ordering of the solving-specific representation of the variables and constraints.

resources in a resource allocation problem or adding more jobs in a scheduling problem. Formally, the concept can be defined as follows.

Definition 5.10 (maCOP Extension). A maCOP $P^+ = \langle X, D^+, C^+, u_1^+, \dots, u_n^+ \rangle$ is an *extension* of a maCOP $P = \langle X, D, C, u_1, \dots, u_n \rangle$ if and only if all the following conditions hold.¹⁹

- (1) The extension contains at least all the original domain values, i.e., for every $x \in X$, $D^+(x) \supseteq D(x)$;
- (2) every solution to the original problem is a solution to the extension, e.g., $\text{sols}(P) \subseteq \text{sols}(P^+)$; and
- (3) every solution shared between the maCOPs have the same utilities in the original problem and the extension, i.e., for every $\mathcal{A} \in \text{sols}(P)$ and every agent k , $\mathcal{A}(u_k) = \mathcal{A}(u_k^+)$.

Using the definition, the final property can be defined formally.

Property 6 (Independence of Irrelevant Alternatives). Let $\mathcal{S}$ be a scheme. Let P and P^+ be any two maCOPs such that P^+ extends P with $\mathcal{S}(P^+) \in \text{sols}(P)$. The scheme $\mathcal{S}$ has the *independence of irrelevant alternatives* property if and only if $\mathcal{S}(P) = \mathcal{S}(P^+)$ holds for all such P and P^+ .

In other words, a scheme independent of irrelevant alternatives will not result in a different solution if disallowing a solution that it would not have picked anyway.

Note that similar properties to those above have also been established for welfare metrics (rather than schemes), to be able to compare and (possibly) conclude what welfare metric is best suited for a particular goal or situation. See [154, 90] for an overview of other, less prevalent properties in the literature concerning schemes and schemes satisfying these.

Because utilitarian efficiency can be seen as a general measure of system efficiency and overall resource utilisation, comparing any fairness scheme with it is interesting. Naturally, the efficiency will be lower (or at most equal) than when using the utilitarian scheme. The difference in efficiency between a fairness scheme and the utilitarian scheme can be thought of as *the price of fairness* [18], usually defined as follows:

¹⁹One could define this concept more broadly, where extending the set of variables is also permitted. However, since only extensions of the domains and relaxations of the constraints are necessary for this thesis, we opt for this simpler definition.

Definition 5.11 (Price of Fairness). Let P be any maCOPs and $\mathcal{S}$ be a scheme. The price of fairness is the relative difference in utilitarian welfare from the utilitarian scheme, i.e., as follows:

$$\frac{\text{Util}(P) - \text{Util}(\mathcal{S}(P))}{\text{Util}(P)}.$$

It provides a relative measure of how much in terms of utilitarian welfare is sacrificed when attaining greater fairness.

Let $\text{ub}^*(u_k)$ denote the maximum utility possible for agent k across all feasible solutions, i.e., $\text{ub}^*(u_k) = \max_{\mathcal{A} \in \text{sols}(P)} u_k(\mathcal{A})$, where P is the problem being solved. This will be called the *ideal utility* of agent k , while $\mathcal{A}(u_k)$ will be called the *actual utility* of agent k , for a given solution $\mathcal{A}$. For clarity, a subscript (e.g., s) can be used (i.e., $\text{ub}_s^*(u_k)$) to denote the ideal utility of maCOP P_s .

Finally, let us discuss two schemes coming from cooperative bargaining theory field as they are now quite well-known in the broader fairness literature. The first scheme is the *Nash bargaining solution*, developed by Nash [106], and the second one is the *Kalai-Smorodinsky bargaining solution*, developed by Kalai and Smorodinsky [85]. These concepts will, again, be presented slightly differently in this chapter, using this thesis's notation, and we will also derive welfare metrics from them. The Nash bargaining solution, or the *Nash scheme* as it will be referred to throughout the rest of this thesis, is based on a standard of comparison where the percentage change in an agent's utility when allocating resources corresponds directly to the overall objective to maximise. This means that transferring resources from one agent to another is justified if the receiving agent gains a larger percentage of utility than the other agent loses. We can define this scheme in the context of this thesis as follows:

Definition 5.12 (Nash Scheme and Welfare). Given a maCOP $\langle X, D, C, u_1, \dots, u_n \rangle$ the *Nash scheme* is obtained by maximising the COP

$$\text{Nash}(P) = \langle X, D, C, f \rangle \text{ where } f = \prod_{k=1}^n u_k,$$

which also denotes the *Nash welfare*, i.e., $\text{Nash}(\mathcal{A}) = f(\mathcal{A})$ (using the above definition of f) for every solution $\mathcal{A}$ to P . Solutions with the highest possible Nash welfare are called *Nash optimal*. Note that the scheme can be equivalently formulated with $f = \sum_{k=1}^n \log u_k$.

The Nash scheme satisfies the properties of Pareto optimality, symmetry, affine invariance, and independence of irrelevant alternatives (as proved in [106]).

The second scheme from cooperative bargaining theory is the Kalai-Smorodinsky bargaining solution, or the *Kalai-Smorodinsky scheme* as it will be referred to throughout the rest of this thesis. It is founded on the principle of giving every agent k the largest possible fraction of their ideal utility $ub^*(u_k)$ as long as every agent gets the same fraction. That is, it maximises utility subject to everybody having the same relative concession. We can define this scheme with the help of the formulation of [27], in the context of this thesis, as follows:

Definition 5.13 (Kalai-Smorodinsky Scheme). Given a maCOP $\langle X, D, C, u_1, \dots, u_n \rangle$ the *Kalai-Smorodinsky scheme* is obtained by maximising the COP

$$KS(P) = \langle X, D, C, f \rangle, \text{ where } f = \begin{cases} \sum_{k=1}^n u_k, & \text{if } \exists b, 0 \leq b \leq 1, \forall i \in 1..n: \\ & u_i = b \cdot ub^*(u_i), \\ 0, & \text{otherwise.} \end{cases}$$

Optimal solutions to the Kalai-Smorodinsky scheme are called *Kalai-Smorodinsky optimal*.

The Kalai-Smorodinsky scheme satisfies Pareto optimality, symmetry, and affine invariance (as proved in [85]). Notice that this is similar to the Nash scheme, except that independence of irrelevant alternatives is exchanged for monotonicity properties (see [85] for details).

The Kalai-Smorodinsky scheme does not lend itself quite easily to be formulated as a welfare metric because it is utilitarian welfare under equal relative concession and 0 otherwise. However, notice its similarity to the proportionality coefficient and the proportionality scheme introduced above, as they both operate on relative concession. Hence, the proportionality coefficient can be viewed as a potential welfare-equivalent of the Kalai-Smorodinsky scheme.

5.3 FAIRNESS IN ONLINE COMBINATORIAL OPTIMISATION

This section formulates the novel connection between fairness, online decision-making, and combinatorial optimisation. The already established concepts of the related fields discussed in the previous section are applied to multiagent online combinatorial optimisation where possible, and new concepts are defined along the way.

In doing so, we highlight many of the challenges of (and possibilities for) addressing fairness in this context.

A dynamic multiagent combinatorial optimisation problem can be expressed as a *dynamic multiagent COP* (dmaCOP), that is, a (possibly infinite) tuple $Q = \langle P_1, P_2, \dots \rangle$ of maCOPs where each maCOP P_s can be expanded as $\langle X_s, D_s, C_s, u_1, \dots, u_n \rangle$. Note that, for simplicity, we assume the set of n agents is fixed across all sessions, i.e., no agents can join or leave, and their utility functions are fully defined a priori, which is why the session subscript is omitted from utility functions. When solving online, two separate kinds of utility emerge from the utility functions as follows. Recall that, at session s , we first observe the problem milieu, creating the current instance P_s with its objects, data points, and incumbent assignment, and we then solve, creating a solution $\mathcal{A}_s \in \text{sols}(P_s)$. Before executing any solution to P_s , i.e., at the start of session s , the (inter-session) *incumbent utility* U_k^s , describes the total, accumulated utility achieved by agent k in the problem milieu P_s . In a solution $\mathcal{A}_s$, the (intra-session) *marginal utility* for agent k , for $1 \leq k \leq n$, denoted u_k^s , defines the utility difference from the incumbent solution. The *ideal incumbent utility* at the start of session s is denoted $\text{ub}^*(U_k^s)$, with the initial utility being zero, that is, $U_k^0 = 0$, for all agents k , $1 \leq k \leq n$, and the *ideal marginal utility* is denoted $\text{ub}^*(u_k^s)$. Under perfect execution, for a series of maCOPs $P_1, \dots, P_s$, the relationships between incumbent and marginal utilities, for each agent k , is $U_k^s = \sum_{i=1}^s u_k^i$ for actual and $\text{ub}^*(U_k^s) = \sum_{i=1}^{s-1} \text{ub}^*(u_k^i)$ for ideal.

5.3.1 Dynamic Multiagent Job-Shop Scheduling

Let us extend the multiagent job-shop scheduling problem (Example 5.1, page 145) to a dynamic one.

Example 5.2 (Dynamic Multiagent Job-shop Scheduling). Recall, from Example 5.1, that we are given a set of machines $MACs$, a set of jobs $JOBS$, a set of tasks $TSKs$, a set of agents A , shipment hour per day h , and storage cost per hour per job fee ; for each job $j \in JOBS$, we are given its owner agent own_j , deadline due_j , completion revenue rev_j , and tardiness cost per day $tard_j$; additionally, for each task $t \in TSKs$, we are given a machine requirement req_{jt} , and a processing duration dur_{jt} . Recall also that we must decide when to start each task (i.e., $start_{jt}$), which jobs to use in the produced schedule (i.e., use_j), and we want to minimise the makespan.

In the dynamic version of this problem, we will assume that the set of machines $MACs$ and the set of agents A are static and known a priori, but new jobs can arrive once a week and be added to $JOBS$. Further, we assume that the uncertainty in execution is negligible. Hence, we solve the problem online, with each session corresponding to a single week. Each session s considers the jobs (of possibly varying quantity and duration) requested by the agents to be processed in week s (called the *demand* of week s). Cancelled jobs are not carried over to the next week but removed from $JOBS$; if agents need them, they can add new jobs to the next period. In each session s , the goal of each agent k is to maximise their marginal utility u_k^s for that session. Recall that we assume utility is individual profit.

The above case study will be used in the rest of the chapter as a concrete example for applying the theories and concepts in the fairness literature to the online multiagent combinatorial optimisation context. Note, however, that the framework presented later in this chapter is problem-independent. Further, while the time periods (weeks) are independent in the example, they can be easily extended to be dependent by, for example, allowing for overlap between periods (as was the case in previous chapters).

5.3.2 Challenges to Addressing Fairness

The dynamic multiagent job-shop scheduling problem (Section 5.1) will be used as an example in this subsection to illustrate different challenges when addressing fairness in dynamic multiagent combinatorial optimisation. Although the problem is solved online, with each session spanning a week, it may be too restrictive to expect a completely fair and balanced solution every week. Allowing unfairness within each week and instead expecting fairness when evaluating across multiple weeks may be more reasonable. It allows the solving to be more flexible, ideally increasing fairness without much decrease in overall efficiency by taking advantage of the differences in demands from one week to another.

Conflicting Demands. The most apparent challenge comes from agents with conflicting demands, such that favouring one agent leads to an overall higher total utility than favouring any other agent. In our example, this might be due to large values returned by the utility functions of a particular agent relative to the agents with which it has conflicting demands. An alternative challenge occurs when an agent requires so many resources that satisfying its demands yields solutions with low utility for all other agents, leading to overall low utility. In our example, this might be due to an agent submitting large jobs that occupy the machines for significant amounts of time,

blocking the jobs of many other agents. The conflicting demands can be addressed in many ways, as reflected by the literature. It is a matter of delicately balancing all the objectives and priorities.

Uncertainty Issues. The dynamic nature of these problems complicates things when it comes to fairness since we must decide what the fairest and best scheduling is each week without knowing the future. Let us assume that, after several weeks (e.g., half a year), we must look back and evaluate how fair the generated schedules are. This introduces a dissonance: decisions need to be made rather myopically (session by session) while evaluation happens over a more extended period (over multiple sessions). Simply maximising fairness within each session will likely lead to lower utilitarian welfare than otherwise possible due to the price of fairness and because each session can look vastly different. Assume, for example, that one agent, owing to conflicting demands, cannot get high utility in the current session but can in the next session. In that case, ignoring this agent in the current session can make sense while favouring it in the next session. This allows for more flexibility, i.e., we can be intentionally unfair now but compensate for this later. Maximising fairness within each session ignores this flexibility, leading to a higher price of fairness: lower utilitarian welfare. Nevertheless, knowing the best time to be unfair and how to be unfair is difficult. The online nature of the solving also imposes a risk of unfairness being magnified over time.

Intractability Issues. In combinatorial optimisation, problems are assumed to be NP-hard. This means that it is intractable to solve such problems to optimality, especially when there are strict time constraints (which is common in online optimisation). This has two major consequences. First, Pareto optimality is unattainable since it typically requires expensive complete search. Likewise, incomplete search makes many other desirable properties of schemes hard to attain, such as symmetry, and independence of irrelevant alternatives. For example, adding irrelevant solutions (i.e., alternatives) to the search space can increase the required solving time, which might result in worse solutions if solving is not taken to completion. Second, it makes approximating the ideal utility $ub^*(\cdot)$ necessary, leading to approximate fairness measures and performance metrics. This approximation may cause the evaluation of fairness to be misleading and attempts to increase fairness to be misplaced. Nevertheless, all metrics and performance measures in the rest of this chapter, especially in the experimental section, are approximates.

Solver Bias. Solvers often try to address intractability issues of finding good solutions by using intelligent inference and heuristics. In doing so, they may unintentionally reinforce some of the intractability issues with respect to fairness discussed above. We introduce the term *solver bias* to refer to any bias towards any agent introduced due to heuristics. One particular consequence of solver bias is that the symmetry property is often violated. This occurs when some agents are prioritised over others based solely on the fact that, e.g., their jobs occurred earlier in the array of jobs or earlier in the search tree. This is because of the prevalence of heuristics that use tie-breaking by array index in solvers, during the search for an optimal solution, and because solving is rarely taken to completion. As a result, finding later an equally efficient solution that favours another agent has no effect, as it does not replace the incumbent solution. Thus, there is a bias towards agents whose jobs appear in earlier index positions. Because the search is rarely complete, many equally good solutions, favouring other agents, will not be found either. Simply using randomised search may not be feasible since it could have worse solving performance than a well-thought-out search heuristic, leading to an overall worse solution even for those agents affected by the very solver bias we just tried to eliminate.

Model Bias. Typically, how the objective is specified affects the solving; for example, different objectives generate different learnt clauses in lazy clause generation solvers (such as Chuffed). We refer to this as *model bias*. In conventional optimisation, this is primarily of interest due to its effect on solving performance, but in our situation, how solving behaves can also affect the fairness of the produced solutions (especially when solving under time constraints). Under some objectives, the solver may find solutions that favour one agent more than what it would do under other objectives merely because of how the objective affects solving, and not because the objective prioritised this agent or not. Model bias was observed in the experiments (Section 5.6). It consisted of primarily subtle cases but also some severe cases (to the point that for some instances, using some methods, no solution could be found within a reasonable time). Note that the instances impacted by the latter were omitted in the experimental evaluation. Other experiments in that section also show that the affine invariance property is violated, implying that the strategy-proofness property is also violated.

5.3.3 Measuring Efficiency and Fairness

The aim here is to evaluate the incumbent utilities after executing a series of solutions $\mathcal{A}_1, \dots, \mathcal{A}_s$ to a dmaCOP $Q = \langle P_1, \dots, P_s \rangle$ with respect to the most prevalent of the welfare and inequality metrics discussed in the previous section. In particular, we use four welfare metrics, utilitarian welfare (Util), Rawlsian welfare (Rawls), Nash welfare (Nash), and the proportionality coefficient (Prop)—and two inequality metrics, coefficient of variation (CV) and the Gini coefficient (Gini).

The definitions of the above welfare and inequality metrics require the use of a utility *performance measure*. Thus far, we have defined the metrics based on utility, but other performance measures can be used to get an alternative view of the fairness of any one situation. In this chapter, we will use the following five performance measures, each of which may be more suitable for different situations: *incumbent utility*, *total marginal relative gap*, *incumbent relative gap*, and *incumbent absolute gap*. Let us discuss these individually.

Incumbent Utility (U). This performance measure simply uses, at the start of each session s , the incumbent utility U_k^s of each agent k . As usual, incumbent utility is a positive performance measure, i.e., each agent wants to maximise theirs.

Incumbent utility is the most typical performance measure when evaluating welfare and fairness. However, it is only focused on what an agent did get, which may be misleading when evaluating or correcting for fairness because it ignores relative concession. This can be a problem if one adopts the philosophy of the Kalai-Smorodinsky scheme. In that case, agents may interject and demand that they all make similar sacrifices, that is, similar relative concessions, in order to cooperate. A performance measure based on relative concession can be formulated in two distinct ways: first, by obtaining the relative concession of actual from the ideal *incumbent* utility, and second, by obtaining the average relative concession of the actual from the ideal *marginal* utility.

Incumbent Relative Gap (IRG). This measure is defined for incumbent utilities, for every agent k and at the start of each session s , as $\frac{\text{ub}^*(U_k^s) - U_k^s}{\text{ub}^*(U_k^s)}$. As before, we will assume 0/0 takes the value 0. IRG is a negative performance measure, i.e., each agent wants to minimise theirs.

Average Marginal Relative Gap (MRG). This measure is defined for marginal utilities, for every agent k and at the start of each session s , as $\frac{1}{s-1} \sum_{i=1}^{s-1} \frac{\text{ub}^*(u_k^i) - u_k^i}{\text{ub}^*(u_k^i)}$.

This can be seen as the relative concession from the ideal utility. As before, 0/0 is assumed to be 0. MRG is also a negative performance measure.

The above two performance measures reflect the relative price of sharing in an overall and per-session sense, respectively. In some situations, however, the absolute price of sharing may be more applicable. This is the case, for example, when utilities are expressed in a transferable setting, e.g., a common currency, and fairness should be achieved over missed profits in absolute terms. The fourth and last performance measure we will use is therefore defined as follows.

Incumbent Absolute Gap (AG). This performance measure is, for every agent k and at the start of each session s , defined as $\text{ub}^*(U_k^s) - U_k^s$. AG is a negative performance measure, i.e., each agent wants to minimise theirs.

The absolute gap makes utilities that only differ by an additive constant comparable, while the two relative gaps also account for different scales and are applicable to different utility definitions where the achieved/missed percentage of the optimum should be equal. To illustrate their differences, consider a solution where agent k gets $u_k = \$50$ of $\text{ub}^*(u_k) = \$100$ and another agent i gets $u_i = \$100$ of $\text{ub}^*(u_i) = \$200$. In terms of the absolute gap, i is worse off as it loses \$100 rather than \$50. However, proportionally speaking, both get 50% of their optimum, as reflected by the relative gap. The presented framework can take both gaps into account.

The welfare and inequality metrics can be calculated using each of these performance measures, for which they would yield different results, except for the proportionality coefficient, which comes out to the same value regardless of performance measure used. For incumbent utility and incumbent absolute gap, an average per period (in our case, per week) is often easier to understand and makes the performance measure independent of the length of the problem. For example, if an agent has a utility of 375 after five weeks and another has 225 after three weeks, then they are more easily compared using a per-week average utility, for which they both have 75 per week. Thus, these averages will be used in the rest of this chapter.

Finally, notice that relative gap has two versions, one based on incumbent utility and one based on average marginal utility. We could also have average marginal utility and average marginal absolute gap, but they would be equivalent to their incumbent counterparts under perfect execution. For relative gap, the two versions are different even under perfect execution (as, mathematically speaking, division does not distribute over sums), making it more interesting to use them both and compare them. In the remainder of this thesis, we will refer to incumbent absolute gap simply

as absolute gap (or AG) and incumbent utility simply as utility (or U) when there is no room for ambiguity.

5.3.4 Possibilities for Addressing Fairness

There are plenty of opportunities for addressing fairness in online optimisation. Essentially, any combination (e.g., scalar) of the metrics (using any of the performance measures) defined above can be used. Because of the online nature of our setting, one can also adaptively change how to mitigate unfairness based on the current performance measurements. Below, the most likely mitigation techniques will be presented.

Mitigating Solver Bias and Asymmetry. Because solving is not taken to completion, the symmetry property cannot always be upheld. Recall that this is because of the prevalence of tie-breaking by array index in solvers. A possible way to mitigate this effect in online optimisation is to randomly permute the order in which the agents are presented to the solver, i.e., to permute the indices randomly. Hopefully, this will cancel (on average) any adverse effect asymmetry has over a longer period. This technique will be referred to as *random index permutation*.

Using Utilitarian Principles. A pure utilitarian approach ignores fairness, as discussed before, but mitigation techniques can be defined to take it into account. One technique is to put bounds on one or more fairness metrics while maximising utility, akin to the ϵ -constraint approach discussed in Section 5.2. The bounds would represent the least fairness that is still acceptable, and the solutions the most utilitarian among those with acceptable fairness. Note that, in some cases, the problem may be over-constrained, and soft constraint methods for the bounds must be employed. Another technique for a utilitarian approach is to have weighted utilities, where the (historically) worst-off agents have a higher weight than those better off. These weights adaptively change depending on the incumbent solutions. Here, what ‘worst-off’ and ‘best-off’ means depend on the performance measure selected. Moreover, one can use subtle weight differences to prioritise utilitarian efficiency and pronounced weight differences to strongly discriminate between the agents to attain some fairness goal.

Using Rawlsian Principles. Mitigation techniques derived from Rawlsianism, including the proportionality coefficient and the leximin scheme, are also possible. As discussed, pure Rawlsianism has its weaknesses in ignoring Pareto optimality. Com-

binning Rawlsianism with utilitarianism is not that difficult: one could simply have a utilitarian objective where the utility of the marginally worst-off agent is prioritised higher (e.g., counted twice). However, this technique would ignore historical performance. To address this, one can use the weighted utilitarian approach presented above as follows: having a higher weight on the historically worst-off agent, and equal weights on all others, would create a scenario similar to Rawlsianism with utilitarianism as a second objective but including historical performance. Note that this technique may overshoot, prioritising the historically worst-off agent too much and making it better off than other agents. Using the incumbent or marginal relative gap to define ‘worst-off’ and ‘best-off’ would create an effect similar to maximising the proportionality coefficient.

The leximin scheme, which can be seen as an obvious extension of Rawlsianism to Pareto optimality, is possible to implement directly but is too computationally intensive to adhere to fully, especially under strict time constraints. It is also difficult to implement. Instead, one could approximate this method, again by using a weighted utilitarian technique: let the (incumbent) worst-off agent have the highest weight, the next worst-off have the second-highest weight, and so on.

Using Inequality Principles. If low inequality is the most desired outcome, then techniques striving for low inequality ought to be considered. However, purely minimising inequality can yield optimal solutions where no agent gets anything. Further, assuming some lower bound on utility, minimising inequality on a session-by-session basis will likely lead to less utilitarian efficient solutions than those possible when unfairness is allowed in any session as long as fairness is achieved over multiple sessions. The reason for this is that such a method will inhibit flexibility. Thus, instead of myopically minimising inequality, it is better to try to minimise inequality over a multi-session scale. The weighted utilitarian method that approximates the leximin principle should achieve this as it drives low inequality on a multi-session scale by trying to lift everyone to the historically highest performing agent.

Using Principles from Bargaining Theory. Finally, techniques derived from the Nash scheme are also worth investigating. The Nash scheme combines efficiency and fairness into one coherent objective. For extra flexibility, we could allow a weighted Nash scheme. The weighting can be achieved by adding a scalar multiplier on each logarithm (of the utilities) in the sum representation or raising each utility to a power in the product representation. Unfortunately, due to the lack of support for arbitrarily large integer or floating-point variables in most solvers (especially CP solvers), both

the product and the sum (logarithm) representations are problematic. This could be addressed by developing a scaled sum representation constraint (similar to that of Chapter 6), which is left for future work.

Techniques derived from the Kalai-Smorodinsky scheme may also require dedicated propagators or soft-constraint methods. However, ensuring low inequality on incumbent or marginal relative gap should mimic the Kalai-Smorodinsky philosophy of equalising relative concession.

5.4 RELATED WORK

Before presenting the novel framework, let us explore related work of combining fairness considerations, online decision-making, and combinatorial optimisation. First, three fields will be discussed in order: *online fair division*, *process scheduling*, and *computer networks*, all of which lack the combinatorial optimisation aspect but are still relevant. Finally, we discuss how fairness have been addressed in optimisation disciplines previously.

5.4.1 Online Fair Division

The fair division field, mentioned in the offline context in Section 5.2, has also been studied in the online context. We call this *online fair division* [4]. In the offline context, the set of agents is fully known, the set of goods is fully known, and all goods are allocated simultaneously. In online fair division, goods are allocated one by one, and the agents and goods may not be entirely known at the start. Online fair division follows the pure sequence paradigm from the online optimisation context.

Fairness in online shared resource allocation, in general, has received significant attention. As an example of a classic method for improving fairness, consider the carpool problem as discussed by [44]: a number of agents decide to form a carpool and must determine who should drive each day, ensuring the driving load is shared evenly. Using ‘drives’ as fairness currency (each passenger of a day with d people owes $1/d$ of a drive to the driver), the authors prove that a greedy re-balancing strategy (the person having accumulated the highest debt needs to drive) has bounded unfairness.

However, online fair division, as currently studied in the literature, does not include NP-hard problems. Methods for online fair division often rely on a simpler, interactive problem structure, e.g., [4], where new items are presented, and the agents provide marginal utilities (i.e., how much their utility will increase if allocated the item). These

methods cannot be applied to, for example, scheduling problems since quantifying the utility of some share of machine hours relies on knowing what the solution space is, given those resources. However, there are impossibility results that can be extracted from the more restrictive setting of online fair division to our more general multiagent online combinatorial optimisation setting [3]. Essentially, strategy-proofness, envy-freeness, and Pareto optimality (including some weakened versions of these three) are impossible to achieve unless strict assumptions apply, such as all identical utility functions or non-zero marginal utilities.

In conclusion, while methods from online fair division cannot directly be applied to the more general multiagent online combinatorial optimisation, the impossibility results therein likely spell bad news for any attempts at formal guarantees on fairness in this more general setting.

5.4.2 Fairness in Process Scheduling and Computer Networks

Fairness considerations also arise in resource allocation for distributed computer systems and operating systems. In operating systems, one must schedule the allocation of time slots of, e.g., CPUs to processes running in parallel [151]. In networks and distributed systems, one must design, maintain, and operate a network to ensure a high quality of service, performance, and fairness. In these scenarios, there are multiple concepts of efficiency considered jointly. These include, most prominently, *throughput*, referring to the rate of successful outputs per any given period, and *latency* (or *delay*), referring to the time between the start and finish of a process.

In the case of the operating system, process scheduling [151] aims for maximal throughput and low latency, while guaranteeing fair shares of resource usage for all competing processes. Typical strategies include round-robin scheduling, which grants a fixed time slice to processes in a fixed ordering, and dynamic priority, which accumulates a process's claim to the processor (see [94] for an overview of these and other scheduling strategies). More generally, in parallel systems theory, one often distinguishes between two concepts of fairness: *weak* and *strong* [34, 59]. A *weak* fairness assumption requires that any process that can always proceed, be enabled eventually to proceed. A *strong* fairness assumption requires that any process that can proceed infinitely often (but not always), be enabled to proceed infinitely often. Simple round-robin scheduling ensures weak fairness, while dynamic priority ensures strong fairness [66].

In computer networks, fair queueing, a family of scheduling algorithms, is used to ensure fairness under limited resources while maintaining high throughput and

low latency. These primarily concern round-robin scheduling and techniques derived therefrom. Many measures for fairness have been established, or borrowed from other fields, and are routinely used in the field of computer networks. These include inequality and welfare (and combined) metrics and fairness properties but are either similar to those presented above or are highly specific to the computer networks field. See [77] for an overview.

These conventional scheduling and queueing techniques cannot directly be applied to the fairness issues considered in this thesis. This is because general multiagent combinatorial optimisation includes a wide variety of problems other than process scheduling problems. Beyond that, application would be problematic when only taking scheduling problems into account. Consider, for example, process scheduling, where tardiness is often controlled to ensure that every job eventually gets scheduled and finished. Controlling tardiness is not enough to enforce other kinds of fairness, as the time a job is processed can heavily (and non-monotonically) influence the utility of processing it. Further, fairness is measured at the agent level in our case rather than at the job- or packet level. Nevertheless, these process scheduling ideas can be turned into tangible heuristics and integrated as a core part of an online fairness framework, as will be seen in later sections.

5.4.3 Fairness in Optimisation Disciplines

Let us briefly return to the dynamic multiagent job-shop scheduling problem. The main challenge is to define what fairness means. One must choose between low inequality (little difference in profits), high Rawlsian welfare (no one gets too little profit), and high utilitarian welfare (making the most of the resources), or any other measure one can come up with. As these goals often conflict with each other, i.e., improving one metric often requires another to worsen, the choice is critical.

Fairness has also been considered in combinatorial optimisation, constraint programming, and operations research, where NP-hardness is common. For example, optimisation models and algorithms that address efficiency and fairness have been introduced and studied for several complex systems in [110]. While effective, they only apply to problems without an underlying NP-hard structure. The authors highlight that for NP-hard problems, approximate or heuristic algorithms are probably needed, strengthening the case for considering online optimisation, as done in this chapter. There are also attempts to address fairness under NP-hardness directly. However, it has often been expressed using problem-specific constraints. For example, [10] proposes fairness and utility constraints when sharing the use of a satellite; [84] discusses

fairness criteria in allocating unpopular jobs in railway scheduling; [26] discusses how to specify fairness criteria in static MIP models, combine machine learning and optimisation for fairness, and to fairly solve simple (not NP-hard) problems agents' preferences revealed online; and [65] implements leximin scheme to schedule a data centre with constraints. For a survey of how metrics and schemes can be formulated or specified in optimisation problems, see [27].

The above problems are static and solved offline. Instead, this chapter aims to formulate a problem-independent framework that can be instantiated easily for any dynamic combinatorial optimisation problem that strives for fairness throughout an online scheduling horizon and is specified using a high-level modelling language.

5.5 TOWARDS AN ONLINE FAIRNESS FRAMEWORK

This section presents a framework for increasing fairness in online multiagent combinatorial optimisation, by allowing modellers to experiment with different methods and principles of fairness and how to attain it. The framework, depicted in Figure 5.2,²⁰ incorporates ideas from process scheduling (such as round-robin and dynamic priority) into a problem- and solver-independent modelling system. Intuitively, the framework performs three core operations per session (e.g., week) of the dynamic problem: first, the problem associated with the current period is *solved*; second, the agents are *ranked* based on their performance so far; and third, the optimisation objective for the next period is *re-balanced* to favour the agents whose ranks show low performance. Therefore, the framework can be seen as a feedback-controlled system over a session loop. Additionally, the fairness of the overall solution is *evaluated* at regular intervals (e.g., twice a year), and the results can be used to modify the metrics that determine the ranking of the agents and the strategies used for re-balancing the objective; thus, establishing an open-loop control over each evaluation interval.

5.5.1 A Detailed Example

A detailed, simple example is presented to demonstrate how this framework operates. Suppose there are three agents; call them a , b , and c . As the full job-shop example is superfluous for this demonstration, suppose that the problem being solved is a simple item allocation problem, where every item out of a given set has to be allocated to exactly one agent. Further, the utility functions of the agents are additive, that is,

²⁰The diagram was jointly created with Alexander Schiendorfer.

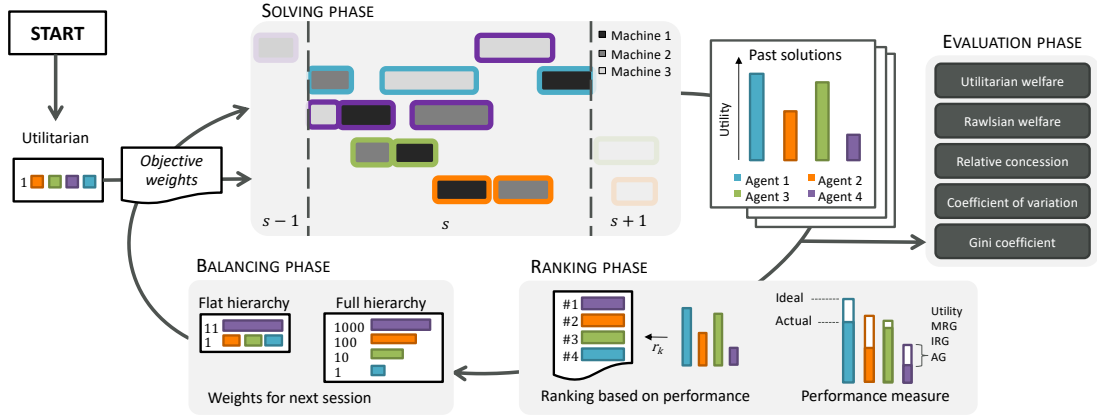


Figure 5.2: An overview of the proposed framework using the job-shop scheduling example with four agents.

the utility of being assigned two items simultaneously is equal to the sum of being assigned one and being assigned the other. Thus, the marginal utilities can be defined on an item-by-item basis.

In the first session, all agents are prioritised equally in a utilitarian objective since there is no incumbent performance with which to re-balance. Suppose there are two items, x_1 and y_1 , each with a marginal utility of 1 for all agents. There are 2^3 possible item allocations. The table below shows the marginal utilities of all three agents:

	u_a	u_b	u_c	max
x_1	1	1	1	1
y_1	1	1	1	1
ub*	2	2	2	2

Each row represents an item to be allocated, with the last row representing all items and thus the ideal utility. Each column represents an agent's marginal utility, with the last column representing the maximum marginal utility possible. Thus, a cell represents an agent's marginal utility of being allocated an item (or all items) or the maximum marginal utility of allocating an item (or all items). According to utilitarianism, all solutions are equal. Assume that a is allocated x_1 and b is allocated y_1 . The resulting performance measures would be as follows:

	a	b	c
ub* (U^2)	2	2	2
U	1	1	0
AG	1	1	2
MRG	$\frac{2-1}{2} = 0.5$	$\frac{2-1}{2} = 0.5$	$\frac{2-0}{2} = 1$
IRG	$\frac{2-1}{2} = 0.5$	$\frac{2-1}{2} = 0.5$	$\frac{2-0}{2} = 1$

Each row represents a performance measure—namely, utility (U), absolute gap (AG), total marginal relative gap (MRG), and incumbent relative gap (IRG). The first row represents the ideal incumbent utility just before session 2 (i.e., after session 1, assuming that the allocation was executed perfectly). Again, each column represents an agent. So far, no matter the performance measure, c is ranked highest (i.e., worst off), while a and b would be joint second.

Suppose we are using a strategy where the worst-off agent gets a weight of 11, and the other agents get a weight of 1. Thus, in the second session, c is prioritised highest with weight 11, and a and b are prioritised second with weight 1. This strategy (the worst-off agent is given a high weight while all others are given the same lower one) is called *flat hierarchy* balancing. Suppose there are three items in this session, x_2 , y_2 , and z_2 , with the following marginal utilities (weighted utilities in parentheses):

	u_a	u_b	u_c	max
x_2	2	3	1 (11)	1 (11)
y_2	7	8	0	8
z_2	1	1	1 (11)	1 (11)
ub*	10	12	2 (22)	12 (30)

The best allocation is thus c getting x_2 and z_2 , and b getting y_2 . Note how there is a cost of fairness due to the imposed weighting: the actual maximum utility achieved for item x_2 is 1, while it would have been 3 if it were not for the weights. The resulting performance measures would be as follows:

	a	b	c
ub*(U^3)	$2 + 10 = 12$	$2 + 12 = 14$	$2 + 2 = 4$
U	$1 + 0 = 1$	$1 + 8 = 9$	$0 + 2 = 2$
AG	$12 - 1 = 11$	$14 - 9 = 5$	$4 - 2 = 2$
MRG	$\frac{2-1}{2} + \frac{10-0}{10} = 1.5$	$\frac{2-1}{2} + \frac{12-8}{12} \approx 0.83$	$\frac{2-0}{2} + \frac{2-2}{2} = 1$
IRG	$\frac{12-1}{12} \approx 0.92$	$\frac{14-9}{14} \approx 0.36$	$\frac{4-2}{4} = 0.5$

At this point, the performance measure used for ranking will make a difference. If one uses raw utility or incumbent relative gap, then a is worst off, followed by c and finally b . If one instead uses absolute gap or marginal relative gap, then the order from worst-off to best-off is a , b , c .

There could be many more iterations, but let us stop and prompt an evaluation phase. For comparison, let us evaluate both after the first session and after the second session. The proportionality coefficient after the first session was $3 \cdot \min\{\frac{1}{2}, \frac{1}{2}, \frac{0}{2}\} = 0$, meaning that someone got nothing, and after the second session it is $3 \cdot \min\{\frac{1}{12}, \frac{9}{12}, \frac{2}{4}\} = 0.25$, meaning that one agent got exactly 25% of a

proportional solution, with all other agents more than or equal to that. The other five welfare and inequality metrics can be used over any of the three performance measures applicable for evaluation. In the tables below (left is after the first session, right is after the second session, and $\uparrow$ denotes that the metric is to be maximised instead of minimised), each of the five evaluation metrics (columns) is evaluated over each of the three performance measures (rows).

	Util	Rawls	Nash	CV	Gini		Util	Rawls	Nash	CV	Gini
U	2 $\uparrow$	1 $\uparrow$	0 $\uparrow$	0.71	0.33	U	6 $\uparrow$	0.5 $\uparrow$	2.25 $\uparrow$	0.89	0.44
AG	2	2	2	0.35	0.17	AG	9	5.5	13.75	0.62	0.33
IRG	0.67	1	0.25	0.35	0.17	IRG	0.60	0.92	0.17	0.40	0.21

Note that U and AG denote per-week average utility and average gap, and note that the above are only approximate figures because of slight rounding, making the numbers brief and easy to understand. If we compare the first and second sessions, then utilitarian welfare got better, except for absolute gap; Rawlsian welfare got worse except for absolute gap; Nash welfare got better except for absolute gap; and both inequality metrics got worse across the board. While the result is quite imbalanced, the framework can achieve fairness if it is run for many more sessions (i.e., ten or more), as will be discussed later.

We can also investigate the price of fairness (Definition 5.11). After the first session, the utilitarian optimum was 2, and the actual utilitarian welfare was also 2; thus, the price of fairness was $\frac{2-2}{2} \approx 0\%$, i.e., there was no price. After the second session, the utilitarian optimum is $2 + 12 = 14$, and the actual was $2 \cdot 6 = 12$; thus, the price of fairness is $\frac{14-12}{14} \approx 14\%$, meaning that the solution is 14% less efficient than maximally possible.

In the next three subsections, the *solving phase*, *ranking phase*, and *balancing phase* are described in more detail.

5.5.2 Solving Phase

Let $Q = \langle P_1, P_2, \dots \rangle$ be a dmaCOP. In this phase, a particular subproblem $P_s = \langle X_s, D_s, C_s, u_1, \dots, u_n \rangle$ is solved (in session s) as a single-objective optimisation problem $\langle X_s, D_s, C_s, f_s \rangle$ with a weighted utilitarian objective, that is, $f_s = \sum_{k=1}^n w_{s,k} u_k$. Recall that the agents' utilities are defined as profit in the dynamic multiagent job-shop example, while the weights represent the relative priorities among agents. In the first period ($s = 1$), all weights are equal, meaning that the pure utilitarian objective $f_1 = \sum_{k=1}^n u_k$ is used. The same definition of utility is used across all weeks, while

the weight $w_{s,k}$, i.e., the relative priority in session s of agent k , is modified by the balancing phase occurring after session $s - 1$.

The aim of these weights is to increase fairness across the whole problem Q by mitigating the issues stemming from the conflicting demands of the agents, thanks to the flexibility of the online nature of the solving. Figure 5.2 shows a solution for an instance P_s of the solving phase, with three machines (represented by the colours black, dark grey and light grey), four agents (represented by the colours blue, orange, green, and purple), and five jobs (one per row, with each task represented by a rectangle filled with the colour of the machine that processed it and lined with the colour of the agent that owns its job).

5.5.3 Ranking Phase

This phase aims to rank the agents from worst-off to best-off, with respect to their results in sessions 1 to s . The next phase will use this ranking to re-balance the relative priorities given to the agents by means of the weights used in the f_{s+1} objective. To achieve this, the framework evaluates a ranking function r_k^s for each agent k on the computed solutions $\mathcal{A}_1, \dots, \mathcal{A}_s$ of sessions 1, $\dots$, s , respectively. While the framework can use any such ranking function, five ranking functions are introduced for the sake of example: a round-robin priority (due to its prevalence in the scheduling literature) and one for each performance metric.

Round-robin Priority (RR). Rank agents via the well-known round-robin strategy where agents take turns being prioritised cyclically. This can be achieved by letting $r_k^s = n_k$, where n_k is the last session where agent k was given the highest priority. Thus, the higher the value, the better off k is assumed to be.

Ranking by Utility (U). The agent with the lowest incumbent utility is assigned the highest rank (as it needs to be prioritised the most), and so on in descending order. That is, $r_k^s = U_k^s$.

Ranking by Absolute Gap (AG). The agent with the highest incumbent absolute gap is assigned the highest rank, and so on in descending order. That is, $r_k^s = U_k^s - \text{ub}^*(U_k^s)$. Notice that this is -1 times the definition of the absolute gap. This is to obtain the desired order, higher being better instead of lower being better.

Ranking by Marginal Relative Gap (MRG). The agent with the historically highest sum of marginal relative gaps is assigned the highest rank, and so on in des-

cending order. That is, $r_k^s = \sum_{i=1}^{s-1} \frac{u_k^i - \text{ub}^*(u_k^i)}{\text{ub}^*(u_k^i)}$. Notice the multiplication by -1 here, as well, and that this is not the per-session average.²¹

Ranking by Incumbent Relative Gap (IRG). The agent with the highest incumbent relative gap is assigned the highest rank, and so on in descending order. That is, $r_k^s = \frac{U_k^s - \text{ub}^*(U_k^s)}{\text{ub}^*(U_k^s)}$. Again, notice the multiplication by -1 .

The above ranking functions yield an ordering $\langle a_1, \dots, a_n \rangle$ among the agents, where $r_{a_k} \leq r_{a_{k+1}}$, for all agents $1 \leq k \leq n$, and ties are broken randomly (adhering to the random index permutation strategy). The ranking phase in Figure 5.2 illustrates some of the ranking functions: when using a raw utility ranking, the purple agent has received the overall lowest utility so far and will be ranked the highest, followed by the orange, green, and blue agents. When using absolute or incumbent relative gap, the blue agent would rank above the green since the green one received almost all the utility they could have wished for.

5.5.4 Balancing Phase

While the ranking phase determines the ranking of the agents, the *balancing phase* regulates *how strongly* we discriminate between them. In simple terms, stronger discrimination leads to more focus on fairness, while a weaker one leads to more focus on efficiency (i.e., overall utility). Objectives are constructed using a basic weighting unit ω for a given ranking of n agents $\langle a_1, a_2, \dots, a_{n-1}, a_n \rangle$. While the framework can use any choice of balancing strategy, the experiments in this chapter use the following two:

Full Hierarchy. A strict weighting of all agents that resembles a lexicographic ordering is imposed. In session s , it sets the weights $\vec{w} = \langle \omega^{n-1}, \omega^{n-2}, \dots, \omega^1, 1 \rangle$ to get a hierarchical objective value where agent a_1 is much more strongly preferred than a_n during session $s+1$. That is, $w_{s+1,k} = \omega^{n-k}$.

Flat Hierarchy. A weaker form of discrimination that considers only two weight levels (rather than n) in the hierarchy is imposed. Agent a_1 is preferred; all others share the same weight in a utilitarian fashion. The resulting weights are $\vec{w} = (\omega + 1, 1, \dots, 1, 1)$. That is, $w_{s+1,k} = 1$ if $k > 1$ and $\omega + 1$ otherwise.

Figure 5.2 shows these two different balancing strategies. In the flat hierarchy on the left, the purple agent (ranked highest) is given a high priority ($\omega + 1$), while all other

²¹We do not need to divide by the number of sessions as it does not affect the ordering.

agents are treated with the same low priority. The full hierarchy on the right assigns decreasing priorities based on decreasing ranks.

The full hierarchy method should favour low inequality since it discriminates between agents more strongly, trying to lift everybody to the best-off agent. The flat hierarchy method should favour high welfare, both utilitarian and Rawlsian, because, while it prioritises the worst-off agent first, it also strongly emphasises high utility as a secondary priority. By this reasoning, the flat hierarchy method should also lead to a high proportionality coefficient.

Again, note that many more strategies are possible in the framework. Nevertheless, we must limit the scope for the experiments in this chapter in one way or another. These two are selected as archetypical examples of strategies that strongly and weakly discriminate between agents.

5.5.5 Evaluation Phase

This phase is performed at regular intervals (e.g., monthly, quarterly, annually) and aims to evaluate the overall solution to the dynamic problem Q . First, a performance measure must be selected for *evaluation purposes*. Note that this can differ from the performance measure used during ranking and the (utility-based) objective function used during solving. Further, we can evaluate multiple metrics over multiple performance measures simultaneously, creating a complete picture of the overall solution's fairness and efficiency. In the experiments performed in the next section, six metrics are used to evaluate and compare the results: the proportionality coefficient (Prop), utilitarian welfare (Util), Rawlsian welfare (Rawls), Nash welfare (Nash), coefficient of variation (CV), and the Gini coefficient (Gini), defined in Section 5.2.

5.6 EXPERIMENTAL EVALUATION

The framework is implemented in Python, taking advantage of the MiniZinc/Python interface²² for the solving phase. To evaluate the framework, the dynamic multiagent job-shop problem described above is modelled in MiniZinc 2.5.5. The Chuffed 0.10.4 solver [29] is used in every iteration of the solving phase (as it proved to be the fastest of those tried for these experiments), with the fixed search specified in the model in order to improve comparability across different runs. All experiments were run on

²²<https://minizinc-python.readthedocs.io>

an Intel Xeon 8260 24-core CPU at 2.4 GHz, with 256 GB RAM, running Ubuntu Linux 20.04 LTS. Each run was limited to a single core and 8 GB RAM.

5.6.1 Instance Generation

To simulate a dynamic problem, 11 sets of instances were generated, each containing instances spanning 26 weeks (half a year), corresponding to 26 sessions. Each instance is (randomly) based on the data contained in one of the five basic job-shop input data files²³ selected from the MiniZinc benchmark repository. Each of these data files contains different values for the required parameters of an offline model,²⁴ such as the number of machines available, the processing time and the machine required by each job's tasks. The five data files were selected to yield instances large enough to be interesting, but not too large to make evaluation impractical. These data files are then randomly picked and ordered to create the 26 weeks for each of the 11 sets of instances. The revenue of completing each job is set to 40 units, the storage cost per hour to 1 unit, the tardiness cost per day to 20 units, and the shipment hour to 8 am, for all instances.

The experiments were run using three, five, and seven agents (yielding $11 \times 3 = 33$ sets of instances). To generate each instance, one of the five input data files was randomly selected with its jobs randomly distributed across the agents. In order to generate a diverse set of instances, the jobs were distributed using four different approaches (yielding $33 \times 4 = 132$ sets of instances): *invariably balanced*, where the agents demand roughly the same number of jobs with roughly the same processing time per week; *variably balanced*, where the agents demand highly different numbers of jobs and processing time per week, but at the end of the 26 weeks, the agents' demands sum up to roughly the same number of jobs and processing time; *imbalanced*, where the agents demand different numbers of jobs and processing time, and at the end of the 26 weeks the imbalance is still there and roughly the same as the weekly imbalances; and *imbalanced load*, which is similar to the previous scenario but only concerning the load, not the job count. For an overview, see Table 5.1.

In addition, two variations of load demand were used (yielding $132 \times 2 = 264$ sets of instances): high load (*HiLd*) and low load (*LoLd*). Since just using the durations from the data files as hours (the smallest time unit) will result in most jobs taking well over a week, the durations were scaled down. In *HiLd*, the durations of all jobs are scaled down to the largest point where all jobs in the instance fit within a week. This

²³swv01.dzn, swv02.dzn, swv03.dzn, la28.dzn, and la29.dzn.

²⁴<https://github.com/MiniZinc/minizinc-benchmarks/tree/master/jobshop>.

Variation for	<i>imb.</i>	<i>imb. load</i>	<i>inv. bal.</i>	<i>var. bal.</i>
Weekly Job Demand	0.18	0.09	0.00	0.31
Weekly Load Demand	0.16	0.18	0.07	0.40
Total Job Demand	0.11	0.02	0.00	0.08
Total Load Demand	0.12	0.13	0.01	0.10

Table 5.1: Median coefficient of variation of job and load demand variation among agents for our four distribution approaches.

yields as much load as possible per job while ensuring that no job is impossible to schedule. For the five input data files we selected, this is achieved by scaling down all durations to 19.3% (rounding up to the nearest integer and interpreting it as hours). In *LoLd*, the durations of all jobs are scaled down to the smallest point where each job still has half or more of its tasks with a duration still longer than 1 (the smallest time unit: one hour). This yields as little load as possible per job while somewhat maintaining the characteristics of the job-shop instances. For the five input data files, this is achieved by scaling down all durations to 5.9% (rounding up).

Finally, each job was assigned a deadline by randomly selecting a value from the first attainable shipment deadline to the last shipment deadline of the week. To ensure comparability across strategies and methods, the same series of random seeds were used across all of them.

5.6.2 Solver Bias and Random Index Permutation

In the initial experiments, *solver bias* was observed. Recall that the symmetry property is not upheld due to index tie-breaking in the solver and because the search is not complete. To mitigate this bias, the indices of the jobs in each week were randomly permuted. An initial experiment was run with and without this random index permutation. It ran for 52 sessions (each representing a week) using the pure utilitarian approach and the round-robin approach with full and flat hierarchies. The results show no noticeable effects on utilitarian welfare regardless of the performance measure. However, proportionality is slightly improved when using random index permutation, and the two inequality metrics are substantially improved for absolute gap and incumbent relative gap, while slightly improving for utility.

Figures 5.3 and 5.4 show more detailed results of the effects of using random index permutation. The figure is split into a grid of 16 subfigures across the two figures, each with two box plots and their underlying (probability) distribution of observations represented by violin plots underneath. Each row uses a different performance

measure: average (avg.) utility in the first row, average absolute gap (avg. AG) in the second, and incumbent relative gap (IRG) in the third row. These are the same across both figures. Each column uses a different welfare or inequality metric: utilitarian welfare (Util) in the first column, Rawlsian welfare (Rawls) in the second, and Nash welfare (Nash) in the third column, all within Figure 5.3; continuing with Figure 5.4, we have the coefficient of variation (CV) in the fourth row, the Gini coefficient (Gini) in the fifth, and the proportionality coefficient (Prop) in the sixth column (which is not split into multiple rows as it is the same for the three performance measures used here).

Let us begin by discussing the first column. In Subfigures (1a), (1b), and (1c), the utilitarian welfare is respectively evaluated over utility, absolute gap, and incumbent relative gap. There is no noticeable difference between using and not using random index permutation, indicating that the overall efficiency of the solutions produced is equivalent. In Subfigures (2a), (2b), and (2c), the Rawlsian welfare is respectively evaluated over the three performance measures. We can see that using random index permutation results in a slight increase in Rawlsian utility, and slight decreases in average gap and relative gap, supporting its use. In Subfigures (3a), (3b), and (3c), the Nash welfare is respectively evaluated over the three performance measures. As for utilitarian welfare, there is no noticeable difference between using and not using random index permutation, indicating that random index permutation does not affect Nash welfare. Over to Figure 5.4: in Subfigure (6), the proportionality coefficients of the obtained solutions are plotted. The two box plots, especially the two violin plots underneath, indicate that slightly higher proportionality can be achieved with random index permutation. Let us investigate the rest of the subfigures row-wise. In Subfigures (4a) and (5a), the coefficient of variation and the Gini coefficient are evaluated over average utility. The figures show that the most unequal solutions have been made less so under random index permutation and that the median and lower quartile are both slightly lowered. This indicates that there is less utility inequality when using random index permutation. In Subfigures (4b) and (5b), the coefficient of variation and the Gini coefficient are evaluated over average absolute gap. These figures also show that the most unequal solutions are improved under random index permutation. Further, the median and lower quartile are significantly lowered, with the mode of the distribution being close to the bottom. This indicates that there is much less inequality of absolute gap when using random index permutation. Finally, in Subfigures (4c) and (5b), the coefficient of variation and the Gini coefficient are evaluated over the incumbent relative gap. Again, the figures indicate there is much less inequality of incumbent relative gap when using random index permutation.

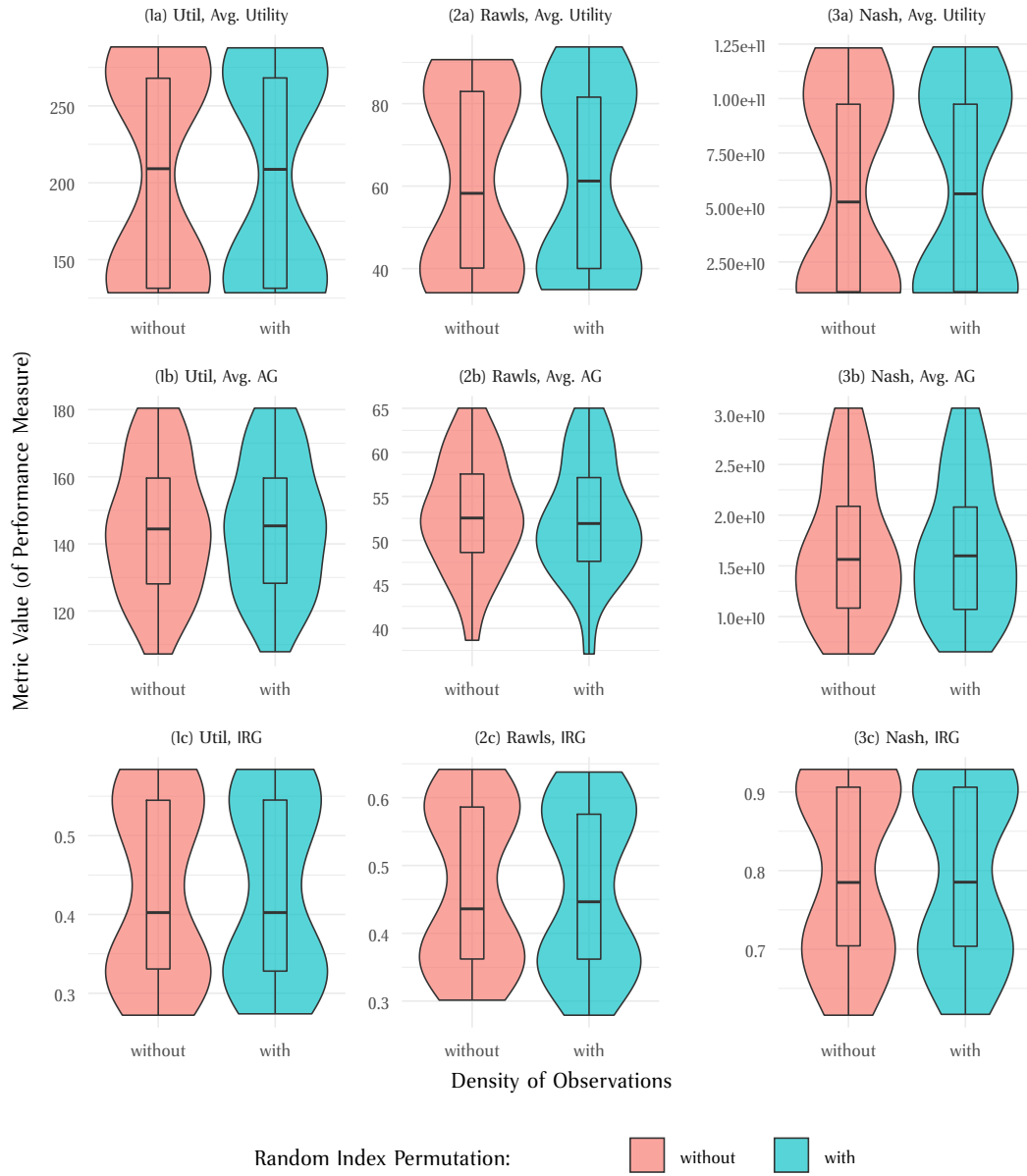


Figure 5.3: For the first nine combinations: box plots with underlying density (violin plots) visualising the differences between solving without random index permutation (right, red) and solving with random index permutation (left, blue).

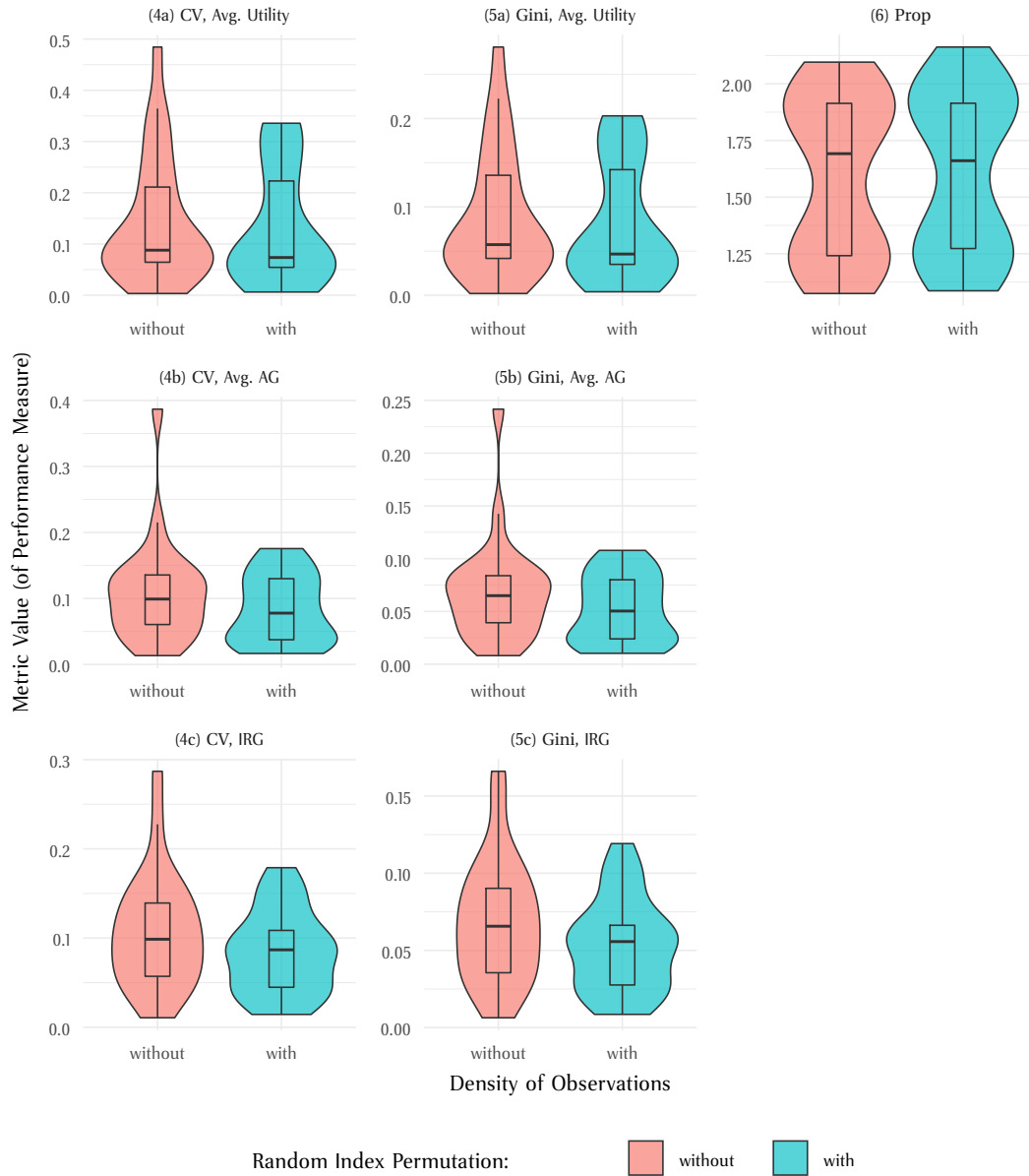


Figure 5.4: For the final seven combinations: box plots with underlying density (violin plots) visualising the differences between solving without random index permutation (right, red) and solving with random index permutation (left, blue).

Based on those figures, all the experimental results shown henceforth were obtained using random index permutation.

5.6.3 Choice of Parameters

The experiment performs the evaluation phase at the end of the 26 weeks. In each session s , the ideal utility $\text{ub}_s^*(u_k)$ for each agent k was estimated by solving P_s in what we call *dictator mode* for agent k , that is, with weight $w_{s,k}$ set to 1 and the weights for all other agents set to 0. This gives sole priority to the utility of the dictator agent. A 20-second timeout was used, which may result in a suboptimal estimation of $\text{ub}_s^*(u_k)$. Once the approximation for each agent was computed, P_s was solved using the weights obtained by ranking and balancing after session $s-1$, again with a 20-second timeout. For balancing purposes, we selected $\omega = 10$. If the actual utility $\mathcal{A}_s(u_k)$ happened to be higher than the estimated ideal one, due to a poor estimate, then $\text{ub}_s^*(u_k)$ was set to $\mathcal{A}_s(u_k)$ instead.

5.6.4 General Results

The average results, obtained across all instances for the different ranking and balancing strategies using the different metrics and performance measures, are shown in Table 5.2. The results show that the right choice of ranking and balancing strategy depends on the kind of fairness one seeks.

Unsurprisingly, when evaluating for utilitarian welfare Util, a pure utilitarian objective (PU) is the best strategy, no matter what performance measure the evaluation uses. This is because there is always a price to pay in utility for increasing fairness. Thus, if fairness is a strictly secondary priority, then mitigating solver bias (by using random index permutation) may be enough. Interestingly, PU is the best when it comes to Nash welfare of incumbent relative gap. However, PU leads to subpar performance with respect to fairness, especially when evaluating for Gini and CV metrics. Note that none of our combinations of ranking and balancing strategies generate average weekly utilitarian welfare that is more than 7% worse than the pure utilitarian approach (bold in the column), and many are within 2%.

By contrast, if we consider Rawlsian welfare (Rawls) or Proportionality (Prop) as the metric, then the flat hierarchy is consistently the best balancing strategy over all ranking functions, and marginal relative gap (i.e., MRG1) is often the best ranking function. In fact, when using the flat hierarchy balancing strategy, MRG almost dominates all other ranking functions, with absolute gap (i.e., AG1) a close second. Compared to

	Avg. Utility					Avg. Abs. Gap					Incumbent Rel. Gap					
	Prop	Util	Rawls	Nash	CV	Gini	Util	Rawls	Nash	CV	Gini	Util	Rawls	Nash	CV	Gini
PU	2.782	464.4	93.8	1.930e+11	0.155	0.094	256.1	61.1	2.279e+10	0.170	0.102	0.385	0.435	0.642	0.165	0.099
RR <i>n</i>	2.550	433.0	88.6	6.162e+10	0.139	0.084	287.6	67.7	1.875e+10	0.141	0.086	0.426	0.474	0.761	0.130	0.079
RR1	2.772	458.2	93.1	1.910e+11	0.144	0.087	262.3	62.4	2.500e+10	0.166	0.100	0.395	0.438	0.646	0.157	0.094
Un	2.098	433.6	80.5	6.295e+10	0.390	0.233	286.9	85.0	1.081e+10	0.408	0.246	0.425	0.561	0.760	0.362	0.218
U1	2.646	458.5	90.3	1.935e+11	0.258	0.152	262.1	69.2	2.201e+10	0.274	0.163	0.395	0.468	0.646	0.241	0.144
AG <i>n</i>	2.585	433.6	89.0	6.412e+10	0.149	0.086	286.9	61.7	1.971e+10	0.037	0.022	0.425	0.466	0.760	0.075	0.044
AG1	2.789	459.0	93.6	1.931e+11	0.148	0.087	261.5	58.0	2.531e+10	0.103	0.061	0.394	0.433	0.645	0.122	0.073
MRC <i>n</i>	2.751	437.6	93.1	7.56e+10	0.078	0.046	283.3	65.2	1.879e+10	0.090	0.053	0.420	0.437	0.749	0.051	0.031
MRC1	2.893	459.1	96.4	1.937e+11	0.089	0.052	261.8	61.4	2.469e+10	0.134	0.080	0.394	0.414	0.643	0.105	0.063
IRG <i>n</i>	2.667	438.9	91.6	7.344e+10	0.101	0.060	282.0	66.5	1.895e+10	0.136	0.082	0.418	0.452	0.746	0.110	0.067
IRG1	2.737	459.0	92.9	1.937e+11	0.159	0.095	261.9	62.6	2.471e+10	0.167	0.100	0.395	0.443	0.643	0.157	0.095

Table 5.2: Averages of different ranking/balancing strategies (rows) and metrics of performance (columns). Optima per column are boldface, worse than median per column are coloured grey. The ranking functions used are: pure utilitarian objective (PU), round-robin scheduling (RR), raw utility (R), absolute gap (AG), marginal relative gap (MRC), and incumbent relative gap (IRG), with full (*n*) and flat (1) balancing hierarchies. Welfare and Inequality metrics used are: the proportionality coefficient (Prop), utilitarian welfare (Util), Rawlsian welfare (Rawls), Nash welfare (Nash), coefficient of variation (CV), and Gini coefficient (Gini), computed over three different performance measures: (weekly average) utility, (weekly average) absolute gap, and incumbent relative gap.

PU	RR n	RR1	Un	U1	AG n	AG1	MRG n	MRG1	IRG n	IRG1
30.9%	37.4%	31.7%	40.3%	31.3%	37.2%	32.0%	38.9%	31.4%	37.1%	34.4%

Table 5.3: Percentage of instances where optimality was proven per balancing strategy: pure utilitarian objective (PU), round-robin scheduling (RR), raw utility (R), absolute gap (AG), marginal relative gap (MRG), and incumbent relative gap (IRG), with full (n) and flat (1) hierarchies.

PU, using MRG for the ranking phase only slightly decreases the utilitarian welfare but significantly improves the other metrics across the board. If striving for high Nash welfare over utility, MRG1 or IRG1 is similarly recommended.

If striving for low Gini and CV inequality, the results suggest using the full hierarchy balancing strategy and, again, the MRG ranking function (i.e., MRG n). If low inequality over the absolute gap is the focus, then the results suggest using AG n .

Interestingly, while using incumbent relative gap (IRG) as a ranking function performs better than PU for fairness, it performs worse than AG and especially MRG, even when evaluating fairness over IRG. This last point is somewhat surprising; it is most likely because IRG inhibits flexibility as a ranking function as opposed to MRG.

Worth noting that U1 and Un have arguably the worst results out of all ranking and balancing strategies, unless Nash welfare of absolute gap is the sole desired metric to maximise.

One can further see that for the round-robin (RR) ranking function, there is not much gain in fairness but still a loss in efficiency, highlighting the need for intentional balancing and ranking.

Overall, using MRG as a ranking function, regardless of strategy, seems to be the best option as a default.

5.6.5 Strategies and Altered Behaviours

In the above experiments, all agents were assumed to derive utility only from their own jobs, ignore what happened to the other agents, and report their utilities truthfully. These assumptions can sometimes be too strong. For example, when deciding the temperature of the apartments in a building, agents would benefit from the temperature of their surrounding apartments being close to their desired temperature since it would then cost them less to maintain said temperature. This section presents additional experiments, using the dynamic job-shop as before, but with agents exhibiting strategic behaviours and reporting utilities affected by what other agents get. Agents

who report their utility as solely their profit, as they all did in previous experiments, are referred to as *sincere agents*. Agents who alter the reporting of their utility in any (non-sincere) way are referred to as *altered agents*.

To do this, we run five experiments with agents exhibiting altered behaviours. For every agent k , let u_k denote their true utility function and $\tilde{u}_k$ denote their altered one used instead. The experiments are constructed as follows:

Adversarial (adv.) Agents. These are agents who report extra utility when another agent does badly. In particular for this experiment, all agents except agent 1 strongly dislike agent 1 and deduct the utility agent 1 gets from their utility function; hence, agent 1 is sincere, and the others are altered agents. More formally, the maCOPs solved have the form $\langle X, D, C, u_1, \tilde{u}_2, \dots, \tilde{u}_n \rangle$, where $\tilde{u}_k = u_k - u_1$.

Coalitional (coal.) Agents. These are agents who report utility as a group. In particular for this experiment, agents 1 and 2 are in a coalition and use the sum of their sincere utilities in their reported utilities; hence, agents 1 and 2 are altered, and the others are sincere agents. More formally, the maCOPs solved have the form $\langle X, D, C, \tilde{u}_1, \tilde{u}_2, u_3, \dots, u_n \rangle$, where $\tilde{u}_1 = \tilde{u}_2 = u_1 + u_2$.

Compassionate (comp.) Agents. These are agents who report extra utility when another agent does well. In particular for this experiment, every agent strongly likes agent 1 and includes the utility that agent 1 receives in their own utility function; hence, agent 1 is sincere, and the others are altered agents. More formally, the maCOPs solved have the form $\langle X, D, C, u_1, \tilde{u}_2, \dots, \tilde{u}_n \rangle$, where $\tilde{u}_k = u_k + u_1$.

Deflationary (defl.) Agents. These are agents who report lower than their true utility. In particular for this experiment, agent 1 reports half the truthful utility; hence, agent 1 is altered, and the others are sincere agents. More formally, the maCOPs solved have the form $\langle X, D, C, \tilde{u}_1, u_2, \dots, u_n \rangle$, where $\tilde{u}_1 = u_1/2$.

Inflationary (infl.) Agents. These are agents who report higher than their true utility. In particular for this experiment, agent 1 reports twice the truthful utility; hence, agent 1 is altered, and the others are sincere agents. More formally, the maCOPs solved have the form $\langle X_s, D_s, C_s, \tilde{u}_1, u_2, \dots, u_n \rangle$, where $\tilde{u}_1 = 2u_1$.

The experiments were run with PU, RRn, and RR1, as baselines; additionally, MRGn and MRG1 were run, as they generally performed the best in previous experiments. The results, showing the average weekly gain (+) or loss (-) in raw (sincere) utility, are shown in Table 5.4. Remaining sincere rarely leads to positive results. Some of

	Sincere Agents					Altered Agents				
	<i>adv.</i>	<i>coal.</i>	<i>comp.</i>	<i>defl.</i>	<i>infl.</i>	<i>adv.</i>	<i>coal.</i>	<i>comp.</i>	<i>defl.</i>	<i>infl.</i>
PU	-62.81	-15.81	-11.39	-13.53	+30.14	+1.89	+30.20	+37.15	-69.53	+405.00
RR n	-52.46	-16.60	-9.08	+50.68	-22.65	+0.07	+16.57	+38.54	-49.64	+221.25
RR1	-69.31	-7.59	-12.00	+2.61	+17.09	+1.53	+20.48	+41.73	-66.19	+396.45
MRC n	-65.61	-6.43	-9.82	-18.79	+36.83	+1.74	+6.10	+39.43	-70.49	+489.78
MRC1	-58.91	-10.21	-8.33	+42.82	-16.03	-1.56	+12.47	+42.32	-53.15	+255.88

Table 5.4: Results showing the per-week average gain/loss in raw utility across different altered behaviours of the agents.

the results are surprising, such as both sincere and altered agents losing under the deflationary situation with PU. This is likely a result of the solver behaving differently in search due to the altered objectives.

5.7 DISCUSSION AND FUTURE WORK

Before concluding this chapter, let us briefly discuss and investigate potential future work. The most obvious avenue for future work is fully addressing the possibilities and challenges presented in Section 5.3. A few other interesting prospects are presented below.

5.7.1 Problems and Solvers

The most pressing issue for future work is considering other problems and other solvers. How do the kinds of dynamisms present in the problem being addressed affect the generally preferred method? For example, it is possible that the results change if there is much overlap between sessions. There is also a need to investigate solver bias more intricately, as it is not clear how much random index permutation mitigates the asymmetry property. Further, there might be more subtle forms of solver bias, not stemming from the index tie-breaking and incomplete search.

5.7.2 Evaluation Frequency

Due to the dynamic nature of the problem we are solving, a natural question is how frequently fairness should be expected and evaluated. This affects the best strategy for ranking and balancing, i.e., we need more aggressive balancing for frequent fairness than infrequent fairness as there is less time to make up for past unfairness. Moreover,

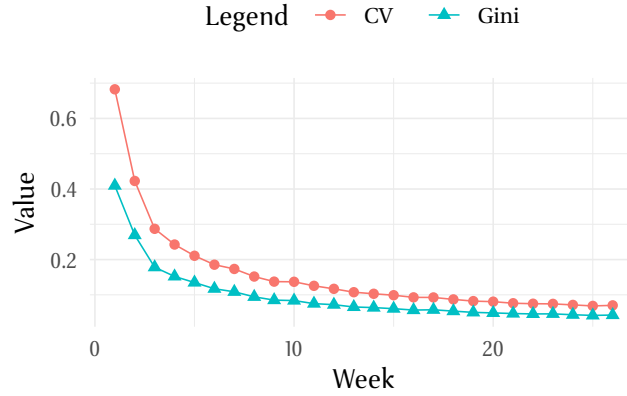


Figure 5.5: Progress of inequality over time

if we expect fairness to be achieved very frequently (say, every week), then we run into complications from the discrete nature of the scheduling problem, and it may negatively impact the overall efficiency of the solutions, yielding poor compromises. Conversely, if we expect fairness to be achieved very infrequently (say, every ten years), then we allow the framework to yield extremely fair end results. This could also yield unacceptable results where, e.g., one agent is disadvantaged for the first eight years and treated beneficially for only the last two. Since later periods often have higher uncertainty, the balancing might come ‘too late’. The presented framework allows for easy experimentation to find a suitable compromise.

We could introduce a similar concept such as the *fairness waiting period*, which is the transient phase before a situation is reached that is considered fair by the agents. Figure 5.5 plots the median value of the coefficient of variation and the Gini coefficient, using the incumbent relative gap as the performance measure. The median is taken over all ranking functions and balancing strategies in Table 5.2 (except the worst three, namely PU, RU n and RU1) when evaluated at different points over the 26 weeks. It takes about 13 weeks before a reasonably fair solution is obtained on this example setting, that is, when both metrics fall below 0.125.

5.7.3 Defining Fairness and Performance

There are many more welfare and inequality metrics in the literature than those considered and experimented with in this chapter. There are also many more ways to configure and combine balancing and ranking functions. Therefore, more experimentation is always a possible avenue for future work. How would more subtle weight

differences impact fairness and utilitarianism, for example? Is it possible to create rules for adaptively applying weights similarly to how the balancing was adaptive? For example, we currently always consider an unbounded look-back horizon when calculating the ranking function, i.e., the solutions to all previous sessions. However, the ranking function may benefit from considering a shorter look-back horizon, such as the past week, month (four weeks), quarter (13 weeks), or year. This leads to the possible introduction of *discounted incumbent performance*: the further back in time, the more discounted (i.e., less weight) is applied. An interesting question is how problem-dependent circumstances affect the choice of best-performing look-back horizons. Regarding balancing strategies, setting ω to 10 is entirely arbitrary in the experiments, for example. Exploring the parameter space would be interesting. How would more subtle weight differences impact fairness and utilitarianism, for example? Is it possible to create rules for adaptively applying weights similarly to how the balancing was adaptive? Regarding ranking strategies, an interesting idea that needs more attention is to combine multiple ranking functions or dynamically change the ranking function throughout the sessions by creating a ‘ranking function scheduler’.

What to evaluate fairness over is a crucial question that must be answered. The definition of utility may vary across agents, e.g., while some agents in a scheduling problem might focus on profit, others might focus on the number of jobs finished on time or the number of machine-hours utilised. Thus, the utilities need not be comparable across agents, complicating the quest to attain fairness. Generally, the definitions of fairness to be used must be agreed upon, e.g., by the agents, before solving commences. Nevertheless, an inquiry into how to address incomparable utilities remains an interesting avenue for future work. Similarly, it remains future work to map the established theories of how to measure fairness from the resource allocation literature (see, e.g., [90]) to the context of this thesis.

Importantly, all these questions and ideas can easily be explored within the presented framework for the combinatorial optimisation problem of interest.

5.7.4 Varying Importance of Fairness

In some applications, fairness could be more critical during some periods than others. Consider an online optimisation application for rostering retail staff, where fairness may be more important and acute during a holiday season or other similar high-sales periods. Because fairness is considered over time, the presented framework is not directly geared to handle such situations. However, one could easily extend the ranking phase to take some fairness weights into account, use more granularity

(shorter periods than weeks) during critical times, or compensate agents outside the system. Similarly, one could introduce fairness constraints when fairness is most critical.

5.7.5 Strategy-proofness and Envy-freeness

Strategy-proofness is an important concept in many disciplines concerning fairness. While we have shown that our framework is not strategy-proof out-of-the-box, are there any modifications or restrictions that can be imposed to make it so? Investigating how to diminish the effect of, or catch, agents that alter their utility functions insincerely is an interesting avenue for future work. There are also weakened concepts of strategy-proofness in the literature. Perhaps some of them can be attained in some cases. The decision-theoretic concept of regret also ties into strategy-proofness in the context of this thesis.

Envy-freeness is another important concept that requires further investigation. This concept is not easily mapped to general optimisation problems, as we have discussed, but is still important for many applications. As for strategy-proofness, there are weakened conditions of envy-freeness in the literature, which may be better suited for the problems considered in this chapter. Investigating these and potentially developing new concepts of envy-freeness for subclasses of optimisation problems is an interesting avenue for future work.

5.8 CHAPTER CONCLUSION

This chapter has presented a novel, problem- and solver-independent algorithmic framework for experimenting and increasing fairness in dynamic multiagent combinatorial optimisation problems, while maintaining high overall utility. Since fairness is a broad concept with many schools of thought, experimental results for multiple welfare and fairness metrics have been shown, namely *utilitarian*, *Rawlsian*, and *Nash welfare*, *coefficient of variation*, the *Gini coefficient*, *relative concession*, and *proportionality*, each evaluated over different performance measures, namely *average raw utility*, *average absolute gap* and *incumbent relative gap*. To address and increase fairness, different ways of ranking the agents were employed, namely *round-robin* scheduling, as well as ranking by *incumbent raw utility*, *incumbent absolute gap*, *average marginal relative gap*, and *incumbent relative gap*—along with two balancing strategies, namely *full* and *flat hierarchy*. General recommendations have been given, depending on what is considered most important for an application. The framework is built on top of the MiniZinc

toolchain, which allows the modeller to develop, use, and easily work with various solvers and models. This makes the framework problem- and solver-independent.

We have presented experiments with the framework that start by using a *pure utilitarian objective* and, in subsequent solving sessions, adaptively re-balance and weigh the objectives of the agents according to ranking functions that look back at the agents' performance in earlier sessions. Selecting the best ranking function and balancing strategy depends on the application and fairness goals. When utilitarian welfare is paramount, the *pure utilitarian objective* is naturally the best. When Rawlsian welfare and proportionality are paramount, and low inequality is important but secondary, prioritising only the worst-off agent over all other agents aggregated by a utilitarian objective for tie-breaking is generally recommended (e.g., *flat hierarchy*). When low inequality is imperative, perhaps at the cost of efficiency, the recommendation is to balance all agents in a fully hierarchical manner, from best-off to worst-off (e.g., *full hierarchy*). Generally, the recommendation is to use a ranking function that measures the agents' core interest (e.g., profit) both in terms of how much and how many times they have conceded utility due to sharing (e.g., *average incumbent relative gap*).

Finally, several challenges and opportunities have been highlighted when it comes to ensuring fairness in online combinatorial optimisation.

EXPLAINING PROPAGATORS FOR GINI AND SPREAD WITH VARIABLE MEAN²⁵

In optimisation problems involving multiple agents, one often wants to ensure that the solution is balanced and fair, at least to some degree. The balance and fairness are often represented as a statistical dispersion metric, for example, variance or the Gini coefficient. The previous chapter discussed methods for achieving fairness in the online setting (i.e., for dynamic problems). However, static problems (i.e., the offline setting) are often quite challenging as well. A static problem in which both fairness and efficiency are essential is often addressed with one as the objective and the other as a constraint. That is, we want to maximise efficiency (i.e., total utility) subject to an upper bound on the statistical dispersion of the utility given to different agents or minimising dispersion subject to some lower bound on utility. These needs arise in many applications, such as those that try to balance tardiness in scheduling, unwanted shifts in rostering, and desired resources in resource allocation, or those that aim to minimise deviation from a baseline in schedule repair. For example, consider a resource allocation problem, where a set A of agents compete over a set R of resources under complex constraints. The utility that an agent $a \in A$ receives from being allocated a resource $r \in R$ may entirely depend on what other resources a is being allocated. For example, a woodworker gets little utility from being allocated nails but not a hammer, or hammer and nails but not planks. In this situation, balancing the utilities (or performance measures derived from the utilities) each agent receives may be essential. A fixed mean is often assumed in the literature, simplifying the respective dispersion constraint; however, fixing the mean is impossible in many contexts. This is why the mean is assumed to be variable throughout this chapter.

To solve the above kinds of problems effectively, a solver must reason about dispersion effectively. Learning CP solvers are strong candidates for solving combinatorial

²⁵The content of this chapter is based on the published work of Paper [III].

problems effectively. However, learning CP solvers require dedicated propagators that can explain their inferences in Boolean logic [III], since these explanations are key to speeding up the search using SAT solving technology.

In this chapter, two log-linear-time dispersion propagators are presented that are capable of explaining their propagations, allowing effective learning solvers to be used. The use of floating-point decision variables, which are usually not supported by learning solvers, is avoided by reasoning over appropriately scaled integer versions of dispersion constraints.

The rest of this chapter is organised as follows. In Section 6.1, the relevant background specific to this chapter is presented. Section 6.2 presents the first explaining propagator: a propagator for spread (variance, and indirectly standard deviation). Much previous work has studied this propagator under the assumption that the mean is fixed. However, this is not possible for many problems, e.g., scheduling where total (and thus mean) utility depends on the final schedule. No previous work has considered a spread propagator that needs to explain its inferences. Section 6.3 presents the second explaining propagator: a propagator for the Gini coefficient. No constraint propagator for the Gini coefficient has been previously studied in the literature. Section 6.4 provides experimental results for the propagators. The experimentation shows that clause learning can substantially improve the solving of problems with a bound on dispersion and total utility as an objective and vice versa. The chapter presents related work in Section 6.5 before concluding in Section 6.6.

6.1 PRELIMINARIES

Before the contributions of this chapter can be presented, preliminary concepts need to be established. We define explaining constraint propagators and learning CP solvers, followed by real relaxations and their advantage. For ease of notation, given a variable x with domain $D(x)$, the term $\text{lb}(x)$ refers to the lower bound of x , i.e., $\min(D(x))$, and $\text{ub}(x)$ refers to the upper bound of x , i.e., $\max(D(x))$.

6.1.1 SAT Solvers and Nogood Learning

Boolean Satisfiability (SAT) solvers have, over the past couple of decades, become remarkably powerful [149]. The three main components that enable this are inference, heuristics, and learning. The final of the three, *learning*, often called *clause learning* or *nogood learning*, is the most relevant for this chapter and will be discussed further. The main aim of learning is for the solver not to redo mistakes (i.e., guesses that

lead to unsatisfiable states) during the search, such that the search is sped up. This is achieved by successively understanding the hidden relations between the variables in a model. A SAT solver learns these hidden relations by deriving redundant clausal constraints, which are then added to the (global) set of constraints, allowing additional inferences to occur. To achieve this, modern SAT solvers first successively construct an *implication graph*, which *explains* the steps and inferences that have taken place up to a given point.

Definition 6.1 (Implication Graph). Let L be a set of literals and C be a set of clauses (the constraints) over these literals. An *implication graph* is a directed acyclic graph $G = \langle V, E \rangle$ defined as follows. Each literal that is currently set to true is a node in the graph. If the truth is a result of search (i.e., a guess), then it has no incoming edges, and if it is a result of inference, then there is an incoming edge from each literal that directly lead to this inference. Each node has an associated search depth (i.e., the number of guesses applied), indicating where in the search it was generated. For each literal $l \in V$, its corresponding *explanation* represented in the implication graph is the formula $(l_1, \dots, l_k) \rightarrow l$ for each $l_i \in V$, $1 \leq i \leq k$, such that $(l_i, l) \in E$.

Once a conflict is reached, modern SAT solvers use the implication graph to generate a *nogood*, that is, a set of literals representing the cause for the conflict. The solver then adds the negation of this nogood as a new clausal constraint, ensuring that the same conflict is not reached again. Finally, the solver *backjumps*, that is, backtracks to where the first decision involved in the conflict (i.e., nogood) was made.

SAT solvers operate on Boolean variables and clausal constraints in CNF. As a result, to use a SAT solver to solve a problem, that problem needs to be encoded into Boolean variables and clausal constraints. Explaining the latter is superfluous for the purposes of this chapter; let us focus just on the former: the encoding of variables into Boolean variables. Recall that the notation $\llbracket \cdot \rrbracket$ is used for the Boolean literal representing the truth value of the relation inside the brackets. Suppose that, in a constraint-based modelling view, there is an integer variable x with domain $l..u$. Since SAT does not operate on integers, this variable must be encoded into Boolean variables representing its assignment. Two prevalent variable encodings are the *equality encoding* (also called the *direct encoding*) and the *order encoding*. In the *equality encoding*, a Boolean variable is introduced for each possible value in the domain of the original variable. Each such Boolean variable denotes whether x is assigned its associated domain value. That is,

the following $l - u + 1$ Boolean variables are introduced:

$$\llbracket x = l \rrbracket, \llbracket x = l + 1 \rrbracket, \dots, \llbracket x = u - 1 \rrbracket, \llbracket x = u \rrbracket.$$

In the *order encoding*, a Boolean variable is similarly introduced for each value in the domain; however, each variable instead denotes whether x is assigned a value greater than or equal to its associated domain value. The following $l - u$ Boolean variables are introduced:

$$\llbracket x \geq l + 1 \rrbracket, \llbracket x \geq l + 2 \rrbracket, \dots, \llbracket x \geq u - 1 \rrbracket, \llbracket x \geq u \rrbracket.$$

An analogous order encoding exists for less than or equal. For more information on variable encodings and encoding constraints into CNF, see, e.g., [21].

6.1.2 Combining CP and SAT: Lazy Clause Generation

One issue that can occur in a CP solver is *thrashing*, which means that the same unsatisfiable subproblem, occurring multiple times across the search tree, is repeatedly explored. The solver spends substantial time inferring that a subproblem is unsatisfiable and backtracks; however, the solver has ‘forgotten’ this inference if and when it reencounters the same subproblem, resulting in the same substantial time being spent. If one can make the solver ‘remember’ inferences, one could significantly speed up the overall search.

Since CP and SAT operate on similar fundamental principles regarding search and inference, they can be combined if one can connect the respective high- and low-level representations. The key observation is that the inferences of constraint propagators can be represented as explanations and put into an implication graph, using SAT encodings of the variables involved [111]. Consider the following simple example.

Example 6.1. Suppose there is a constraint $x + y \leq z$ with initial domains $D(x) = 0..7$, $D(y) = 0..7$, and $D(z) = 0..14$. Suppose that the domain of y is updated to $D(y) = 5..7$. A constraint propagator could infer that the domain of z must then be changed to $D(z) = 5..14$ because the sum on the left-hand side now has a minimum value of 5; thus, the minimum value of the right-hand side (i.e., z), must also have a minimum value of 5. This inference can be explained in SAT as follows: $\llbracket y \geq 5 \rrbracket \rightarrow \llbracket z \geq 5 \rrbracket$.

The antecedents (e.g., $y \geq 5$) of the generated explanations are facts implied by the current domains (e.g., $D(x) = 0..7$, $D(y) = 5..7$, and $D(z) = 0..14$), and the consequent (e.g., $z \geq 5$) is the inference made.

Using this connection, one can enhance a SAT solver with inferences only possible in CP. Similarly, it solves the problem of thrashing in CP solvers. It is, however, not efficient to fully and eagerly encode a CP model, including all inferences, into SAT directly. Instead, one can successively and lazily introduce these explanations into an implication graph. This kind of solver is called a *lazy clause generation* (LCG) solver. For two possible ways of implementing LCG, see [III] and [48]. A popular LCG solver is Chuffed [29], which we will use and extend in this paper.

In summary, an LCG solver connects Boolean variables concerning bounds (and fixed values) to the integer variables. For example, the Boolean $\llbracket x \geq d \rrbracket$ holds if the integer variable x takes a value greater than or equal to d . An LCG solver tracks the reasons justifying all propagated domain changes during search as explanations in an implication graph. Just like in pure SAT, in case of a failure, these explanations are used to find *nogoods*, which extract the reason for the failure as a new (previously implicit) constraint, representing a wrong choice made earlier during search and preventing it from reoccurring. We can then backjump to the point just before this choice was made and add the new constraint explicitly to prevent making the same erroneous choice again.

6.1.3 Real Relaxation

The dispersion constraints will be examined over a set X of integer-valued variables. It will often be helpful to relax the integrality constraint when creating lower bounds on dispersion values. This relaxation may result in weaker bounds than otherwise possible but will allow calculations to be faster. Real-valued relaxations will be used as follows.

Definition 6.2 (Real Relaxation). An $\mathbb{R}$ -assignment of a set of variables X is a mapping θ from X to $\mathbb{R}$ such that $\text{lb}(x) \leq \theta(x) \leq \text{ub}(x), \forall x \in X$.

Another concept used in this chapter is that of centring an assignment around a given real value [117]. Given a value v to centre around, all variables will be assigned a value as close as possible to this v . To do this, we construct an assignment that assigns each variable to a value in its domain as close as possible to v . In particular, variables

whose domain contains v are assigned to v . Formally, this concept is defined as follows.

Definition 6.3 (Centred Assignment). Given a real value v , a v -centred $\mathbb{R}$ -assignment, denoted θ_v , is defined as:

$$\theta_v(x) = \begin{cases} \text{ub}(x) & \text{if } \text{ub}(x) \leq v \\ \text{lb}(x) & \text{if } \text{lb}(x) \geq v \\ v & \text{otherwise} \end{cases}$$

6.2 THE SPREAD CONSTRAINT

The first measure of dispersion we consider is *variance*, or *spread*. The spread constraint (introduced in [117]) was initially defined as $\text{spread}(X, \mu, \sigma, \tilde{x})$, where variable μ is constrained to be the mean of the array of (say n) variables $X = [x_1, \dots, x_n]$, σ the standard deviation, and $\tilde{x}$ the median. A slightly different form will be examined in this chapter.

Let $\text{spread}(X, M, v)$ be defined to constrain variable M to be the sum of the variables in array X , i.e., $\sum_{i=1}^n x_i$, and variable v to be equal to n^2 times the variance of X , i.e., $n^2 \sigma^2$. The advantage of this form is that each variable takes integer values. By making use of the alternate expression of variance (see Section 2.5, page 60), the following equations hold:

$$v = n \sum_{i=1}^n x_i^2 - M^2 \quad \wedge \quad M = \sum_{i=1}^n x_i, \quad (6.1)$$

Since all variables involved are integer, we can use this formula within a solver that does not support floating-point variables.

Unfortunately, when n is large, the size of $n^2 \sigma^2$ may often push the boundaries of integers that most solvers can deal with. Thus, this chapter also introduces a relaxed version of the spread constraint that works with a given fixed positive scaling factor s . Avoiding multiplication by n , $\text{spread}(X, M, v, s)$ constrains M to be the sum of X and $v = \lfloor \sigma^2 \cdot s \rfloor$, where σ^2 is the variance of X scaled (by s) and rounded down. For instance, $s = 1$ means whole number and $s = 100$ means percentage. Note that when this is used as a lower bound, it is always correct. The benefit of this relaxed form of the constraint is that it, again, only involves integer variables.

The constraint can be defined as a decomposition in MiniZinc using Equation (6.1), as follows.

```

1 predicate spread(array[int] of var int:X, var int:M, var int:v,
2                 int:s) =
3     let { int: n=length(X);
4         array[int] of var 0..infinity: sq = [x * x | x in X];
5         var 0..n*ub(sum(sq)): v1 = n*sum(sq);
6         var 0..max(lb(M)*lb(M), ub(M)*ub(M)): Msq = M*M;
7         var 0..ub(v1): v2 = v1 - Msq;
8         var 0..ub(v2)*s: v3 = v2*s; }
9     in v = v3 div (n*n) /\ M = sum(X);
    
```

Note that `lb` and `ub` return the best known, *at compile time*, lower and upper bound respectively of the argument expression. Note, also, that the first version of `spread` is implemented by `spread(X,M,v) = spread(X,M,v,n2)`.

6.2.1 Lower Bound on Variance

This section presents a propagator for filtering the lower bound of the variance of a set of decision variables. The propagator uses a binary chop algorithm; thus, we arrive at a log-linear time complexity.

The following formula defines a real-value relaxed lower bound for the variance:

$$\text{LBV}(X, M) = \min_{m=\text{lb}(M)}^{\text{ub}(M)} \text{LBV}'(X, \frac{m}{n}), \quad \text{LBV}'(X, v) = \frac{1}{n} \sum_{i=1}^n (\theta_v(x_i) - v)^2$$

Essentially, we pick the v -centred assignment with the lowest variance. We take inspiration from [117], which also proposes a propagator for variance. Note, however, that the definition herein is slightly different from the one in [117], to allow log-linear filtering. The difference is that, in LBV' , v is used in place of the mean of $x_1, \dots, x_n$. This gives us some helpful properties, as we will prove below. First, the following lemma is required for us to prove that LBV is a lower bound on the variance.

Lemma 6.1. $\text{LBV}(X, M)$ is a lower bound on σ^2 , i.e., $\text{LBV}(X, M) \leq \sigma^2$.

Proof. Let $Y = \{y_1, \dots, y_n\}$ be an arbitrary assignment of the variables in X where $y_i \in D(x_i)$ and $v = M_Y/n$ be the mean of Y where $M_Y = \sum_{i=1}^n y_i$. Now, we only need to prove that $\text{LBV}'(X, v) \leq \frac{1}{n} \sum_{i=1}^n (y_i - v)^2$, i.e., the variance of the v -centred assignment is less than or equal to the variance of the assignment Y .

We do so by proving $(\theta_v(x_i) - v)^2 \leq (y_i - v)^2$ for every i . By construction of the v -centred assignment (see Definition 6.3), for any i , where $1 \leq i \leq n$, it holds either that $y_i \leq \theta_v(x_i) \leq v$ for the case $\text{ub}(x_i) \leq v$, that $v \leq \theta_v(x_i) \leq y_i$ for the case $\text{lb}(x_i) \geq v$, or that $v = \theta_v(x_i)$ (i.e., $(\theta_v(x_i) - v)^2 = 0$ otherwise). Thus, it holds $(\theta_v(x_i) - v)^2 \leq (y_i - v)^2$ in all cases. Hence, the lemma holds. $\square$

Next, we show that we need not investigate all values of v in LBV to find the lower bound. Proving that LBV' is convex in v will suffice. The following lemma is proved using the well-known fact that a sum of convex functions is convex.

Lemma 6.2. $\text{LBV}'(X, v)$ is a convex function in v .

Proof. Since a sum of convex functions is convex, we only need to prove for each i , where $1 \leq i \leq n$, that the function $f(v) = (\theta_v(x_i) - v)^2$ is convex in v . By construction of θ_v (see Definition 6.3), the difference $\theta_v(x_i) - v$ is strictly decreasing until $v = \text{lb}(x_i)$, then zero until $v = \text{ub}(x_i)$, and strictly decreasing again. Since the difference is squared, the function f is a quadratic function with a flat bottom of some length. Hence, it is convex and $\text{LBV}'(X, v)$ as well. $\square$

Thanks to Lemma 6.2, one can use binary search on the values m in M to find a minimum value for LBV' . And thanks to Lemma 6.1, only the mean values $v = \frac{m}{n}$ with $\text{lb}(M) \leq m \leq \text{ub}(M)$ have to be considered. This follows from the proof of Lemma 6.1 since, in it, we set v to be the mean of an arbitrary assignment.

6.2.2 Algorithm and Clause Learning Explanations

Algorithm 2 defines the proposed approach to finding and asserting a lower bound on (scaled) variance. Note that when all $x \in X$ are fixed, one can simply calculate the actual variance in $\mathcal{O}(n)$ time.²⁶ The algorithm returns a clause that is a consequence of the constraint and whose right-hand side gives the new propagation of the lower bound; otherwise, it returns the trivial true clause.

The algorithm searches through the integer range $L..R$ (minimal sum to maximal sum of the variables X) of values for the sum m , resulting in the minimum value for $\text{LBV}'(X, m/n)$. The while-block on lines 3–14 searches for the minimum spread assignment, where lines 4–6 calculates the spread values of the central two v values (to obtain a slope towards the minima), lines 7–14 checks whether a minimum has been found, and if not, continues the search on the half that will contain the minimum

²⁶In this case, we still scale and round down for consistency.

```

1: procedure SpreadLb( $X, M, v, s$ )
2:    $L, R \leftarrow \text{lb}(M), \text{ub}(M)$ 
3:   while  $L < R$  do
4:      $m_L \leftarrow L + \lfloor (R - L) / 2 \rfloor$ 
5:      $m_R \leftarrow m_L + 1$ 
6:      $V_L, V_R \leftarrow \text{LBV}'(X, m_L/n), \text{LBV}'(X, m_R/n)$ 
7:     if  $\min(V_L, V_R) = 0$  then return true
8:     if  $V_L = V_R$  then
9:        $V, m \leftarrow V_L, m_L$ 
10:    break
11:    if  $V_L < V_R$  then
12:       $V, m, R \leftarrow V_L, m_L, m_L$ 
13:    else if  $V_L > V_R$  then
14:       $V, m, L \leftarrow V_R, m_R, m_R$ 
15:   $Vs \leftarrow \lfloor V \times s \rfloor$ 
16:  if  $Vs \leq \text{lb}(v)$  then return true
17:   $EX \leftarrow \{ \llbracket x_i \leq \text{ub}(x_i) \rrbracket \mid \text{ub}(x_i) < m/n \} \cup \{ \llbracket x_i \geq \text{lb}(x_i) \rrbracket \mid m/n < \text{lb}(x_i) \}$ 
18:  if  $m = \text{ub}(M)$  then  $EX \leftarrow EX \cup \{ \llbracket M \leq \text{ub}(M) \rrbracket \}$ 
19:  if  $m = \text{lb}(M)$  then  $EX \leftarrow EX \cup \{ \llbracket M \geq \text{lb}(M) \rrbracket \}$ 
20:  if  $Vs > \text{ub}(v)$  then
21:    return  $\llbracket v < Vs \rrbracket \wedge \bigwedge_{l \in EX} l \rightarrow \text{false}$ 
22:  else
23:    return  $\bigwedge_{l \in EX} l \rightarrow \llbracket v \geq Vs \rrbracket$ 

```

Algorithm 2: Only run if not all fixed. $n = |X|$

(depending on the slope). The algorithm computes the values V_L of LBV' for the (integer) midpoint m_L of $L..R$, and V_R for the position one to the right of m_L , namely m_R . If either V_L or V_R gives 0, then the overall variance bound is 0, and we return since no propagation is possible. If V_L and V_R are equal, then we have found the lowest value and break from the loop since, thanks to convexity, two variance-equal and neighbouring points can only occur if they are both minima. Otherwise, the relative values of V_L and V_R will be used to decide which half of the interval to keep. The algorithm stores the best value found V and the sum value m where it occurs and updates the appropriate bound L or R . If the interval shrinks to a singleton, then the loop (lines 3–14) exits. The lower bound Vs is computed on the scaled version of the variance v . If the lower bound is already subsumed, then a trivial clause *true* is returned. Otherwise, the algorithm collects the literals that will appear in the explanation for the bound change or failure in EX . In the explanation, all the lower and upper bounds are collected that appear in the v -centred valuation, but not the bounds of the overlapping variables (i.e., those with v between their upper and lower bounds). If the optimal m value resides at one of the extremes of the sum M , then the appropriate literal is collected. Finally, if the new bound will cause unsatisfiability,

then the algorithm adds in the current upper bound for v and returns an explanation for failure. Otherwise, an explanation for the new bound $v \geq Vs$ is returned.

This algorithm runs in $\mathcal{O}(n \log d)$ time, where $n = |X|$ and $d = \text{ub}(M) - \text{lb}(M)$. We can assume this algorithm is $\mathcal{O}(n \log n)$ when d is of size $\mathcal{O}(n^k)$ for any reasonably small constant k .

The following example illustrates how the explanations are formed.

Example 6.2. Consider an execution of the algorithm where $X = [x_1, x_2, x_3, x_4]$ with current domains $[0, 0.4, 2.3, 4.6]$, M has domain $6..10$, v has domain $0..400$ and $s = 100$. We start with $L = 6$, $R = 10$. Then $m_L = 8$ and $m_R = 9$. The next step is to compute $V_L = \text{LBV}'(X, 2) = 2$ and $V_R = \text{LBV}'(X, 2.25) = 2.015625$. We set $V = V_L$, $m = 8$ and $R = 8$. The next step is now to compute $m_L = 7$ and $m_R = 8$, and $V_L = \text{LBV}'(X, 1.75) = 2.0123$ and $V_R = \text{LBV}'(X, 2) = 2$. We set $V = V_R$, $m = 8$ and $L = 8$. We exit the loop, computing $Vs = 200$. The explanation we ultimately collect is $EX = \{\llbracket x_1 \leq 0 \rrbracket, \llbracket x_4 \geq 4 \rrbracket\}$ returning the explanation clause $\llbracket x_1 \leq 0 \rrbracket \wedge \llbracket x_4 \geq 4 \rrbracket \rightarrow \llbracket v \geq 200 \rrbracket$. Note that we do not collect $\llbracket x_3 \geq 2 \rrbracket$. This is because if the domain of x_3 is extended arbitrarily much in either or both directions, then it would still overlap with 2, and if it were reduced arbitrarily much from either or both directions, then the resulting minimal variance would increase. Thus, the lower bound holds regardless of the value of x_3 . $\square$

It remains to prove that the algorithm and the generated explanations are correct.

Lemma 6.3. Algorithm 2 is correct and returns a clause that is a consequence of the constraint $\text{spread}(X, M, v, s)$.

Proof. The result clearly holds for the trivial clause true. The correctness of the value $V = \text{LBV}'(X, m/n)$ computed by Algorithm 2 follows from Lemma 6.1 and Lemma 6.2. We now show that the explanation collected in EX is correct, i.e., $EX \rightarrow s \times \sigma^2 \geq \lfloor s \times V \rfloor$ is universally true, which is the clausal explanation in both non-trivial cases. Assume to the contrary that there is a solution η of $\text{spread}(X, M, v, s)$ satisfying EX that has smaller variance. We can assume η is v -centred for some v and bounds on X (not necessarily the current bounds) [117]. But then η must set all x_i variables not appearing in EX to v ; otherwise, there is an assignment allowed by EX which would be better. But η as a function of v is identical to $\text{LBV}'(X, v)$ around the minima m/n . That is, all variables with an upper bound below m/n are set to their upper bound, which is part of EX ,

and similarly, for variables with a lower bound above m/n ; the remaining variables take value v . If m is not the upper or lower bound of the sum M , then m/n is a local (and global) minima of $LBV'(X, v)$, and hence also of η . If m is the upper or lower bound of M , then this bound is included in EX , and again m/n is the minima for $LBV'(X, v)$ and η . Contradiction. $\square$

6.3 THE GINI COEFFICIENT CONSTRAINT

The second measure of dispersion we consider is the *Gini coefficient*. The *Gini coefficient* of X is conventionally formulated as:

$$\text{Gini}(X) = \frac{\sum_{i=1}^n \sum_{j=i+1}^n |x_i - x_j|}{n \sum_{i=1}^n x_i}. \quad (6.2)$$

Calculating this naïvely takes $\mathcal{O}(|X|^2)$ time, but an alternative linear-time formulation is well-known if X is sorted in a non-decreasing order [41]:

$$\text{Gini}(X) = \frac{\sum_{i=1}^n (2i - n - 1)x_i}{n \sum_{i=1}^n x_i}. \quad (6.3)$$

Note that the Gini coefficient is only meant to apply when $x_i > 0$ since the sum divisor can be zero otherwise.

Suppose we have an array X of n variables $x_1, \dots, x_n$. Define the Gini constraint $\text{gini}(X, M, g, s)$ to constrain variable M to be the sum of X (i.e., $\sum_{i=0}^n x_i$) and $g = \lfloor \text{Gini}(X) \times s \rfloor$ to be the Gini coefficient variable expressed as an integer scaled by integer value s and rounded down. Using Equation (6.2), this constraint can be defined as a decomposition in MiniZinc as follows:

```

1 predicate gini(array[int] of var int:X, var int:M, var int:g,
2               int:s) =
3   let { int: n=length(X);
4         int: range_size = ub_array(X) - lb_array(X);
5         array[int] of var 0..range_size: diffs
6           = [ abs(X[i]-X[j]) | i,j in index_set(X) where i<j];
7         var 0..s*(n div 2 + 1)*(n div 2)*(range_size): tot_diff
8           = sum(diffs) * s;
9         var 0..ub(tot_diff) div n: result_ = tot_diff div n;
10        var 0..ub(result_): result = result_ div M }
11   in v = result /\ M = sum(X);

```

Note that this tries to bound the initial domains reasonably tightly to reduce overhead in the solver. The function `lb_array` (and `ub_array`) returns the least (and greatest) possible value known *at compile time* of an array of decision variables. The upper bound on the sum of differences of numbers in the range $(\min X)..\!(\max X)$ is to have half at each end, thus $\lceil \frac{n}{2} \rceil \times \lfloor \frac{n}{2} \rfloor \times (\text{range_size})$ where *range_size* is the difference between the minimum and maximum possible values of X .

Before going into the details of the propagator algorithm, we will prove a lemma that will help us disregard any non- ν -centred assignment. The formulation from Equation (6.3) will be used since it is faster to calculate for ν -centred assignments and will simplify the proofs in this section. The following result shows that the minimum Gini coefficient of a set of variables arises for some ν -centred assignment of these variables.

Lemma 6.4. For a given fixed mean value μ of variables X , a ν -centred assignment leads to the minimal value of the sum of absolute values of pairwise differences.

Proof. We prove the lemma by contradiction. Assume w.l.o.g., that a non- ν -centred assignment ϕ for X is presented in non-decreasing order, i.e., $\phi(x_i) \leq \phi(x_{i+1})$ for $1 \leq i < n$, with the mean $\mu = \frac{1}{n} \sum_{i=1}^n \phi(x_i)$, and let $A = \sum_{i=1}^n (2i - n - 1)\phi(x_i)$ be the sum of the absolute values (by Equation (6.3)). Assume to the contrary that ϕ leads to a minimum sum A . Since the mean μ is fixed, we need not analyse the denominator of the Gini coefficient.

At least one variable must be unfixed; otherwise, ϕ must be ν -centred. There must be at least one variable x_j with $\phi(x_j) < \text{ub}(x_j)$, otherwise ϕ is ν -centred with $\nu = \max_{i=1}^n \phi(x_i)$. Furthermore, there must be at least one variable x_k with $\text{lb}(x_k) < \phi(x_k)$ (analogously) and $1 \leq j < k \leq n$ (otherwise ϕ must be ν -centred). Without loss of generality, we can assume that $\phi(x_j) < \phi(x_{j+1})$ and $\phi(x_{k-1}) < \phi(x_k)$, because one can reorder the variables having the same value in ϕ , so that indices j and k are the greatest and least one with the values $\phi(x_j)$ and $\phi(x_k)$, respectively.

We construct a new assignment ϕ' for X , having the same mean as ϕ , as follows: $\phi'(x_i) = \phi(x_i)$ for $1 \leq i \leq n$ with $i \notin \{j, k\}$, $\phi'(x_j) = \phi(x_j) + \delta$, and $\phi'(x_k) = \phi(x_k) - \delta$ where $\delta \in \mathbb{R}$ and $\delta > 0$ is chosen, so that $\phi(x_j) + \delta \leq \min\{\phi(x_{j+1}), \text{ub}(x_j)\}$ and $\max\{\phi(x_{k-1}), \text{lb}(x_k)\} \leq \phi(x_k) - \delta$. Note that ϕ' is also in non-decreasing order, thus, its sum of absolute values is $A' = \sum_{i=1}^n (2i - n - 1)\phi'(x_i)$ by Equation (6.3). If A is the minimal sum of absolute values then $0 \geq A - A'$. Since ϕ and ϕ' only differ in

the variable values for x_j and x_k , it holds $0 \geq (2j - n - 1)(\phi(x_j) - \phi'(x_j)) + (2k - n - 1)(\phi(x_k) - \phi'(x_k))$, which is $0 \geq (2j - n - 1)(-\delta) + (2k - n - 1)\delta = 2(k - j)\delta$. Due to $j < k$, we have $2(k - j)\delta > 0$, which contradicts the assumption that A is the minimal sum of absolute values. $\square$

Since fixing the mean fixes the divisor of the Gini coefficient, the following is an obvious consequence. Given a fixed mean μ , the ν -centred assignment θ_ν where $\mu = \frac{1}{n} \sum_{i=1}^n \theta_\nu(x_i)$ leads to minimal Gini coefficient. And this must hold for any mean.

Corollary 6.1. A Gini-minimal assignment of X must be ν -centred.

6.3.1 A Lower Bound on the Gini Coefficient

This section presents a log-linear time propagator for filtering the lower bound of the Gini coefficient using a binary chop approach.

Similarly to the approach used for variance, the first step is to show how to calculate a lower bound on the Gini Coefficient. Again, the key to this is finding the best ν -centred assignment, which would have to be a lower bound.

Let $L = \min_{i=1}^n \text{lb}(x_i)$ be the least lower bound and $U = \max_{i=1}^n \text{ub}(x_i)$ be the greatest upper bound, then we can define a real-value relaxed lower bound on the Gini coefficient of X as follows.

$$\text{LBG}(X) = \min_{\beta=L}^U \text{Gini}(X, \beta), \quad \text{Gini}(X, \nu) = \frac{\sum_{i=1}^n \sum_{j=i+1}^n |\theta_\nu(x_i) - \theta_\nu(x_j)|}{n \sum_{i=1}^n \theta_\nu(x_i)}.$$

By Equation (6.3) and given a non-decreasingly sorted list of the bounds of X , one can calculate $\text{Gini}(X, \nu)$ in linear time over $|X|$. The question, then, is whether all values between L and U have to be considered in LBG to guarantee that a lower bound is found. The next result shows that this is not the case.

Lemma 6.5. The minimum Gini coefficient occurs at a ν -centred assignment where $\nu \in \cup_{i=1}^n \{\text{ub}(x_i), \text{lb}(x_i)\}$.

Proof. By Corollary 6.1, we know that the minimum Gini coefficient occurs with a ν -centred assignment. We need to show that there are no local minima in any segment between two consecutive bounds, which, if true, will allow us to disregard any point that is not a bound of one of the variables when searching for the lower

bound on Gini. Consider a segment between two consecutive bounds l and u . Since no other bound occurs in between l and u , the three sets of variables in X that will be set to their upper bounds, their lower bounds, and v , respectively, will remain constant across all v -centred assignments when $l \leq v \leq u$. Assume these variables in X , and what they are set to, occur sorted such that $B = 1..j-1$ are the indices of the variables below the segment—i.e., where $\text{ub}(x_i) \leq l$; the indices $C = j..j+k-1$ are of the variables that cover the segment—i.e., where $\text{lb}(x_i) \leq l \leq u \leq \text{ub}(x_i)$; and $A = j+k..n$ are the indices of the variables above the segment—i.e., where $\text{lb}(x_i) \geq u$. Using Equation (6.3), the formula for the Gini coefficient across this segment $G(v)$ is hence:

$$G(v) \equiv \frac{\sum_{i \in B} (2i - n - 1) \text{ub}(x_i) + \sum_{i \in C} (2i - n - 1) v + \sum_{i \in A} (2i - n - 1) \text{lb}(x_i)}{n (\sum_{i \in B} \text{ub}(x_i) + \sum_{i \in A} \text{lb}(x_i) + kv)}$$

Let $D = \sum_{i \in B} (2i - n - 1) \text{ub}(x_i) + \sum_{i \in A} (2i - n - 1) \text{lb}(x_i)$, $N = \sum_{i \in B} \text{ub}(x_i) + \sum_{i \in A} \text{lb}(x_i)$ and $c = \sum_{i \in C} (2i - n - 1)$. Differentiating $G(v)$ with respect to v , the following is obtained by the quotient rule

$$\frac{\partial}{\partial v} \frac{D + cv}{n(N + kv)} = \frac{cn(N + kv) - (D + cv)nk}{n^2(N + kv)^2} = \frac{cN - kD}{n(N + kv)^2}$$

Note that the sign of the slope is determined by the numerator, and does not depend on v . Hence, there can be no local minima in the segment. $\square$

From the above result, it follows that there is no need even to consider the real-relaxed case for Gini, as all values v of importance are integers. Next, we prove that one can use binary chop to find the v -centred assignment that minimises the Gini coefficient.

Lemma 6.6. If we go through the segments of LBG in increasing order of the sorted end-points, then a positive slope is not (directly or indirectly) followed by a negative slope.

Proof. Let us revisit the derivative of each segment, i.e., $\frac{cN - kD}{n(N + kv)^2}$. The sign of its slope is determined by the numerator $cN - kD$. We can rewrite c to a closed form formula because it is an arithmetic sum. We get $k \frac{((2j - n - 1) + (2(j + k - 1) - n - 1))}{2}$ which simplifies to $k(k + 2j - n - 2)$. Rewriting the numerator we get $k(k + 2j - n - 2)N - kD$, which we can simplify to $k((k + 2j - n - 2)N - D)$. Because k is always non-negative, the outer k cannot determine the sign of the slope, only its

magnitude, thus we can safely ignore it for our purposes. Ignoring the outer k and expanding N and D we get $(\sum_{i \in B} (k + 2j - n - 2) \text{ub}(x_i) - (2i - n - 1) \text{ub}(x_i)) + (\sum_{i \in A} (k + 2j - n - 2) \text{lb}(x_i) - (2i - n - 1) \text{lb}(x_i))$. This simplifies to $\sum_{i \in B} (k + 2j - 2i - 1) \text{ub}(x_i) + \sum_{i \in A} (k + 2j - 2i - 1) \text{lb}(x_i)$. Let us denote the first term by T_B and the second by T_A . The maximum value of $i \in B$ is $j - 1$; thus, all terms of T_B are non-negative; and the minimum value of $i \in A$ is $j + k$; thus, all terms of T_A are non-positive. At any segment where the slope is positive, we must have that $T_B > |T_A|$. If we move from one segment to the next, variables could move from A to the overlap part, from the overlap part to B , or both. Let us consider two cases: (1) at least one variable moves from A to the overlap, and (2) at least one variable moves from the overlap to B .

Case (1): In this case, $|T_A|$ will decrease because at least one negative term will be removed, and k will increase by at least one. Moreover, T_B will increase because k will increase by at least one.

Case (2): In this case, $|T_A|$ will decrease because $k + 2j$ will increase by at least one (while k decreases, j must increase the same amount) and T_B will increase because at least one more positive term will be introduced, and $k + 2j$ will increase by at least one.

As a result, moving from one segment where $T_B > |T_A|$ to the next, we must maintain that $T_B > |T_A|$. Hence, once the slope is positive it cannot become negative. $\square$

The following corollary is an obvious consequence of this result:

Corollary 6.2. LBG has no local minimum that is not a global minimum.

Given this result, we can again use binary chop to find the global minimum of $\text{Gini}(X, v)$.

6.3.2 Algorithm and Clause Learning Explanations

The algorithm for computing a lower bound on the (scaled) Gini coefficient is given in Algorithm 3. It does a binary chop across the sorted list of bounds of the x variables: $\beta_1, \dots, \beta_{2n}$. L and R hold the left and right bound of indices into this array, for where the *rightmost* minimum lies. The algorithm computes the midpoint l of L and R and its neighbour r and compares the Gini values. If they are identical in value, the interval $l..r$ is extended to the left until a difference is found, or the entire region

Require: $B = [\beta_1, \dots, \beta_{2n}]$ is a sorted array of bounds of X

```

1: procedure GiniLb( $X, M, g, s$ )
2:    $L, R \leftarrow 1, 2n$ 
3:   while  $L < R$  do
4:      $l \leftarrow L + \lfloor (R - L) / 2 \rfloor$ 
5:      $r \leftarrow l + 1$ 
6:      $G_l \leftarrow \text{Gini}(X, \beta_l)$ 
7:      $G_r \leftarrow \text{Gini}(X, \beta_r)$ 
8:     while  $G_l = G_r$  do
9:       if  $l > L$  then
10:         $l, G_l \leftarrow l - 1, \text{Gini}(X, \beta_{l-1})$ 
11:      else
12:         $L \leftarrow r$ 
13:      break
14:     if  $G_l < G_r$  then
15:        $R \leftarrow l$ 
16:     else
17:        $L \leftarrow r$ 
18:    $m, G \leftarrow R, \text{Gini}(X, \beta_R)$ 
19:    $Gs \leftarrow \lfloor G \times s \rfloor$ 
20:   if  $Gs \leq \text{lb}(g)$  then return true
21:    $EX \leftarrow \{ \lfloor x_i \rfloor \mid \text{ub}(x_i) \leq \beta_m \} \cup \{ \lceil x_i \rceil \mid \beta_m \leq \text{lb}(x_i) \}$ 
22:   if  $Gs > \text{ub}(g)$  then
23:     return  $\llbracket g < Gs \rrbracket \wedge \bigwedge_{l \in EX} l \rightarrow \text{false}$ 
24:   else
25:     return  $\bigwedge_{l \in EX} l \rightarrow \llbracket g \geq Gs \rrbracket$ 

```

Algorithm 3: Only run if not all fixed. $n = |X|$

from L to r is filled, in which case a chop is performed, setting $L = r$. We can incrementally compute $\text{Gini}(X, \beta_{i-1})$ from $\text{Gini}(X, \beta_i)$ in constant time with a little care. If they are not identical in value, an update of L or R takes place to keep the rightmost minimum between them. The loop terminates when $L = R$. The algorithm calculates the Gini value G and its scaled version Gs . Note that the *rightmost* v -value is used when finding a range of bounds with the same Gini value. If the new lower bound is not stronger than the current bound, a *true* clause is returned. Otherwise, an explanation antecedent is collected as follows: all lower bounds where x is less than *or equal to* the v value β_m and all upper bounds where x is greater than *or equal to* the v -value β_m . If the new bound causes a violation, the algorithm returns an explanation for the violation; otherwise, it returns an explanation for the new bound. Overall the algorithm is $\mathcal{O}(n \log n)$ since the initial sorting is of this complexity, the while loop can only execute $\mathcal{O}(\log n)$ times, and the Gini calculations are linear (using Equation (6.3)).

Note also that the scaling and rounding to integer values occur after the binary chop has terminated. This is important since not having the best accuracy when calculating the slopes can lead to the algorithm terminating with a solution that is only optimal given the rounding to integer values. Using this can then lead to incorrect explanations.

Lemma 6.7. Algorithm 3 is correct and returns a clause which is a consequence of $\text{gini}(X, M, g, s)$.

Proof. The result clearly holds for the trivial clause true. The correctness of the value G computed by Algorithm 2 follows from Lemma 6.4, Lemma 6.5 and Lemma 6.6. We now show that the explanation collected in EX is correct. Assume to the contrary that there is a solution η of $\text{gini}(X, M, g, s)$ satisfying EX that has Gini coefficient $G' < G$. We can assume η is v -centred for some v and some bounds on X (not necessarily the current bounds) by Lemma 6.4. But then η must set all x_i variables not appearing in EX to v ; otherwise, there is an assignment allowed by EX which would be no worse. So we can consider η as defined by v and the bounds appearing in EX . Hence, it has the same properties as the $\text{Gini}(X, v)$ but with fewer bounds: importantly, it has negatively sloped and flat segments, followed by positively sloped and flat segments. Let us increase v smoothly from β_m . Because the slope of the curve $G(v)$ defined in Lemma 6.5 is positive by construction (this is why it is crucial to take the rightmost point with the least Gini value), and this slope only depends on the bounds in EX , the Gini value for η must increase. If we move v smoothly from β_m downwards then the slope on $G(v)$ defined in Lemma 6.5 is either negative or zero by construction. Since the slope only depends on the bounds in EX , if it is negative, then the Gini value of η must increase. If it is zero, then the Gini value must stay the same. This holds until we cross a lower bound in $\text{lb}(x_i)$ not in EX , until then the Gini values for η are the same as $\text{Gini}(X, v)$. Nevertheless, once we cross this bound, the sum defined by solution η is smaller than that for $\text{Gini}(X, v)$, and, overall, the Gini value must be larger. Contradiction. $\square$

6.4 EXPERIMENTAL EVALUATION

This section presents an experimental evaluation of the spread and Gini propagators introduced above.²⁷ We use four configurations for each propagator: their decompos-

²⁷Models, data, and raw results are available at <https://github.com/aekh/CP22-Extras>

prob	prop	proved			best			no solution	
		total	first	sole	total	first	sole	total	sole
Spread	Dcmp	38	17	2	43	22	7	127	127
Spread	Simp	25	2	0	85	60	58	0	0
Spread	LB-TL	38	12	0	59	30	14	0	0
Spread	LB	40	25	3	115	96	71	0	0
Gini	Dcmp	33	11	0	37	17	6	97	97
Gini	Simp	25	10	0	171	154	144	0	0
Gini	LB-TL	40	15	0	42	32	11	0	0
Gini	LB	47	40	6	26	20	2	0	0

Table 6.1: Summary of the results of the propagators for the 195 dispersion-only instances

ition (Dcmp), from page 201 or 205; a simple minimal propagator (Simp), which only propagates on the dispersion once all variables X are fixed; the proposed binary-chop lower-bound propagator with learning (LB); and a variation of it with trivial learning (LB-TL). The trivial learning simply uses all bounds of all the variables as explanations, which leads to larger, and thus less general and weaker nogoods. The above propagators are implemented in Chuffed.²⁸ The below experiments were run using MiniZinc 2.5.5 [107] on an Intel Xeon 8260 CPU (24 cores) with 268.55 GB of RAM. A single core, 8 GB of RAM, and a 20-minutes timeout were allocated to solve each instance.

6.4.1 Dispersion Only

Let us begin with a simple problem to illustrate the general performance. Assume we have n variables $x_1, \dots, x_n \in X$, each with domain $l_i..u_i$. For each i , two integers are uniformly randomly drawn from the range $-n^2..n^2$ (if equal, the second gets redrawn until not equal). The lower number becomes l_i , and the higher becomes u_i . For each $n \in \{2, 3, \dots, 40\}$, we create five instances, resulting in $5 \times 39 = 195$ instances. The goal is to minimise dispersion. Since Gini is ill-defined for negative values, all numbers of all Gini instances are uniformly incremented (if needed) until the least one reaches 1.

We will see that for these simple benchmarks, using learning with the global propagators simply adds overhead since the search is so simple that little is repeated. Learning does improve the decomposition, though, since it can learn on intermediate variables.

²⁸<https://github.com/aekh/chuffed>, forked from the develop branch (commit hash dc0775e)

The results are shown in Table 6.1. The table shows, for each method, the *total* number of instances out of the 195 where it *proved* optimality, the number of times the algorithm proved optimality *first* (i.e., was the fastest one to prove optimality among the algorithms tested) on an instance, and the number of instances where it was the *sole* algorithm to prove optimality. It then shows for all instances the *total* number of instances where the algorithm found the *best* solution of any found, the number of instances where it did this *first* of all algorithms, and the number of times it was the *sole* method to find this best solution. Finally, the *total* number of instances is shown where the method found *no solution*, along with how many it was the *sole* method to do so. Clearly, for *spread*, LB is the best method in all categories. It finds the best solution for all instances and is the fastest on almost all. The decomposition fails on most instances because the domain sizes of the intermediate values get too large for the solver to handle. For *gini*, perhaps somewhat surprisingly, the simple propagator is best at finding solutions. This is because the best solutions are usually found early in the search, so the fast, simple propagator will find good solutions very quickly compared to more expensive propagators LB and LB-TL. Also, intermediate values are smaller since numbers are not squared. The decomposition is competitive in terms of optimality but still fails on many instances.

6.4.2 Job-Shop Scheduling

The dispersion-only problems are not complex enough to illustrate why learning is needed for dispersion propagators. Let us, instead, run experiments on a type of problem where using a learning solver is often desirable, namely, a scheduling problem.

Consider the job-shop scheduling problem of Figure 2.4 (page 44) but with modifications to have multiple agents as follows. In addition to the jobs *JOBS*, machines *MACs*, and tasks *TSKs*, we also have a set of agents *A*, such that each job $j \in \text{JOBS}$ belongs to precisely one agent $a \in A$.

The experiments consider a limited horizon (after which scheduling jobs is pointless due to zero utility) with optional jobs (because not all jobs will fit within the limited horizon). The utility of a given agent is the machine-hours used by their jobs. Further, assume that each agent submits more jobs than can be accommodated. This results in comparable utility functions.

The *objective* of the overall problem is to either maximise efficiency (use as many machine-hours as possible) or minimise dispersion (give similar amounts of machine-hours to each agent), respectively denoted MaxEff and MinDis. In either case, a bound

on the other metric is used to ensure some solution quality with respect to it. For example, giving no agent any machine-hours is a minimal dispersion schedule but not very efficient.

The large instances (30 or more jobs) of [150] are used (resulting in 50 instances),²⁹ and for each instance we split the jobs uniformly across n agents, for every n so that each agent has between 10 and 20 jobs (resulting in $20 \times 2 + 20 \times 3 + 10 \times 6 = 160$ instances). This uniform split is performed three times to generate a more diverse instance space (resulting in $160 \times 3 = 480$ instances). The experiments include runs of these on both configurations (MaxEff and MinDis), making the resulting number of instances to test each propagator 960. As will be explained below, two setup phases are needed to obtain some necessary parameters.

The first setup phase runs, with a 20 minute timeout for each instance, a variation of the problem where the objective is to minimise the makespan when all jobs are scheduled. The result of this will be a good indication of how long to set the limited horizon for each instance such that no agent can fit all their jobs. The limited horizon for the instance in question is set to be half of this minimum makespan.

The second setup phase runs, again with a 20 minute timeout for each instance, a variation of the problem where the objective is to maximise machine-hour utilisation H given the limited horizon. The result will give a good indication of the optimal utilisation possible for each instance. The actual experiments will then use this number H to calculate bounds for machine-hour utilisation and dispersion. For spread, a scale of 1 is used because the numbers are already large, and for Gini, a scale of 10,000 is used. For both spread and Gini with MinDis, $\lceil H \times 0.8 \rceil$ is a lower bound on efficiency, allowing 20% slack on efficiency; for spread with MaxEff, $\lfloor (0.2 \times H/|A|)^2 \rfloor$ is an upper bound on spread, allowing a 20% standard deviation from the per-agent average machine-hour utilisation. For Gini and MaxEff, 2,000 is an upper bound on Gini, allowing a 20% unequal Gini distribution.

The cactus plots in Figure 6.1 show the number of problems solved to optimality at different times for the different variations: using the spread problems on the top, the Gini problems on the bottom, the MaxEff objective on the left, and the MinDis objective on the right. In general, solving with the MaxEff objective is much more challenging than with MinDis.

²⁹There are 20 instances with 30 jobs (the first half of which have 15 machines and the rest have 20 machines), 20 instances with 50 jobs (again, the first half of which have 15 machines and the rest have 20 machines), and 10 instances with 100 jobs, all with 20 machines.

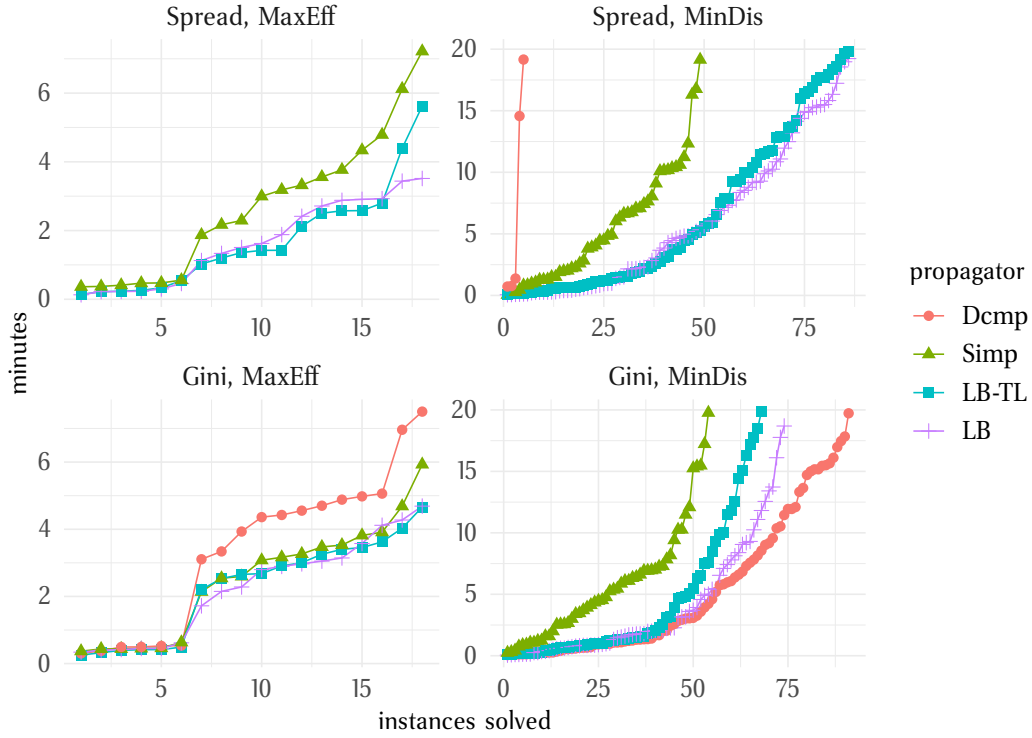


Figure 6.1: Cactus plot of instances solved to proof of optimality of the propagators for jobshop

Clearly, for spread the decomposition is very weak; this is because the size of intermediates involved is enormous and often goes outside the range that solvers can handle. The simple propagator is much better, and the full spread propagator is the best. Interestingly the small explanations are only slightly more effective than the trivial explanations.

The results for gini are quite different. The decomposition proved optimality more often than other methods on the minimisation problems. This is likely because the decomposition can use learning on the intermediate variables in the decomposition. Having a richer language of learning can often improve proofs of optimality/unsatisfiability. The propagator with the small explanations is the second best. For the MaxEff problems, the decomposition is performing the worst, and the other methods are roughly equivalent.

For a more detailed comparison we examine Table 6.2 which is organised like Table 6.1. Table 6.2 clearly illustrates the power of the global propagators. For spread, the propagator with the small explanations dominates all methods. For gini, while

prob	prop	proved			best			no solution	
		total	first	sole	total	first	sole	total	sole
Spread	Dcmp	5	3	0	10	8	5	927	904
Spread	Simp	67	14	0	270	137	100	15	0
Spread	LB-TL	102	48	12	433	250	148	18	0
Spread	LB	104	53	16	695	558	412	18	0
Gini	Dcmp	109	46	15	264	123	82	104	86
Gini	Simp	69	11	0	249	130	108	16	1
Gini	LB-TL	85	25	1	345	197	131	19	0
Gini	LB	92	30	2	620	500	402	19	0

Table 6.2: Summary of the results of the propagators for the 960 jobshop instances

the decomposition is best for proving optimality, overall, it is much weaker than the global propagator, which finds the best solution in over twice as many instances, and almost always finds a solution. It also more clearly illustrates the importance of small explanations; LB is far better than LB-TL in terms of best solutions. Additional experiments consisted of runs with no learning across all four methods, but it led to timeouts on all instances.

6.5 RELATED WORK

The spread constraint was introduced by [117]. Many previous methods assume that the mean is fixed, e.g. [133]. A fixed mean is no impediment when the problem is about allocating a given set of resources without mutual exclusions and uncertainties, but not for the general case where the resource usage can vary across solutions. [116] introduce a domain-consistent propagator for dispersion constraints (including spread) but again assume a fixed/given mean. While an extension for a bounded (variable) mean is proposed, the main criticism is that that algorithm runs the domain propagator for each value in the mean, essentially adding another factor to the time complexity. The resulting time complexity is essentially $\mathcal{O}(|X|^3 \cdot |D(X)| \cdot |D(\mu)|)$, which is impractical in many cases. A multi-valued decision diagram (MDD) formulation of the spread constraint is presented in [115]. They, again, assume a fixed mean. Although they also extend their algorithm to work on variable means, it does not apply to the scenarios considered in this chapter as it relies on probability density functions and cannot guarantee correct propagation when the mean has an unknown distribution.

An earlier paper by [117] introduces a bounds-consistent propagator for the spread constraint, which covers mean, median, and standard deviation (and thus indirectly variance). They assume finite-domain variables X and bounded continuous domains

for mean and standard deviation. They introduce two propagators: (i) one which runs in $\mathcal{O}(|X| + |D(X)|^2)$ time and (ii) one that is bounds consistent and runs in $\mathcal{O}(|X|^2)$. One criticism (posed in [133] which has one author in common with [117]) is that (ii) has never been implemented, is hard to understand, and contains mathematical errors. Thus, we do not compare our propagator against it. The propagator (i) assumes that $|X|$ dominates $|D|^2$, but it does not hold in use cases explored in this chapter. In the job-shop instances used in the experiments of this chapter, $|X|$ (between 2 and 10) is much smaller than $|D(X)|$ (between 0 and 47,186). This renders propagator (i) asymptotically worse than the propagator presented in this chapter, and thus we do not compare our propagator against it, either.

Propagators for several other statistical constraints have been introduced in the literature. These include, the deviation constraint [132], mean, median, and weighted mean constraints [22], an occurrence balancing constraint [20], the two-sum constraint [104] (which subsumes spread), and many other constraints [130]. [112] provide a review of when to use what balancing/dispersion constraint, depending on the nature of the situation. However, none of these spread propagators generate explanations of their propagation, as required by learning CP solvers [111].

We are not aware of any propagators for the Gini coefficient to the best of our knowledge. Bounds on the Gini coefficient have been established in the statistics literature [63, 102]; however, these approaches concern obtaining bounds on the actual Gini coefficient of an existing, large population from relatively few samples, not about filtering possibilities of an unknown population.

6.6 CHAPTER CONCLUSION

This chapter has defined efficient propagators for propagating the lower bound on variance and the Gini coefficient that also generate explanations of their inferences, allowing them to be used in learning CP solvers. Only the case of filtering the lower bound has been considered because, usually, minimisation of dispersion or keeping dispersion under some upper bound is of interest. The introduction of floating-point variables, which are usually not supported by clause learning solvers, is avoided by reasoning over appropriately scaled integer versions of the constraints. Selecting a scale is application-specific: there is a trade-off between the size of integers and fidelity.

Problems that require maximising dispersion or ensuring sufficient dispersion need filtering algorithms for the upper bound, and they require other types of approximations and roundings than presented here, which is left for future work.

Propagating the bounds of the variables X being measured is not considered. This is because preliminary experimentation indicated that such propagation happens very late in the search tree and cannot lead to significant speedups. However, the explanations generated by the presented propagators could result in some propagation on the X .

THESIS CONCLUSION

This thesis has aimed to (a) systematise the high-level modelling and online solving of dynamic combinatorial optimisation problems and (b) understand how to improve fairness and balance in multiagent combinatorial optimisation problems, dynamic or not. This chapter summarises the contributions towards these two aims and summarises directions for future research not addressed in this thesis.

7.1 MODELLING AND SOLVING

Current approaches to solving dynamic optimisation problems are *problem-specific*, meaning that they are tailored to one kind of problem, cannot be applied to other problems, and are not easily generalised. This is despite substantial similarities between all online optimisation methods. Furthermore, over time, many problem variables progressively become realised, i.e., their values became fixed in the past, and they can no longer affect the solution. If these realised variables and associated data are not removed, then the performance slows down significantly. As simply removing them can be incorrect, the correct removal of these variables has primarily remained a complex, manual, problem-specific, and time-consuming task. A task that requires understanding how time and data interact with the variables and constraints used to model the problem.

7.1.1 Summary and Main Contributions

The first main contribution of this thesis is a problem-independent framework for automatically solving dynamic combinatorial problems with online (including real-time) optimisation. In doing so, we have formalised many of the aspects that arise in online optimisation. Modellers use the framework by declaratively modelling the static version of a dynamic problem and then annotating it to specify how the problem inter-

acts with time. Several predicates and annotations have been added to the framework that enable modellers to formulate, at a high level, the dynamic aspects of their problem. This annotated static model is then transformed into the *update-model* needed to solve the problem online, taking into account new data and the results of previously executed decisions. The same framework can be applied to automatically create rolling horizon decompositions for large problems. The framework was implemented by extending the MiniZinc toolchain. Proof-of-concept experiments have shown that we can automatically create efficient online and rolling-horizon solutions to optimisation problems based on the annotated static models. Therefore, the framework makes online combinatorial optimisation more efficient for experienced modellers and more accessible for novices.

The second main contribution of this thesis is a novel problem-independent *garbage collection* mechanism for the aforementioned framework that ensures the internal representation of the evolving and growing problem remains as small as possible. This includes a systematic way of modelling and inferring when each part of a model is realised and how to utilise this. Importantly, the mechanism only requires the modeller to provide minimal information about the relation between time, data, and variables, thanks to the use of uniquely named objects, ensuring consistency of information across solving sessions. The garbage collection mechanism performs three main steps. First, the modeller's information is used to analyse each constraint to determine what is irrelevant or can be made irrelevant. Second, the constraints are reformulated to summarise as much of the realised information as possible. For several kinds of constraints, we have presented methods of summarising the effect that realised parts have on the future without keeping the actual parts, by reformulating the model. Third, the reformulated high-level constraint-based model is analysed to identify which parts are never used again in a given instance, before they are safely removed from future solving sessions. Empirical evaluation of the mechanism showed substantially improved model performance. An analysis has also been presented regarding what kind of constraints can and cannot be summarised. The end product is an automatic garbage collection mechanism for generic online (and real-time) combinatorial optimisation that is effective without being burdensome for the modeller.

7.1.2 Directions for Future Work

One direction for future work is to address the lack of support for modelling any a priori known stochastic aspects of dynamic problems in the presented framework. With such information, we could strengthen the solving by taking it into account, either

by solving for the robustness of the solution (e.g., leaving gaps between scheduled jobs for anticipated delays) or by optimising for the expected value (e.g., leaving room for anticipated difficult jobs, leading to a better schedule later). Relevant work include [125, 73, 158]. Related to robustness, allowing a modeller to specify stability criteria (see, e.g., [31]) is also a potential avenue for future work, where we want the incumbent solution to change as little as possible session to session. Another interesting direction for future work is understanding how to integrate methods like pattern learning and active prediction (e.g., some form of machine learning) as described in [15] with the presented framework.

On the solving side, there are also several potential avenues for future work. The first question regards how to utilise warm-starts most effectively along with this framework. A warm-start is when the solver is provided with a variable assignment (not necessarily a solution) as a starting point, and then searches for similar solutions before searching for solutions more generally. Warm-starts can be very effective as it may be wasteful to search for solutions anew, blind to any previous insights and incumbent assignments, and it can also help with stability criteria. The second question regards adaptive online optimisation, where the sessions are dynamically sized, perhaps as a response to the predicted difficulty of the session. This approach would aim to find an optimal meticulousness-quickness trade-off.

Regarding aggregation and garbage collection, the most obvious direction for future work is to classify (into the different aggregation types) more constraints and expressions that are available in MiniZinc. This requires formulating aggregation rules for these, which may be interesting to investigate in their own right. The goal would be to have a library of aggregation rules for typical MiniZinc constraints and functions.

The ultimate aim is to arrive at a general and complete theory of aggregation and garbage collection. This is required to answer questions such as how to aggregate any arbitrarily nested expression and how to make symmetry-breaking constraints and aggregation co-exist. Once a general theory, or at least close thereto, is formulated, implementing the aggregation and garbage collection framework in MiniZinc, automating the reformulations, is a natural next step.

See Sections 3.6 and 4.8 for a full discussion of future work.

7.2 FAIRNESS AND BALANCE

In optimisation problems involving multiple stakeholders, there is often a requirement (or at least a desire) for the solution to be balanced and fair. However, the fairness

literature often ignores problems of a dynamic nature, with an underlying combinatorial structure, or both, while the combinatorial optimisation literature often ignores issues of fairness and balance.

7.2.1 Summary and Main Contributions

The third main contribution of this thesis is a novel, problem- and solver-independent algorithmic framework for experimenting and increasing fairness in dynamic multi-agent combinatorial optimisation problems, while maintaining high overall utility. Since fairness is a broad concept with many schools of thought, the framework allows easy experimentation with different welfare and fairness matrices (and ways of achieving these). To show this, we provide experimental results for *utilitarian*, *Rawlsian*, and *Nash welfare*, *coefficient of variation*, the *Gini coefficient*, *relative concession*, and *proportionality*, each evaluated over three ways of measuring. To address and increase fairness, five ways of ranking the agents were employed, combined with two ways of balancing them according to their ranks. The experiments start by using a *pure utilitarian objective* and, in subsequent solving sessions, adaptively re-balance and weigh the objectives of the agents according to ranking functions that look back at the agents' performance in earlier sessions. The selection of the best ranking function and the best balancing strategy depends on the application and fairness goals. Generally, based on our experiments, the recommendation is, as a default, to use a ranking function that measures the agents' core interest (e.g., profit) both in terms of how much and how many times they have conceded utility due to sharing (e.g., *average marginal relative gap*). The framework is built on top of the MiniZinc toolchain, which allows the modeller to develop, use, and easily work with various solvers and models. This makes the framework problem- and solver-independent. Finally, several challenges and opportunities have been highlighted when it comes to ensuring fairness in online combinatorial optimisation.

The fourth and final main contribution of this thesis concerns the need for fairness in offline optimisation (including within a specific session instead of across sessions). In particular, this thesis has defined efficient propagators for the lower bound on variance and the Gini coefficient that also generate explanations of their propagations, allowing them to be used in learning CP solvers. The focus on filtering the lower bound is because minimisation of dispersion or keeping dispersion under some upper bound is typically of interest. Further, propagating the bounds of the set of variables X being measured is not considered. This is because preliminary experimentation indicated that such propagation happens very late in the search tree and cannot

lead to significant speedups. However, the explanations generated by the presented propagators could result in some propagation on the set X . The introduction of floating-point variables, which are usually not supported by clause learning solvers, is avoided by reasoning over appropriately scaled integer versions of the constraints. Note that selecting a scale is application-specific as there is a trade-off between the size of integers and fidelity.

7.2.2 Directions for Future Work

For the online fairness framework, the most obvious avenue for future work is fully addressing the possibilities and challenges presented in Section 5.3. Other than that, the most pressing issue for future work is considering other problems and other solvers. How do the kinds of dynamisms present in the problem being addressed affect the generally preferred method? For example, the results may change if there is much overlap between sessions.

There are many more welfare and inequality metrics in the literature than considered and experimented with in this thesis. There are also many more ways to configure and combine balancing and ranking functions. Therefore, more experimentation is always a possible avenue for future work. Similarly, it remains future work to map the established theories of how to measure fairness from the resource allocation literature (see, e.g., [90]) to the context of this thesis.

Strategy-proofness and envy-freeness are important concepts in many disciplines concerning fairness and should be considered for future work. While we have shown that our framework is not strategy-proof out-of-the-box, are there any modifications or restrictions that can be imposed to make it so? Further, envy-freeness, as we have discussed, is not easily applied to general optimisation problems, but it is still important to understand under what circumstances it applies.

See Section 5.7 for a full discussion of future work relevant to the above.

Regarding the dispersion propagators, in addition to investigating other dispersion metrics, there is one important direction for future work. It concerns problems that require maximising dispersion or ensuring sufficient dispersion, which need filtering algorithms for the upper bound. Filtering upper bounds require different types of approximations and roundings than presented in this thesis.

REVIEW OF NOTATION

For clarity and consistency, herein is a review of the notation used throughout this thesis presented.

Meta-variables are written with the normal mathematical fonts, including any super- or subscripts or prime marks. Lower-case italics $a, b, c, \dots$ (including ℓ) represent meta-variables ranging over any atomic constructs such as parameters, decision variables, constants, functions, relations, loop indices, elements of arrays, etc. The one it refers to will be clear by the context. Typically, a ranges over objects, c ranges over constraints, d ranges over domain values, f ranges over objective functions, j ranges over jobs (job-shop scheduling, JSP), k ranges over agents, l ranges over literals (SAT), ℓ ranges over locations (vehicle routing, VRP), m ranges over machines (JSP), n ranges over nodes (VRP), p ranges over data points, s ranges over sessions, t ranges over tasks (JSP), u ranges over utility functions, v ranges over vehicles (VRP), and x, y , and z ranges over arbitrary decision variables. Upper-case italics $A, B, C, \dots$ represent meta-variables ranging over sets, entire arrays, and entire lists. Again, context will make clear what a given meta-variable represents.

Well-known and pre-defined mathematical functions, including global constraints, are written with upright, strongly serified, lower-case Latin letters (e.g., `max`, `alldifferent`, and so on). Some, perhaps less obvious, of these functions are:

- `sols(·)` the set of solutions to a COP/CSP or constraint.
- `vars(·)` the set of variables occurring in a COP/CSP or the variables constrained by a given constraint.
- `lb(x)`, `ub(x)` respectively the lower and upper bound of a decision variable x with domain $D(x)$, i.e., $\min(D(x))$ and $\max(D(x))$.

Well-known and pre-defined mathematical classes are written in upright, strongly serified, upper-case Latin letters (e.g., NP-hard), while the well-known numeral sets are written in blackboard bold as:

- $\emptyset$ the empty set
- $\mathbb{B}$ the binary set containing true and false, (or 1 and 0, respectively, if numerals are needed).
- $\mathbb{N}$ the (non-negative) natural numbers $0, 1, 2, \dots$
- $\mathbb{Z}$ the integers $\dots, -2, -1, 0, 1, 2, \dots$
- $\mathbb{Z}^+$ the positive integers $1, 2, \dots$
- $\mathbb{Z}^-$ the negative integers $\dots, -2, -1$
- $\mathbb{Q}$ the rational numbers.
- $\mathbb{R}$ the real numbers.

MiniZinc code is written in typewriter font as in the MiniZinc IDE, e.g., `constraint`, `forall`, etc.

Greek letters and curly upper-case Latin letters are used sparingly, and only for the following purposes.

- Σ represent an iterated sum, often with a given range or condition.
- Π represents an iterated multiplication, often with a given range or condition.
- τ variables ranging over time relevant to decision-problems and solving of such problems. There are τ^{beg} , τ^{len} , and τ^{end} .
- δ, Δ are meta-variables that, respectively, ranges over possible instantiations (assignments to data points and provided object sets) and instance spaces (i.e., all possible instantiations).
- μ represents the arithmetic mean over a set of values.
- ν represents a value to centre a variable assignment around. See Chapter 6.
- σ represents the standard deviation of a set of values, and σ^2 is the variance.
- $\mathcal{L}, \mathcal{C}, \mathcal{F}$ functions that respectively returns the truth value of whether its argument decision variable or domain value is locked, committed, and forbidden.

- $\mathcal{R}, \neg\mathcal{R}$ functions that respectively return the truth value of whether its argument data point, decision variable, or object is realised and non-realised. In some contexts, they are extended to take an object set as input, respectively returning the realised and non-realised objects of that set.
- $\mathcal{G}, \neg\mathcal{G}$ functions that respectively return the truth value of whether its argument data point, decision variable, or object is garbage or realised but not garbage.
- $\mathcal{A}$ is an assignment function from variables to their values.
- $\mathcal{O}$ denotes *big O notation*, i.e., $\mathcal{O}(f(x))$ is a set of functions that grows at least as fast as $f(x)$. Formally, this is a set defined as $\{g(x) \mid \exists c, k \in \mathbb{R}, \forall y > k: |f(y)| \leq |cg(y)|\}$

The following set theoretical notation is used:

- $\in, \notin$ set inclusion and exclusion, e.g., $1 \in \mathbb{N}$. Can also be written leftward, e.g., $\mathbb{N} \ni 0$.
- $\subseteq$ subset, with possible equality, e.g., $K \subseteq L$.
- $\subset$ proper (strict) subset, not equal, e.g., $K \subset L$
- $\bigcup K$ union of all sets of K . Suppose $K = \{K_1, K_2, \dots\}$, then $K_1 \cup K_2 \cup \dots$ is the result.
- $l..u$ detones the inclusive range of integers from l to u , i.e., $\{l, l+1, \dots, u-1, u\}$.
- 2^S denotes the powerset (set of all subsets) of a given set S .
- $\langle i_1, \dots, i_n \rangle$ denotes an n -tuple or list (i.e., ordered multiset) of objects.

The logic symbols used are:

- $\exists i \in S$ existential quantification, i.e., ‘there exists an element i in S such that...’
- $\forall i \in S$ universal quantification, i.e., ‘for all elements i in S ... holds’
- $\wedge$ conjunction, i.e., “and”. It can also be used as universal quantification when limits are provided.
- $\vee$ disjunction, i.e., “or”. It can also be used as existential quantification when limits are provided.
- $\rightarrow$ implication, i.e., “if ... then ...”, may also be written $\leftarrow$ for implication in the reverse direction.

$\llbracket r(x, y) \rrbracket$ the Boolean literal (or 0-1) representation (i.e., the Iverson bracket) of the mathematical relation $r(x, y)$ within the brackets. For example, $\llbracket x \leq y + 5 \rrbracket$ is a Boolean true if $x \leq y + 5$ holds and false if the negation holds.

BIBLIOGRAPHY

- [1] Miklos Ajtai, James Aspnes, Moni Naor, Yuval Rubani, Leonard J. Schulman and Orli Waarts. ‘Fairness in Scheduling’. In: *J. Algorithms* 29 (1998), pp. 306–357 (cit. on p. 156).
- [2] Özgür Akgün, Ian P. Gent, Christopher Jefferson, Ian Miguel, Peter Nightingale and András Z. Salamon. ‘Automatic Discovery and Exploitation of Promising Subproblems for Tabulation’. In: *Principles and Practice of Constraint Programming*. Ed. by John Hooker. Springer, 2018, pp. 3–12 (cit. on p. 66).
- [3] Martin Aleksandrov and Toby Walsh. ‘Monotone and Online Fair Division’. In: *KI 2019: Advances in Artif. Intell.* Ed. by Christoph Benzmüller and Heiner Stuckenschmidt. Springer, 2019, pp. 60–75 (cit. on p. 172).
- [4] Martin Aleksandrov and Toby Walsh. ‘Online Fair Division: A Survey’. In: *AAAI’20*. AAAI Press, 2020, pp. 13557–13562 (cit. on p. 171).
- [5] Valentin Antuori and Florian Richoux. ‘Constrained optimization under uncertainty for decision-making problems: Application to Real-Time Strategy games’. In: *2019 IEEE Congress on Evolutionary Computation (CEC)*. 2019, pp. 458–465 (cit. on p. 68).
- [6] Haris Aziz and Simon Mackenzie. ‘A discrete and bounded envy-free cake cutting protocol for any number of agents’. In: *57th IEEE Ann. Symp. Foundations of Comp. Sci.* IEEE. 2016, pp. 416–427 (cit. on p. 156).
- [7] E. Balas, S. Ceria, G. Cornuéjols and N. Natraj. ‘Gomory cuts revisited’. In: *Operations Research Letters* 19.1 (1996), pp. 1–9 (cit. on p. 25).
- [8] Roman Barták. ‘Dynamic Global Constraints in Backtracking Based Environments’. In: *Annals of Operations Research* 118 (2003), pp. 101–119 (cit. on p. 140).
- [9] Roman Barták. ‘History of Constraint Programming’. In: *Wiley Encyclopedia of Operations Research and Management Science* (2010). url: <https://onlinelibrary.wiley.com/doi/abs/10.1002/9780470400531.eorms0382> (cit. on p. 26).

- [10] Nicolas Bataille, Michel Lemaitre and Gerard Verfaillie. 'Efficiency and Fairness when Sharing the Use of a Satellite'. In: *5th Intl. Symp. Artif. Intel., Robotics and Automation in Space*. Vol. 440. ESA SP. 1999, pp. 465–470 (cit. on p. 173).
- [11] EML Beale and RE Small. 'Mixed integer programming by a branch and bound technique'. In: *Proceedings of the IFIP Congress*. Vol. 2. 1965, pp. 450–451 (cit. on p. 25).
- [12] Nicolas Beldiceanu and Helmut Simonis. 'ModelSeeker: Extracting Global Constraint Models from Positive Examples'. In: *Data Mining and Constraint Programming: Foundations of a Cross-Disciplinary Approach*. Ed. by Christian Bessiere, Luc De Raedt, Lars Kotthoff, Siegfried Nijssen, Barry O'Sullivan and Dino Pedreschi. Springer, 2016, pp. 77–95 (cit. on p. 66).
- [13] David E. Bell. 'Regret in Decision Making under Uncertainty'. In: *Operations research* 30.5 (1982), pp. 961–981 (cit. on p. 155).
- [14] Gleb Belov, Natashia Boland, Martin W. P. Savelsbergh and Peter J. Stuckey. 'Local Search for a Cargo Assembly Planning Problem'. In: *CPAIOR'14*. LCNS 8451. Springer, 2014, pp. 159–175 (cit. on pp. 16, 67, 98).
- [15] Russell Bent and Pascal Van Hentenryck. 'Scenario-Based Planning for Partially Dynamic Vehicle Routing with Stochastic Customers'. In: *Operations Research* 52.6 (2004), pp. 977–987 (cit. on pp. 99, 221).
- [16] Russell Bent and Pascal Van Hentenryck. 'The Value of Consensus in Online Stochastic Scheduling'. In: *ICAPS'04*. Ed. by Shlomo Zilberstein, Jana Koehler and Sven Koenig. AAAI, 2004, pp. 219–226 (cit. on p. 68).
- [17] Patrizia Beraldi, Antonio Violi, Nadia Scordino and Nicola Sorrentino. 'Short-term electricity procurement: A rolling horizon stochastic programming approach'. In: *Applied Mathematical Modelling* 35.8 (2011), pp. 3980–3990 (cit. on p. 68).
- [18] Dimitris Bertsimas, Vivek F. Farias and Nikolaos Trichakis. 'The Price of Fairness'. In: *Operations Research* 59.1 (2011), pp. 17–31 (cit. on pp. 146, 159, 160).
- [19] Dimitris Bertsimas, Patrick Jaillet and Sébastien Martin. 'Online Vehicle Routing: The Edge of Optimization in Large-Scale Applications'. In: *Operations Research* 67.1 (2019), pp. 143–162 (cit. on pp. 53, 67).
- [20] Christian Bessiere, Emmanuel Hebrard, George Katsirelos, Zeynep Kiziltan, Émilie Picard-Cantin, Claude-Guy Quimper and Toby Walsh. 'The Balance Constraint Family'. In: *Principles and Practice of Constraint Programming*. Ed. by Barry O'Sullivan. Springer, 2014, pp. 174–189 (cit. on p. 217).

-
- [21] Armin Biere, Marijn Heule and Hans van Maaren, eds. *Handbook of Satisfiability*. Vol. 185. IOS press, 2009 (cit. on pp. 13, 23, 25, 198).
- [22] Alessio Bonfietti and Michele Lombardi. ‘The Weighted Average Constraint’. In: *Principles and Practice of Constraint Programming*. Ed. by Michela Milano. Springer, 2012, pp. 191–206 (cit. on p. 217).
- [23] Felix Brandt, Vincent Conitzer, Ulle Endriss, Jérôme Lang and Ariel D Procaccia. *Handbook of computational social choice*. Cambridge University Press, 2016 (cit. on pp. 153–155).
- [24] Kenneth N. Brown and Ian Miguel. ‘Uncertainty and Change’. In: *Handbook of Constraint Programming*. Elsevier, 2006. Chap. 21, pp. 731–760 (cit. on pp. 48, 66).
- [25] Michael R. Bussieck and Alex Meeraus. ‘General Algebraic Modeling System (GAMS)’. In: *Modeling Languages in Mathematical Optimization*. Ed. by Josef Kallrath. Vol. 88. APOP. Boston, MA: Springer, 2004, pp. 137–157 (cit. on p. 42).
- [26] Violet X. Chen. ‘Fairness Methods in Optimization and Artificial Intelligence’. PhD thesis. Carnegie Mellon University, 2022 (cit. on p. 174).
- [27] Violet X. Chen and J. N. Hooker. ‘A Guide to Formulating Equity and Fairness in an Optimization Model’. Preprint; submitted. 2022. url: <http://public.tepper.cmu.edu/jnh/papers.html> (cit. on pp. 162, 174).
- [28] Steve Chien, Russell Knight, Andre Stechert, Rob Sherwood and Gregg Rabideau. ‘Using Iterative Repair to Improve the Responsiveness of Planning and Scheduling’. In: *Proc. the 5th Intl. Conf. Artif. Intell. Planning Systems*. AAAI Press, 2000, pp. 300–307 (cit. on p. 67).
- [29] Geoffrey Chu. ‘Improving Combinatorial Optimization’. Available at <http://hdl.handle.net/11343/36679>; the Chuffed solver and MiniZinc backend are available at <https://github.com/chuffed/chuffed>. PhD thesis. Department of Computing and Information Systems, University of Melbourne, Australia, 2011 (cit. on pp. 33, 180, 199).
- [30] Alistair R. Clark and Simon J. Clark. ‘Rolling-horizon lot-sizing when set-up times are sequence-dependent’. In: *Intl. J. Production Research* 38.10 (2000), pp. 2287–2307 (cit. on p. 67).
- [31] Laura Climent, Richard J. Wallace, Miguel A. Salido and Frederico Barber. ‘Robustness and Stability in Constraint Programming under Dynamism and Uncertainty’. In: *J. Artif. Intell. Research* 49 (2014), pp. 49–78 (cit. on pp. 51, 55, 99, 221).

- [32] Carlos A. Coello Coello, Clarisse Dhaenens and Laetitia Jourdan. 'Multi-Objective Combinatorial Optimization: Problematic and Context'. In: *Advances in Multi-Objective Nature Inspired Computing*. Ed. by Carlos A. Coello Coello, Clarisse Dhaenens and Laetitia Jourdan. Vol. 272. Studies in Computational Intelligence. Springer, 2010, pp. 1–21 (cit. on p. 150).
- [33] Stephen A. Cook. 'The complexity of theorem-proving procedures'. In: *3rd annual ACM symposium on Theory of computing*. 1971, pp. 151–158 (cit. on p. 25).
- [34] Gerardo Costa and Colin Stirling. 'Weak and strong fairness in CCS'. In: *Information and Computation* 73.3 (1987), pp. 207–244 (cit. on p. 172).
- [35] George Dantzig, Ray Fulkerson and Selmer Johnson. 'Solution of a large-scale traveling-salesman problem'. In: *J. the operations research society of America* 2.4 (1954), pp. 393–410 (cit. on p. 24).
- [36] George B Dantzig. 'Maximization of a linear function of variables subject to linear inequalities'. In: *Activity analysis of production and allocation* 13 (1951), pp. 339–347 (cit. on p. 24).
- [37] George B. Dantzig. 'Linear Programming under Uncertainty'. In: *Management Science* 1.3/4 (1955), pp. 197–206. url: <http://www.jstor.org/stable/2627159> (cit. on p. 50).
- [38] Martin Davis, George Logemann and Donald Loveland. 'A machine program for theorem-proving'. In: *Communications of the ACM* 5.7 (1962), pp. 394–397 (cit. on p. 25).
- [39] Rina Dechter and Avi Dechter. 'Belief Maintenance in Dynamic Constraint Networks'. In: *AAAI/88*. AAAI Press, 1988, pp. 37–42 (cit. on p. 66).
- [40] Jip J. Dekker, Maria Garcia de la Banda, Andreas Schutt, Peter J. Stuckey and Guido Tack. 'Solver-Independent Large Neighbourhood Search'. In: *Principles and Practice of Constraint Programming*. Ed. by John N. Hooker. LNCS 11008. Springer, 2018, pp. 81–98 (cit. on pp. 68, 78).
- [41] Philip M. Dixon, Jacob Weiner, Thomas Mitchell-Olds and Robert A. Woodley. 'Bootstrapping the Gini coefficient of inequality'. In: *Ecology* 68 (1987). (Erratum in *Ecology*, 69:1307, 1987), pp. 1548–1551 (cit. on p. 205).
- [42] Fabian Dunke and Stefan Nickel. 'A general modeling approach to online optimization with lookahead'. In: *Omega* 63 (2016), pp. 134–153 (cit. on pp. 55, 70).

-
- [43] Matthias Ehrgott. *Multicriteria optimization*. Vol. 491. Springer Science & Business Media, 2005 (cit. on p. 152).
 - [44] Ronald Fagin and John H. Williams. ‘A Fair Carpool Scheduling Algorithm’. In: *IBM J. Res. Devel.* 27.2 (1983), pp. 133–139 (cit. on p. 171).
 - [45] Boi Faltings and Santiago Macho-Gonzalez. ‘Open constraint programming’. In: *Artif. Intell.* 161.1-2 (2005), pp. 181–208 (cit. on p. 140).
 - [46] Hélène Fargier and Jérôme Lang. ‘Uncertainty in Constraint Satisfaction Problems: a Probabilistic Approach’. In: *Symbolic and Quantitative Approaches to Reasoning and Uncertainty, European Conf. LNCS 747*. Springer, 1993, pp. 97–104 (cit. on p. 69).
 - [47] Hélène Fargier, Jérôme Lang and Thomas Schiex. ‘Mixed Constraint Satisfaction: A Framework for Decision Problems under Incomplete Knowledge’. In: *AAAI’96*. AAAI Press / The MIT Press, 1996, pp. 175–180 (cit. on p. 69).
 - [48] Thibaut Feydy and Peter J. Stuckey. ‘Lazy Clause Generation Reengineered’. In: *Principles and Practice of Constraint Programming*. Ed. by Ian P. Gent. Springer, 2009, pp. 352–366 (cit. on p. 199).
 - [49] Pierre Flener, Justin Pearson and Magnus Ågren. ‘Introducing ESRA, a Relational Language for Modelling Combinatorial Problems’. In: *Logic Based Program Synthesis and Transformation*. Ed. by Maurice Bruynooghe. LNCS 3018. Springer, 2004, pp. 214–232 (cit. on p. 43).
 - [50] Lance Fortnow and Steve Homer. ‘A Short History of Computational Complexity’. In: *Bull. of the EATCS* 80.01 (2003), pp. 95–133 (cit. on p. 25).
 - [51] Robert Fourer, David M. Gay and Brian W. Kernighan. *AMPL: A Modeling Language for Mathematical Programming*. 2nd. Cengage Learning, 2002 (cit. on pp. 14, 42, 66).
 - [52] David W. Fowler and Kenneth N. Brown. ‘Branching Constraint Satisfaction Problems for Solutions Robust under Likely Changes’. In: *Principles and Practice of Constraint Programming*. Ed. by Rina Dechter. LNCS 1894. Springer, 2000, pp. 500–504 (cit. on p. 69).
 - [53] Eugene C. Freuder. ‘In Pursuit of the Holy Grail’. In: *Constraints* 2.1 (1997), pp. 57–61 (cit. on p. 66).
 - [54] Eugene C. Freuder. ‘Constraints: The Ties that Bind’. In: *AAAI’06*. AAAI Press, 2006, pp. 1520–1523 (cit. on p. 11).

- [55] Eugene C. Freuder. ‘Progress towards the Holy Grail’. In: *Constraints* 23.2 (2018), pp. 158–171 (cit. on p. 66).
- [56] Eugene C. Freuder and Barry O’Sullivan. ‘Grand challenges for constraint programming’. In: *Constraints* 19.2 (2014), pp. 150–162 (cit. on p. 66).
- [57] Eugene C. Freuder and Richard J. Wallace. ‘Partial constraint satisfaction’. In: *Artif. Intell.* 58.1-3 (1992), pp. 21–70 (cit. on p. 153).
- [58] Alan M. Frisch, Warwick Harvey, Chris Jefferson, Bernadette Martínez-Hernández and Ian Miguel. ‘Essence: A constraint language for specifying combinatorial problems’. In: *Constraints* 13.3 (2008), pp. 268–306 (cit. on pp. 14, 43, 66).
- [59] Dov Gabbay, Amir Pnueli, Saharon Shelah and Jonathan Stavi. ‘On the Temporal Analysis of Fairness’. In: *Proc. the 7th ACM SIGPLAN-SIGACT Symp. Principles of Programming Languages*. 1980, pp. 163–173 (cit. on p. 172).
- [60] George Gamow and Marvin Stern. *Puzzle-Math*. Viking Books, 1958 (cit. on p. 157).
- [61] Graeme Gange. *Geas*. Repository. Available at <https://bitbucket.org/gkgange/geas/src/master>. 2022 (cit. on p. 33).
- [62] Partha Gangopadhyay. ‘On the Coase Theorem and Coalitional Stability: The Principle of Equal Relative Concession’. In: *Theory and Decision* 48.2 (2000), pp. 179–191 (cit. on p. 157).
- [63] Joseph L Gastwirth. ‘The estimation of the Lorenz curve and Gini index’. In: *The review of economics and statistics* (1972), pp. 306–316 (cit. on p. 217).
- [64] Gecode Team. *Gecode: A Generic Constraint Development Environment*. <http://www.gecode.org>. 2020 (cit. on pp. 33, 104, 114, 140).
- [65] Ali Ghodsi, Matei Zaharia, Scott Shenker and Ion Stoica. ‘Choosy: Max-Min Fair Sharing for Datacenter Jobs with Constraints’. In: *8th ACM Europ. Conf. Computer Systems*. ACM, 2013, pp. 365–378 (cit. on p. 174).
- [66] Rob van Glabbeek and Peter Höfner. ‘Progress, justness, and fairness’. In: *ACM Comp. Surveys* 52.4 (2019), pp. 1–38 (cit. on p. 172).
- [67] Ralph E. Gomory. *An algorithm for the mixed integer problem*. Tech. rep. RM-2597-PR. The RAND Corp., 1960 (cit. on p. 25).
- [68] Ralph E. Gomory. ‘An algorithm for integer solutions to linear programs’. In: *Recent advances in mathematical programming* 64.260-302 (1963) (cit. on p. 25).
- [69] Google Optimization Team. *OR-Tools: Google’s Software Suite for Combinatorial Optimization*. Available at <https://developers.google.com/optimization>. 2022 (cit. on p. 33).

-
- [70] Martin Grötschel, Sven O. Krumke, Jörg Rambau, Thomas Winter and Uwe T. Zimmermann. ‘Combinatorial Online Optimization in Real Time’. In: *Online Optimization of Large Scale Systems*. Ed. by Martin Grötschel, Sven O. Krumke and Jörg Rambau. Springer, 2001, pp. 679–704 (cit. on pp. 53, 55, 58, 60, 66).
- [71] Yacov Y. Haimes, Leon S. Lasdon and David A. Wismer. ‘On a bicriterion formulation of the problems of integrated system identification and system optimization’. In: *IEEE Transactions on Systems, Man, and Cybernetics* 1.3 (1971), pp. 296–297 (cit. on p. 153).
- [72] Shan He, Mark Wallace, Graeme Gange, Ariel Liebman and Campbell Wilson. ‘A Fast and Scalable Algorithm for Scheduling Large Numbers of Devices Under Real-Time Pricing’. In: *Principles and Practice of Constraint Programming*. LNCS 11008. Springer, 2018, pp. 649–666 (cit. on p. 68).
- [73] David Hemmi. ‘Modelling and Solving Techniques for Stochastic Combinatorial Optimisation Problems’. PhD thesis. Caulfield School of Information Technology, Monash University, Australia, 2018 (cit. on pp. 99, 221).
- [74] Frank L Hitchcock. ‘The distribution of a product from several sources to numerous localities’. In: *J. mathematics and physics* 20.1-4 (1941), pp. 224–230 (cit. on p. 24).
- [75] Brahim Hnich, Roberto Rossi, S. Armagan Tarim and Steven Prestwich. ‘A Survey on CP-AI-OR Hybrids for Decision Making Under Uncertainty’. In: *Hybrid Optimization: The Ten Years of CPAIOR*. Ed. by Pascal van Hentenryck and Michela Milano. Springer, 2011, pp. 227–270 (cit. on p. 66).
- [76] Edgar M Hoover. ‘The measurement of industrial localization’. In: *The Review of Economic Statistics* (1936), pp. 162–171 (cit. on p. 149).
- [77] SHI Huaizhou, R Venkatesha Prasad, Ertan Onur and IGMM Niemegeers. ‘Fairness in wireless networks: Issues, measures and challenges’. In: *IEEE Communications Surveys & Tutorials* 16.1 (2013), pp. 5–24 (cit. on p. 173).
- [78] Shyh-In Hwang and Sheng-Tzong Cheng. ‘Combinatorial Optimization in Real-Time Scheduling: Theory and Algorithms’. In: *J. Combinatorial Optimization* 5.3 (2001), pp. 345–375 (cit. on p. 67).
- [79] IBM Analytics Team. *IBM ILOG CP Optimizer*. Webpage. Available at <https://www.ibm.com/au-en/analytics/cplex-cp-optimizer>. 2022 (cit. on p. 33).

- [80] Joxan Jaffar and J-L Lassez. ‘Constraint logic programming’. In: *14th ACM SIGACT-SIGPLAN symp. on Principles of programming languages*. 1987, pp. 111–119 (cit. on p. 26).
- [81] Joxan Jaffar, Spiro Michaylov, Peter J Stuckey and Roland HC Yap. ‘The CLP(R) language and system’. In: *ACM Transactions on Programming Languages and Systems (TOPLAS)* 14.3 (1992), pp. 339–395 (cit. on p. 26).
- [82] Patrick Jaillet and Michael R. Wagner. ‘Online Vehicle Routing Problems: A Survey’. In: *The Vehicle Routing Problem: Latest Advances and New Challenges*. Vol. 43. OR/CS Interfaces. Springer, 2008, pp. 221–237 (cit. on pp. 67, 140).
- [83] Richard E. Jones, Antony L. Hosking and J. Eliot B. Moss. *The Garbage Collection Handbook: The art of automatic memory management*. Chapman and Hall / CRC Applied Algorithms and Data Structures Series. CRC Press, 2011 (cit. on pp. 104, 139).
- [84] Silke Jütte, Daniel Müller and Ulrich W. Thonemann. ‘Optimizing railway crew schedules with fairness preferences’. In: *J. Scheduling* 20.1 (2017), pp. 43–55 (cit. on p. 173).
- [85] Ehud Kalai and Meir Smorodinsky. ‘Other solutions to Nash’s bargaining problem’. In: *Econometrica* (1975), pp. 513–518 (cit. on pp. 161, 162).
- [86] Leonid V Kantorovich. ‘The mathematical method of production planning and organization’. In: *Management Science* 6.4 (1939), pp. 363–422 (cit. on p. 24).
- [87] Richard M Karp. ‘Reducibility among combinatorial problems’. In: *Complexity of computer computations*. Springer, 1972, pp. 85–103 (cit. on p. 25).
- [88] Philip Kilby, Patrick Prosser and Paul Shaw. *Dynamic VRPs: A Study of Scenarios*. Tech. rep. Report APES-06-1998. University of Strathclyde, 2002 (cit. on pp. 55, 56).
- [89] Daniel Kroening and Ofer Strichman. *Decision Procedures: An Algorithmic Point of View*. second. Texts in Theoretical Computer Science, an EATCS Series. Springer, 2016 (cit. on p. 13).
- [90] Tian Lan, David Kao, Mung Chiang and Ashutosh Sabharwal. ‘An Axiomatic Theory of Fairness in Network Resource Allocation’. In: *29th IEEE Intl. Conf. Comp. Communications*. IEEE. 2010, pp. 1343–1351 (cit. on pp. 160, 192, 223).
- [91] AH Land and AG Doig. ‘An Automatic Method of Solving Discrete Programming Problems’. In: *Econometrica* 28.3 (1960), pp. 497–520 (cit. on p. 24).

-
- [92] Javier Larrosa and Thomas Schiex. ‘Solving weighted CSP by maintaining arc consistency’. In: *Artif. Intell.* 159.1-2 (2004), pp. 1–26 (cit. on p. 153).
 - [93] Kevin Leo, Christopher Mears, Guido Tack and Maria Garcia de la Banda. ‘Globalizing constraint models’. In: *Artif. Intell.* 302 (2022) (cit. on p. 66).
 - [94] Joseph Y-T Leung. *Handbook of Scheduling: Algorithms, Models, and Performance Analysis*. London: London: CRC Press LLC, 2004 (cit. on p. 172).
 - [95] Leonid Anatolevich Levin. ‘Universal sequential search problems’. In: *Probl. Pere-dachi Inf.* 9.3 (1973), pp. 115–116 (cit. on p. 25).
 - [96] Chu Min Li and Felip Manyà. ‘MaxSAT, hard and soft constraints’. In: *Handbook of satisfiability*. IOS Press, 2009, pp. 613–631 (cit. on p. 36).
 - [97] BoonPing Lim, Hassan Hijazi, Sylvie Thiébaux and Menkes van den Briel. ‘On-line HVAC-Aware Occupancy Scheduling with Adaptive Temperature Control’. In: *Principles and Practice of Constraint Programming*. LNCS 9892. Springer, 2016, pp. 683–700 (cit. on pp. 53, 67).
 - [98] Yongkui Liu, Lihui Wang, Xi Vincent Wang, Xun Xu and Lin Zhang. ‘Scheduling in Cloud Manufacturing: State-of-the-art and Research Challenges’. In: *Intl. J. Production Research* 57.15–16 (2019), pp. 4854–4879 (cit. on p. 143).
 - [99] Harry M Markowitz and Alan S Manne. ‘On the solution of discrete programming problems’. In: *Econometrica* (1957), pp. 84–110 (cit. on p. 24).
 - [100] Julien F Marquant, Ralph Evins and Jan Carmeliet. ‘Reducing computation time with a rolling horizon approach applied to a MILP formulation of multiple urban energy hub system’. In: *Procedia Computer Science* 51 (2015), pp. 2137–2146 (cit. on pp. 16, 68).
 - [101] Christopher Mears, Maria Garcia de la Banda, Mark Wallace and Bart Demoen. ‘A method for detecting symmetries in constraint models and its generalisation’. In: *Constraints* 20.2 (2015), pp. 235–273 (cit. on p. 39).
 - [102] Farhad Mehran. ‘Bounds on the Gini index based on observed points of the Lorenz curve’. In: *J. the American Statistical Association* 70.349 (1975), pp. 64–66 (cit. on p. 217).
 - [103] Amit Metodi, Michael Codish and Peter J. Stuckey. ‘Boolean Equi-propagation for Concise and Efficient SAT Encodings of Combinatorial Problems’. In: *J. Artif. Intell. Research* 46 (2013), pp. 303–341 (cit. on p. 140).

- [104] Jean-Noël Monette, Nicolas Beldiceanu, Pierre Flener and Justin Pearson. ‘A parametric propagator for pairs of Sum constraints with a discrete convexity property’. In: *AJ* 241 (2016), pp. 170–190 (cit. on p. 217).
- [105] Hervé Moulin. *Fair Division and Collective Welfare*. MIT Press, 2004 (cit. on p. 156).
- [106] John F Nash Jr. ‘The bargaining problem’. In: *Econometrica* (1950), pp. 155–162 (cit. on pp. 161, 162).
- [107] Nicholas Nethercote, Peter J. Stuckey, Ralph Becket, Sebastian Brand, Gregory J. Duck and Guido Tack. ‘MiniZinc: Towards a Standard CP Modelling Language’. In: *Principles and Practice of Constraint Programming*. LNCS 4741. Springer, 2007, pp. 529–543 (cit. on pp. 14, 43, 66, 212).
- [108] Barry O’Sullivan. ‘Automated Modelling and Solving in Constraint Programming’. In: *AAAI’10*. Ed. by Maria Fox and David Poole. AAAI Press, 2010, pp. 1493–1497 (cit. on p. 66).
- [109] Barry O’Sullivan. ‘Opportunities and Challenges for Constraint Programming’. In: *AAAI’12*. Ed. by Jörg Hoffmann and Bart Selman. AAAI Press, 2012, pp. 2148–2152 (cit. on p. 66).
- [110] Włodzimierz Ogryczak, Hanan Luss, Michał Pióro, Dritan Nace and Artur Tomaszewski. ‘Fair Optimization and Networks: A Survey’. In: *J. Appl. Math.* 2014 (2014), pp. 1–25 (cit. on p. 173).
- [111] Olga Ohrimenko, Peter Stuckey and Michael Codish. ‘Propagation via lazy clause generation’. In: *Constraints* 14 (3 2009), pp. 357–391 (cit. on pp. 18, 196, 198, 199, 217).
- [112] Philippe Olivier, Andrea Lodi and Gilles Pesant. ‘Measures of balance in combinatorial optimization’. In: *4OR* (2021) (cit. on p. 217).
- [113] William Orchard-Hays. ‘History of the development of LP solvers’. In: *Interfaces* 20.4 (1990), pp. 61–73 (cit. on p. 24).
- [114] Fanny Pascual, Krzysztof Rządca and Piotr Skowron. ‘Collective Schedules: Scheduling Meets Computational Social Choice’. In: *17th Intl. Conf. Autonomous Agents and Multi-Agent Systems*. 2018, pp. 667–675 (cit. on p. 154).
- [115] Guillaume Perez and Jean-Charles Régin. ‘MDDs are Efficient Modeling Tools: An Application to Some Statistical Constraints’. In: *CPAIOR’17*. Ed. by Domenico Salvagnin and Michele Lombardi. Springer, 2017, pp. 30–40 (cit. on p. 216).

-
- [116] Gilles Pesant. ‘Achieving Domain Consistency and Counting Solutions for Dispersion Constraints’. In: *INFORMS J. Comput.* 27.4 (2015), pp. 690–703 (cit. on p. 216).
- [117] Gilles Pesant and Jean-Charles Régin. ‘SPREAD: A Balancing Constraint Based on Statistics’. In: *Principles and Practice of Constraint Programming*. Ed. by Peter van Beek. LNCS 3709. Springer, 2005, pp. 460–474 (cit. on pp. 199–201, 204, 216, 217).
- [118] Warren B. Powell. *Reinforcement Learning and Stochastic Optimization: A Unified Framework for Sequential Decisions*. Wiley-Blackwell, 2022 (cit. on p. 67).
- [119] Charles Prud’homme, Jean-Guillaume Fages and Xavier Lorca. *Choco: A Free and Open-Source Java Library for Constraint Programming*. Available at <http://www.choco-solver.org>. 2014 (cit. on p. 33).
- [120] Kirk R. Pruhs, Jivri Sgall and Eric Torng. ‘Online Scheduling’. In: *Handbook of Scheduling — Algorithms, Models, and Performance Analysis*. Chapman and Hall/CRC, 2004 (cit. on pp. 53, 67, 140).
- [121] Gregg Rabideau, Russell Knight, Steve Chien, Alex Fukunaga and Anita Govindjee. ‘Iterative repair planning for spacecraft operations using the ASPEN system’. In: *Artif. Intell., Robotics and Automation in Space*. Vol. 440. 1999, pp. 99–109 (cit. on p. 67).
- [122] Katayoun Rahbar, Jie Xu and Rui Zhang. ‘Real-Time Energy Storage Management for Renewable Integration in Microgrid: An Off-Line Optimization Approach’. In: *IEEE Transactions on Smart Grid* 6.1 (2015), pp. 124–134 (cit. on p. 67).
- [123] John Rawls. *A Theory of Justice*. 2nd ed. Belknap Press, 1999 (cit. on p. 147).
- [124] Andrea Rendl, Tias Guns, Peter J. Stuckey and Guido Tack. ‘MiniSearch: A Solver-Independent Meta-Search Language for MiniZinc’. In: *Principles and Practice of Constraint Programming*. LNCS 9255. Springer, 2015, pp. 376–392 (cit. on p. 78).
- [125] Andrea Rendl, Guido Tack and Peter J. Stuckey. ‘Stochastic MiniZinc’. In: *Principles and Practice of Constraint Programming*. Ed. by Barry O’Sullivan. LNCS 8656. Springer, 2014, pp. 636–645 (cit. on pp. 99, 221).
- [126] Florian Richoux, Alberto Uriarte and Jean-François Baffier. ‘GHOST: A Combinatorial Optimization Framework for Real-Time Problems’. In: *IEEE Trans. Comput. Intell. AI Games* 8.4 (2016), pp. 377–388 (cit. on pp. 56, 68).
- [127] John Alan Robinson. ‘A machine-oriented logic based on the resolution principle’. In: *J. the ACM* 12.1 (1965), pp. 23–41 (cit. on p. 25).

- [I28] Marcel Roelofs and Johannes Bisschop. *AIMMS: The Language Reference*, AIMMS 4. March 3, 2022. Available at: www.aimms.com. 2022 (cit. on pp. 42, 68, 140).
- [I29] Francesca Rossi, Peter van Beek and Toby Walsh, eds. *Handbook of Constraint Programming*. Elsevier, 2006 (cit. on pp. 13, 23, 26).
- [I30] Roberto Rossi, Özgür Akgün, Steven Prestwich and S. Armagan Tarim. *Declarative Statistics*. 2017. arXiv: 1708.01829 [cs.AI] (cit. on p. 217).
- [I31] Zsofi Ruttkay. ‘Fuzzy constraint satisfaction’. In: *Proc. of 1994 IEEE 3rd Intl. Fuzzy Systems Conf.* IEEE, 1994, pp. 1263–1268 (cit. on p. 153).
- [I32] Pierre Schaus, Yves Deville and Pierre Dupont. ‘Bound-Consistent Deviation Constraint’. In: *Principles and Practice of Constraint Programming*. Ed. by Christian Bessière. Springer, 2007, pp. 620–634 (cit. on p. 217).
- [I33] Pierre Schaus and Jean-Charles Régin. ‘Bound-consistent spread constraint’. In: *EURO J. Comput. Optim.* 2.3 (2014), pp. 123–146 (cit. on pp. 216, 217).
- [I34] Alexander Schiendorfer. ‘Soft Constraints in MiniBrass: Foundations and Applications’. PhD thesis. University of Augsburg, Germany, 2019 (cit. on p. 153).
- [I35] Alexander Schiendorfer and Wolfgang Reif. ‘Reducing Bias in Preference Aggregation for Multiagent Soft Constraint Problems’. In: *Principles and Practice of Constraint Programming*. LNCS 11802. Springer, 2019, pp. 510–526 (cit. on p. 154).
- [I36] Alexander Schrijver. *Theory of Linear and Integer Programming*. John Wiley & Sons, 1998 (cit. on pp. 13, 23, 24).
- [I37] Alexander Schrijver. *Combinatorial Optimization: Polyhedra and Efficiency*. Ed. by W.J. Cook, R.L. Graham, B. Korte, L. Lovász, A. Wigderson and G.M. Ziegler. Vol. 24. Algorithms and Combinatorics. Springer, 2003 (cit. on pp. 11, 13, 23, 25).
- [I38] Alexander Schrijver. ‘On the History of Combinatorial Optimization (Till 1960)’. In: *Discrete Optimization*. Elsevier, 2005, pp. 1–68 (cit. on p. 24).
- [I39] Christian Schulte and Peter J. Stuckey. ‘Dynamic variable elimination during propagation solving’. In: *Principles and Practice of Declarative Programming*. Ed. by Sergio Antoy and Elvira Albert. ACM, 2008, pp. 247–257 (cit. on p. 140).
- [I40] Steven S. Seiden. ‘On the Online Bin Packing Problem’. In: *J. ACM* 49.5 (2002), pp. 640–671 (cit. on pp. 56, 60).
- [I41] Amartya Sen. *On economic inequality*. Annexe. Oxford New York: Clarendon Press Oxford University Press, 1997 (cit. on pp. 17, 146).
- [I42] Suresh Sethi and Gerhard Soerger. ‘A theory of rolling horizon decision making’. In: *Annals of Operations Research* 29 (1991), pp. 387–415 (cit. on p. 53).

-
- [143] Lloyd S. Shapley. *Notes on the n -Person Game II: The Value of an n -Person Game*. Tech. rep. Rand Corporation, 1951 (cit. on p. 154).
- [144] Piotr Skowron and Krzysztof Rzdca. ‘Fair Share Is Not Enough: Measuring Fairness in Scheduling with Cooperative Game Theory’. In: *Rev. Sel. Papers 10th Intl. Conf. Parallel Processing and Applied Mathematics, Part II*. Ed. by Roman Wyrzykowski, Jack J. Dongarra, Konrad Karczewski and Jerzy Wasniewski. LNCS 8385. Berlin: Springer, 2013, pp. 38–48 (cit. on p. 154).
- [145] Alice E. Smith, David W. Coit, Thomas Baeck, David Fogel and Zbigniew Michalewicz. ‘Penalty Functions’. In: *Handbook of Evolutionary Computation* 97.1 (1997), p. C5 (cit. on p. 153).
- [146] A. Sprecher, R. Kolisch and A. Drexl. ‘Semi-active, active and non-delay schedules for the resource-constrained project scheduling problem’. In: *European J. Operational Research* 80 (1995), pp. 94–102 (cit. on p. 97).
- [147] Hugo Steinhaus. ‘The problem of Fair Division’. In: *Econometrica* 16 (1948), pp. 101–104 (cit. on p. 156).
- [148] Peter J Stuckey, Thibaut Feydy, Andreas Schutt, Guido Tack and Julien Fischer. ‘The MiniZinc challenge 2008–2013’. In: *AI Magazine* 35.2 (2014), pp. 55–60 (cit. on pp. 18, 46).
- [149] Peter J. Stuckey. ‘Lazy Clause Generation: Combining the Power of SAT and CP (and MIP?) Solving’. In: *CPAIOR’10*. Ed. by Andrea Lodi, Michela Milano and Paolo Toth. Springer, 2010, pp. 5–9 (cit. on p. 196).
- [150] Eric Taillard. ‘Benchmarks for basic scheduling problems’. In: *European J. Operational Research* 64.2 (1993), pp. 278–285 (cit. on p. 214).
- [151] Andrew S. Tanenbaum and Herbert Bos. *Modern Operating Systems*. Pearson, 2015 (cit. on p. 172).
- [152] S. Armagan Tarim, Suresh Manandhar and Toby Walsh. ‘Stochastic Constraint Programming: A Scenario-Based Approach’. In: *Constraints* 11.1 (2006), pp. 53–80 (cit. on p. 70).
- [153] Henri Theil. *Economics and information theory*. Tech. rep. 1967 (cit. on p. 149).
- [154] William Thomson. ‘Chapter 35 Cooperative models of bargaining’. In: *Handbook of Game Theory with Economic Applications*. Handbook of game theory and economic applications. Elsevier, 1994, pp. 1237–1284 (cit. on pp. 159, 160).

- [155] E. L. Ulungu and J. Teghem. 'Multi-objective combinatorial optimization problems: A survey'. In: *J. Multi-Criteria Decision Analysis* 3.2 (1994), pp. 83–104 (cit. on p. 150).
- [156] P. Van Hentenryck, L. Michel, L. Perron and J. -C. Régin. 'Constraint Programming in OPL'. In: *Intl. Symp. on Principles and Practice of Declarative Programming*. LNCS 1702. Springer, 1999, pp. 98–116 (cit. on pp. 14, 43).
- [157] Pascal Van Hentenryck. *Constraint satisfaction in logic programming*. MIT press, 1989 (cit. on p. 26).
- [158] Pascal Van Hentenryck and Russell Bent. *Online Stochastic Combinatorial Optimization*. MIT Press, 2009 (cit. on pp. 23, 53, 66, 67, 99, 221).
- [159] Pascal Van Hentenryck and Michela Milano, eds. *Hybrid Optimization: The Ten Years of CPAIOR*. Vol. 45. Springer Optimization and Its Applications. Springer, 2011 (cit. on p. 11).
- [160] Gérard Verfaillie and Narendra Jussien. 'Constraint Solving in Uncertain and Dynamic Environments: A Survey'. In: *Constraints* 10.3 (2005), pp. 253–281 (cit. on pp. 66, 69).
- [161] Gérard Verfaillie, Cédric Pralet and Michel Lemaître. 'How to model planning and scheduling problems using constraint networks on timelines'. In: *Knowl. Eng. Rev.* 25.3 (2010), pp. 319–336 (cit. on p. 70).
- [162] Mark Wallace. 'Practical Applications of Constraint Programming'. In: *Constraints* 1.1/2 (1996), pp. 139–168 (cit. on p. 11).
- [163] Toby Walsh. 'Stochastic constraint programming'. In: *ECAI'02*. Ed. by F. van Harmelen. Vol. 2. 2002, pp. 111–115 (cit. on p. 70).
- [164] Toby Walsh. 'Fair Division: The Computer Scientist's Perspective'. In: *Proc. of the 29th Intl. Joint Conf. Artif. Intell. IJCAI'20*. Yokohama, Yokohama, Japan, 2021 (cit. on p. 156).
- [165] Guiyi Wei, Athanasios V. Vasilakos, Yao Zheng and Naixue Xiong. 'A game-theoretic method of fair resource allocation for cloud computing services'. In: *J. Supercomputing* 54.2 (2010), pp. 252–269 (cit. on pp. 154, 156).
- [166] Insoon Yang. 'Online Combinatorial Optimization for Interconnected Refrigeration Systems: Linear Approximation and Submodularity'. In: *IFAC-PapersOnLine* 50.1 (2017). 20th IFAC World Congress, pp. 7389–7396 (cit. on p. 56).

- [167] Chunyang Yu, Xun Xu, Shiqiang Yu, Zhiqian Sang, Cheng Yang and Xuanlin Jiang. 'Shared Manufacturing in the sharing Economy: Concept, definition and service operations'. In: *Comp. & Industr. Eng.* 146 (2020) (cit. on p. 143).

